

XCPC 算法模板

至圣 && Nrtusea && Hiraethsoul

目录

1	Prepare 准备工作	1
1.1	Testing 测试	1
1.1.1	Floyd 传递闭包	1
1.1.2	Floyd with Bitset 传递闭包 bitset 优化	1
1.1.3	Basic operations 基本运算	1
1.1.4	Float operations 浮点运算	2
1.1.5	STL performance 1e6 STL 性能	2
1.2	Shell script 编译脚本	2
1.2.1	main.sh 编译运行	2
1.2.2	checker.sh 对拍脚本	2
1.2.3	checker_spj.sh SPJ 对拍脚本	3
2	Data structure 数据结构	3
2.1	Disjoint union set 并查集	3
2.2	Disjoint union set rollback 可撤销并查集	3
2.3	Disjoint union set range 倍增并查集	3
2.4	Sparse table ST 表	4
2.5	Fenwick tree 树状数组	4
2.6	Segment tree 懒标记线段树	4
2.7	Segment tree persistent 可持久化线段树	5
2.8	Segment tree beats 区间 min/max 线段树	6
2.9	Segment tree line 李超树	7
2.10	Query tree 线段树分治	7
2.11	Chtholly tree 珂朵莉树	8
2.12	Bit trie 01 字典树	8
2.13	Xor basis 线性基	9
2.14	Prefix xor basis 前缀线性基	10
2.15	PBDS tree PBDS 平衡树	10

3 Graph 图论	10
3.1 Graph DP & Tree Techniques 图论选摘	10
3.2 一些概念	10
3.3 树上问题:	11
3.3.1 树的重心	11
3.3.2 树的直径	11
3.3.3 树链求交	12
3.3.4 重链剖分	12
3.3.5 点分治	14
3.3.6 树上启发式合并	14
3.3.7 基环树与基环树森林	16
3.3.8 虚树	17
3.3.9 树哈希	18
3.3.10 快速 LCA	19
3.3.11 长链剖分	19
3.4 拓扑排序	20
3.5 最短路	21
3.5.1 单源最短路	21
3.5.2 全源最短路	26
3.5.3 多源最短路	28
3.5.4 最长路	29
3.6 生成树	29
3.6.1 最小生成树	29
3.6.2 生成树计数	31
3.6.3 最小度限制生成树	31
3.6.4 曼哈顿最小生成树	31
3.6.5 最小直径生成树	32
3.6.6 动态 MST 问题	33
3.6.7 kruskal 重构树	34
3.6.8 最小斯坦纳树	35
3.7 连通性	37
3.7.1 有向图连通性	37

3.7.2	无向图连通性	39
3.8	欧拉图	43
3.8.1	无向图	43
3.8.2	有向图	44
3.8.3	混合图	45
3.9	拉姆齐定理	45
3.10	竞赛图	45
3.11	建图	45
3.11.1	线段树优化建图	45
3.11.2	前后缀优化建图	46
3.11.3	ST 表优化建图	47
3.11.4	网格建图	49
3.12	网络流	50
3.12.1	拆点	50
3.12.2	最大流	50
3.12.3	最小费用最大流	51
3.12.4	原始对偶优化	55
3.12.5	上下界网络流	59
3.12.6	Stoer-Wagner 无向图全局最小割	62
3.12.7	其他总结	63
3.13	二分图	63
3.13.1	二分图的判断	64
3.13.2	二分图上博弈	64
3.13.3	删边二分图	64
3.14	图匹配	65
3.14.1	基础概念	65
3.14.2	二分图上的匹配	65
3.14.3	一般图最大匹配	71
3.15	环计数问题	73
3.15.1	无向图三元环计数	73
3.15.2	无向图四元环计数	73
3.16	最大团与最大独立集	74

3.17 LCA 最近公共祖先	78
3.18 Cycle count 普通环计数	79
3.19 Cut edge 割边	79
3.20 Cut node 割点	79
3.21 SCC 强连通分量	80
3.22 EBCC 边双连通分量	80
3.23 VBCC 点双连通分量 (点集)	80
3.24 VBCC 点双连通分量 (边集)	81
3.25 2-SAT 2-SAT	81
3.26 Virtual tree 虚树	82
4 Dynamic Programming 动态规划	82
4.1 Subsequence DP 子序列问题	82
4.1.1 LIS / LDS / 不升 / 不降子序列	82
4.1.2 LCS 最长公共子序列	82
4.2 Partition of integers 自然数划分问题	83
4.2.1 可重复	83
4.2.2 不可重复	83
4.3 Knapsack DP 背包 DP	83
4.3.1 Multiple knapsack with monotone queue 多重背包 (单调队列优化)	83
4.3.2 Tree knapsack 树上背包	83
4.4 Interval DP 区间 DP	84
4.5 Monotone queue optimization 单调队列优化	84
4.6 Monge decision optimization 决策单调性优化	84
4.6.1 2D form	84
4.6.2 1D form	85
4.7 Slope trick / Convex hull optimization 斜率优化	85
4.7.1 Monotone queue version	86
4.7.2 Binary-search queue version	86
4.8 WQS binary / weight binary search WQS 二分	87
4.9 Dynamic DP 动态 DP	87
4.10 Bitmask DP 状压 DP	87

4.10.1 SOS DP (子集和 DP)	87
4.11 Digit DP 数位 DP	87
4.12 Tree DP 树形 DP	88
4.12.1 Rerooting DP 换根 DP	88
4.13 DP with after-effect 带后效性的 DP	89
4.13.1 Method of undetermined coefficients 待定系数法	89
4.13.2 Gaussian elimination 高斯消元	89
5 String 字符串	89
5.1 String Hash 字符串哈希	89
5.1.1 Ordered string hash 有序字符串哈希	89
5.1.2 Unordered string hash 无序字符串哈希	89
5.1.3 2D ordered hash 二维有序字符串哈希	90
5.2 Minimal representation 最小表示法	90
5.3 KMP	90
5.4 Z-function (扩展 KMP)	91
5.5 Manacher	91
5.6 Minimal period 最小循环节	91
5.7 Lyndon decomposition Lyndon 分解	92
5.8 Subsequence automaton 子序列自动机	92
5.8.1 Subsequence automaton with President tree 主席树优化子序列自动机	92
5.9 AC automaton AC 自动机	93
5.10 Suffix automaton 后缀自动机	94
5.10.1 Dynamic SAM with LCT (sketch) LCT 维护 SAM 链 / 子树信息	95
5.11 Generalized SAM 广义后缀自动机	95
5.12 Palindromic automaton 回文自动机	96
5.13 Suffix array 后缀数组	97
6 Number Theory 数论	97
6.1 ModInt 模运算	97
6.2 Prime sieve 素数筛	98
6.3 Qpow block 快速幂分块	98

6.4	$O(1)$ GCD $O(1)$ 最大公约数	98
6.5	Number theory basics 数论基础知识	99
6.5.1	关于狄利克雷卷积的定义	99
6.5.2	质数表	99
6.6	Number theory theorems 数论定理	102
6.6.1	三平方和定理	102
6.6.2	拉格朗日四平方和定理	102
6.6.3	雅可比四平方和定理	103
6.6.4	毕达哥拉斯三元组	103
6.6.5	预处理因数	103
6.6.6	欧拉筛	103
6.6.7	欧拉函数	104
6.6.8	莫比乌斯函数	105
6.6.9	欧拉定理	106
6.6.10	扩展欧拉定理	106
6.6.11	扩展欧几里得算法	106
6.6.12	扩展中国剩余定理	107
6.6.13	扩展大步小步算法	107
6.6.14	卢卡斯定理	108
6.6.15	原根	108
6.6.16	Pell 方程	110
6.6.17	类欧几里得算法	110
6.6.18	整除分块	111
6.6.19	莫比乌斯反演	111
6.6.20	杜教筛	111
6.6.21	Min25 筛	112
6.6.22	常见的积性函数	113
6.7	Combination preprocessing 组合数预处理	113
6.7.1	组合数预处理	113
6.7.2	取模类	114
6.7.3	枚举固定大小子集	116
6.8	Tricks 技巧	116

6.8.1	Miller Rabin	116
6.8.2	Pollard Rho	117
6.8.3	随机数生成	117
6.9	Classic counting models 几个经典计数模型	117
6.9.1	卡特兰数	117
6.9.2	卡特兰数	118
6.9.3	卡特兰数数表	118
6.9.4	k 阶卡特兰数	119
6.9.5	欧拉错排数	119
6.9.6	莫茨金数 (Motzkin Numbers)	119
6.9.7	贝尔数	119
6.9.8	斐波那契数列	119
6.9.9	斐波那契数表	119
6.10	Tree counting 树计数	121
6.10.1	无标号树计数	121
6.10.2	Prufer 序列	121
6.10.3	Cayley 定理	122
6.10.4	** 扩展 Cayley 定理 **	122
6.10.5	** 矩阵树定理 **	122
6.11	Randomized algorithms 随机化算法	123
6.11.1	KP 算法	123
6.11.2	MinHash	123
6.11.3	模拟退火	123
6.11.4	拉格朗日插值	123
6.11.5	原根表	124
6.11.6	多项式全家桶	124
6.11.7	集合幂级数全家桶	127
6.11.8	分治 NTT	129
6.11.9	Chirp-Z-Transform	129
6.11.10	高精度运算	129
6.11.11	乘法卷积	131
6.11.12	循环卷积	132

6.11.13 NTT 优化字符串通配符匹配	132
6.11.14 NTT 优化有限字符集的匹配	134
6.11.15 公平组合游戏	134
6.11.16 公平组合游戏下的经典模型	134
6.11.17 Nim 游戏	134
6.11.18 巴什博弈	134
6.11.19 威佐夫博弈	134
6.11.20 斐波那契博弈	134
6.11.21 广义斐波那契博弈	135
6.11.22 Nim-K 博弈	135
6.11.23 Anti-Nim 博弈	135
6.11.24 Anti-SG 博弈	135
6.11.25 Staircase-Nim 博弈	135
6.11.26 Lasker's-Nim 博弈	135
6.11.27 树上删边博弈	135
6.11.28 无向图删边游戏	135
6.11.29 一般图上博弈	136
6.11.30 翻硬币游戏	136
6.11.31 不公平游戏	136
6.11.32 超现实数 (Surreal Numbers)	136
6.11.33 Blue-Red Hackenbush 博弈	137
6.11.34 Termites	137
6.11.35 Game of Sorting	137
6.11.36 Xormites	137
6.11.37 矩阵与矩阵快速幂	137
6.11.38 行列式	137
6.11.39 加法高斯消元	138
6.11.40 异或高斯消元	138
6.11.41 动态维护异或方程组	139
6.11.42 二维平面上的距离	139
6.11.43 自适应辛普森法	139
6.11.44 平面最近点对	140

6.11.45 最小圆覆盖和最小球覆盖	140
6.11.46 单峰函数与三分	141
7 Computational geometry 计算几何.cpp	142
7.1 几何基础	142
7.2 圆与多边形面积交	146
7.3 凸包	146
7.4 半平面交	146
7.5 点集形成的最小最大三角形	151
7.6 平面最近点对	151
7.7 线段扫线判交	152
7.8 多边形面积并	152
7.9 圆面积并	153
8 Misc 杂项	154
8.1 Bitset 暴力滴神	154
9 Temp 临时式子整理	154
9.1 交换律	154
9.2 结合律	154
9.3 分配律	154
10 Summation Transformation 和式的变换技巧	154
10.1 替换条件式	154
10.2 替换指标变量	154
10.3 改变求和次序	154
10.3.1 基础	154
10.3.2 复杂的改变求和次序	155
10.4 分离变量	155
10.5 改变枚举范围	155
10.5.1 扩大枚举范围	155
10.5.2 缩小枚举范围	155
10.6 示例：组合数	155

10.7 和式的变换公式	155
10.7.1 公式 1	155
10.7.2 公式 2	155
10.8 阿贝尔部分求和公式	156

1 Prepare | 准备工作

1.1 Testing | 测试

1.1.1 Floyd | 传递闭包

- Codeforces: 1421 ms
- Atcoder: 476 ms
- QOJ: 328 ms

```

1 constexpr int N = 1000;
2
3 vector<vector<int>> f(N + 1, vector<int>(N + 1));
4 for (int i = 1; i <= N; i++) {
5     f[i][i] = 1;
6 }
7 for (int i = 1; i <= N; i++) {
8     int u = rand() % N + 1;
9     int v = rand() % N + 1;
10    f[u][v] = 1;
11 }
12
13 for (int i = 1; i <= N; i++) {
14     for (int k = 1; k <= N; k++) {
15         for (int j = 1; j <= N; j++) {
16             f[i][j] |= f[i][k] | f[k][j];
17         }
18     }
19 }
20
21 for (int i = 1; i <= N; i++) {
22     for (int j = 1; j <= N; j++) {
23         cout << f[i][j];
24     }
25     cout << '\n';
26 }

```

1.1.2 Floyd with Bitset | 传递闭包 bitset 优化

- Codeforces: 2077 ms
- Atcoder: 1065 ms
- QOJ: 1146 ms

```

1 constexpr int N = 5000;
2

```

```

3 vector<bitset<N + 1>> f(N + 1, bitset<N + 1>());
4 for (int i = 1; i <= N; i++) {
5     f[i][i] = 1;
6 }
7 for (int i = 1; i <= N * N / 2; i++) {
8     int u = rand() % N + 1;
9     int v = rand() % N + 1;
10    f[u][v] = 1;
11 }
12
13 for (int i = 1; i <= N; i++) {
14     for (int k = 1; k <= N; k++) {
15         if (f[i][k]) f[i] |= f[k];
16     }
17 }
18
19 for (int i = 1; i <= N; i++) {
20     cout << f[i] << '\n';
21 }

```

1.1.3 Basic operations | 基本运算

- Codeforces: 3015 ms
- Atcoder: 1512 ms
- QOJ: 1132 ms

```

1 constexpr int N = 1e8;
2
3 volatile i64 res = 0;
4 volatile i64 add = rand();
5 volatile i64 sub = rand();
6 volatile i64 mul = rand();
7 volatile i64 div = rand();
8 volatile i64 mod = rand();
9 volatile i64 XOR = rand();
10 for (int i = 1; i <= N; i++) {
11     res += add;
12     res -= sub;
13     res *= mul;
14     res /= div;
15     res %= mod;
16     res ^= XOR;
17 }
18
19 cout << res << '\n';

```

1.1.4 Float operations | 浮点运算

- Codeforces: 1327 ms
- Atcoder: 1324 ms
- QOJ: 1233 ms

```

1 constexpr int N = 1e8;
2
3 volatile double res = 1.0;
4 volatile double add = 1.0000001;
5 volatile double sub = 0.0000001;
6 volatile double mul = 1.0000002;
7 volatile double div = 1.0000003;
8
9 for (int i = 1; i <= N; i++) {
10     res += add;
11     res -= sub;
12     res *= mul;
13     res /= div;
14 }
15
16 cout << fixed << setprecision(10) << res << '\n';

```

1.1.5 STL performance | 1e6 STL 性能

- Codeforces: 2484 ms
- Atcoder: 1526 ms
- QOJ: 1221 ms

```

1 constexpr int N = 1e6;
2
3 mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
4
5 vector<int> vec;
6 deque<int> dq;
7 set<int> set;
8 map<int, int> map;
9
10 for (int i = 1; i <= N; i++) {
11     vec.push_back(rng());
12     dq.push_back(rng());
13     dq.push_front(rng());
14     set.insert(rng());
15     map[rng()] = rng();
16 }
17
18 cout << vec[N / 2] << '\n';

```

```

19 cout << dq[N / 2] << '\n';
20 cout << *set.rbegin() << '\n';
21 cout << map.rbegin()->second << '\n';

```

1.2 Shell script | 编译脚本

1.2.1 main.sh | 编译运行

```

1 #!/bin/sh
2 g++ -std=c++20 -DLOCAL -O0 -g0 -x c++-header main.hpp -o main.hpp.gch
3 g++ -std=c++20 -DLOCAL -O0 -g0 -o ./case/main.out main.cpp -ftime-report
4
5 for in in ./case/*.in; do
6     echo "-----"
7     ans="{in%.in}.ans"
8     echo "cerr: "
9     ./case/main.out < "$in" > "$ans"
10    echo
11    echo "$ans:"
12    cat $ans
13    echo "-----"
14 done

```

1.2.2 checker.sh | 对拍脚本

```

1 #!/bin/sh
2 set -e
3 g++ -std=c++20 -O0 -g0 -o test.out test.cpp
4 g++ -std=c++20 -O0 -g0 -o main.out main.cpp
5 g++ -std=c++20 -O0 -g0 -o true.out true.cpp
6
7 d="temp"; n="{1:-0}"; i=0
8
9 while ;; do
10     i=$((i+1))
11     in="{d/$i.in}"
12     out="{d/$i.out}"
13     ans="{d/$i.ans}"
14     ./test.out > "$in"
15     ./main.out < "$in" > "$out"
16     ./true.out < "$in" > "$ans"
17     if ! diff -Z -q "$out" "$ans"; then
18         echo "Test $i WA!"; echo ""
19         echo "$in:"; cat "$in"; echo ""
20         echo "$out:"; cat "$out"; echo ""
21         echo "$ans:"; cat "$ans"; echo ""
22     exit 1

```

```

23 fi
24 echo "Test $i Correct!"
25 if [ "$i" -ge "$n" ]; then
26     exit 0
27 fi
28 done

```

1.2.3 checker_spj.sh | SPJ 对拍脚本

```

1 #!/bin/sh
2 set -e
3 g++ -std=c++20 -O0 -g0 -o test.out test.cpp
4 g++ -std=c++20 -O0 -g0 -o main.out main.cpp
5 g++ -std=c++20 -O0 -g0 -o true.out true.cpp
6 g++ -std=c++20 -O0 -g0 -o checker.out checker.cpp
7
8 d="temp"; n="${1:-0}"; i=0
9
10 while ;; do
11     i=$((i+1))
12     in="$d/$i.in"
13     out="$d/$i.out"
14     ans="$d/$i.ans"
15     ./test.out > "$in"
16     ./main.out < "$in" > "$out"
17     ./true.out < "$in" > "$ans"
18     res1=$(cat "$in" "$out" | ./checker.out | tail -n1 || true)
19     res2=$(cat "$in" "$ans" | ./checker.out | tail -n1 || true)
20     if [ "$res1" != "$res2" ]; then
21         echo "Test $i WA!"; echo ""
22         echo "$in:"; cat "$in"; echo ""
23         echo "$out:"; cat "$out"; echo ""
24         echo "$ans:"; cat "$ans"; echo ""
25         exit 1
26     fi
27     echo "Test $i Correct!"
28     if [ "$i" -ge "$n" ]; then
29         exit 0
30     fi
31 done

```

```

2 vector<int> data;
3
4 disjoint_union_set(int _n) : data(_n + 1, -1) {}
5
6 int find(int x) {
7     return data[x] < 0 ? x : data[x] = find(data[x]);
8 }
9
10 bool merge(int x, int y) {
11     if (x = find(x), y = find(y); x == y) return false;
12     if (data[x] < data[y]) data[x] += data[y], data[y] = x;
13     else data[y] += data[x], data[x] = y;
14     return true;
15 }

```

2.2 Disjoint union set rollback | 可撤销并查集

```

1 struct disjoint_union_set {
2     vector<int> set;
3     vector<array<int, 2>> stk;
4
5     disjoint_union_set(int n) : set(n + 1, -1), stk{} {}
6
7     int find(int x) {
8         while (~set[x]) x = set[x];
9         return x;
10    }
11
12    bool merge(int x, int y) {
13        if (x = find(x), y = find(y); x == y) return false;
14        if (set[x] < set[y]) stk.push_back({y, set[y]}), set[x] += set[y], set[y] = x;
15        else stk.push_back({x, set[x]}), set[y] += set[x], set[x] = y;
16        return true;
17    }
18
19    void rollback() {
20        auto [u, s] = stk.back();
21        stk.pop_back();
22        set[set[u]] -= s;
23        set[u] = s;
24    }
25 };

```

2 Data structure | 数据结构

2.1 Disjoint union set | 并查集

```

1 struct disjoint_union_set {

```

2.3 Disjoint union set range | 倍增并查集

```

1 struct disjoint_union_set {
2     vector<int> data[20];
3
4     disjoint_union_set(int n) {

```

```

5   for (int i = 0; i <= __lg(n); i++) data[i].assign(n - (1 << i) + 2, -1);
6   }
7
8   int find(int k, int x) {
9       return data[k][x] < 0 ? x : data[k][x] = find(k, data[k][x]);
10  }
11  bool merge(int k, int x, int y) {
12      if (x = find(k, x), y = find(k, y); x == y) return false;
13      if (data[k][x] < data[k][y]) data[k][x] += data[k][y], data[k][y] = x;
14      else data[k][y] += data[k][x], data[k][x] = y;
15      return true;
16  }
17  void merge_range(int k, int x, int y, auto&& call) {
18      if (find(k, x) == find(k, y)) return;
19      if (k == 0) {
20          call(find(0, x), find(0, y));
21      } else {
22          merge(k - 1, x, y, call);
23          merge(k - 1, x + (1 << (k - 1)), y + (1 << (k - 1)), call);
24      }
25      merge(k, u, v);
26  }
27  };

```

2.4 Sparse table | ST 表

```

1  template <typename meta, auto comp>
2  struct sparse_table {
3      vector<meta> data[20];
4
5      void build(int n, auto&& call) {
6          for (int i = 0; i <= __lg(n); i++)
7              data[i].assign(n - (1 << i) + 2, meta{});
8          for (int i = 1; i <= n; i++)
9              data[0][i] = call(i);
10         for (int k = 1; k <= __lg(n); k++) {
11             for (int i = 1; i + (1 << k) - 1 <= n; i++) {
12                 data[k][i] = comp(data[k - 1][i], data[k - 1][i + (1 << (k - 1))]);
13             }
14         }
15     }
16     meta ask(int l, int r) {
17         int k = __lg(r - l + 1);
18         return comp(data[k][l], data[k][r - (1 << k) + 1]);
19     }
20 };

```

2.5 Fenwick tree | 树状数组

```

1  template <typename meta>
2  struct fenwick_tree {
3      int n;
4      vector<meta> data;
5
6      fenwick_tree(int _n) : n(_n), data(_n + 1) {}
7
8      void add(int u, const meta& x) {
9          for (int i = u; i <= n; i += i & -i) data[i] += x;
10     }
11     meta ask(int u, meta res = meta{}) {
12         for (int i = u; ~i; i &= i - 1) res += data[i];
13         return res;
14     }
15 };

```

2.6 Segment tree | 懒标记线段树

```

1  template <typename meta>
2  struct segment_tree {
3      vector<meta> data;
4
5      segment_tree(int n) : data(4 << __lg(n + 1)) {}
6
7      inline int lson(int p) { return p << 1; }
8      inline int rson(int p) { return p << 1 | 1; }
9      void build(auto&& call, int r, int p = 1, int l = 1) {
10         if (l + 1 == r) return data[p] = call(l), void();
11         int m = (l + r) >> 1;
12         build(call, m, lson(p), l);
13         build(call, r, rson(p), m);
14         data[p].pull(data[lson(p)], data[rson(p)], l, r);
15     }
16     void add(int L, int R, const auto& x, int r, int p = 1, int l = 1) {
17         if (L <= l && r <= R) return data[p].add(x, l, r);
18         data[p].push(data[lson(p)], data[rson(p)], l, r);
19         int m = (l + r) >> 1;
20         if (L < m) add(L, R, x, m, lson(p), l);
21         if (R > m) add(L, R, x, r, rson(p), m);
22         data[p].pull(data[lson(p)], data[rson(p)], l, r);
23     }
24     meta ask(int L, int R, int r, int p = 1, int l = 1) {
25         if (R <= l || r <= L) return meta{};
26         if (L <= l && r <= R) return data[p];
27         data[p].push(data[lson(p)], data[rson(p)], l, r);
28         int m = (l + r) >> 1;
29         return ask(L, R, m, lson(p), l) + ask(L, R, r, rson(p), m);

```

```

30 }
31 };
32
33 struct meta {
34     meta() {}
35     void add(const auto& x, int l, int r) {}
36     void pull(meta& lhs, meta& rhs, int l, int r) {}
37     void push(meta& lhs, meta& rhs, int l, int r) {}
38     friend meta operator+ (const meta& lhs, const meta& rhs) {}
39 };

```

2.7 Segment tree persistent | 可持久化线段树

split(p, x, y, l, r, L, R):

- p: 当前需要划分的根
- x: 划分后 [L, R) 的部分
- y: 划分后剩余的部分

merge(x, y, l, r):

- x: 需要合并的树根
- y: 需要合并的树根

```

1  template <typename meta, int N>
2  struct segment_tree {
3      struct node {
4          meta val;
5          int son[2];
6          node() : val{}, son{} {}
7      };
8      int lst;
9      node data[N << 6];
10
11     void add(int& p, int q, int u, const auto& x, int l, int r) {
12         data[p = ++lst] = data[q];
13         if (l + 1 == r) return data[p].val.add(x, l, r);
14         int m = (l + r) >> 1;
15         if (u < m) add(data[p].son[0], data[q].son[0], u, x, l, m);
16         else add(data[p].son[1], data[q].son[1], u, x, m, r);
17         data[p].val.pull(data[data[p].son[0]].val, data[data[p].son[1]].val, l, r);
18     }
19     meta ask(int p, int l, int r, int L, int R) {
20         if (!p) return meta{};
21         if (L <= l && r <= R) return data[p].val;
22         int m = (l + r) >> 1;
23         meta res{};
24         if (L < m) res = res + ask(data[p].son[0], l, m, L, R);
25         if (R > m) res = res + ask(data[p].son[1], m, r, L, R);

```

```

26     return res;
27 }
28 meta ask_with_diff(int p, int q, int l, int r, int L, int R) {
29     if (!p) return meta{};
30     if (L <= l && r <= R) return data[p].val - data[q].val;
31     int m = (l + r) >> 1;
32     meta res{};
33     if (L < m) res = res + ask_with_diff(data[p].son[0], data[q].son[0], l, m, L, R);
34     if (R > m) res = res + ask_with_diff(data[p].son[1], data[q].son[1], m, r, L, R);
35     return res;
36 }
37 void split(int p, int& x, int& y, int l, int r, int L, int R) {
38     if (!p) return x = y = 0, void();
39     if (R <= l || r <= L) return x = 0, y = p, void();
40     if (L <= l && r <= R) return x = p, y = 0, void();
41     int m = (l + r) >> 1;
42     int xl = 0, xr = 0, yl = 0, yr = 0;
43     split(data[p].son[0], xl, yl, l, m, L, R);
44     split(data[p].son[1], xr, yr, m, r, L, R);
45     if (xl || xr) {
46         x = ++lst;
47         data[x] = data[p];
48         data[x].son[0] = xl;
49         data[x].son[1] = xr;
50         data[x].val.pull(data[data[x].son[0]].val, data[data[x].son[1]].val, l, r);
51     } else x = 0;
52     if (yl || yr) {
53         y = ++lst;
54         data[y] = data[p];
55         data[y].son[0] = yl;
56         data[y].son[1] = yr;
57         data[y].val.pull(data[data[y].son[0]].val, data[data[y].son[1]].val, l, r);
58     } else y = 0;
59 }
60 int merge(int x, int y, int l, int r) {
61     if (!x || !y) return x || y;
62     int p = ++lst;
63     if (l + 1 == r) {
64         data[p].son[0] = data[p].son[1] = 0;
65         data[p].val = data[x].val + data[y].val;
66         return p;
67     }
68     int m = (l + r) >> 1;
69     data[p].son[0] = merge(data[x].son[0], data[y].son[0], l, m);
70     data[p].son[1] = merge(data[x].son[1], data[y].son[1], m, r);
71     data[p].val.pull(data[data[p].son[0]].val, data[data[p].son[1]].val, l, r);
72     return p;
73 }
74 };
75
76 struct meta {

```



```

77 meta() {}
78 void add(const auto& x, int l, int r) {}
79 void pull(meta& lhs, meta& rhs, int l, int r) {}
80 void push(meta& lhs, meta& rhs, int l, int r) {}
81 friend meta operator+ (const meta& lhs, const meta& rhs) {}
82 friend meta operator- (const meta& lhs, const meta& rhs) {}
83 };

```

2.8 Segment tree beats | 区间 min/max 线段树

```

1 template <typename meta>
2 struct segment_tree {
3     vector<meta> data;
4
5     segment_tree(int n) : data(4 << __lg(n + 1)) {}
6
7     inline int lson(int p) { return p << 1; }
8     inline int rson(int p) { return p << 1 | 1; }
9     void build(auto&& call, int r, int p = 1, int l = 1) {
10         if (l + 1 == r) return data[p] = call(l), void();
11         int m = (l + r) >> 1;
12         build(call, m, lson(p), l);
13         build(call, r, rson(p), m);
14         data[p].pull(data[lson(p)], data[rson(p)], l, r);
15     }
16     void add(int L, int R, const auto& x, int r, int p = 1, int l = 1) {
17         if (L <= l && r <= R) return data[p].add(x, l, r);
18         data[p].push(data[lson(p)], data[rson(p)], l, r);
19         int m = (l + r) >> 1;
20         if (L < m) add(L, R, x, m, lson(p), l);
21         if (R > m) add(L, R, x, r, rson(p), m);
22         data[p].pull(data[lson(p)], data[rson(p)], l, r);
23     }
24     void upd(int L, int R, const auto& x, int r, int p = 1, int l = 1) {
25         if (L <= l && r <= R && data[p].upd(x, l, r)) return;
26         data[p].push(data[lson(p)], data[rson(p)], l, r);
27         int m = (l + r) >> 1;
28         if (L < m) upd(L, R, x, m, lson(p), l);
29         if (R > m) upd(L, R, x, r, rson(p), m);
30         data[p].pull(data[lson(p)], data[rson(p)], l, r);
31     }
32     meta ask(int L, int R, int r, int p = 1, int l = 1) {
33         if (R <= l || r <= L) return meta{};
34         if (L <= l && r <= R) return data[p];
35         data[p].push(data[lson(p)], data[rson(p)], l, r);
36         int m = (l + r) >> 1;
37         return ask(L, R, m, lson(p), l) + ask(L, R, r, rson(p), m);
38     }
39 };

```

```

40 struct meta {
41     meta() {}
42     static meta merge(const meta& lhs, const meta& rhs) {}
43     void add(const auto& x, int l, int r) {}
44     bool upd(const auto& x, int l, int r) {}
45     void pull(meta& lhs, meta& rhs, int l, int r) {}
46     void push(meta& lhs, meta& rhs, int l, int r) {}
47     friend meta operator+ (const meta& lhs, const meta& rhs) {}
48 };
49
50
51 /*
52 区间 +, 区间 max 操作示例
53 1. a[i] <- a[i] + x
54 2. a[i] <- max(a[i], x)
55 min_num: 最小值
56 sec_num: 次小值
57 min_cnt: 最小值出现次数
58 tag: 加法标记
59 */
60
61 struct meta {
62     static constexpr int INF = 1e9;
63
64     int min_num;
65     int sec_num;
66     int min_cnt;
67     int sum;
68     int tag;
69
70     meta()
71         : min_num(INF), sec_num(INF), min_cnt(0), sum(0), tag(0) {}
72     meta(int val)
73         : min_num(val), sec_num(INF), min_cnt(1), sum(val), tag(0) {}
74
75     static meta merge(const meta& lhs, const meta& rhs) {
76         if (lhs.min_num == INF) return rhs;
77         if (rhs.min_num == INF) return lhs;
78
79         meta res{};
80         res.sum = lhs.sum + rhs.sum;
81
82         if (lhs.min_num < rhs.min_num) {
83             res.min_num = lhs.min_num;
84             res.min_cnt = lhs.min_cnt;
85             res.sec_num = min(lhs.sec_num, rhs.min_num);
86             return res;
87         }
88
89         if (lhs.min_num > rhs.min_num) {
90             res.min_num = rhs.min_num;

```

```

91     res.min_cnt = rhs.min_cnt;
92     res.sec_num = min(lhs.min_num, rhs.sec_num);
93     return res;
94 }
95
96 res.min_num = lhs.min_num;
97 res.min_cnt = lhs.min_cnt + rhs.min_cnt;
98 res.sec_num = min(lhs.sec_num, rhs.sec_num);
99
100 return res;
101 }
102 void add(const auto& x, int l, int r) {
103     min_num += x;
104     sec_num += x;
105     sum += x * (r - l);
106     tag += x;
107 }
108 bool upd(const auto& x, int l, int r) {
109     if (x <= min_num) return true;
110     if (x < sec_num) {
111         sum += (x - min_num) * min_cnt;
112         min_num = x;
113         return true;
114     }
115     return l + 1 == r;
116 }
117 void pull(meta& lhs, meta& rhs, int l, int r) {
118     *this = merge(lhs, rhs);
119 }
120 void push(meta& lhs, meta& rhs, int l, int r) {
121     int m = (l + r) >> 1;
122     lhs.add(tag, l, m);
123     rhs.add(tag, m, r);
124     lhs.upd(min_num, l, m);
125     rhs.upd(min_num, m, r);
126     tag = 0;
127 }
128 friend meta operator+ (const meta& lhs, const meta& rhs) {
129     return merge(lhs, rhs);
130 }
131 };

```

```

6     segment_tree(int n) : data(4 << __lg(n + 1)) {}
7
8     inline int lson(int p) { return p << 1; }
9     inline int rson(int p) { return p << 1 | 1; }
10    void push(meta line, int r, int p = 1, int l = 1) {
11        int m = (l + r) >> 1;
12        if (comp(f[p], line, m)) swap(f[p], line);
13        if (l + 1 == r) return;
14        if (comp(f[p], line, l)) push(line, m, lson(p), l);
15        if (comp(f[p], line, r)) push(line, r, rson(p), m);
16    }
17    void add(meta line, int L, int R, int r, int p = 1, int l = 1) {
18        if (L <= l && r <= R) return push(line, r, p, l);
19        int m = (l + r) >> 1;
20        if (L < m) add(line, L, R, m, lson(p), l);
21        if (R > m) add(line, L, R, r, rson(p), m);
22    }
23    type ask_max(int u, int r, int p = 1, int l = 1) {
24        if (l + 1 == r) return data[p](u);
25        int m = (l + r) >> 1;
26        type res = type{};
27        if (u < m) res = max(res, ask_max(u, m, lson(p), l));
28        else res = max(res, ask_max(u, r, rson(p), m));
29        return res;
30    }
31    type ask_min(int u, int r, int p = 1, int l = 1) {
32        if (l + 1 == r) return data[p](u);
33        int m = (l + r) >> 1;
34        type res = type{};
35        if (u < m) res = min(res, ask_min(u, m, lson(p), l));
36        else res = min(res, ask_min(u, r, rson(p), m));
37        return res;
38    }
39 };
40
41 template <typename T>
42 struct meta {
43     using type = T;
44     T k, b;
45     meta() : k{}, b{} {}
46     constexpr T operator() (int x) const {
47         return k * x + b;
48     }
49 };

```

2.9 Segment tree line | 李超树

```

1 template <typename meta, auto comp>
2 struct segment_tree {
3     using type = typename meta::type;
4     vector<meta> data;
5

```

2.10 Query tree | 线段树分治

```

1 struct query_tree {
2     vector<vector<int>> data;

```

```

3 query_tree(int n) : data(4 << __lg(n + 1)) {}
4
5
6 inline int lson(int p) { return p << 1; }
7 inline int rson(int p) { return p << 1 | 1; }
8 void add(int L, int R, int x, int r, int p = 1, int l = 1) {
9     if (L <= l && r <= R) return data[p].push_back(x);
10    int m = (l + r) >> 1;
11    if (L < m) add(L, R, x, m, lson(p), l);
12    if (R > m) add(L, R, x, r, rson(p), m);
13 }
14 void ask(auto&& _add, auto&& _del, auto&& call, int r, int p = 1, int l = 1) {
15     _add(data[p]);
16     if (l + 1 == r) {
17         call(l);
18     } else {
19         int m = (l + r) >> 1;
20         ask(_add, _del, call, m, lson(p), l);
21         ask(_add, _del, call, r, rson(p), m);
22     }
23     _del(data[p]);
24 }
25 };

```

2.11 Chtholly tree | 珂朵莉树

```

1 template <typename meta>
2 struct chtholly_tree {
3     using iter = map<int, meta>::iterator;
4     map<int, meta> data;
5
6     chtholly_tree(int l, int r) {
7         data[l] = meta{};
8         data[r + 1] = meta{};
9     }
10
11     iter split(int p) {
12         iter it = prev(data.upper_bound(p));
13         if (it->first == p) return it;
14         return odt.insert(it, {p, it->second});
15     }
16     void alloc(int l, int r, const meta& x) {
17         iter L = split(l);
18         iter R = split(r + 1);
19         data.erase(L, R);
20         data[l] = x;
21     }
22     void apply(int l, int r, auto&& call) {
23         iter L = split(l);

```

```

24         iter R = split(r + 1);
25         while (L != R) {
26             iter P = L++;
27             call(P, L);
28         }
29     }
30 };

```

2.12 Bit trie | 01 字典树

```

1 template <int N, int B>
2 struct bit_trie {
3     static constexpr int M = []() -> int {
4         int res = 1;
5         int siz = 1;
6         for (int i = 0; i < B; i++) {
7             siz = min(N, siz << 1);
8             res += siz;
9         }
10        return res;
11    }();
12    int lst;
13    int rot[N << 1];
14    int cnt[M << 4];
15    int son[M << 4][2];
16
17    void clear() {
18        memset(cnt, 0, sizeof(cnt[0]) * (lst + 1));
19        memset(son, 0, sizeof(son[0]) * (lst + 1));
20        memset(rot, 0, sizeof(rot));
21        lst = 0;
22    };
23    int nxt(int& p) {
24        return p ? p : p = ++lst;
25    }
26    void add(int x, int u = 0) {
27        u = nxt(rot[u]);
28        for (int i = B - 1; i >= 0; i--) cnt[u = nxt(son[u][x >> i & 1])]++;
29    }
30    void del(int x, int u = 0) {
31        u = rot[u];
32        for (int i = B - 1; i >= 0; i--) cnt[u = son[u][x >> i & 1]]--;
33    }
34    void merge(int& u, int v) {
35        if (!u || !v) return u |= v, void();
36        cnt[u] += cnt[v];
37        merge(son[u][0], son[v][0]);
38        merge(son[u][1], son[v][1]);
39    }

```

```

40 void split(int& u, int& v, int p, int val, int dep) {
41     if (!p) return u = v = 0, void();
42     if (dep--) {
43         if (val >> dep & 1) {
44             u = p, v = ++lst;
45             son[v][0] = 0;
46             split(son[u][1], son[v][1], son[u][1], val, dep);
47         } else {
48             v = p, u = ++lst;
49             son[u][1] = 0;
50             split(son[u][0], son[v][0], son[v][0], val, dep);
51         }
52         cnt[u] = cnt[son[u][0]] + cnt[son[u][1]];
53         cnt[v] = cnt[son[v][0]] + cnt[son[v][1]];
54     } else {
55         u = p;
56         v = 0;
57     }
58 }
59 };

```

2.13 Xor basis | 线性基

```

1  template <int B>
2  struct xor_space {
3      int cnt;
4      i64 data[B];
5      constexpr xor_space() { reset(); }
6      constexpr void reset() {
7          cnt = zro = 0;
8          fill(data, data + B, 0);
9      }
10     void rebuild() {
11         for (int i = B - 1; i >= 0; i--) {
12             if (!data[i]) continue;
13             for (int j = i - 1; j >= 0; j--)
14                 if (data[i] >> j & 1) data[i] ^= data[j];
15         }
16     }
17     bool add(i64 x) {
18         for (int i = B - 1; i >= 0 && x; i--) {
19             if (~x >> i & 1) continue;
20             if (!data[i]) return data[i] = x, cnt++, true;
21             x ^= data[i];
22         }
23         return false;
24     }
25     i64 ask_max(i64 x = 0) {
26         for (int i = B - 1; i >= 0; i--)

```

```

27         if (~x >> i & 1) x ^= data[i];
28         return x;
29     }
30     i64 ask_min(i64 x = 0) {
31         for (int i = B - 1; i >= 0; i--)
32             if (x >> i & 1) x ^= data[i];
33         return x;
34     }
35     i64 ask_kth(i64 k = 0) {
36         if (k <= 0 || k >= (1ULL << cnt)) return -1;
37         i64 res = 0;
38         for (int i = 0; i < B; i++) {
39             if (!data[i]) continue;
40             if (k & 1) res ^= data[i];
41             k >>= 1;
42         }
43         return res;
44     }
45     i64 aks_rnk(i64 x = 0) {
46         i64 res = 0;
47         i64 k = 1;
48         for (int i = 0; i < B; i++) {
49             if (!data[i]) continue;
50             if (x >> i & 1) res |= k;
51             k <<= 1;
52         }
53         return res;
54     }
55     friend constexpr xor_space operator|(const xor_space &lhs, const xor_space &rhs)
56     {
57         xor_space res{lhs};
58         for (int i = 0; i < B; i++)
59             res.add(rhs.data[i]);
60         return res;
61     }
62     friend constexpr xor_space operator&(const xor_space &lhs, const xor_space &rhs)
63     {
64         xor_space<B * 2> tmp;
65         for (int i = 0; i < B; i++) {
66             tmp.add((lhs.data[i] << B) | lhs.data[i]);
67             tmp.add(rhs.data[i] << B);
68         }
69         xor_space res{};
70         for (int i = 0; i < B; i++)
71             if (tmp.data[i]) res.data[i] = tmp.data[i], res.cnt++;
72         return res;
73     }
74 };

```

2.14 Prefix xor basis | 前缀线性基

```

1 template <int B>
2 struct xor_space {
3     int cnt;
4     i64 data[B];
5     int time[B];
6     constexpr xor_space() { reset(); }
7     constexpr void reset() {
8         cnt = 0;
9         fill(data, data + B, 0);
10        fill(time, time + B, -1);
11    }
12    bool add(i64 x, int t) {
13        for (int i = B - 1; i >= 0 && x; i--) {
14            if (!(x >> i & 1)) continue;
15            if (!data[i]) return data[i] = x, time[i] = t, cnt++, true;
16            if (time[i] < t) {
17                swap(data[i], x);
18                swap(time[i], t);
19            }
20            x ^= data[i];
21        }
22        return false;
23    }
24    i64 ask_max(i64 x = 0, int l = 0) {
25        for (int i = B - 1; i >= 0; i--) {
26            if (time[i] < l) continue;
27            if (~x >> i & 1) x ^= data[i];
28        }
29        return x;
30    }
31    i64 ask_min(i64 x = 0, int l = 0) {
32        for (int i = B - 1; i >= 0; i--) {
33            if (time[i] < l) continue;
34            if (x >> i & 1) x ^= data[i];
35        }
36        return x;
37    }
38    friend constexpr xor_space operator|(const xor_space &lhs, const xor_space &rhs)
39    {
40        xor_space res{lhs};
41        for (int i = 0; i < B; i++)
42            res.add(rhs.data[i], res.time[i]);
43        return res;
44    }
45    friend constexpr xor_space operator&(const xor_space &lhs, const xor_space &rhs)
46    {
47        xor_space<B * 2> tmp;
48        for (int i = 0; i < B; i++) {
49            tmp.add((lhs.data[i] << B) | lhs.data[i], lhs.time[i]);

```

```

48        tmp.add(rhs.data[i], rhs.time[i]);
49    }
50    xor_space res{};
51    for (int i = 0; i < B; i++) {
52        if (tmp.data[i]) {
53            res.data[i] = tmp.data[i];
54            res.time[i] = tmp.time[i];
55            res.cnt++;
56        }
57    }
58    return res;
59 }
60 };

```

2.15 PBDS tree | PBDS 平衡树

inset(x): 插入一个元素 x

erase(it): 删除一个迭代器 it

order_of_key(x): 返回严格小于 x 的数量

find_by_order(x): 返回 cmp_fn 排名为 x 的迭代器

join(x, y): 合并两个集合 x 和 y, 合并到 x 上, y 被清空

split(x, k): 以 cmp_fn 比较, 小于等于 x 的属于当前树, 其余的属于 b 树

```

1 #include <bits/stdc++.h>
2 #include <bits/extc++.h>
3 // #include <ext/pb_ds/assoc_container.hpp>
4 // #include <ext/pb_ds/tree_policy.hpp>
5 using namespace std;
6 using namespace __gnu_pbds;
7
8 template <typename T>
9 using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
    tree_order_statistics_node_update>;

```

3 Graph | 图论

3.1 Graph DP & Tree Techniques | 图论选摘

3.2 一些概念

基图: 即把有向图中的有向边改为无向边得到的图称作基图

弱连通: 对于有向图中两点 (u, v) , 如果我们转化为基图后 (u, v) 联通, 那么称 (u, v) 弱联通

完全图: 任意两点间都有一条无向边的图称作完全图, n 个点的完全图记作 K_n

子图: 对于一个图 $G = (V, E)$, 如果另一个图 $G' = (V', E')$ 满足 $V' \subseteq V$ 且 $E' \subseteq E$ 则 G' 称作 G 的子图

导出子图: 选定一个点集 V , 再加上两端都在 V 中的边, 即为点集 V 的导出子图

平面图: 将所有点和边铺在平面上, 可以满足每条边之间两两不相交的图称作平面图, 对于平面图 $G = (V, E)$ 满足 $E \leq 3V - 6$ 且根据欧拉公式, 平面图满足 $V - E + F = C + 1$ 其中 F 是面的个数, C 是图连通块的个数

3.3 树上问题:

3.3.1 树的重心

我们不妨设树的大小为 n

树的重心定义为, 当树以重心为根时, 每个子树的大小不超出 $n/2$ 。

性质: 一棵树最多只能有两个重心。

求重心的方法:

在我们求出以任意一点为根的子树大小后, 我们从根开始 dfs 。如果某个点的所有儿子的子树大小都 $\leq \frac{n}{2}$, 那么该点为重心, 直接返回。否则我们递归子树大小超过 $n/2$ 的儿子即可 (显然只有一个)。

```
1 struct TreeCentroid
2 {
3     int n;
4     std::vector<std::vector<std::pair<int, int>>> g; // 邻接表
5     std::vector<int> siz; // 子树大小
6
7     TreeCentroid(int n) : n(n), g(n + 1), siz(n + 1) {}
8
9     void addEdge(int u, int v, int w)
10    {
11        g[u].emplace_back(v, w);
12        g[v].emplace_back(u, w);
13    }
14
15    void dfsSize(int u, int fa)
16    {
17        siz[u] = 1;
18        for (const auto &[v, w] : g[u])
19        {
20            if (v != fa)
21            {
22                dfsSize(v, u);
23                siz[u] += siz[v];
24            }
25        }
26    }
27
28    int findCentroid(int u)
29    {
30        dfsSize(u, 0);
```

```
31        return getRoot(u, 0, siz[u]);
32    }
33
34 private:
35     int getRoot(int u, int fa, int total_size)
36     {
37         for (const auto &[v, w] : g[u])
38         {
39             if (v != fa && 2 * siz[v] > total_size)
40             {
41                 return getRoot(v, u, total_size);
42             }
43         }
44         return u;
45     }
46 };
```

3.3.2 树的直径

直径求法:

我们通过二次 BFS 解决直径问题, 也就是说我们先以任意点为根, 找到最深的点, 再次从最深的点开始二次 BFS , 得到最大深度即直径。正确性显然。

```
1
2 template <class T>
3 struct TreeDiameter
4 {
5     int n;
6     std::vector<std::vector<std::pair<int, int>>> g; // 邻接表 (无向图)
7     std::vector<int> fa; // 父节点记录 (用于路径回溯)
8
9     TreeDiameter(int n) : n(n), g(n + 1), fa(n + 1) {}
10
11     // 添加无向边 u-v
12     void addEdge(int u, int v, int w)
13     {
14         g[u].emplace_back(v, w);
15         g[v].emplace_back(u, w);
16     }
17
18     // BFS 求最远节点 (返回节点编号)
19     std::pair<int, T> bfs(int start)
20     {
21         std::vector<T> dep(n + 1, -1);
22         std::queue<int> q;
23         q.push(start);
24         dep[start] = 0;
25         fa[start] = -1;
26
27         while (!q.empty())
```

```

28 {
29     int u = q.front();
30     q.pop();
31     for (auto [v, w] : g[u])
32     {
33         if (dep[v] == -1)
34         {
35             dep[v] = dep[u] + w;
36             fa[v] = u;
37             q.push(v);
38         }
39     }
40 }
41 int maxPos = std::max_element(dep.begin(), dep.end()) - dep.begin();
42 return std::make_pair(maxPos, dep[maxPos]);
43 }
44
45 // 求树的直径 (返回直径长度和端点 {u, v})
46 std::pair<T, std::pair<int, int>> getDiameter()
47 {
48     auto [x, dis1] = bfs(1); // 第一次BFS
49     auto [y, dis2] = bfs(x); // 第二次BFS
50     return {dis2, {x, y}};
51 }
52 };

```

直径的性质

- 1) 树上任意一点，离它最远的一点必然是直径两 endpoint 之一
- 2) 两个联通块合并，新联通块的直径两 endpoint 一定是原先的四个直径 endpoint 中的其中两个
- 3) 我们可以扩展 2 中的结论，我们将直径的定义扩展为点权 + 边权的情况下仍然成立
- 4) 一棵树可以拥有多条直径，其所有直径共用一个中点，中点有可能在边上

3.3.3 树链求交

我们对于一棵树的两条链 (u, v) 和 (x, y) ，我们考虑求出它们的交链 endpoint

首先可以证明的是最终交链的 endpoint 只可能在集合 S 中，其中 S 是四个点任选两个点求 LCA 以后的点，显然这样的点最多只有 $C(4, 2)$ 个，我们可以通过求距离的方法判断每个点是否在链上，那么同时在 (u, v) 和 (x, y) 上的点可以成为候选点，若候选点个数 $|S| = 0$ ，那么树链完全无交，若候选点个数 $|S| = 1$ ，那么该候选点为唯一交点，若 $|S| \geq 2$ ，那么取这些点的直径即可。如果精细实现 LCA 的话可以做到常数时间复杂度

```

1 bool cmp(int a, int b) { return dep[a] < dep[b]; }
2 path merge(path u, path v)
3 {
4     int d[4], c[2];
5     if (!u.x || !v.x)
6         return path(0, 0);
7     d[0] = lca(u.x, v.x);

```

```

8     d[1] = lca(u.x, v.y);
9     d[2] = lca(u.y, v.x);
10    d[3] = lca(u.y, v.y);
11    c[0] = lca(u.x, u.y);
12    c[1] = lca(v.x, v.y);
13    sort(d, d + 4, cmp);
14    sort(c, c + 2, cmp);
15    if (dep[c[0]] <= dep[d[0]] && dep[c[1]] <= dep[d[2]])
16        return path(d[2], d[3]);
17    else
18        return path(0, 0);
19 }

```

3.3.4 重链剖分

重链剖分可以将树上的任意一条路径划分成不超过 $O(\log n)$ 条连续的链，每条链上的点深度互不相同。同时会给树的点重新编号为 dfn 序。

dfn 序的定义：按先遍历重儿子的顺序得到的 dfs 序。

重链剖分的性质：

- 1: 每条重链上的 dfn 序连续。
- 2: 每个点子树内的 dfn 序连续，节点 u 子树内的 dfn 序范围是 $[dfn[u], dfn[u] + siz[u] - 1]$ 。
- 3: 每个点到根节点的路径上，最多只会有 $O(\log n)$ 个轻儿子。
- 4: 每条路径能被划分为最多 $O(\log n)$ 条重链。

关于第三个和第四个性质的证明：每次经过轻儿子，子树大小至少会减半，因此最多下降 $O(\log n)$ 次。

```

1 template <class T>
2 class TreePre
3 {
4
5 private:
6     int n;
7     int idx = 0;
8     int root;
9     std::vector<std::vector<std::pair<int, int>>> &g;
10    void dfs1(int u, int f)
11    {
12        fa[u] = f;
13        siz[u] = 1;
14        for (auto [v, w] : g[u])
15        {
16            if (v != f)
17            {
18                dep[v] = dep[u] + 1;
19                dis[v] = dis[u] + w;
20                dfs1(v, u);
21                siz[u] += siz[v];
22                if (siz[son[u]] < siz[v])
23                    son[u] = v;

```

```

24         son[u] = v;
25     }
26 }
27 }
28 }
29 void dfs2(int u, int tp)
30 {
31     dfn[u] = ++idx;
32     idfn[idx] = u;
33     top[u] = tp;
34     if (son[u])
35     {
36         dfs2(son[u], tp);
37     }
38     for (auto [v, w] : g[u])
39     {
40         if (v != fa[u] and v != son[u])
41         {
42             dfs2(v, v);
43         }
44     }
45 }
46
47 public:
48     TreePre(std::vector<std::vector<std::pair<int, int>>> &g, int root)
49         : g(g), n(g.size() - 1), root(root), dep(n + 1), top(n + 1), fa(n
          + 1), siz(n + 1), dfn(n + 1), idfn(n + 1), dis(n + 1)
50     {
51         dep[root] = 1;
52         dis[root] = 0;
53         dfs1(root, 0);
54         dfs2(root, root);
55     }
56     std::vector<int> dfn, idfn, siz, fa, dep, top, son;
57     std::vector<T> dis;
58     int kthAncestor(int u, int k) // k级祖先
59     {
60         if (dep[u] <= k)
61         {
62             return -1;
63         }
64         int d = dep[u] - k;
65         while (dep[top[u]] > d)
66         {
67             u = fa[top[u]];
68         }
69         return idfn[dfn[u] - dep[u] + d];
70     }
71     int getLca(int u, int v) // u和v在指定根下的lca
72     {
73         while (top[u] != top[v])

```

```

74     {
75         if (dep[top[u]] > dep[top[v]])
76         {
77             u = fa[top[u]];
78         }
79         else
80         {
81             v = fa[top[v]];
82         }
83     }
84     return dep[u] < dep[v] ? u : v;
85 }
86 T getDistance(int u, int v) // u和v之间的距离
87 {
88     return dis[u] + dis[v] - 2 * dis[getLca(u, v)];
89 }
90 bool isAncestor(int u, int v) // u是不是v的祖先
91 {
92     return dfn[u] <= dfn[v] and dfn[u] + siz[u] - 1 >= dfn[v];
93 }
94
95 auto getRoad(int u, int v) // 得到uv之间路径的dfn序
96 {
97     std::vector<std::pair<int, int>> res;
98     while (top[u] != top[v])
99     {
100         if (dep[top[u]] < dep[top[v]])
101         {
102             std::swap(u, v);
103         }
104         res.push_back({dfn[top[u]], dfn[u]});
105         u = fa[top[u]];
106     }
107     if (dep[u] > dep[v])
108     {
109         std::swap(u, v);
110     }
111     res.push_back({dfn[u], dfn[v]});
112     return res;
113 }
114
115 int rootedFather(int u, int root) // u在root为根的情况下的父节点
116 {
117     if (!isAncestor(u, root)) // u不是root的祖先 那么父节点不变
118     {
119         return fa[u];
120     }
121     auto it = std::upper_bound(g[u].begin(), g[u].end(), root, [&](int x, int y)
122         { return dfn[x] < dfn[y]; });
123     --it;
124     return *it;

```



```

125 // 把u的邻点按dfn序排序后 dfn最大且 dfn[u]<=dfn[root]的点
126 }
127
128 int rootedSize(int u, int root) // root为根的情况下u的子树大小
129 {
130     if (root == u)
131     {
132         return n;
133     }
134     if (!isAncestor(u, root)) // u不是根的祖先
135     {
136         return siz[u];
137     }
138     return n - siz[rootedFather(u, root)];
139 }
140 int rootedLca(int root, int u, int v)
141 {
142     return getLca(u, v) ^ getLca(u, root) ^ getLca(v, root);
143 }
144 };

```

3.3.5 点分治

点分治的思想是统计经过根的路径的贡献，对于不经过根的路径，我们分治下去直至每个点都做为根被统计贡献。在这样的情况下，不断选择树的重心分治是最优的选择，可以证明在该条件下访问点的数量为 $n \log n$ 级别的。也就是说，我们通过一种特殊的遍历顺序得到了所有路径的信息。

特别的，如果我们存下来点分治过程中经过的节点，我们称作点分治序，每个点对应的可结合的点的点分治序都是一个连续的区间。

在以每个点为根统计答案时，我们考虑两种不同的方法进行求解。

不妨设当前根为 u ,

1 对于 u 的每一颗子树，我们挨个遍历并且和之前遍历到的子树信息进行结合统计答案。在遍历完一颗子树后要加入该子树的信息。在遍历完所有子树后，要清空信息。这样的方法能保证在统计时就不重不漏。请注意清空信息！

2 我们直接把所有 u 的信息统计出来并计算答案，注意到这时候可能有单个子树之中的点结合，这部分是非法答案，我们考虑容斥减掉对应的贡献。具体的说，考虑一个函数 $calc(int\ root, int\ val)$ 表示以 $root$ 为根，当前权值为 val 的贡献，我们要加上 $calc(u, 0)$ ，减去所有的 $calc(v, w(u, v))$

```

1 struct PointerDivide
2 {
3     std::vector<std::vector<std::pair<int, int>>> &g;
4     std::vector<int> siz, dfn, vis;
5     std::vector<std::vector<int>> pdt;
6     int root;
7
8     void dfsSize(int u, int fa)
9     {
10         siz[u] = 1;

```

```

11     for (auto [v, w] : g[u])
12     {
13         if (v != fa and !vis[v])
14         {
15             dfsSize(v, u);
16             siz[u] += siz[v];
17         }
18     }
19 }
20 int getRoot(int u, int fa, int Size)
21 {
22     for (auto [v, w] : g[u])
23     {
24         if (v != fa and !vis[v] and 2 * siz[v] > Size)
25         {
26             return getRoot(v, u, Size);
27         }
28     }
29     return u;
30 }
31 void build(int u)
32 {
33     vis[u] = true;
34     dfn.push_back(u);
35     for (auto [v, w] : g[u])
36     {
37         if (!vis[v])
38         {
39             dfsSize(v, -1);
40             v = getRoot(v, -1, siz[v]);
41             // pdt[u].push_back(v);
42             build(v);
43         }
44     }
45 }
46 PointerDivide(std::vector<std::vector<std::pair<int, int>>> &g) : g(g), siz(g.size()), vis(g.size())
47 {
48     dfsSize(1, 0);
49     root = getRoot(1, 0, siz[1]);
50     build(root);
51 }
52 };

```

3.3.6 树上启发式合并

这是一个离线做法，本质就是对于每个节点都通过 DFS 求答案，但是通过保留重儿子的答案使复杂度降为优秀的 $n \log n$

具体做法是，对于以 u 为根的子树，我们先递归其轻儿子，但是信息我们不做保留，注意这次 dfs 并不

是在求轻儿子的对 u 结点的答案，而是递归轻儿子求其自己的答案，递归结束后回到当前节点，我们再递归其重儿子，标记重儿子并且保留信息，然后我们开始统计该节点 u 的答案，由于重儿子的信息被保留，我们只需要计算轻儿子的贡献即可，这里实际上加上了轻儿子的贡献，统计完后在此更新该节点的答案，如果该节点 u 不是其父节点的重儿子，还需要清除贡献，清除贡献前置重儿子为空，这样会把该节点轻重儿子的贡献全部删去，保证了正确性

```

1 // 对于以u为根的子树
2 // 1先统计其轻儿子树的答案 统计完后删除信息
3 // 2再统计其重儿子树的答案 统计完后保留信息
4 // 3将重儿子树的信息合并到u上
5 // 4再遍历u的轻子树 把轻子树的信息合并到u上
6 // 5判断u的信息是否需要传给父节点（如果u是其父节点的重儿子就需要传递信息）
7
8
9 // 以求子树颜色种类为例
10 template <class T>
11 struct DSUOnTree
12 {
13     int n;
14     std::vector<std::vector<std::pair<int, int>>> g; // 邻接表
15     std::vector<int> siz, son;
16     std::vector<T> ans; // 子树大小、重儿子、每个节点的答案
17     int root;
18     int heavyNode = 0; // 当前处理的重儿子节点
19
20
21     int typeCnt = 0; // 颜色种类数量
22     std::vector<int> col, colorCnt; // 原始颜色数组 颜色出现次数数组
23
24     DSUOnTree(std::vector<std::vector<std::pair<int, int>>> &g, std::vector<int> &col,
25               , int root) : n(g.size() - 1), g(g), siz(n + 1), son(n + 1), ans(n + 1), root(
26               root), col(col), colorCnt(n + 1, 0) {}
27
28     // 预处理重儿子
29     void preProcess(int u, int f)
30     {
31         siz[u] = 1;
32         for (auto [v, w] : g[u])
33         {
34             if (v != f)
35             {
36                 preProcess(v, u);
37                 siz[u] += siz[v];
38                 if (siz[son[u]] < siz[v])
39                 {
40                     son[u] = v;
41                 }
42             }
43         }
44     }
45
46     // 添加/删除节点信息（需根据题目实现）

```

```

44 void update(int u, int f, int val)
45 {
46     // 示例：统计子树颜色种类数
47     if (val == 1)
48     {
49         colorCnt[col[u]]++;
50         typeCnt += (colorCnt[col[u]] == 1);
51     }
52     else
53     {
54         colorCnt[col[u]]--;
55         typeCnt -= (colorCnt[col[u]] == 0);
56     }
57 }
58
59 // 遍历子树统计/清除信息
60 void traverse(int u, int f, int val)
61 {
62     update(u, f, val);
63     for (auto [v, w] : g[u])
64     {
65         if (v != f && v != heavyNode)
66         {
67             traverse(v, u, val);
68         }
69     }
70 }
71
72 // 主算法
73 void solve(int u, int f, bool keep)
74 {
75     // 1. 先处理轻儿子（不保留信息）
76     for (auto [v, w] : g[u])
77     {
78         if (v != f && v != son[u])
79         {
80             solve(v, u, false);
81         }
82     }
83
84     // 2. 处理重儿子（保留信息）
85     if (son[u])
86     {
87         solve(son[u], u, true);
88         heavyNode = son[u];
89     }
90
91     // 3. 再次遍历轻儿子并合并信息
92     for (auto [v, w] : g[u])
93     {
94         if (v != f && v != son[u])

```

```

95     {
96         traverse(v, u, 1);
97     }
98 }
99 update(u, f, 1); // 添加当前节点信息
100 // 4. 此处可保存当前节点的答案
101 ans[u] = typeCnt;
102
103 heavyNode = 0;
104
105 // 5. 如果需要清除信息
106 if (!keep)
107 {
108     traverse(u, f, -1);
109     typeCnt = 0;
110 }
111 // 对外接口
112 void compute()
113 {
114     preProcess(root, 0); // 假设根节点是1
115     solve(root, 0, true);
116 }
117 }
118 };

```

3.3.7 基环树与基环树森林

基环树又名环套树，是 n 个点 n 条边的无向连通图，我们看待基环树的思路往往是环和树分开，这个思想是处理基环树的关键，也就是说，基环树实际上是一个环，且环上每个点都悬挂着一颗以该点为根的子树，如果我们破坏成链，那么形状将是一条长链，链上每个节点都有一棵子树（可能为空）

我们往往对基环树问题有两种处理方法

- 1 对于在基环树上贪心的问题，我们往往先在每棵小树上 dfs 合并信息到环上，然后再对环单独处理。
- 2 对于基环树一些性质的题目，以基环树直径为例，我们往往要破坏成链，分类讨论。具体的说，我们任选环上某条边记为 (u, v) ，我们将其断开，那么剩下的结构将是一棵树，我们可以朴素的求出它的直径，这是不经过 (u, v) 边的最大直径，然后我们考虑一定经过 (u, v) 边的最大直径，我们不难发现，直径一定是破坏后的链的一个前缀 + 一个后缀 + 该边拼凑而成，我们要先对每棵小树处理出它的最大深度 dep ，那么 $pre[u] = \max(pre[u-1], dep[u] + len[u])$ ， $len[u]$ 是链左边端点到该点的长度，我们求出前后缀 max 后，遍历取最大即可

基环树

```

1 struct RingTree
2 {
3     std::vector<std::vector<std::pair<int, int>>> g; // 基环树上的小树
4     std::vector<int> ring, ringVal; // 环, 环的出点的权值
5     // ringVal[i] 对应的是 ring[i] 和 ring[i+1] 的权值
6
7     RingTree(std::vector<std::vector<std::pair<int, int>>> e) : g(e.size())

```

```

8 {
9     int n = e.size() - 1;
10    std::vector<int> vis(n + 1, 0), deg(n + 1, 0);
11    for (int u = 1; u <= n; ++u)
12    {
13        for (auto [v, w] : e[u])
14        {
15            deg[v]++;
16        }
17    }
18    auto dfs1 = [&](auto dfs1, int u) -> void
19    {
20        vis[u] = 1;
21        for (auto [v, w] : e[u])
22        {
23            if (!vis[v])
24            {
25                // dsu.merge(u, v);
26                g[v].emplace_back(u, w);
27                deg[v]--;
28                if (deg[v] == 1)
29                {
30                    dfs1(dfs1, v);
31                }
32            }
33        }
34    };
35    for (int i = 1; i <= n; ++i)
36    {
37        if (!vis[i] and deg[i] == 1) // 从叶子开始剥离树的结构
38        {
39            dfs1(dfs1, i);
40        }
41    }
42    auto dfs2 = [&](auto dfs2, int u) -> void
43    {
44        ring.push_back(u);
45        vis[u] = 1;
46        for (auto [v, w] : e[u])
47        {
48            if (!vis[v])
49            {
50                ringVal.push_back(w);
51                dfs2(dfs2, v);
52            }
53        }
54    };
55    for (int i = 1; i <= n; ++i)
56    {
57        if (!vis[i]) // 一定是环点
58        {

```

```

59     dfs2(dfs2, i); // 找到整个环
60     for (auto [v, w] : e[ring.back()])
61     {
62         if (v == ring.front())
63         {
64             ringVal.push_back(w); // 把环起点和终点的边权加入
65             break;
66         }
67     }
68 }
69 }
70 }
71 };

```

基环树森林是一个更特殊的情况，它往往比基环树更加常见，其题目往往有特征为每个点只有唯一的入点或唯一的出点，也就是说我们可以理解为，在树上它有唯一的父亲，那么如果我们连边 $i - fa[i]$ ，将会得到一个基环树森林，基环树的处理手法和基环树如出一辙，无非就是对多棵树处理。

注意有些时候题目可能不是单纯的基环树森林，它可能是树森林和基环树森林的并集，需要分类讨论

基环树森林

```

1 struct RingTreeForest
2 {
3     int cnt = 0;
4     int n;
5     std::vector<int> &fa; // 基环树的森林的出边/父边
6     std::vector<int> onRing, root, ringLen, pos, posOnRing;
7     // onRing[i]表示i是否在环上, root[i]表示i所在的树的根, ringLen存所有环的长度, pos
8     // [i]表示i点在哪个环上, posOnRing[i]表示i在环上的位置
9     std::vector<std::vector<int>> ring, id;
10    // ring[i]存环i的所有点, 下标从0开始
11    std::vector<std::vector<int>> g;
12    RingTreeForest(std::vector<int> &b) : n(b.size()), fa(b), g(n), onRing(n, 0),
13        root(n, 0), pos(n, 0), posOnRing(n, 0), ring(n)
14    {
15        std::vector<int> vis(n, 0);
16        for (int u = 1; u <= n - 1; ++u)
17        {
18            int v = u;
19            while (!vis[v] and v) // 防止跳到0
20            {
21                vis[v] = u;
22                v = fa[v];
23            }
24            if (vis[v] == u and v) // 跳到的不是0于是成环
25            {
26                ++cnt;
27                int k = v, cur = 0;
28                do
29                {
30                    ring[cnt].push_back(k);
31                    onRing[k] = 1;

```

```

30        pos[k] = cnt;
31        root[k] = k;
32        posOnRing[k] = cur++;
33        k = fa[k];
34    } while (k != v);
35    ringLen.push_back(cur);
36    // update a new ring
37    }
38 }
39 for (int i = 1; i <= n - 1; ++i)
40 {
41     if (!onRing[i] and fa[i]) // 不在环上且fa不为空
42     {
43         g[fa[i]].push_back(i);
44     }
45 }
46 }
47 };

```

3.3.8 虚树

虚树是指我们在原树的基础上抽离出一颗新的树，虚树只由关键点以及关键点的 LCA 构成，在预处理一些信息以后我们可以在虚树上求解只涉及关键点的答案。复杂度由 $O(n)$ 降为 $O(V)$ ，其中 V 为关键点的数量。

构建方法：

因为多个点的 LCA 可能是同一个点，为了保证复杂度我们不能多次加入。

我们首先将关键点集合按 dfn 序排序，然后将相邻点的 lca 加入点集。

之后再按 dfn 序排序并去重。然后把相邻的两个点 (u, v) 连边 $lca(u, v) -> v$

```

1 template <class T>
2 struct VirtualTreePre
3 {
4     TreePre<T> pre;
5     int root;
6     std::vector<std::vector<std::pair<int, int>>> vt;
7     VirtualTreePre(std::vector<std::vector<std::pair<int, int>>> &g, int root) : pre(
8         g, root), root(root), vt(g.size()) {}
9
10    void build(std::vector<int> a)
11    {
12        a.push_back(root);
13        std::sort(begin(a), end(a), [&](int u, int v)
14            { return pre.dfn[u] < pre.dfn[v]; });
15        for (int i = a.size() - 1; i; --i)
16        {
17            a.push_back(pre.getLca(a[i], a[i - 1]));
18        }
19        std::sort(begin(a), end(a), [&](int u, int v)
20            { return pre.dfn[u] < pre.dfn[v]; });

```

```

20     a.erase(std::unique(begin(a), end(a)), end(a));
21     for (auto u : a)
22     {
23         vt[u].clear();
24     }
25     for (int i = 1; i < a.size(); ++i)
26     {
27         int lca = pre.getLca(a[i], a[i - 1]);
28         vt[lca].emplace_back(a[i], pre.dis[a[i]] - pre.dis[lca]);
29     }
30 }
31 };

```

3.3.9 树哈希

树哈希主要解决树的同构问题，即快速判断两颗树的结构是否相同。

首先我们得有一个哈希函数 f ，树哈希想要不被卡关键在于哈希函数的设置不能是线性函数 $ax + b$ ，我们可以设置成 $ax^3 + b$ ，对于整棵树哈希值的计算，我们的公式是

$$\text{hash}[u] = 1 + \sum_{v \in \text{son}(u)} f(\text{hash}[v]).$$

有了这两个前置知识，我们分有根树和无根树来讨论。

1 有根树。这种情况十分简单，我们从指定的根开始进行深度优先搜索（记为 dfs ），求出根节点的哈希值，如果两个根节点的哈希值相同，那么两棵有根树同构。

2 无根树。对于无根树的同构，我们有两种方法：

其一是先求出树的一个或两个重心，以重心为根做 dfs 求出哈希值；如果有两个重心就把两个哈希值异或起来，然后对比即可。需要注意的是，如果存在两个重心，就必须求两个重心为根的哈希值，否则会出错。

其二是求出以每个点为根的哈希值，然后异或起来得到最终哈希值，操作方法是朴素的换根 DP ，模板代码给出的就是换根 DP 做法。

```

1 using u64 = unsigned long long;
2
3 template <class T>
4 struct Rand
5 {
6     std::mt19937 myrand;
7     Rand(const long long seed = time(0)) : myrand(seed) {}
8     T operator()(T l, T r)
9     {
10         return std::uniform_int_distribution<T>(l, r)(myrand);
11     }
12 };
13 Rand<u64> rd;
14
15 u64 f(u64 x)
16 {

```

```

17     const static u64 r1 = rd(1 << 20, 1 << 24);
18     const static u64 r2 = rd(1 << 25, 1 << 30);
19     const static u64 mask = (1ll << 31) - 1;
20
21     auto h = [&](u64 y)
22     {
23         return (u64)y * y * y * r1 + r2;
24     };
25     return h(x & mask) + h(x >> 31);
26 }
27
28 struct Treehash // 得到每个点为根的hash值 On
29 {
30     int n;
31     std::vector<std::vector<int>> g;
32     std::vector<u64> hash;
33     u64 ans = 0;
34     void dfs1(int u, int fa)
35     {
36         hash[u] = 1;
37         for (auto v : g[u])
38         {
39             if (v == fa)
40             {
41                 continue;
42             }
43             dfs1(v, u);
44             hash[u] += f(hash[v]);
45         }
46     }
47     void dfs2(int u, int fa)
48     {
49         for (auto v : g[u])
50         {
51             if (v == fa)
52             {
53                 continue;
54             }
55             hash[v] = hash[v] + f(hash[u] - f(hash[v]));
56             dfs2(v, u);
57         }
58     }
59     Treehash(std::vector<std::vector<int>> &e) : n(e.size() - 1), g(e), hash(n + 1)
60     {
61         dfs1(1, 0);
62         dfs2(1, 0);
63         for (int i = 1; i <= n; ++i)
64         {
65             ans ^= hash[i];
66         }
67     }

```

68 };

3.3.10 快速 LCA

我们考虑用 dfs 序求两个点的 LCA

结论：不妨设 $dfn[x] < dfn[y]$ ，那么 $lca(x, y)$ 是 dfn 序上 $[dfn[x] + 1, dfn[y]]$ 区间内深度最小的点的父亲

正确性显然：按 x, y 是否有祖先关系讨论即可。

用返回最小位置的 ST 表预处理可以做到 $O(1)$ 查询。

```
1 template <class T>
2 class FastLCA
3 {
4     int n;
5     std::vector<std::vector<std::pair<int, int>>> g;
6     TreePre<T> pre;
7     ST<T> *st;
8
9 public:
10     FastLCA(std::vector<std::vector<std::pair<int, int>>> &g, int root) : n(g.size()
11         - 1), g(g), pre(g, root)
12     {
13         std::vector<int> reLabel(n + 1);
14         for (int i = 1; i <= n; ++i)
15         {
16             reLabel[pre.dfn[i]] = pre.dep[i];
17         }
18         st = new ST(reLabel);
19     }
20     int getLca(int u, int v)
21     {
22         if (u == v)
23         {
24             return u;
25         }
26         u = pre.dfn[u], v = pre.dfn[v];
27         if (u > v)
28         {
29             std::swap(u, v);
30         }
31         return pre.fa[pre.idfn[(*st)(u + 1, v)]];
32     }
33 };
```

3.3.11 长链剖分

类似于重链剖分，其区别在于重儿子的定义。在此处重儿子的定义为最大深度最深的儿子。

用于快速求树上 k 级祖先， $O(n \log n)$ 预处理， $O(1)$ 查询。

```
1 // 长链剖分类（用于高效处理树上的k级祖先查询）
2 struct LongChainSplit
3 {
4     int n; // 节点数量（编号1~n）
5     std::vector<std::vector<int>> up; // 存储每个链顶向上的节点序列
6     std::vector<std::vector<int>> down; // 存储每个链顶向下的节点序列（沿重儿子）
7     std::vector<std::vector<int>> &g; // 树的邻接表（引用传递）
8     std::vector<std::vector<int>> f; // 倍增表，f[u][i]表示u的2^i级祖先
9     std::vector<int> len; // len[u]: u为根的子树中最长链长度
10    std::vector<int> son; // son[u]: u的重儿子（最长链方向）
11    std::vector<int> dep; // dep[u]: u的深度（根为1）
12    std::vector<int> top; // top[u]: u所在长链的链顶节点
13    std::vector<int> lg; // lg数组：预处理log2值
14    int bit; // 倍增的最大位数
15
16    // 第一次DFS：预处理深度、重儿子、倍增表
17    void dfs1(int u, int fa)
18    {
19        f[u][0] = fa; // u的1级祖先是fa
20        // 预处理倍增表（2^i级祖先）
21        for (int i = 1; i <= bit; i++)
22        {
23            f[u][i] = f[f[u][i - 1]][i - 1]; // 2^i = 2^(i-1) + 2^(i-1)
24        }
25        // 遍历子节点
26        for (auto v : g[u])
27        {
28            if (v != fa)
29            {
30                dep[v] = dep[u] + 1; // 更新深度
31                dfs1(v, u);
32                // 更新重儿子（选择子树链最长的）
33                if (len[v] > len[son[u]])
34                {
35                    son[u] = v;
36                }
37            }
38        }
39        len[u] = len[son[u]] + 1; // u的最长链长度 = 重儿子的最长链长度 + 1
40    }
41
42    // 第二次DFS：构建长链（剖分链）
43    void dfs2(int u, int tp)
44    {
45        top[u] = tp; // 记录u所在链的链顶
46        // 如果是链顶，预处理该链的向上和向下序列
47        if (u == tp)
48        {
49            // 向上序列：从u开始向上跳len[u]次（到根或链外）
50            for (int i = u, j = 0; j < len[u]; j++, i = f[i][0])
51            {
```

```

52         up[tp].push_back(i);
53     }
54     // 向下序列：从u开始沿重儿子走len[u]次
55     for (int i = u, j = 0; j <= len[u]; j++, i = son[i])
56     {
57         down[tp].push_back(i);
58     }
59 }
60 // 优先处理重儿子（保证长链连续）
61 if (son[u])
62 {
63     dfs2(son[u], tp);
64 }
65 // 处理其他轻儿子（新链的起点）
66 for (auto v : g[u])
67 {
68     if (v != f[u][0] && v != son[u])
69     {
70         dfs2(v, v);
71     }
72 }
73 }
74
75 // 查询x的k级祖先
76 int query(int x, int k)
77 {
78     if (!k)
79         return x; // 0级祖先是自己
80     // 先跳到x的2^lg[k]级祖先
81     x = f[x][lg[k]];
82     k -= (1 << lg[k]); // 剩余需要跳的步数
83     k -= dep[x] - dep[top[x]]; // 调整到链顶的相对位置
84     x = top[x];
85     // 根据k的正负决定向上/向下查找
86     if (k >= 0)
87     {
88         return up[x][k]; // 从链顶向上跳k步
89     }
90     else
91     {
92         return down[x][-k]; // 从链顶向下跳-k步
93     }
94 }
95
96 // 构造函数：初始化并预处理
97 LongChainSplit(std::vector<std::vector<int>> &g, int root) : g(g), //
98     // 树的邻接表
99     n(g.size() - 1), //
100     // 节点数（假设编号
101     // 1~n）
102     up(n + 1), //

```

```

100     // 初始化up数组
101     down(n + 1), //
102     // 初始化down数组
103     len(n + 1), //
104     // 初始化len数组
105     son(n + 1), //
106     // 初始化son数组
107     dep(n + 1), //
108     // 初始化dep数组
109     top(n + 1), //
110     // 初始化top数组
111     lg(n + 1), //
112     // 初始化lg数组
113
114 {
115     // 预处理log2值（查询时快速计算二进制最高位）
116     lg[0] = lg[1] = 0;
117     for (int i = 2; i <= n; i++)
118     {
119         lg[i] = lg[i >> 1] + 1;
120     }
121     // 计算倍增的位数（最大2^bit >= n）
122     bit = std::__lg(n) + 1;
123     // 初始化倍增表
124     f.assign(n + 1, std::vector<int>(bit + 1));
125     dep[root] = 1; // 根节点深度为1
126     dfs1(root, 0); // 第一次DFS预处理
127     dfs2(root, root); // 第二次DFS构建长链
128 }
129 };

```

3.4 拓扑排序

拓扑排序是处理 DAG 的算法，其本质是按顺序处理每个点，处理该点时它的所有前驱点已经处理完毕，从而保证信息不会遗漏。它有如下的性质：

只有 DAG 才存在拓扑序，拓扑序是一个点的有序排列，其大小等于原图总点数。拓扑序不止一种，但在没有特殊性质下，拓扑序计数没有多项式时间解，但我们可以通过改队列为优先队列求出最大/最小拓扑序，当然也能判断图是不是 DAG

在 DAG 上跑拓扑排序 DP，可以解决 DAG 最长路、DAG 路径计数、DAG 点对可达情况等等。

对于 DAG 点对可达情况我们可以用 bitset 优化，即 $f(u) \mid= f(v)$ 。

```

1 struct Toposort
2 {
3     int n; // 节点数量
4     std::vector<std::vector<int>> g; // 邻接表
5     std::vector<int> order; // 存储拓扑序
6
7     // 构造函数，初始化节点数量
8     Toposort(int n) : n(n), g(n + 1) {}

```

```

9
10 // 添加有向边 u->v
11 void addEdge(int u, int v)
12 {
13     g[u].push_back(v);
14 }
15
16 // 执行拓扑排序, 返回是否成功 (无环)
17 bool sort()
18 {
19     std::vector<int> deg(n + 1, 0);
20     order.clear();
21     order.reserve(n);
22
23     // 计算入度
24     for (int u = 1; u <= n; ++u)
25     {
26         for (int v : g[u])
27         {
28             ++deg[v];
29         }
30     }
31     std::queue<int> q;
32     // 找出所有入度为0的节点
33     for (int u = 1; u <= n; ++u)
34     {
35         if (deg[u] == 0)
36         {
37             q.push(u);
38         }
39     }
40
41     while (!q.empty())
42     {
43         int u = q.front();
44         q.pop();
45         order.push_back(u);
46         // 减少相邻节点的入度
47         for (int v : g[u])
48         {
49             if (--deg[v] == 0)
50             {
51                 q.push(v);
52             }
53         }
54     }
55
56     // 如果拓扑序包含所有节点, 则无环
57     return order.size() == n;
58 }
59

```

```

60 // 获取拓扑排序结果
61 const std::vector<int> &getOrder() const
62 {
63     return order;
64 }
65 };

```

3.5 最短路

3.5.1 单源最短路

首先我们最短路算法的精髓主要是松弛, 松弛的定义为对于一条边 (u, v, w) , 如果满足 $d[u] + w < d[v]$ 我们让 $d[v] = d[u] + w$ 就叫松弛成功

BFS 由于队列先进先出的特性, 在边权只有 0/1 时我们可以采用 *BFS* 求解最短路。

对于边权为 0 的情况, 我们放到队列的头, 对于边权为 1 的情况, 我们放到队列的尾。

BellmanFord *Bellmanford* 是暴力的松弛算法, 根据松弛的定义我们知道每个点达到最优态最多需要 $n - 1$ 轮, 每轮松弛需要遍历所有的边松弛。特别的, 如果到了第 n 轮还能松弛成功的话, 就说明有负环。

复杂度 $O(nm)$

但是根据算法流程, 我们发现 *Bellmanford* 算法可以计算最多经过 k 条边的最短路, 只需要松弛 k 轮即可

```

1 template <class T>
2 struct BellmanFord
3 {
4     int n; // 节点数量
5     const T INF = std::numeric_limits<T>::max() / 2; // 无穷大值
6     std::vector<std::vector<std::pair<int, int>>> g; // 邻接表
7     std::vector<T> dis; // 最短距离
8     std::vector<int> pre; // 前驱节点
9
10    // 构造函数, 初始化节点数量
11    BellmanFord(int n)
12        : n(n),
13          g(n + 1),
14          dis(n + 1, INF),
15          pre(n + 1, -1) {}
16
17    // 添加有向边 u->v, 权重为 w
18    void addEdge(int u, int v, int w)
19    {
20        g[u].emplace_back(v, w);
21    }
22    // 执行 Bellman-Ford 算法
23    // 参数: 起点 st, 可选参数 k 表示最多经过 k 条边

```



```

24 [[nodiscard]] bool run(int st, int k = -1)
25 {
26     dis.assign(n + 1, INF);
27     pre.assign(n + 1, -1);
28     dis[st] = 0;
29
30     bool cycle = false; // 标记负环
31     int nums = (k == -1) ? n : k; // 迭代轮数
32
33     for (int i = 1; i <= nums; ++i)
34     {
35         bool relaxed = false;
36         auto lastDist = dis; // 保存上一轮距离，防止串联
37
38         for (int u = 1; u <= n; ++u)
39         {
40             if (lastDist[u] == INF)
41                 continue;
42             for (const auto &[v, w] : g[u])
43             {
44                 if (lastDist[u] + w < dis[v])
45                 {
46                     dis[v] = lastDist[u] + w;
47                     pre[v] = u;
48                     relaxed = true;
49                 }
50             }
51         }
52         // 如果第 n 轮仍有松弛操作，说明存在负权环
53         if (i == nums && relaxed && k == -1)
54         {
55             cycle = true;
56         }
57         // 如果本轮没有松弛操作，可以提前终止
58         if (!relaxed)
59             break;
60     }
61     return cycle;
62 }
63 // 获取最短距离数组
64 const std::vector<T> &getDistance() const
65 {
66     return dis;
67 }
68 };

```

Spfa 由于 *Bellmanford* 的复杂度不够优秀，我们引入它的优化版 *SPFA*，我们发现每轮松弛有很多没必要的状态被操作，这大大浪费了时间。

所以 *SPFA* 算法在 *Bellmanford* 的基础上采用了一个队列，队列里记录了可能会松弛成功的点，加

快了算法的执行速度。同样 *SPFA* 也能判断负环，我们用一个 *cnt* 记录松弛次数即可。

作为处理负环的最短路算法，在一般的图上十分高效，但仍可以被毒瘤数据卡成 $O(nm)$ 的复杂度。尽管如此，*SPFA* 仍然是我们在面对负数边权时最好的选择。

```

1 template <class T>
2 class SPFA
3 {
4 private:
5     int n; // 节点数量
6     const T INF = std::numeric_limits<T>::max() / 2; // 无穷大值
7     std::vector<std::vector<std::pair<int, int>>> g; // 邻接表
8     std::vector<T> dis; // 最短距离
9     std::vector<int> pre; // 前驱节点
10    std::vector<int> inQueue; // 节点是否在队列中
11    std::vector<int> relaxCount; // 节点松弛次数
12
13 public:
14     // 构造函数，初始化节点数量
15     SPFA(int n)
16     : n(n),
17       g(n + 1),
18       dis(n + 1, INF),
19       pre(n + 1, -1),
20       inQueue(n + 1, 0),
21       relaxCount(n + 1, 0) {}
22
23     // 添加有向边 u->v，权重为 w
24     void addEdge(int u, int v, int w)
25     {
26         g[u].emplace_back(v, w);
27     }
28
29     // 执行 SPFA 算法
30     // 参数：起点 st
31     // 返回值：pair<bool, bool>
32     // first: 是否成功找到最短路径
33     // second: 是否存在负权环
34     [[nodiscard]] std::pair<bool, bool> run(int st)
35     {
36         // 初始化
37         dis.assign(n + 1, INF);
38         pre.assign(n + 1, -1);
39         inQueue.assign(n + 1, 0);
40         relaxCount.assign(n + 1, 0);
41         dis[st] = 0;
42         std::queue<int> q;
43         q.push(st);
44         inQueue[st] = 1;
45         while (!q.empty())
46         {
47             int u = q.front();

```

```

48     q.pop();
49     inQueue[u] = 0;
50
51     for (const auto &[v, w] : g[u])
52     {
53         if (dis[u] + w < dis[v])
54         {
55             dis[v] = dis[u] + w;
56             pre[v] = u;
57             relaxCount[v] = relaxCount[u] + 1;
58
59             // 如果松弛次数超过n次, 说明存在负权环
60             if (relaxCount[v] >= n)
61             {
62                 return {false, true};
63             }
64
65             if (!inQueue[v])
66             {
67                 q.push(v);
68                 inQueue[v] = 1;
69             }
70         }
71     }
72 }
73
74 return {true, false};
75 }
76
77 // 获取最短距离数组
78 const std::vector<T> &getDistance() const
79 {
80     return dis;
81 }
82
83 // 获取从起点到终点的最短路径
84 std::vector<int> getPath(int target) const
85 {
86     std::vector<int> path;
87     if (pre[target] == -1 && dis[target] == INF)
88     {
89         return path; // 不可达
90     }
91
92     for (int u = target; u != -1; u = pre[u])
93     {
94         path.push_back(u);
95     }
96     std::reverse(path.begin(), path.end());
97     return path;
98 }

```

```

99
100 // 检查图中是否存在负权环
101 bool has_negative_cycle()
102 {
103     // 需要重新初始化
104     dis.assign(n + 1, 0); // 使用0初始化以检测所有可能的负环
105     inQueue.assign(n + 1, 1); // 初始所有节点都在队列中
106     relaxCount.assign(n + 1, 0);
107
108     std::queue<int> q;
109     for (int i = 1; i <= n; ++i)
110     {
111         q.push(i);
112     }
113
114     while (!q.empty())
115     {
116         int u = q.front();
117         q.pop();
118         inQueue[u] = 0;
119
120         for (const auto &[v, w] : g[u])
121         {
122             if (dis[u] + w < dis[v])
123             {
124                 dis[v] = dis[u] + w;
125                 relaxCount[v] = relaxCount[u] + 1;
126
127                 if (relaxCount[v] >= n)
128                 {
129                     return true;
130                 }
131
132                 if (!inQueue[v])
133                 {
134                     q.push(v);
135                     inQueue[v] = 1;
136                 }
137             }
138         }
139     }
140     return false;
141 }
142 };

```

Dijkstra Dijkstra 是我们最为常用的最短路径算法之一, 可以处理无负边图的最短路径, 常规堆优化的最短路径复杂度为 $O(m \log m)$, 如果我们使用 *pbds* 中的斐波那契堆优化, 那么复杂度将是 $O(n \log n + m)$ 特别的, 当边权只有 0 和 1 时, 我们使用 *BFS* 求解最短路径

由于 *pbds* 中的配对堆不支持配模板, 因此该板子没写类模板封装, 需要手动改数据类型。

```

1 struct Dijkstra
2 {
3     using Edge = std::pair<int, int>; // first: 目标节点, second: 边权重
4
5     // 自定义比较函数, 按距离排序
6     struct Compare
7     {
8         bool operator()(const Edge &a, const Edge &b) const
9         {
10             return a.second > b.second; // 小根堆
11         }
12     };
13
14     // 使用配对堆 (最接近斐波那契堆的实现)
15     using Heap = __gnu_pbds::priority_queue<
16         Edge,
17         Compare,
18         __gnu_pbds::pairing_heap_tag>;
19
20     std::vector<int> dis;
21     const int INF = std::numeric_limits<int>::max() / 2;
22     int n;
23
24     // 初始化
25     Dijkstra(int n) : n(n), dis(n + 1) {}
26
27     // 执行Dijkstra算法
28     void run(int S, const std::vector<std::vector<Edge>> &g)
29     {
30         dis.assign(n + 1, INF);
31         std::vector<bool> vis(n + 1, false);
32         std::vector<Heap::point_iterator> handles(n + 1);
33         Heap pq;
34
35         dis[S] = 0;
36         handles[S] = pq.push({S, 0});
37
38         while (!pq.empty())
39         {
40             auto [u, d] = pq.top();
41             pq.pop();
42
43             if (vis[u])
44                 continue;
45             vis[u] = true;
46
47             for (const auto &[v, w] : g[u])
48             {
49                 if (!vis[v] && dis[v] > dis[u] + w)
50                 {
51                     dis[v] = dis[u] + w;

```

```

52             if (handles[v] != nullptr)
53             {
54                 pq.modify(handles[v], {v, dis[v]});
55             }
56             else
57             {
58                 handles[v] = pq.push({v, dis[v]});
59             }
60         }
61     }
62
63     // 获取结果
64     const std::vector<int> &getDistance() const
65     {
66         return dis;
67     }
68 };
69
70

```

还能不能优化呢? 如果还要追求更极致的效率, 需要发掘一些性质

比如: 如果边权总和有限/最短路不会很长, 我们可以用桶配合距离实现最小堆的功能, 这样可以做到 $O(n + m)$

最短路计数 我们不妨设源点为 1, 我们要求源点到其他点最短路的条数, 考虑 *dijkstra* 的过程, 我们开一个 *cnt* 数组记录最短路条数, 如果 $d[u] + w < d[v]$, 那么 $d[u] + w$ 将成为新的最短路, 所以我们让 $cnt[v] = cnt[u]$, 而如果是 $d[u] + w = d[v]$, 那么 v 的最短路将会更多, 我们让 $cnt[v] = cnt[v] + cnt[u]$ 注意先判 *vis*, 无论是大于还是等于都要入队

差分约束 差分约束用来求解一类形如 $x_1 \leq x_2 + c$ 的不等式组的可行解问题, 我们通过建立有向图来解决。

我们把每个未知数看作图上的一个点, 建立虚拟源点 x_0 , 从 x_0 向每个点连一条边权为 0 的有向边。

对于一个未知数 x , 从超级源点 x_0 到它的最短路距离就是一个可行解。

我们接下来详细介绍如何建图, 分求最大解和最小解两种情况。

1) 求最大可行解

未知数 x 的最大可行解为, 从源点到 x 的最短路 $dis(x)$ 。

首先我们将所有不等式转化为 $x_1 \leq x_2 + C$, 我们连边 $x_2 \rightarrow x_1$, 边权为 C 。

如果不等式形如 $x_1 < x_2 + C$, 我们转化为 $x_1 \leq x_2 + C - 1$ 。

如果不等式形如 $C \leq x$, 我们转化为 $x_0 \leq x - C$, 其中 x_0 为超级源点。

2) 求最小可行解

未知数 x 的最小可行解为, 从源点到 x 的最长路 $dis(x)$

首先我们将所有不等式转化为 $x_1 + C \leq x_2$ 我们连边 $x_2 \rightarrow x_1$ 边权为 C

如果不等式形如 $x_1 + C < x_2$ 我们转化为 $x_1 + C + 1 \leq x_2$

如果不等式形如 $x \leq C$ 我们转化为 $x - C \leq x_0$ 其中 x_0 为超级源点

我们选择合适的最短路算法求解即可如果存在负环, 那么原不等式组无解 (一般用 SPFA)

分层图最短路 分层图最短路是将原本的一层图复制变成多层图帮助我们求答案的一个 *Trick*, 题目往往形如, 对一个边带权图 $G = (V, E)$, 要求 $S - T$ 的最短路, 但是我们可以免费走 K 条边, 我们复制 K 层相同的图, 在层与层之间连边权为 0 的边, 这样就达到了免费走边的目的, 如果当前在第 i 层图, 相当于走了 $i - 1$ 条免费边

最短路树和最短路 DAG 最短路树:

在我们求解以 S 为起点的单源最短路时, 我们每次对一个点成功松弛时, 即 $d[u] + w - > d[v]$ 时, 我们令 $fa[v] = u$, 也就是说我们记录每个点的前驱, 最终会形成一棵树的结构, 称作最短路树. 最短路树时处理变形最短路问题的关键方法.

最短路 DAG:

我们求解完单源最短路径之后, 假设起点为 S , 我们建立一张新图, 我们对于每条边 $u -> [v, w]$, 如果有 $dis[u] + w = dis[v]$, 那么我们在新图上连边 $u -> v$, 这样我们将得到一张 *DAG*, 对于这张 *DAG* 的所有生成树, 都是一个合法的最短路树, 因此我们可以套用矩阵树定理求最短路树数量

删边/带修最短路 这是一个非常经典的问题, 题目形式为 q 次询问, 每次询问删去某条边 $S - T$ 的最短路, 删除非永久化.

每次询问的 $S - T$ 相同.

我们用 $path(x, y)$ 表示最短路树上 (x, y) 的路径, D 表示原最短路长度

我们先求出 $path(S, T)$, 注意到 $path(S, T)$ 是一段连续的边的区间

如果询问删去的边不属于 $path(S, T)$, 那么 $ans = D$

如果询问删去的边属于 $path(S, T)$, 我们预处理出每条非最短路边的贡献, 对于一条非最短路边 $u -> [v, w]$, 我们把经过这条边的最短路表示为 $dis(S, u) + w(u, v) + dis(v, T)$, 不难发现 $w(u, v)$ 有贡献的区间是 $[L, R]$, 其中 L 为 $path(S, T)$ 和 $path(S, u)$ 第一个非重合点的位置, R 为 $path(T, S)$ 和 $path(T, v)$ 第一个非重合点的位置. 我们可以通过最短路树上 *DP* 求出所有点的 L, R , 这里的贡献的含义是, 删去这个区间的边后我可以用 $w(u, v)$ 代替, 因此我们使用区间 *min* 线段树直接维护最短路值 $dis(S, u) + w(u, v) + dis(v, T)$, 每次询问单点查询即可

```
1 template <class G>
2 class EdgeDeleteShortestPath
3 {
4 public:
5     SegmentTree<G> *Tree; // 区间min线段树
6     const G inf = std::numeric_limits<G>::max() / 2; // 上限
7     std::map<std::pair<int, int>, int> mp;
8     std::vector<G> disS, disT; // S为起点的dis数组 T为起点的dis数组
9     std::vector<int> preS, preT; //
10    G before; // before表示最开始的最短路
11    EdgeDeleteShortestPath(int S, int T, std::vector<std::vector<std::pair<int, G>>>
12        &g)
13    {
14        int n = g.size() - 1;
15        Dijkstra dijk(n);
16        dijk.run(S, g);
17        before = dijk.dis[T];
```

```
18 preS = dijk.getPre();
19 disS = dijk.getDistance();
20 dijk.run(T, g);
21 preT = dijk.getPre();
22 disT = dijk.getDistance();
23 std::vector<int> path; // 从s->t
24 int now = S;
25 while (now != -1)
26 {
27     path.push_back(now);
28     now = preT[now];
29     if (path.size() >= 2)
30     {
31         mp[{path[path.size() - 2], path.back()}] = path.size() - 1;
32         mp[{path.back(), path[path.size() - 2]}] = path.size() - 1; // 记录路
33         // 径是第几条边
34     }
35 }
36 std::vector<std::vector<int>> treeS(n + 1), treeT(n + 1); // 最短路树
37 for (int i = 1; i <= n; ++i)
38 {
39     if (preS[i] != -1)
40     {
41         treeS[preS[i]].push_back(i);
42     }
43     if (preT[i] != -1)
44     {
45         treeT[preT[i]].push_back(i);
46     }
47 }
48 int PathSum = path.size() - 1;
49
50 std::vector<int> dpS(n + 1, 0), dpT(n + 1, 0);
51 auto dfsS = [&](auto dfsS, int u) -> void
52 {
53     for (auto v : treeS[u])
54     {
55         if (mp.contains(std::make_pair(u, v))) // 说明还在干线上
56         {
57             dpS[v] = mp[{std::make_pair(u, v)}] + 1;
58             dfsS(dfsS, v);
59         }
60         else
61         {
62             dpS[v] = dpS[u];
63             dfsS(dfsS, v);
64         }
65     }
66 };
67 dpS[S] = 1;
68 dfsS(dfsS, S);
```

```

68 auto dfsT = [&](auto dfsT, int u) -> void
69 {
70     for (auto v : treeT[u])
71     {
72         if (mp.contains(std::make_pair(u, v))) // 说明还在线上
73         {
74             dpT[v] = mp[{std::make_pair(u, v)}] - 1;
75             dfsT(dfsT, v);
76         }
77         else
78         {
79             dpT[v] = dpT[u];
80             dfsT(dfsT, v);
81         }
82     }
83 };
84 dpT[T] = path.size() - 1;
85 dfsT(dfsT, T);
86 std::vector<G> cur(path.size(), inf);
87 Tree = new SegmentTree<G>(cur);
88 for (int u = 1; u <= n; ++u)
89 {
90     for (auto [v, w] : g[u])
91     {
92         if (!mp.contains({u, v})) // 说明不是最短路边需要更新答案
93         {
94             if (dpS[u] <= dpT[v] and dpT[v] <= path.size() - 1 and dpS[u] >=
95                 1)
96             {
97                 Tree->rangeGetMin(dpS[u], dpT[v], disS[u] + disT[v] + w);
98             }
99         }
100     }
101 }
102 G getShortestPath(int u, int v)
103 {
104     if (!mp.contains({u, v}))
105     {
106         return before;
107     }
108     else
109     {
110         return Tree->rangeQuerySum(mp[{u, v}], mp[{u, v}]);
111     }
112 }
113 };

```

平面图最小割 我们有如下定理:

对于一个平面图来说, $S - T$ 的最小割等价于在其对偶图上的最短路。

实际上我们最大的困难在于把平面图转化为其对偶图

同余最短路 当出现形如给定 n 个整数, 求这 n 个整数能拼凑出多少个其他整数 (n 个整数可以重复取), 以及给定 n 个整数, 求这 n 个整数不能拼凑出的最小 (最大) 的整数, 或者至少要拼几次才能拼出模 P 余 K 的数的问题时可以使用同余最短路的方法。

我们用例题来说明, 给你 x, y, z , 请你求出 $ax + by + cz (a, b, c \geq 0)$ 能构成 $[1, h]$ 区间内多少个不同的数字?

我们不妨设 $x < y < z$, 我们考虑不使用最小数字 x 的情况下, 求出该表达式 $\text{mod } x = i$ 的最小数字, 具体的说, 假如余数为 i 的最小数为 d_i , 那么所有的 $d_i + kx$ 都能构成。那么怎么求出最小数呢, 我们可以进行的操作为 $i \rightarrow (i + y) \text{ mod } x$ 和 $i \rightarrow (i + z) \text{ mod } x$, 不难发现这是一个有向边, 边权分别为 y 和 z , 这启发我们使用最短路算法得到所有的 d_i , 那么最终答案为 $\sum_{i=0}^{x-1} ((h - d_i) / x + 1)$, 注意源点为 0 且 $d[0] = 0$

3.5.2 全源最短路

全源最短路实际上是求出图上任意两点间的最短路。

我们分稠密图和稀疏图来分别考虑。

对于稠密图而言, *Floyd* 算法是最好的选择, 因为时间复杂度与 m 无关, 我们可以在 $O(n^3)$ 的时间内解决该问题。

对于稀疏图来说, 我们有两种选择: 对于没有负边的图, 我们可以跑 n 次 *dijkstra* 算法来实现, 复杂度 $n^2 \log n$; 对于有负权的图, 我们只能采用全源最短路算法 *johnson* 算法, 复杂度为 $nm \log n$ 。

Floyd

```

1 template <class T>
2 class Floyd
3 {
4 private:
5     int n; // 节点数量
6     const T INF = std::numeric_limits<T>::max() / 2; // 防止加法溢出
7     std::vector<std::vector<T>> dp; // 距离矩阵
8     std::vector<std::vector<int>> next; // 路径重建矩阵
9
10 public:
11     // 构造函数, 初始化节点数量
12     Floyd(int n)
13         : n(n),
14           dp(n + 1, std::vector<T>(n + 1, INF)),
15           next(n + 1, std::vector<int>(n + 1, -1))
16     {
17
18         // 初始化对角线为0
19         for (int i = 1; i <= n; ++i)
20         {
21             dp[i][i] = 0;
22             next[i][i] = i;
23         }

```

```

24 }
25
26 // 添加无向边 u-v, 权重为 w (自动处理重边)
27 void addEdge(int u, int v, T w)
28 {
29     dp[u][v] = std::min(dp[u][v], w);
30     dp[v][u] = std::min(dp[v][u], w);
31     next[u][v] = v;
32     next[v][u] = u;
33 }
34
35 // 执行 Floyd-Warshall 算法
36 void run()
37 {
38     for (int k = 1; k <= n; ++k)
39     {
40         for (int i = 1; i <= n; ++i)
41         {
42             for (int j = 1; j <= n; ++j)
43             {
44                 if (dp[i][k] + dp[k][j] < dp[i][j])
45                 {
46                     dp[i][j] = dp[i][k] + dp[k][j];
47                     next[i][j] = next[i][k];
48                 }
49             }
50         }
51     }
52 }
53
54 // 获取距离矩阵
55 const std::vector<std::vector<T>> &getDistanceMatrix() const
56 {
57     return dp;
58 }
59
60 // 获取两点间的最短距离
61 T getDistance(int u, int v) const
62 {
63     return dp[u][v];
64 }
65
66 // 获取两点间的最短路径
67 std::vector<int> getPath(int u, int v) const
68 {
69     std::vector<int> path;
70
71     // 不可达情况
72     if (next[u][v] == -1)
73     {
74         return path;

```

```

75 }
76 path.push_back(u);
77 while (u != v)
78 {
79     u = next[u][v];
80     path.push_back(u);
81 }
82 return path;
83 }
84
85 // 检查图中是否存在负权环
86 bool hasNegativeCycle() const
87 {
88     for (int i = 1; i <= n; ++i)
89     {
90         if (dp[i][i] < 0)
91         {
92             return true;
93         }
94     }
95     return false;
96 }
97 };

```

Johnson *Johnson* 算法实际上也是跑 n 轮 *dijkstra* 算法的做法, 但我们提前处理了负边权, 方法如下。

我们建立超级虚拟源点 0, 向每个点连接边权为 0 的有向边, 然后我们跑 *SPFA* 算法得到 0 向每个点的最短路 $D(i)$, 然后我们重新设定边权, 对原先的边 (u, v, w) , 我们改为 $(u, v, w + d[u] - d[v])$, 可以证明这种操作可以使新的边权为非负数, 然后我们利用新边权跑 n 轮 *dijkstra* 算法, 最终两点之间实际的最短路为 $Dis(u, v) = dis(u, v) + d[u] - d[v]$, 其中 $dis(u, v)$ 是 *dijkstra* 的结果。实现完全基于 *SPFA* 和 *Dijkstra* 算法, 在此不给出代码。

无向图最小环 我们不难想到有朴素的 *dijkstra* 算法, 即对每条边 (u, v, w) , 我求出删去该边后 u 到 v 的最短路 $dis(u, v)$, 然后此时的最小环为 $dis(u, v) + w$, 对每条边处理答案取小即可, 但复杂度十分爆炸, 高达 $m^2 \log m$

于是我们考虑改进 *floyd* 算法, 我们注意到 *floyd* 算法的执行过程, 当外层枚举到 k 时, 此时所有点之间的最短路只经过 $[1, k]$ 区间内的点。我们记 $mp[u][v]$ 为原始边权, $dis[u][v]$ 为最短路, 我们不难想到外层枚举到 k 时, 处理经过点 k 的环路, 其中 k 是最大点, 于是我们双层循环枚举 i, j , i, j 分别为 k 的两个邻点, 取环路长度为 $dis[i][j] + mp[k][i] + mp[j][k]$, 然后正常跑剩下的 *floyd* 算法

```

1 template <class T>
2 class FloydMinCycle
3 {
4 private:
5     int n;
6     const T INF = std::numeric_limits<T>::max() / 2; // 无穷大值
7     std::vector<std::vector<T>> dist; // 最短距离矩阵

```

```

8      std::vector<std::vector<T>> g;          // 原始图邻接矩阵
9      T mincycle;                          // 最小环长度
10
11 public:
12     // 构造函数, 初始化节点数量
13     FloydMinCycle(int n)
14     : n(n),
15       dist(n + 1, std::vector<T>(n + 1, INF)),
16       g(n + 1, std::vector<T>(n + 1, INF)),
17       mincycle(INF)
18     {
19
20         // 初始化对角线为0
21         for (int i = 1; i <= n; ++i)
22         {
23             dist[i][i] = g[i][i] = 0;
24         }
25     }
26
27     // 添加无向边 u-v, 权重为 w (自动处理重边)
28     void addEdge(int u, int v, T w)
29     {
30         g[u][v] = g[v][u] = std::min(g[u][v], w);
31         dist[u][v] = dist[v][u] = g[u][v];
32     }
33
34     // 执行 Floyd 最小环算法
35     void findMinCycle()
36     {
37         mincycle = INF;
38         for (int k = 1; k <= n; ++k)
39         {
40             // 检查以k为最大节点的环
41             for (int i = 1; i < k; ++i)
42             {
43                 for (int j = i + 1; j < k; ++j)
44                 {
45                     // 防止三个INF相加溢出
46                     if (dist[i][j] < INF && g[i][k] < INF && g[k][j] < INF)
47                     {
48                         mincycle = std::min(
49                             mincycle,
50                             dist[i][j] + g[i][k] + g[k][j]);
51                     }
52                 }
53             }
54             // 更新最短路径
55             for (int i = 1; i <= n; ++i)
56             {
57                 for (int j = 1; j <= n; ++j)
58                 {

```

```

59                     if (dist[i][k] < INF && dist[k][j] < INF)
60                     {
61                         dist[i][j] = std::min(
62                             dist[i][j],
63                             dist[i][k] + dist[k][j]);
64                     }
65                 }
66             }
67         }
68     }
69
70     // 获取最小环长度
71     T getMinCycleLength() const
72     {
73         return mincycle;
74     }
75
76     // 检查图中是否存在环
77     bool hasCycle() const
78     {
79         return mincycle < INF;
80     }
81 };

```

有向图最小平均值环

```

1 // 点标号为 1,2,...,n, 0 为虚拟源点向其他点连权值为0的单向边。
2 // f[i][v] : 从 0 到 v 恰好经过 i 条路的最短路
3 ll f[N][N] = {Inf};
4 int u[M], v[M], w[M];
5 f[0][0] = 0;
6 for (int i = 1; i < n + 1; i++)
7     for (int j = 0; j < m; j++)
8         f[i][v[j]] = min(f[i][v[j]], f[i - 1][u[j]] + w[j]);
9
10 double ans = Inf;
11 for (int i = 1; i <= n; i++)
12 {
13     double t = -Inf;
14     for (int j = 1; j < n; j++)
15         t = max(t, (f[n][i] - f[j][i]) / (double)(n - j));
16     ans = min(t, ans);
17 }

```

3.5.3 多源最短路

这里的多源是指有多个起点, 对于非源点的节点, 我们只需求出到其最近源点的距离, 称为多源最短路问题

朴素的最短路算法在多源最短路仍然使用，以 *dijkstra* 算法为例，我们只需要把初始的多个源点都设置 $d(u) = 0$ ，并全部 *push* 到优先队列中即可，然后跑正常的 *dijkstra* 算法，同理我们也有多源 *BFS* 算法

3.5.4 最长路

我们分情况考虑，如果图是一张 *DAG*，我们在 *DAG* 上拓扑排序朴素 *DP* 即可，复杂度 $O(n + m)$ ，但如果是一张普通图，由于 *dijkstra* 的特性只能处理最短路，我们考虑用 *SPFA* 来解决这个问题，操作方法很简单，只需要边权取反跑最短路即可，如果存在负环，说明最长路无穷大，复杂度最坏 $O(nm)$

3.6 生成树

生成树问题是在一个无向图 $G = (V, E)$ 中找到 $V - 1$ 条边，并且这 $V - 1$ 条边满足联通这 V 个点，然后我们会得到一个树形结构称为生成树。常见问题有最小生成树，次小生成树等

3.6.1 最小生成树

我们来讨论最小生成树问题，有三种主流算法 *kruskal*, *prim*, *boruvka*

我们仍然分稠密图和稀疏图来考虑

在稀疏图中，*kruskal* 拥有 $m \log m$ 的复杂度，相比于另外二者压倒式的好用。

其思想基于按边权排序，贪心的选，正确性显然

Kruskal

```
1 template <class T>
2 struct Kruskal
3 {
4     int n;
5     int comp;
6     DSU dsu;
7     std::vector<std::array<int, 3>> g;
8
9     Kruskal(int n) : n(n), dsu(n), comp(n) {}
10
11     void addEdge(int u, int v, int w)
12     {
13         g.push_back({w, u, v});
14     }
15
16     T getMstWeight()
17     {
18         // 排序边，按照边的权重升序排序
19         std::sort(g.begin(), g.end());
20
21         T mstWeight = 0;
22         for (auto &[w, u, v] : g)
23         {
24             if (!dsu.iscon(u, v))
```

```
25         {
26             dsu.merge(u, v);
27             mstWeight += w;
28             --comp;
29         }
30     }
31     if (comp != 1)
32     {
33         return -1;
34     }
35     return mstWeight;
36 }
37 };
```

在稠密图中，边权 m 达到了 $O(n^2)$ 级别，我们就考虑使用堆优化的 *prim* 算法，*prim* 算法类似于 *dijkstra* 算法，我们不断从堆中取出当前 *dis* 最小的点更新答案，并遍历这个点的出边更新其他点的 *dis*，朴素算法是 $O(n^2 + m)$ 的，我们用 *pbds* 中的配对堆做到 $O(1)$ 修改 *key* 值，复杂度 $(n \log n + m)$ ，需要注意的是这里的 *dis*[*u*] 表示 *u* 点到已经选定的点的最小距离

prim 算法的正确性显然，相当于我们从任意一点开始，不断选最小的边开枝散叶，直至结束为止。

Prim

```
1 struct PrimMST
2 {
3     const int INF = 1e9; // 定义无穷大常量
4     int n; // 顶点数量(下标从1开始)
5     std::vector<std::vector<std::pair<int, int>>> g; // 邻接表存储图
6
7     // 构造函数，初始化顶点数量
8     PrimMST(int n) : n(n), g(n + 1) {}
9
10    // 添加无向边
11    void addEdge(int u, int v, int w)
12    {
13        g[u].emplace_back(v, w);
14        g[v].emplace_back(u, w);
15    }
16
17    // 执行Prim算法，返回最小生成树权值和，无法构成返回-1
18    int compute()
19    {
20        int Sum = 0;
21        int Num = 0;
22
23        // 定义配对堆类型，存储(距离, 顶点)对，使用小根堆
24        using PairingHeap = __gnu_pbds::priority_queue<
25            std::pair<int, int>,
26            std::greater<std::pair<int, int>>,
27            __gnu_pbds::pairing_heap_tag>;
28
29        PairingHeap pq;
```



```

30 std::vector<PairingHeap::point_iterator> handlers(n + 1);
31
32 // 初始化优先队列，所有节点距离设为INF，起点距离为0
33 handlers[1] = pq.push({0, 1});
34 for (int i = 2; i <= n; ++i)
35 {
36     handlers[i] = pq.push({INF, i});
37 }
38
39 while (!pq.empty())
40 {
41     auto [dist, u] = pq.top();
42     pq.pop();
43
44     if (dist == INF)
45     {
46         return -1; // 存在不连通节点
47     }
48
49     Sum += dist;
50     Num++;
51     handlers[u] = nullptr; // 标记已处理
52
53     // 遍历邻接节点
54     for (const auto &[v, w] : g[u])
55     {
56         if (handlers[v] != nullptr && w < handlers[v]->first)
57         {
58             pq.modify(handlers[v], {w, v});
59         }
60     }
61
62     return (Num == n) ? Sum : -1;
63 }
64
65 };

```

我们还有第三种 *MST* 算法 *boruvka*，适用于其他特殊情况，在特殊条件下也十分好用

其核心思想是减少联通块的个数，我们每次遍历所有边 (u, v, w) 如果 u 和 v 不在同一联通块内，就用 w 更新两个联通块之间的最小边，然后每次我们将这些边加入答案，并合并联通块。不难发现每次联通块个数减少一半，复杂度 $O(m \log n)$

常规题目我们一般不考虑此算法，但在某些题目中，两点间的边权由特定关系决定，如果我们能快速得到最小边，就会考虑采用 *boruvka* 算法了

```

Boruvka
1 template <class T>
2 struct BoruvkaMST
3 {
4     int n; // 顶点数量(下标从1开始)
5     int comp; // 当前连通块数量

```

```

6     std::vector<std::vector<std::pair<int, int>>> g; // 邻接表存储图
7     DSU dsu; // 并查集结构
8
9     // 构造函数，初始化顶点数量和并查集
10    BoruvkaMST(int n) : n(n), comp(n), dsu(n + 1), g(n + 1) {}
11
12    // 添加无向边
13    void addEdge(int u, int v, int w)
14    {
15        g[u].emplace_back(v, w);
16        g[v].emplace_back(u, w);
17    }
18
19    // 执行Borůvka算法，返回最小生成树权值和，无法构成返回-1
20    T compute()
21    {
22        constexpr T inf = std::numeric_limits<T>::max() / 2;
23        T totalWeight = 0;
24        bool hasChanges = true;
25
26        while (hasChanges && comp > 1)
27        {
28            hasChanges = false;
29            std::vector<T> min(n + 1, inf); // 当前连通块的最小边权值
30            std::vector<int> match(n + 1, 0); // 最小边对应的相邻连通块
31
32            // 第一步：寻找每个连通块的最小出边
33            for (int u = 1; u <= n; ++u)
34            {
35                int rootU = dsu.root(u);
36                for (const auto &[v, w] : g[u])
37                {
38                    int rootV = dsu.root(v);
39                    if (rootU != rootV && w < min[rootU])
40                    {
41                        min[rootU] = w;
42                        match[rootU] = rootV;
43                    }
44                }
45            }
46
47            // 第二步：合并连通块
48            for (int u = 1; u <= n; ++u)
49            {
50                if (match[u] != 0)
51                {
52                    int rootU = dsu.root(u);
53                    int rootV = dsu.root(match[u]);
54                    if (rootU != rootV)
55                    {
56                        dsu.merge(rootU, rootV);

```

```

57         totalWeight += min[u];
58         --comp;
59         hasChanges = true;
60     }
61 }
62 }
63 }
64 return comp == 1 ? totalWeight : -1;
65 }
66 };

```

3.6.2 生成树计数

对于常规的生成树计数，我们可以通过矩阵树定理求解，代码模板在组合数学中的生成树计数中。

最小生成树计数 考虑 kruskal 算法配合矩阵树定理，我们在对边排序以后，依次考虑每一组权值相同的边，对于这一组的边，我们知道最终连完之后的连通性是相同的，无论其顺序如何，也就是说联通块的情况是相同的，我们将这一组的所有边全部连接，对每个联通块都跑矩阵树定理，并把每个联通块得到的答案相乘，之后将每个联通块都缩成一个点，然后考虑下一组边，直至只剩一个联通块即可，正确性是显然的

3.6.3 最小度限制生成树

题目形如求一个图的 MST ，但需要满足点 $Deg(S) = k$ ，我们不难发现这个问题具有凸完全单调性，我们可以通过朴素的 wqs 二分在 $m \log m \log v$ 的时间内解决

3.6.4 曼哈顿最小生成树

考虑平面上有 n 个二维数点，定义两点间距离为曼哈顿距离 $|x1 - x2| + |y1 - y2|$ ，求最小生成树通过对绝对值的拆分，最终该距离去掉绝对值符号仅有四种可能

$(x1 + y1) - (x2 + y2)$ 或 $(x1 - y1) - (x2 - y2)$

$(x2 + y2) - (x1 + y1)$ 或 $(x2 - y2) - (x1 - y1)$

这一变形的关键意义在于：曼哈顿距离可以表示为点在 4 个“变换坐标系”下的投影差值

在每种变换坐标系中，每个点都能找到离自己最近的几个候选点作为可能的边。细节不过多阐述

```

1 #include <bits/stdc++.h>
2 using ll = long long;
3 const int N = 200010;
4 int n, m, i, j, w[N], c[N], bit[N], f[N];
5 ll ans;
6 struct P
7 {
8     int x, y, p;
9     P() {}
10     P(int _x, int _y, int _p) { x = _x, y = _y, p = _p; }
11 } a[N], b[N], e[N << 2];

```

```

12
13 bool cmp(const P &a, const P &b)
14 {
15     return a.x == b.x ? a.y < b.y : a.x < b.x;
16 }
17 bool cmpe(const P &a, const P &b)
18 {
19     return a.p < b.p;
20 }
21 int lower(int x)
22 {
23     int l = 1, r = n, t, mid;
24     while (l <= r)
25     {
26         mid = (l + r) >> 1;
27         if (c[mid] <= x)
28         {
29             l = (t = mid) + 1;
30         }
31         else
32         {
33             r = mid - 1;
34         }
35     }
36     return t;
37 }
38 int dis(int x, int y)
39 {
40     return std::abs(a[x].x - a[y].x) + std::abs(a[x].y - a[y].y);
41 }
42 void ins(int x, int p)
43 {
44     for (; x <= n; x += x & -x)
45     {
46         if (w[p] <= w[bit[x]])
47         {
48             bit[x] = p;
49         }
50     }
51 }
52 int ask(int x)
53 {
54     int t = 0;
55     for (; x; x -= x & -x)
56     {
57         if (w[bit[x]] <= w[t])
58         {
59             t = bit[x];
60         }
61     }
62     return t;

```

```

63 }
64
65 int F(int x)
66 {
67     return f[x] == x ? x : f[x] = F(f[x]);
68 }
69 void solve()
70 {
71     for (sort(b + 1, b + n + 1, cmp), sort(c + 1, c + n + 1), i = 1; i <= n; i++)
72     {
73         if (j = ask(lower(b[i].y)))
74         {
75             e[++m] = P(b[i].p, j, dis(b[i].p, j));
76         }
77         ins(lower(b[i].y), b[i].p);
78     }
79 }
80 std::vector<std::pair<int, int>> res;
81 ll ManhattanMst()
82 {
83     for (w[0] = ~0U >> 1, m = ans = 0, i = 1; i <= n; i++)
84     {
85         f[i] = i;
86     }
87     for (i = 1; i <= n; i++)
88     {
89         b[i] = P(-a[i].x, a[i].x - a[i].y, i);
90         c[i] = b[i].y;
91         w[i] = a[i].x + a[i].y;
92         bit[i] = 0;
93     }
94     solve();
95     for (i = 1; i <= n; i++)
96     {
97         b[i] = P(-a[i].y, a[i].y - a[i].x, i);
98         c[i] = b[i].y;
99         bit[i] = 0;
100     }
101     solve();
102     for (i = 1; i <= n; i++)
103     {
104         b[i] = P(a[i].y, -a[i].x - a[i].y, i);
105         c[i] = b[i].y;
106         w[i] = a[i].x - a[i].y;
107         bit[i] = 0;
108     }
109     solve();
110     for (i = 1; i <= n; i++)
111     {
112         b[i] = P(-a[i].x, a[i].y + a[i].x, i);
113         c[i] = b[i].y;

```

```

114         bit[i] = 0;
115     }
116     solve();
117     sort(e + 1, e + m + 1, cmpe);
118     for (ans = 0, i = 1; i <= m; i++)
119     {
120         if (F(e[i].x) != F(e[i].y))
121         {
122             res.push_back({e[i].x, e[i].y});
123             f[f[e[i].x]] = f[e[i].y];
124             ans += e[i].p;
125         }
126     }
127     return ans;
128 }
129 int main()
130 {
131     std::ios::sync_with_stdio(0);
132     std::cin.tie(0);
133     std::cin >> n;
134     for (i = 1; i <= n; i++)
135     {
136         std::cin >> a[i].x >> a[i].y;
137     }
138     std::cout << ManhattanMst() << '\n';
139     for (auto [x, y] : res)
140     {
141         std::cout << x << ' ' << y << '\n';
142     }
143     return 0;
144 }

```

3.6.5 最小直径生成树

首先我们有结论，一个无向图的最小直径生成树即以该图的绝对中心为起点跑出的最短路树

因此我们首先的问题在于找到图的绝对中心（该点到图上所有点的最远距离最小）

问题在于图的绝对中心可能在点也可能在边上，我们需要分别讨论这两种情况

具体过程如下：

- 1 使用多源最短路求出任意两点间最短路
- 2 求出 $rk[i][j]$ 表示离 i 点第 j 近的点是谁
- 3 我们考虑中心在某个节点的情况，暴力遍历更新答案即可
- 4 我们考虑中心在边上的情况，我们不妨设 ans 为直径，那么我们枚举所有的边 $w(u, v)$ ，如果出现 $d(v, rk(u, i)) > \max_{j=i+1}^n d(v, rk(u, j))$ ，我们用 $d(u, rk(u, i)) + \max_{j=i+1}^n d(v, rk(u, j)) + w$ 更新答案。

复杂度为 $O(n^3 + nm + O(n^2 \log n))$ ，具体看使用什么算法求全源最短路

1 // 比较函数：用于对节点按距离进行排序

```

2 // 功能: 当val[a] < val[b]时, a的排名在b之前 (升序排序)
3 // 参数: a, b为待比较的节点编号
4 bool cmp(int a, int b)
5 {
6     return val[a] < val[b];
7 }
8
9 // Floyd-Warshall算法: 求解图中所有节点对之间的最短路径
10 // 时间复杂度 $O(n^3)$ , 适用于节点数n不太大的情况
11 void Floyd()
12 {
13     // k为中间节点: 枚举所有可能的中转节点
14     for (int k = 1; k <= n; k++)
15         // i为起点: 枚举所有可能的起点
16         for (int i = 1; i <= n; i++)
17             // j为终点: 枚举所有可能的终点
18             for (int j = 1; j <= n; j++)
19                 // 松弛操作: 通过k中转是否能缩短i到j的距离
20                 d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
21 }
22 void solve()
23 {
24     // 第一步: 先计算所有节点对的最短路径
25     Floyd();
26
27     // 第二步: 对每个节点i, 将其他节点按到i的距离排序
28     for (int i = 1; i <= n; i++)
29     {
30         for (int j = 1; j <= n; j++)
31         {
32             rk[i][j] = j;
33             val[j] = d[i][j];
34         }
35         sort(rk[i] + 1, rk[i] + 1 + n, cmp);
36     }
37
38     // 第三步: 初始化答案
39     // 修正1: 将ans的类型从int改为double, 以精确存储半径
40     double ans = 1e9;
41
42     // 记录绝对中心位置的变量
43     bool ok = true;
44     int idx = -1;
45     int au = -1, av = -1;
46     double t = 0.0;
47     // 情况1: 绝对中心在某个节点上
48     for (int i = 1; i <= n; i++)
49     {
50         int dis = d[i][rk[i][n]];
51         if (dis < ans)
52         {

```

```

53             ok = true;
54             ans = dis; // 将半径存入ans
55             idx = i;
56         }
57     }
58     // 情况2: 绝对中心在某条边上 (u-v, 权重为w)
59     for (int i = 1; i <= m; i++)
60     {
61         int u = a[i].u;
62         int v = a[i].v;
63         int w = a[i].w;
64
65         for (int p = n, j = n - 1; j >= 1; j--)
66         {
67             // 这个双指针优化的逻辑是: 对于每个j, 找到一个p, 使得(j, p)这对最远点构成
68             // 的半径最小。
69             // p需要指向从j到n的节点中, 距离v最远的那个。
70             // 首先更新p, 使其成为从当前j到n的范围内离v最远的点
71             if (d[v][rk[u][j]] > d[v][rk[u][p]])
72             {
73                 p = j;
74             }
75
76             // 然后用当前的j和最优的p来计算路径长度
77             int cur = d[u][rk[u][j]] + d[v][rk[u][p]] + w;
78
79             // 修正2: 用计算出的新半径(cur/2.0)与ans(最小半径)进行比较
80             if (cur / 2.0 < ans)
81             {
82                 // 修正3: ans存储的应该是半径, 而不是路径长度
83                 ans = cur / 2.0;
84                 ok = false;
85                 au = u;
86                 av = v;
87                 t = (d[v][rk[u][p]] + w - d[u][rk[u][j]]) / 2.0;
88                 if (t < 0)
89                     t = 0;
90                 if (t > w)
91                     t = w;
92             }
93         }
94     }

```

3.6.6 动态 MST 问题

LCT 是维护动态 *MST* 的利器, 在原有的 *MST* 上, 如果我们想要再加一条边 (u, v, w) , 我们只需要利用 *LCT* 找到链 (u, v) 中最大边权的值 w' 和位置, 如果有 $w < w'$, 我们把原先的边断开, 链接新边即可, 否则跳过这条边。

次小生成树 严格次小生成树和非严格次小生成树没有做法上的区别，都是 *LCT* 维护生成树的裸题，只需要把非生成树边不断尝试即可

3.6.7 kruskal 重构树

用于瓶颈路相关问题，可以在 $\log n$ 时间内求出图上两点之间最大最小边权问题。但重构树并不一定只能维护边权，例如，我们可以把边出现的时间当作边权维护，从而达到维护连通性的效果，我们适当的把其他元素转化为边权能起到很好的效果。

```

1 class KruskalTree
2 {
3 private:
4     DSU dsu;
5     std::vector<int> lc, rc, val;
6     int idx;
7     std::vector<std::vector<int>> fa;
8     std::vector<int> dep;
9     int lg;
10
11 public:
12     // 构造函数，初始化并构建重构树
13     KruskalTree(const std::vector<std::array<int, 3>> &g, int n)
14         : lc(2 * n + 1),
15           rc(2 * n + 1),
16           val(2 * n + 1),
17           idx(n),
18           dsu(2 * n),
19           dep(2 * n + 1)
20     {
21
22         // 计算二进制提升的最大层数
23         lg = std::__lg(2 * n + 1);
24         fa.assign(2 * n + 1, std::vector<int>(lg + 1, 0));
25
26         // 按边权从大到小排序
27         auto e = g;
28         std::sort(e.begin(), e.end(),
29                 [](const auto &a, const auto &b)
30                 { return a[2] < b[2]; });
31
32         // 构建Kruskal重构树
33         for (const auto &[u, v, w] : e)
34         {
35             if (dsu.root(u) != dsu.root(v))
36             {
37                 ++idx;
38                 int rootU = dsu.root(u);
39                 int rootV = dsu.root(v);
40                 lc[idx] = rootU;
41                 rc[idx] = rootV;

```

```

42         dsu.pre[rootU] = idx;
43         dsu.pre[rootV] = idx;
44         val[idx] = w;
45     }
46     if (idx == 2 * n - 1)
47     {
48         break;
49     }
50 }
51
52 // DFS预处理二进制提升表
53 auto dfs = [&](auto &&self, int u, int parent) -> void
54 {
55     fa[u][0] = parent;
56     for (int i = 1; i <= lg; ++i)
57     {
58         fa[u][i] = fa[fa[u][i - 1]][i - 1];
59     }
60     if (lc[u])
61     {
62         dep[lc[u]] = dep[u] + 1;
63         self(self, lc[u], u);
64     }
65     if (rc[u])
66     {
67         dep[rc[u]] = dep[u] + 1;
68         self(self, rc[u], u);
69     }
70 };
71
72 // 从根节点开始DFS
73 for (int i = idx; i >= 1; --i)
74 {
75     if (!dep[i])
76     {
77         dep[i] = 1;
78         dfs(dfs, i, 0);
79     }
80 }
81
82 // 查询两个节点的最近公共祖先
83 int getLca(int u, int v) const
84 {
85     if (dep[u] < dep[v])
86     {
87         std::swap(u, v);
88     }
89     // 提升u到与v相同深度
90     for (int i = lg; i >= 0; --i)
91     {
92

```

```

93     if (dep[fa[u][i]] >= dep[v])
94     {
95         u = fa[u][i];
96     }
97 }
98 if (u == v)
99 {
100     return u;
101 }
102 // 同时提升u和v
103 for (int i = lg; i >= 0; --i)
104 {
105     if (fa[u][i] != fa[v][i])
106     {
107         u = fa[u][i];
108         v = fa[v][i];
109     }
110 }
111 return fa[u][0];
112 }
113 int getVal(int x)
114 {
115     return val[x];
116 }
117 bool isConnected(int u, int v)
118 {
119     return dsu.root(u) == dsu.root(v);
120 }
121 };

```

3.6.8 最小斯坦纳树

从某种程度上讲斯坦纳树也是生成树的一种，只不过是一个关键点集的生成树，而不是对于所有节点。问题形如，给你一个无向带权图 $G = (V, E)$ ，并且给你 k 个关键点，求一个边集 E 使得这 k 个关键点联通且代价最小，显然最终形态应该是一棵树，但不保证只包含这 k 个节点。

我们利用状压 DP 解决这个问题。由于 k 的数据范围为 10，我们状压关键点的状态，设 $DP[i][S]$ 为以点 i 为根、联通点状态为 S 的最小代价，利用最短路 + 枚举子集 DP 更新状态，复杂度 $O(3^k)$ 。

```

1  /**
2  * @class SteinerTreeSolver
3  * @brief 求解无向图中连接指定关键点的最小斯坦纳树问题
4  *
5  * @details
6  * 使用动态规划结合SPFA松弛的方法求解最小斯坦纳树问题
7  * - 节点编号从1开始
8  * - 关键点存储在数组中下标从0开始
9  * - 时间复杂度:  $O(n * 3^k + m * 2^k * \log n)$ 
10  */
11 template <class T>

```

```

12 class SteinerTreeSolver
13 {
14 public:
15     using Edge = std::tuple<int, int, int>; // (u, v, weight)
16     const T inf = std::numeric_limits<T>::max() / 2;
17     /**
18      * @brief 构造函数
19      * @param n 图中节点总数
20      * @param e 图的边集合
21      * @param key 关键点列表(下标从0开始)
22      */
23     SteinerTreeSolver(int n,
24                       const std::vector<Edge> &e,
25                       const std::vector<int> &key)
26     : n(n),
27       k(key.size()),
28       key(key)
29     {
30
31         // 构建邻接表
32         BuildGraph(e);
33         // 初始化DP数组
34         InitializeDP();
35     }
36
37     /**
38      * @brief 计算最小斯坦纳树的权值和
39      * @return 连接所有关键点的最小边权值和
40      * @throws std::runtime_error 如果没有可行解
41      */
42     T ComputeMinSteinerTree()
43     {
44         // 动态规划处理所有状态
45         for (int state = 0; state < (1 << k); ++state)
46         {
47             // 处理子集合并
48             ProcessSubsetUnions(state);
49             // 使用SPFA松弛当前状态
50             RelaxWithSPFA(state);
51         }
52
53         // 在所有关键点都连通的状态中寻找最小值
54         int fullMask = (1 << k) - 1;
55         T minCost = inf;
56         for (int u = 1; u <= n; ++u)
57         {
58             if (dp[u][fullMask] < minCost)
59             {
60                 minCost = dp[u][fullMask];
61             }
62         }

```

```

63     return minCost;
64 }
65
66 private:
67     int n; // 节点总数
68     int k; // 关键点数量
69     std::vector<int> key; // 关键点列表
70     std::vector<std::vector<std::pair<int, int>>> g; // 邻接表
71     std::vector<std::vector<T>> dp; // DP数组
72     std::queue<int> q; // SPFA使用的队列
73     std::vector<bool> inq; // SPFA标记数组
74
75 /**
76  * @brief 构建图的邻接表表示
77  * @param e 图的边集合
78  */
79 void BuildGraph(const std::vector<Edge> &e)
80 {
81     g.resize(n + 1); // 节点编号从1开始
82
83     for (const auto &[u, v, w] : e)
84     {
85         g[u].emplace_back(v, w);
86         g[v].emplace_back(u, w); // 无向图
87     }
88 }
89
90 /**
91  * @brief 初始化DP数组
92  */
93 void InitializeDP()
94 {
95     // dp[i][s] 表示以i为根，连通状态为s的最小权值
96     dp.resize(n + 1, std::vector<T>(1 << k, inf));
97
98     // 初始化每个关键点单独连通的情况
99     for (int i = 0; i < k; ++i)
100     {
101         dp[key[i]][1 << i] = 0;
102     }
103
104     // 初始化SPFA相关数据结构
105     inq.resize(n + 1, false);
106 }
107
108 /**
109  * @brief 处理状态子集的合并
110  * @param state 当前状态掩码
111  */
112 void ProcessSubsetUnions(int state)
113 {

```

```

114     for (int u = 1; u <= n; ++u)
115     {
116         // 枚举state的所有非空子集subset
117         for (int subset = state; subset > 0; subset = (subset - 1) & state)
118         {
119             // 避免整数溢出
120             if (dp[u][subset] < inf &&
121                 dp[u][state ^ subset] < inf)
122             {
123                 T newCost = dp[u][subset] + dp[u][state ^ subset];
124                 if (newCost < dp[u][state])
125                 {
126                     dp[u][state] = newCost;
127                 }
128             }
129         }
130
131         // 如果当前节点的状态被更新，加入队列准备SPFA松弛
132         if (dp[u][state] < inf)
133         {
134             q.push(u);
135             inq[u] = true;
136         }
137     }
138 }
139
140 /**
141  * @brief 使用SPFA算法松弛当前状态
142  * @param state 当前状态掩码
143  */
144 void RelaxWithSPFA(int state)
145 {
146     while (!q.empty())
147     {
148         int u = q.front();
149         q.pop();
150         inq[u] = false;
151
152         for (const auto &[v, weight] : g[u])
153         {
154             if (dp[u][state] < inf &&
155                 dp[u][state] + weight < dp[v][state])
156             {
157                 dp[v][state] = dp[u][state] + weight;
158                 if (!inq[v])
159                 {
160                     q.push(v);
161                     inq[v] = true;
162                 }
163             }
164         }
165     }

```

```

165     }
166 }
167 };

```

3.7 连通性

3.7.1 有向图连通性

我们只要研究强联通分量以及其应用，一个强连通分量内的点两两可达，强联通具有传递性。

强连通分量

```

1 class SCC
2 {
3     const std::vector<std::vector<int>> &e; // 图的邻接表引用（输入图）
4     std::vector<int> stk; // Tarjan算法使用的栈
5     int tt = 0; // 栈顶指针
6     int idx = 0; // 时间戳计数器
7
8     // Tarjan算法的DFS核心实现
9     void dfs(int x)
10    {
11        dfn[x] = low[x] = ++idx; // 初始化当前节点的dfn和low值
12        stk[++tt] = x; // 节点入栈
13
14        // 遍历所有邻接节点
15        for (int y : e[x])
16        {
17            if (!dfn[y]) // 未访问过的节点
18            {
19                dfs(y); // 递归搜索
20                low[x] = std::min(low[x], low[y]); // 更新low值
21            }
22            else if (!bel[y]) // 已访问但未确定SCC的节点（在栈中）
23            {
24                low[x] = std::min(low[x], dfn[y]); // 更新low值
25            }
26        }
27
28        // 发现SCC的根节点
29        if (dfn[x] == low[x])
30        {
31            cntBlock += 1; // SCC计数器增加
32            int y;
33            do
34            {
35                y = stk[tt--]; // 弹出栈顶节点
36                p[cntBlock].push_back(y); // 将节点加入当前SCC
37                bel[y] = cntBlock; // 标记节点所属SCC编号
38            } while (y != x); // 直到处理到当前根节点
39        }

```

```

40    }
41
42 public:
43     // 原始图的访问信息
44     std::vector<int> dfn; // DFS访问序号（时间戳）
45     std::vector<int> low; // 能回溯到的最早节点的时间戳
46     std::vector<int> bel; // 节点所属的SCC编号
47
48     // SCC相关数据
49     std::vector<std::vector<int>> p; // 每个SCC包含的节点列表
50     std::vector<std::vector<int>> g; // 缩点后的DAG图（邻接表）
51     std::vector<int> din; // 缩点后各节点的入度
52     std::vector<int> siz; // 各SCC的大小（节点数）
53     int cntBlock = 0; // SCC总数计数器
54
55     // 构造函数：输入图的邻接表
56     SCC(const std::vector<std::vector<int>> &e)
57     : e(e), // 初始化图引用
58       dfn(e.size()), // 分配dfn数组空间
59       low(e.size()), // 分配low数组空间
60       bel(e.size()), // 分配bel数组空间
61       p(e.size()) // 分配SCC节点列表空间
62    {
63        int n = e.size() - 1; // 节点数（假设节点从1开始编号）
64        stk.assign(n + 1, 0); // 初始化栈
65
66        // 对每个未访问节点执行Tarjan算法
67        for (int i = 1; i <= n; i++)
68        {
69            if (!dfn[i]) // 未访问过的节点
70            {
71                dfs(i); // 开始DFS
72            }
73        }
74
75        // 构建缩点后的DAG图
76        g.resize(cntBlock + 1); // 调整缩点图大小
77        din.resize(cntBlock + 1); // 调整入度数组大小
78        siz.resize(cntBlock + 1); // 调整SCC大小数组
79
80        // 遍历原始图构建缩点图
81        for (int x = 1; x <= n; x++)
82        {
83            siz[bel[x]]++; // 统计各SCC大小
84            for (int y : e[x]) // 遍历邻接节点
85            {
86                if (bel[x] == bel[y]) // 同一SCC内部边跳过
87                {
88                    continue;
89                }
90                g[bel[x]].push_back(bel[y]); // 添加缩点后的边

```



```

91         din[bel[y]]++;           // 更新目标SCC的入度
92     }
93 }
94 }
95 };
96 // 如果belong[u]<belong[v]且u v联通 那么缩点以后边的方向是v->u

```

2-Sat 2-Sat 问题是给定你一些二元布尔表达式，我们要求出满足所有表达式的一个解，或者报告无解。

需要注意的是如果一个变量恒成立那么我们连边 $a + n - > a$

相对应的，如果一个变量恒不成立，我们连边 $a - > a + n$

如何判断无解？存在某个变量 b 满足 b 和 $b + n$ 在同一个 SCC 内

如何输出方案？对于 b 和 $b + n$ 我们取拓扑序大的为真，那么在 SCC 中拓扑序越大的所在 SCC 编号越小，因此输出方案为 $bel[i] < bel[i + n]$ ，

支配树 $idom[i]$ 为 i 的直接支配点，在支配树上是 i 的父节点， $sdom[i]$ 是半支配点，除了辅助求支配点外无实际意义。支配树上一一点的所有祖先都是该点的支配点。

```

1 struct DominatorTree
2 {
3     int n, r;
4     std::vector<std::vector<int>> G, rG; // 原图和反图
5     std::vector<int> dfn, idfn, parent; // dfn[u] = DFS序, idfn[i] = DFS序为i的节点
6     // parent[u] = DFS树父节点
7     std::vector<int> sdom, idom; // 半支配点和直接支配点
8     std::vector<int> fa, mn; // 并查集结构
9     std::vector<std::vector<int>> bucket; // 分组处理sdom
10
11     DominatorTree(int n, int r) : n(n), r(r), G(n + 1), rG(n + 1) {}
12
13     void addEdge(int u, int v)
14     {
15         G[u].push_back(v);
16         rG[v].push_back(u); // 同时构建反图
17     }
18
19     void dfs(int u)
20     {
21         dfn[u] = idfn.size();
22         idfn.push_back(u);
23         for (int v : G[u])
24         {
25             if (!dfn[v])
26             { // 未访问过
27                 parent[v] = u;
28                 dfs(v);
29             }
30         }
31     }
32 }

```

```

30 }
31
32 // 并查集：路径压缩 + 维护sdom最小值
33 int find(int u)
34 {
35     if (fa[u] == u)
36         return u;
37     int root = find(fa[u]);
38     if (dfn[sdom[mn[fa[u]]]] < dfn[sdom[mn[u]]])
39     {
40         mn[u] = mn[fa[u]]; // 更新最小值
41     }
42     return fa[u] = root; // 路径压缩
43 }
44
45 // 构建成支配树，返回idom数组（idom[u] = u的直接支配点）
46 std::vector<int> build()
47 {
48     // Step 1: 初始化
49     dfn.assign(n + 1, 0);
50     parent.assign(n + 1, 0);
51     idfn.push_back(0); // 占位，使id[1..n]有效
52     dfs(r); // 从根节点开始DFS
53
54     // Step 2: 初始化sdom和idom
55     sdom.resize(n + 1);
56     idom.resize(n + 1);
57     for (int i = 1; i <= n; ++i)
58     {
59         sdom[i] = i;
60         idom[i] = 0; // 初始化为0（无效值）
61     }
62
63     // Step 3: 并查集初始化
64     fa.resize(n + 1);
65     mn.resize(n + 1);
66     for (int i = 1; i <= n; ++i)
67     {
68         fa[i] = mn[i] = i;
69     }
70
71     bucket.resize(n + 1);
72     // Step 4: 按DFS序逆序处理（从大到小）
73     for (int i = (int)idfn.size() - 1; i >= 2; --i)
74     { // 跳过根节点（i=1）
75         int u = idfn[i];
76
77         // Step 4.1: 计算sdom[u]
78         for (int v : rG[u])
79         { // 遍历反图（即原图的前驱）
80             if (!dfn[v])

```

```

81         continue; // 跳过不可达节点
82     if (dfn[v] < dfn[u])
83     {
84         sdom[u] = dfn[v] < dfn[sdom[u]] ? v : sdom[u];
85     }
86     else
87     {
88         find(v); // 路径压缩
89         sdom[u] = dfn[sdom[mn[v]]] < dfn[sdom[u]] ? sdom[mn[v]] : sdom[u];
90     }
91 }
92 bucket[sdom[u]].push_back(u);
93
94 // Step 4.2: 合并到父节点
95 if (parent[u])
96 {
97     merge(parent[u], u);
98 }
99
100 // Step 4.3: 处理bucket中的节点
101 for (int v : bucket[parent[u]])
102 {
103     find(v);
104     if (sdom[mn[v]] == parent[u])
105     {
106         idom[v] = parent[u];
107     }
108     else
109     {
110         idom[v] = mn[v]; // 暂时存储, 后续修正
111     }
112 }
113 bucket[parent[u]].clear();
114 }
115
116 // Step 5: 修正idom
117 for (int i = 2; i < idfn.size(); ++i)
118 { // 跳过根节点 (i=1)
119     int u = idfn[i];
120     if (idom[u] != sdom[u])
121     {
122         idom[u] = idom[idom[u]];
123     }
124 }
125 // 根节点idom设为0 (表示无支配点)
126 idom[r] = 0;
127 return idom;
128 }
129
130 private:

```

```

131 // 并查集合并操作
132 void merge(int u, int v)
133 {
134     fa[v] = u;
135 }
136 };

```

3.7.2 无向图连通性

我们主要研究点双连通分量和边双连通分量，前者对应割点，后者对应割边，割边又称桥。特别的，孤立点和孤立边的两个端点都不是割点，但孤立边是割边。

我们有如下结论：

- 1 对于 (u, v) 两点间任意一条简单路径，其路径上的割边就是两点间的所有必经边，所以割边也可以看做必经边。
 - 2 对于两个点双连通分量，如果二者有公共点，那么该点一定是割点，反过来讲，该点是割点当且仅当它属于两个以上点双。
 - 3 边双具有传递性，但点双不具有传递性。
 - 4 如果两个点属于一个点双，那么这两点之间所有简单路径的并集恰好完全等于该点双。
- 对于边双，我们可以通过缩点进一步刻画其连通性，但是点双的形态不适用于缩点，我们用广义圆方树刻画。

点双连通分量 在模板中我们有一个割度数组 *Cutdeg*，割度表示删去该点后增加的联通分量数。割度不为 0 的点均为割点。

```

1 // 用于计算图的顶点双连通分量和割点（关节点）的类
2 class VertexBC
3 {
4     // 图的邻接表引用（const引用避免拷贝）
5     const std::vector<std::vector<int>>> &adj;
6     // DFS时间戳计数器，用于记录访问顺序
7     int cur = 0;
8
9     // DFS遍历函数，用于发现割点
10    // x: 当前遍历的顶点
11    // root: 当前连通分量的根节点
12    void dfs(int x, int root)
13    {
14        // 初始化当前顶点的发现时间和low值
15        dfn[x] = low[x] = cur++;
16
17        // 记录在DFS树中的子节点数量
18        int sonNum = 0;
19
20        // 遍历所有邻接顶点
21        for (int y : adj[x])
22        {
23            if (dfn[y] == -1) // 如果邻接顶点未被访问过

```

```

24     {
25         sonNum++; // 增加子节点计数
26         dfs(y, root); // 递归访问
27         // 回溯时更新low值
28         low[x] = std::min(low[x], low[y]);
29
30         // 割点判定条件:
31         // 1. 子节点的low值 >= 当前节点的dfn值
32         // 2. 当前节点不是根节点
33         if (low[y] >= dfn[x] && x != root)
34         {
35             cutDeg[x] += 1; // 增加当前节点的割点度数
36         }
37     }
38     else // 如果邻接顶点已被访问过 (可能是回边)
39     {
40         // 通过回边更新low值
41         low[x] = std::min(low[x], dfn[y]);
42     }
43 }
44
45 // 特殊处理根节点的情况
46 if (x == root)
47 {
48     // 根节点的割点度数 = 子树数量 - 1
49     cutDeg[x] = std::max(sonNum - 1, 0);
50 }
51 }
52
53 public:
54     // 存储每个顶点的发现时间 (DFS序)
55     std::vector<int> dfn;
56     // 存储每个顶点的low值 (能回溯到的最早祖先)
57     std::vector<int> low;
58     // 存储每个顶点作为割点时的度数 (删除后会增加的连通分量数)
59     std::vector<int> cutDeg;
60     // 记录图的连通分量数量
61     int componentNum = 0;
62
63     // 构造函数, 接收邻接表并计算割点信息
64     VertexBC(const std::vector<std::vector<int>> &adj)
65     : adj(adj), // 初始化邻接表引用
66       dfn(adj.size(), -1), // 初始化dfn数组 (-1表示未访问)
67       low(adj.size()), // 初始化low数组
68       cutDeg(adj.size()) // 初始化割点度数数组
69     {
70         int n = adj.size();
71
72         // 遍历所有顶点 (从1开始, 假设顶点编号为1~n-1)
73         for (int i = 1; i < n; i++)
74         {

```

```

75         if (dfn[i] == -1) // 遇到未访问的顶点
76         {
77             componentNum++; // 增加连通分量计数
78             dfs(i, i); // 从该顶点开始DFS
79         }
80     }
81 }
82 };

```

广义圆方树有十分多好的性质, 我们单独讨论

1 广义圆方树上没有两个邻点形状相同

2 两个圆点在圆方树上的路径, 以及经过的所有方点相邻圆点的集合, 等于原图中两点间所有简单路径的并集

3 根据 2 的结论, 如果我们圆点赋值 -1 , 方点赋值点双大小, 那么路径权值和等于简单路径并集大小 -2

广义圆方树

```

1  class ExRSTree
2  {
3      const std::vector<std::vector<int>> &adj; // 原始图的邻接表
4      int cur = 0; // DFS时间戳计数器
5      std::vector<int> stk; // 用于Tarjan算法的栈
6      int tt = 0; // 栈顶指针
7
8      // 在圆方树中添加边的辅助函数
9      void add(int u, int v)
10     {
11         tree[u].push_back(v);
12         tree[v].push_back(u);
13     }
14
15     // Tarjan算法实现, 用于构建圆方树
16     // x: 当前遍历的顶点
17     // root: 当前连通分量的根节点
18     void dfs(int x, int root)
19     {
20         dfn[x] = low[x] = cur++; // 初始化发现时间和low值
21         stk[++tt] = x; // 将当前节点压栈
22
23         int sonNum = 0; // DFS树中子节点计数
24
25         for (int y : adj[x]) // 遍历所有邻接节点
26         {
27             if (dfn[y] == -1) // 未访问过的节点
28             {
29                 sonNum += 1;
30                 dfs(y, root);
31                 low[x] = std::min(low[x], low[y]); // 更新low值
32
33                 // 发现割点条件

```

```

34     if (low[y] >= dfn[x])
35     {
36         ++cnt; // 新增一个方点
37
38         // 弹出栈中元素直到遇到y, 构建点双连通分量
39         while (stk[tt] != y)
40         {
41             Size[cnt]++; // 更新点双大小
42             add(cnt, stk[tt--]); // 将圆点连接到方点
43         }
44
45         // 处理剩余部分
46         add(cnt, stk[tt--]); // 连接y到方点
47         add(cnt, x); // 连接当前割点到方点
48         Size[cnt] += 2; // 更新点双大小 (加上x和y)
49     }
50 }
51 else
52 {
53     low[x] = std::min(low[x], dfn[y]); // 遇到已访问节点, 更新low值
54 }
55 }
56 }
57
58 // 第二次DFS, 计算子树大小和所属根节点
59 // u: 当前节点
60 // fa: 父节点
61 // r: 所属连通分量的根节点
62 void dfs1(int u, int fa, int r)
63 {
64     if (u <= n) // 如果是圆点 (原始节点)
65     {
66         siz[u] = 1; // 初始化大小为1
67     }
68
69     bel[u] = r; // 记录所属根节点
70
71     for (auto v : tree[u]) // 遍历圆方树中的邻接节点
72     {
73         if (v != fa) // 避免回溯父节点
74         {
75             dfs1(v, u, r);
76             siz[u] += siz[v]; // 累加子树大小
77         }
78     }
79 }
80
81 public:
82     // original graph
83     int n; // 原始图的顶点数
84     int cnt; // 圆方树中的节点计数器 (圆点+方点)

```

```

85     std::vector<int> dfn, low; // Tarjan算法用的发现时间和low值
86     std::vector<std::vector<int>> tree; // 圆方树
87     int componentNum = 0; // 连通分量数量
88     std::vector<int> root; // 所有圆方树的根节点
89     std::vector<int> bel; // 节点所属的圆方树根节点
90     std::vector<int> siz; // 子树大小
91     // 在圆方树中, 一个圆点的割度为度数-1
92     std::vector<int> Size; // 点双连通分量的大小
93
94     // 构造函数: 从原始图构建圆方树
95     ExRSTree(const std::vector<std::vector<int>> &adj)
96     : adj(adj),
97       n(adj.size() - 1), // 假设顶点编号从1开始
98       dfn(n + 1, -1), // 初始化dfn数组
99       low(n + 1), // 初始化low数组
100       tree(2 * n + 1), // 圆方树最多有2n个节点 (n圆点+n方点)
101       bel(2 * n + 1), // 所属根节点数组
102       siz(2 * n + 1) // 子树大小数组
103     {
104         Size.assign(2 * n + 1, 0); // 初始化点双大小数组
105         stk.assign(n + 1, 0); // 初始化Tarjan栈
106         cnt = n; // 方点从n+1开始编号
107
108         // 遍历所有顶点构建圆方树
109         for (int i = 1; i <= n; i++)
110         {
111             if (dfn[i] == -1) // 未访问过的顶点
112             {
113                 root.push_back(i); // 记录根节点
114                 componentNum += 1; // 增加连通分量计数
115                 dfs(i, i); // 第一次DFS构建圆方树
116                 dfs1(i, i, i); // 第二次DFS计算子树信息
117             }
118         }
119     }
120 };

```

边双连通分量

```

1 class EdgeBC
2 {
3     // 输入的无向图的邻接表 (引用, 避免拷贝)
4     const std::vector<std::vector<int>> &e;
5
6     // Tarjan算法用的栈
7     std::vector<int> stk; // stack
8     int tt = 0; // 栈顶指针
9     int cur = 0; // 全局时间戳, 用于dfn和low的标记
10
11     // Tarjan算法求边双连通分量 (EBCC)
12     void dfs(int x, int fa)

```

```

13 {
14     // 初始化当前节点的dfn和low值为当前时间戳
15     dfn[x] = low[x] = ++cur;
16     // 将当前节点压入栈
17     stk[++tt] = x;
18
19     // 遍历所有邻接节点
20     for (int y : e[x])
21     {
22         // 跳过父节点（处理无向图的重复边）
23         if (y == fa)
24         {
25             fa = ~fa; // 特殊处理，确保每条边只处理一次
26             continue;
27         }
28
29         // 如果邻接节点y未被访问过
30         if (!dfn[y])
31         {
32             dfs(y, x); // 递归访问y
33             low[x] = std::min(low[x], low[y]); // 更新low值
34         }
35         else
36         {
37             // y已被访问，更新low值
38             low[x] = std::min(low[x], dfn[y]);
39         }
40     }
41
42     // 如果dfn[x] == low[x]，说明x是一个边双连通分量的根
43     if (dfn[x] == low[x])
44     {
45         int y;
46         cntBlock += 1; // 边双连通分量计数
47         // 弹出栈中元素，直到x被弹出，这些元素构成一个边双连通分量
48         do
49         {
50             y = stk[tt--];
51             ebcc[cntBlock].push_back(y); // 将y加入当前边双连通分量
52             bel[y] = cntBlock; // 记录y所属的边双连通分量编号
53         } while (y != x);
54     }
55 }
56
57 public:
58     // 原始图的属性
59     std::vector<int> dfn; // 访问时间戳（DFS序）
60     std::vector<int> low; // 通过回边能到达的最小dfn值
61     std::vector<int> bel; // 每个节点所属的边双连通分量编号
62     std::vector<std::vector<int>> ebcc; // 存储每个边双连通分量的节点列表
63

```

```

64 // 缩点后的图（边双连通分量构成的树/森林）
65 std::vector<std::vector<int>> g; // 缩点后的邻接表
66 std::vector<int> siz; // 每个边双连通分量的大小
67 std::vector<int> val; // 每个边双连通分量的权值（可选）
68 int cntBlock = 0; // 边双连通分量的总数
69
70 // 构造函数：输入无向图的邻接表e（节点编号从1开始）
71 EdgeBC(const std::vector<std::vector<int>> &e) :
72     e(e), // 初始化图的引用
73     dfn(e.size()), // 初始化dfn数组
74     low(e.size()), // 初始化low数组
75     bel(e.size()), // 初始化bel数组
76     stk(e.size()), // 初始化栈
77     ebcc(e.size()) // 初始化ebcc数组
78 {
79     int n = e.size() - 1; // 节点数（假设节点编号为1~n）
80
81     // 遍历所有节点，对未访问的节点调用Tarjan算法
82     for (int i = 1; i <= n; i++)
83     {
84         if (!dfn[i])
85         {
86             dfs(i, -1); // 从i开始DFS，初始父节点为-1
87         }
88     }
89
90     // 调整ebcc的大小为实际边双连通分量的数量
91     ebcc.resize(cntBlock + 1);
92
93     // 初始化缩点后的图
94     g.resize(cntBlock + 1);
95     siz.resize(cntBlock + 1);
96     // val.resize(cntBlock + 1); // 如果需要权值，可以取消注释
97
98     // 遍历所有节点，构建缩点后的图
99     for (int x = 1; x <= n; x++)
100     {
101         siz[bel[x]]++; // 统计每个边双连通分量的大小
102         // val[bel[x]] += a[x]; // 如果需要权值，可以累加
103
104         // 遍历x的邻接节点y
105         for (int y : e[x])
106         {
107             // 如果x和y属于不同的边双连通分量，则在缩点后的图中添加边
108             if (bel[x] != bel[y])
109             {
110                 g[bel[x]].push_back(bel[y]);
111             }
112         }
113     }
114 }

```

115 };

构造边双连通图 即给你一个无向图，加最少的边使其成为边双连通图。首先我们 *EBCC* 缩点，这样一定会形成一颗森林，

我们假如说这是一棵树的话，记叶子结点数量为 *leaf*，那么答案为 $(leaf + 1)/2$ ，构造方法是，我们把 *dfs* 得到的叶子结点按遍历顺序放到数组中，然后不断首尾匹配即可。

那么如果是森林，我们考虑先把它连成一棵树使得该树的叶子数量最少，在不考虑孤立点的情况下，每棵树至少会有两个叶子，那么我们把这两个叶子找出来，把这些森林按顺序首尾链接即可，注意需要特判孤立点的情况。

假如有 C 个联通块，那么这里消耗边数 $C - 1$

3.8 欧拉图

欧拉路径：经过每条边恰好一次的路径

欧拉回路：经过每条边恰好一次并回到起点的路径

欧拉图：有欧拉回路的图被称作欧拉图

半欧拉图：没有欧拉回路但有欧拉路径的图被称作半欧拉图

欧拉图判断条件：

对于连通图 G ，如下三个性质等价：

1: G 是欧拉图。

2: 对于无向图， G 中所有顶点的度数都是偶数；对于有向图，每个点入度等于出度。

3: G 可以被分解成若干条不共边的回路。

半欧拉图判断条件：

对于有向图：存在一个点 s 使得 $\deg^+(s) = \deg^-(s) + 1$ ，存在一个点 t 使得 $\deg^-(t) = \deg^+(t) + 1$ ，其余所有点 v 满足 $\deg^+(v) = \deg^-(v)$ 。

对于无向图：恰好有两个点的度为奇数。

显然对于有向图和无向图，两个特殊点都是欧拉通路的两端点。

3.8.1 无向图

注意代码求出的是字典序最小的欧拉路。

```
1 class UndirectedEulerRoad
2 {
3
4 public:
5     UndirectedEulerRoad(int _n) : n(_n), g(_n + 1)
6     {
7         cnt.assign(n + 1, 0);
8         deg.assign(n + 1, 0);
9         revTot.assign(n + 1, 0);
10    }
11
```

```
12 void addEdge(int u, int v)
13 {
14     if (u != v)
15     {
16         g[u].push_back({v, 1, 0});
17         g[v].push_back({u, 1, 0});
18     }
19     else
20     {
21         g[u].push_back({v, 1, 0});
22     }
23     deg[v]++;
24     deg[u]++;
25 }
26 void hierholzer(int x)
27 {
28
29     for (int &i = cnt[x]; i < g[x].size(); i++)
30     {
31         auto &e = g[x][i];
32         if (e.exist)
33         {
34             e.exist = false;
35             if (e.v != x)
36             {
37                 g[e.v][e.revCnt].exist = false;
38             }
39             ++i;
40             hierholzer(e.v);
41         }
42         else
43             ++i;
44     }
45     road.push_back(x);
46 }
47 int work()
48 {
49     for (int i = 1; i <= n; ++i)
50     {
51         std::sort(begin(g[i]), end(g[i]));
52     }
53     for (int i = 1; i <= n; ++i)
54     {
55         for (int j = 0; j < g[i].size(); ++j)
56         {
57             g[i][j].revCnt = revTot[g[i][j].v]++;
58         }
59     }
60     int start = 0;
61     int tot = 0;
62     for (int i = n; i >= 1; --i)
```

```

63 {
64     tot += (deg[i] & 1);
65     if (deg[i] & 1)
66     {
67         start = i;
68     }
69     else if (deg[i] and deg[start] {
70         start = i;
71     }
72 }
73 if (tot != 2 and tot != 0)
74 {
75     return 0;
76 }
77 hierholzer(start);
78 return (tot ? 1 : 2);
79 }
80 auto getRoad()
81 {
82     std::reverse(begin(road), end(road));
83     return road;
84 }
85
86 private:
87 struct Edge
88 {
89     int v; // 到达的点是谁
90     bool exist; // 该边是否被用过
91     int revCnt; // 反边在对应vector中的编号
92     bool operator<(const Edge &e) const
93     {
94         return v < e.v;
95     }
96 };
97 int n;
98 std::vector<std::vector<Edge>> g; // 存图
99 std::vector<int> cnt; // 遍历到哪条边了
100 std::vector<int> deg;
101 std::vector<int> road;
102 std::vector<int> revTot;
103 };

```

3.8.2 有向图

注意代码求出的是字典序最小的欧拉路。

```

1 class DirectedEulerRoad
2 {
3
4

```

```

5 public:
6     DirectedEulerRoad(int _n) : n(_n), g(_n + 1)
7     {
8         cnt.assign(n + 1, 0);
9         in.assign(n + 1, 0), out.assign(n + 1, 0);
10    }
11    void addEdge(int u, int v)
12    {
13        g[u].push_back({v, 1});
14        in[v]++;
15        out[u]++;
16    }
17    void hierholzer(int x)
18    {
19        for (int &i = cnt[x]; i < g[x].size(); )
20        {
21            auto &e = g[x][i];
22            if (e.exist)
23            {
24                e.exist = false;
25                ++i;
26                hierholzer(e.v);
27            }
28            else
29                ++i;
30        }
31        road.push_back(x);
32    }
33    int work()
34    {
35        for (int i = 1; i <= n; ++i)
36        {
37            std::sort(begin(g[i]), end(g[i]));
38        }
39        int start = 0;
40        int tot = 0, tot1 = 0, tot2 = 0;
41        for (int i = n; i >= 1; --i)
42        {
43            tot += (in[i] != out[i]);
44            tot1 += (in[i] == out[i] + 1);
45            tot2 += (in[i] == out[i] - 1);
46            if (out[i] != in[i])
47            {
48                start = i;
49            }
50            else if (out[start] == in[start] and in[i] == out[i] and in[i])
51            {
52                start = i;
53            }
54        }
55        if (tot != 2 and tot != 0)

```

```

56     {
57         return 0;
58     }
59     if (tot1 > 1 or tot2 > 1)
60     {
61         return 0;
62     }
63     hierholzer(start);
64     return (tot ? 1 : 2);
65 }
66 auto getRoad()
67 {
68     std::reverse(begin(road), end(road));
69     return road;
70 }
71
72 private:
73     struct Edge
74     {
75         int v; // 到达的点是誰
76         bool exist; // 该边是否被用过
77         bool operator<(const Edge &e) const
78         {
79             return v < e.v;
80         }
81     };
82     int n;
83     std::vector<std::vector<Edge>> g; // 存图
84     std::vector<int> cnt; // 遍历到哪条边了
85     std::vector<int> in, out;
86     std::vector<int> road;
87 };

```

3.8.3 混合图

混合图即既有有向边又有无向边，之所以不能转化为有向图，是因为在该问题下，无向边被视作可定向的有向边

判断是否有欧拉回路：

首先给每条无向边任意定向，设 $deg[x] = dout[x] - din[x]$ 。

若存在 $deg[x]$ 为奇数或者图不联通，直接无解。

否则我们建立源点 S 和汇点 T

对于一个点 x ，如果 $deg[x] > 0$ ，那么 S 向 x 流量为 $deg[x]/2$ 的边，否则 x 向 T 连流量为 $-deg[x]/2$ 的边，对于定向的无向边 $x \rightarrow y$ ，连边 $(x, y, 1)$ ，跑最大流算法

如果与 S 和 T 相连的所有边均满流，那么说明有解。

同时该条件等价于 $maxFlow = \sum max((din - dout)/2, 0)$ ，即最大流等于所有入度大于出度节点对应的入度减出度除以二的和

构造方案：根据网络流的结果调整无向边的方向。将网络流中加入的定向边流量为 1 的边翻转

3.9 拉姆齐定理

有 X 个人，任意两两之间认识或不认识，我们定义 $R(n, m)$ 为 保证“存在一个 m 人的朋友小团体（红色 K_m （红色边表示朋友））”或“一个 n 人的陌生人小团体（蓝色 K_n （蓝色边表示陌生））”所需要的最小派对人数

朋友小团体指该团体内两两互为朋友，陌生人即该团体内两两均不认识。

$R(3, 3) = 6, R(4, 4) = 18$

3.10 竞赛图

任意两点之间有且仅有一条有向边的图为竞赛图，由于连边方式类似于任意两人都打一场单循环赛，得名竞赛图，特别的，我们一般认为有向边的起点是获胜者，因此每个点的出度为对应选手的得分。

得分序列：把每个点的出度拿出来升序排序后的序列。

兰道定理 对于任意一个单调不减的序列，该序列可以为竞赛图的得分序列当且仅当：

1 对任意的 $i \leq n$ ，前 i 个得分和一定不小于 $i * (i - 1) / 2$

2 总和必须等于 $n * (n - 1) / 2$

若满足兰道定理，我们一定可以构造出满足条件的竞赛图。

构造方法：我们不妨设每个选手有一个需求值 $a[i] - b[i]$ ，其中 a 是需要赢多少场， b 是已经赢多少场，我们每次让需求最大的选手打赢需求最小的选手即可。

3.11 建图

3.11.1 线段树优化建图

一定要注意函数参数的位置！

in 数组用于单点连多点， out 数组用于多点连单点

对于多点连多点的情况，我们新开节点，然后分为多连一 + 一连多即可

对于单点连单点的情况，我们直接连就可以了

```

1 class SegmentTreeGraph
2 {
3     #define lc u << 1
4     #define rc u << 1 | 1
5
6 public:
7     int tot;
8     int n;
9     std::vector<std::vector<std::pair<int, int>>> g;
10    SegmentTreeGraph(int n) : n(n), tot(n), in((n << 2) + 1), out((n << 2) + 1), g((n
        << 2) + 1) // 传入实际的点数
11    {
12        auto build = [&](auto build, int u, int l, int r) -> void
13        {
14            if (l == r)

```



```

15     {
16         in[u] = l;
17         out[u] = l;
18         return;
19     }
20     int mid = l + r >> 1;
21     build(build, lc, l, mid);
22     build(build, rc, mid + 1, r);
23     in[u] = ++tot;
24     out[u] = ++tot;
25     g[in[u]].emplace_back(in[lc], 0);
26     g[in[u]].emplace_back(in[rc], 0);
27     g[out[lc]].emplace_back(out[u], 0);
28     g[out[rc]].emplace_back(out[u], 0);
29 };
30 build(build, 1, 1, n);
31 }
32 void singleToRange(int u, int l, int r, int w)
33 {
34     singleToRange(1, 1, n, u, l, r, w);
35 }
36 void rangeToSingle(int u, int l, int r, int w)
37 {
38     rangeToSingle(1, 1, n, u, l, r, w);
39 }
40 void singleToSingle(int u, int v, int w)
41 {
42     g[u].emplace_back(v, w);
43 }
44 void rangeToRange(int l, int r, int x, int y, int w)
45 {
46     ++tot;
47     if (g.size() <= tot)
48     {
49         g.resize(tot + 1);
50     }
51     rangeToSingle(tot, l, r, 0);
52     singleToRange(tot, x, y, w);
53 }
54
55 private:
56 std::vector<int> in, out;
57 void singleToRange(int u, int l, int r, int st, int x, int y, int w) // st为起点
    向[x,y]连边 不要边权就随便传
58 {
59     if (x <= l and y >= r)
60     {
61         g[st].emplace_back(in[u], w);
62         return;
63     }
64     int mid = l + r >> 1;

```

```

65     if (x <= mid)
66     {
67         singleToRange(lc, l, mid, st, x, y, w);
68     }
69     if (y > mid)
70     {
71         singleToRange(rc, mid + 1, r, st, x, y, w);
72     }
73 }
74 void rangeToSingle(int u, int l, int r, int ed, int x, int y, int w) //[x,y]向终
    点ed连边
75 {
76     if (x <= l and y >= r)
77     {
78         g[out[u]].emplace_back(ed, w);
79         return;
80     }
81     int mid = l + r >> 1;
82     if (x <= mid)
83     {
84         rangeToSingle(lc, l, mid, ed, x, y, w);
85     }
86     if (y > mid)
87     {
88         rangeToSingle(rc, mid + 1, r, ed, x, y, w);
89     }
90 }
91 };

```

3.11.2 前后缀优化建图

```

1 struct BuildEdge
2 {
3     int n;
4     int tot;
5     std::vector<std::vector<std::pair<int, int>>> g;
6     BuildEdge(int n) : n(n), g(5 * n + 1), tot(5 * n)
7     {
8         for (int i = 2; i <= n; ++i)
9         {
10             g[n + i].emplace_back(n + i - 1, 0); // 连前缀
11             g[2 * n + i - 1].emplace_back(2 * n + i, 0); // 连后缀
12             g[3 * n + i].emplace_back(3 * n + i - 1, 0);
13             g[4 * n + i - 1].emplace_back(4 * n + i, 0);
14         }
15         for (int i = 1; i <= n; ++i)
16         {
17             g[n + i].emplace_back(i, 0);
18             g[2 * n + i].emplace_back(i, 0);
19             g[i].emplace_back(3 * n + i, 0);

```

```

20     g[i].emplace_back(4 * n + i, 0);
21 }
22 }
23 void norm(int tot)
24 {
25     if (g.size() <= tot)
26     {
27         g.resize(tot + 1);
28     }
29 }
30 void linkPre(int x, int y, int w) // x->[1,y]
31 {
32     g[x].emplace_back(n + y, w);
33 }
34 void linkSuf(int x, int y, int w) // x->[y,n]
35 {
36     g[x].emplace_back(2 * n + y, w);
37 }
38 void preLink(int x, int y, int w) //[1,x]->y
39 {
40     g[3 * n + x].emplace_back(y, w);
41 }
42 void sufLink(int x, int y, int w) //[x,n]->y
43 {
44     g[4 * n + x].emplace_back(y, w);
45 }
46 void preToPre(int x, int y, int w) //[1,x]->[1,y]
47 {
48     ++tot;
49     norm(tot);
50     preLink(x, tot, w);
51     linkPre(tot, y, 0);
52 }
53 void preToSuf(int x, int y, int w) //[1,x]->[y,n]
54 {
55     ++tot;
56     norm(tot);
57     preLink(x, tot, w);
58     linkSuf(tot, y, 0);
59 }
60 void sufToPre(int x, int y, int w) //[x,n]->[1,y]
61 {
62     ++tot;
63     norm(tot);
64     sufLink(tot, x, w);
65     linkPre(tot, y, 0);
66 }
67 void sufToSuf(int x, int y, int w) //[x,n]->[y,n]
68 {
69     ++tot;
70     norm(tot);

```

```

71     sufLink(x, tot, w);
72     linkSuf(tot, y, 0);
73 }
74 };

```

3.11.3 ST 表优化建图

我们考虑一种操作即 $[a, b]$ 向 $[s, t]$ 连边，保证两区间长度相等，连法是一一对应位置连边，我们考虑像 ST 表一样建出 $n \log n$ 个区间作为并查集所有的节点，然后可以把 $merge$ 次数转为 $\log n$ 个并查集节点相连，在所有 $merge$ 操作结束以后，由于每个区间可以拆成对应的左右子区间，把祖先节点对应的 $root$ 标记下传给子区间即可。

具体的说，我们按层考虑，对于同一层内所有 $root$ 相同的节点，我们按顺序把它们的左儿子 $merge$ ，再按顺序把他们的右儿子 $merge$ 即可。这样下传到最后一层，考虑并查集的连通性即可。

正向连正向 (比如 $[1, 2]$ 连 $[8, 9]$)

```

1
2 struct BuildGraph
3 {
4     int n, log;
5     DSU *dsu;
6     std::vector<std::vector<int>> f;
7     std::vector<int> lc, rc;
8     BuildGraph(int n) : n(n)
9     {
10         log = std::__lg(n) + 1;
11         f.assign(log, std::vector<int>(n + 1, 0));
12         for (int i = 1; i <= n; ++i)
13         {
14             f[0][i] = i;
15         }
16         int idx = n;
17         for (int i = log - 1; i >= 1; --i)
18         {
19             for (int j = 1; j + (1 << i) - 1 <= n; ++j)
20             {
21                 f[i][j] = ++idx;
22             }
23         }
24         lc.assign(idx + 1, -1);
25         rc.assign(idx + 1, -1);
26         dsu = new DSU(idx + 1);
27     }
28     void merge(int l1, int r1, int l2, int r2)
29     {
30         int len = r1 - l1 + 1;
31         for (int i = log - 1; i >= 0; --i)
32         {
33             if (len >= (1 << i))
34                 {

```

```

35     dsu->merge(f[i][l1], f[i][l2]);
36     len -= (1 << i);
37     l1 += (1 << i);
38     l2 += (1 << i);
39 }
40 }
41 }
42 void pushall()
43 {
44     for (int i = log - 1; i >= 1; --i)
45     {
46         for (int j = 1; j + (1 << i) - 1 <= n; ++j)
47         {
48             int root = dsu->root(f[i][j]);
49             if (lc[root] != -1)
50             {
51                 dsu->merge(lc[root], f[i - 1][j]);
52                 dsu->merge(rc[root], f[i - 1][j + (1 << i - 1)]);
53             }
54             else
55             {
56                 lc[root] = f[i - 1][j];
57                 rc[root] = f[i - 1][j + (1 << i - 1)];
58             }
59         }
60         for (int j = 1; j + (1 << i) - 1 <= n; ++j)
61         {
62             int root = dsu->root(f[i][j]);
63             lc[root] = rc[root] = -1;
64         }
65     }
66 }
67 std::vector<int> calc()
68 {
69     pushall();
70     std::vector<int> ans(n + 1);
71     for (int i = 1; i <= n; ++i)
72     {
73         ans[i] = dsu->root(i);
74     }
75     return ans;
76 } //
77 };

```

正向连反向 (比如 [1, 2] 连 [9, 8])

```

1 struct BuildGraph
2 {
3     int n, log;
4     DSU *dsu;
5     std::vector<std::vector<int>> f, g;
6     std::vector<int> lc, rc;

```

```

7 BuildGraph(int n) : n(n)
8 {
9     log = std::__lg(n) + 1;
10    f.assign(log, std::vector<int>(n + 1, 0));
11    g.assign(log, std::vector<int>(n + 1, 0));
12
13    for (int i = 1; i <= n; ++i)
14    {
15        f[0][i] = i;
16        g[0][i] = n - i + 1;
17    }
18    int idx = n;
19    for (int i = log - 1; i >= 1; --i)
20    {
21        for (int j = 1; j + (1 << i) - 1 <= n; ++j)
22        {
23            f[i][j] = ++idx;
24            g[i][j] = ++idx;
25        }
26    }
27    lc.assign(idx + 1, -1);
28    rc.assign(idx + 1, -1);
29    dsu = new DSU(idx + 1);
30 }
31 void merge(int l1, int r1, int l2, int r2)
32 // [l2, r2] 反向
33 {
34     l2 = n - l2 + 1;
35     r2 = n - r2 + 1;
36     std::swap(l2, r2);
37     int len = r1 - l1 + 1;
38     for (int i = log - 1; i >= 0; --i)
39     {
40         if (len >= (1 << i))
41         {
42             dsu->merge(f[i][l1], g[i][l2]);
43             len -= (1 << i);
44             l1 += (1 << i);
45             l2 += (1 << i);
46         }
47     }
48 }
49 void pushall()
50 {
51     for (int i = log - 1; i >= 1; --i)
52     {
53         for (int j = 1; j + (1 << i) - 1 <= n; ++j)
54         {
55             int root = dsu->root(f[i][j]);
56             if (lc[root] != -1)
57             {

```

```

58         dsu->merge(lc[root], f[i - 1][j]);
59         dsu->merge(rc[root], f[i - 1][j + (1 << i - 1)]);
60     }
61     else
62     {
63         lc[root] = f[i - 1][j];
64         rc[root] = f[i - 1][j + (1 << i - 1)];
65     }
66     root = dsu->root(g[i][j]);
67     if (lc[root] != -1)
68     {
69         dsu->merge(lc[root], g[i - 1][j]);
70         dsu->merge(rc[root], g[i - 1][j + (1 << i - 1)]);
71     }
72     else
73     {
74         lc[root] = g[i - 1][j];
75         rc[root] = g[i - 1][j + (1 << i - 1)];
76     }
77 }
78 for (int j = 1; j + (1 << i) - 1 <= n; ++j)
79 {
80     int root = dsu->root(f[i][j]);
81     lc[root] = rc[root] = -1;
82     root = dsu->root(g[i][j]);
83     lc[root] = rc[root] = -1;
84 }
85 }
86 }
87 std::vector<int> calc()
88 {
89     pushall();
90     std::vector<int> ans(n + 1);
91     for (int i = 1; i <= n; ++i)
92     {
93         ans[i] = dsu->root(i);
94         //std::cerr << ans[i] << " \n"[i == n];
95     }
96     return ans;
97 } //
98 };

```

3.11.4 网格建图

```

1 template <class Element, class Vector, class AdjEle>
2 //元素类型(int char) 一行的类型(string vector) 第三个默认为int
3 class GridPre
4 {
5     const int h, w, n;
6     static constexpr int D = 4;

```

```

7     static constexpr int dx[D] = {0, 0, -1, 1};
8     static constexpr int dy[D] = {-1, 1, 0, 0};
9
10 public:
11     std::vector<std::vector<AdjEle>> g;
12     // std::vector<Element> a;
13     std::vector<int> reachableVertexSet;
14
15     GridPre(const std::vector<Vector> &grid, const std::set<Element> &reachableSet)
16         : h{(int)grid.size() - 1}, w{(int)grid[1].size() - 1}, n{h * w + 1}, g{n} //,
17           a(n)
18     {
19         reachableVertexSet.reserve(n);
20
21         auto ok = [&](int x, int y)
22         {
23             return 1 <= x and x <= h and 1 <= y and y <= w and reachableSet.count(
24                 grid[x][y]);
25         };
26
27         for (int i = 1; i <= h; i++)
28         {
29             for (int j = 1; j <= w; j++)
30             {
31                 if (reachableSet.count(grid[i][j]) == 0)
32                 {
33                     continue;
34                 }
35
36                 int x = (i - 1) * w + j;
37                 // a[x] = grid[i][j];
38
39                 reachableVertexSet.push_back(x);
40
41                 for (int k = 0; k < D; k++)
42                 {
43                     int next_i = i + dx[k];
44                     int next_j = j + dy[k];
45
46                     if (ok(next_i, next_j))
47                     {
48                         g[x].emplace_back((next_i - 1) * w + next_j);
49                     }
50                 }
51             }
52         }
53     }
54 };

```

3.12 网络流

网络流是一种建好图以后，内置反悔机制的一种贪心策略，不需要我们人为去寻找贪心的反悔策略

容量: 当前边最多经过的流量记作 $C(u, v)$

流量: 当前边实际经过的流量记作 $F(u, v)$

残量: $C(u, v) - F(u, v)$

残量网络: 把残量为 0 的边删去，剩下的图称作残量网络，对应边的边权也变为残量

增广路: 与二分图中增广路定义不同，增广路指残量网络上源点到汇点的一条简单路径

割: 割是一个边的集合，一个边集是割当且仅当去掉这些边后网络不在流通，即最大流为 0

最大流最小割定理: 一个网络的最大流等于其最小割

我们根据定义发现，只要存在增广路，流量就能不断增大，因此求最大流是一个求增广路的过程，并且这条增广路我们遵循能流满就流满的贪心策略

我们选择一条增广路，假设流量为 f ，我们正向边容量 $-f$ ，反向边容量 $+f$ ，这就是最开始我们说的内置反悔机制，也就是说我们可以通过反边流量进行反悔

由于 *EK* 算法不优于 *dinic*，我们只介绍 *dinic* 算法

dinic 是多路增广 + 当前弧优化的增广路算法

我们通过 *bfs* 给图分层，分层后多路增广，也就是同时累加多条增广路的流量

当前弧优化是指，流量已经流满的边无用，我们没必要对每个点每次都从头开始遍历，而是记录一个当前弧，直接从当前弧开始遍历

贪心的正确性全部基于退流反边对流量不断调整直至最优状态

3.12.1 拆点

注意到在流网络中，我们对于限制都是加在边上的。那么对于点的限制我们往往考虑拆点。具体的说，我们把一个点 U 拆成对应的 In_u 和 Out_u ，然后连边 $In_u -> Out_u$ ，边的流量限制即为该点的流量限制。

3.12.2 最大流

```
1 template <class T>
2 struct Flow
3 {
4     const T inf = std::numeric_limits<T>::max() / 2;
5     const int n;
6
7     std::vector<std::pair<int, T>> e;
8     std::vector<std::vector<int>> g;
9     std::vector<int> cur, dep;
10
11     Flow(int n) : n(n), g(n + 1) {}
12
13     bool bfs(int s, int t)
14     {
15         dep.assign(n + 1, -1);
```

```
16         std::queue<int> q;
17         dep[s] = 0;
18
19         q.push(s);
20         while (!q.empty())
21         {
22             const int u = q.front();
23             q.pop();
24
25             for (int i : g[u])
26             {
27                 auto [v, c] = e[i];
28
29                 if (c > 0 and dep[v] == -1)
30                 {
31                     dep[v] = dep[u] + 1;
32                     if (v == t)
33                     {
34                         return true;
35                     }
36                     q.push(v);
37                 }
38             }
39         }
40
41         return false;
42     }
43
44     T dfs(int u, int t, T f)
45     {
46         if (u == t)
47         {
48             return f;
49         }
50         T res = f;
51         for (int &i = cur[u]; i < g[u].size(); i++)
52         {
53             const int j = g[u][i];
54             auto [v, c] = e[j];
55
56             if (c > 0 and dep[v] == dep[u] + 1)
57             {
58                 T out = dfs(v, t, std::min(res, c));
59                 e[j].second -= out;
60                 e[j ^ 1].second += out;
61
62                 res -= out;
63                 if (res == 0)
64                 {
65                     return f;
66                 }
67             }
68         }
69     }
70 }
```

```

67     }
68 }
69 return f - res;
70 }
71
72 void add(int u, int v, T c)
73 {
74     g[u].push_back(e.size());
75     e.emplace_back(v, c);
76     g[v].push_back(e.size());
77     e.emplace_back(u, 0);
78 }
79
80 T work(int s, int t)
81 {
82     T ans = 0;
83     while (bfs(s, t))
84     {
85         cur.assign(n + 1, 0);
86         ans += dfs(s, t, inf);
87     }
88     return ans;
89 }
90 std::vector<std::tuple<int, int, T>> getFlowEdges()
91 {
92     std::vector<std::tuple<int, int, T>> edge;
93     for (int i = 0; i < e.size(); i += 2) // 只检查正向边
94     {
95         int u = e[i ^ 1].first; // 反向边的起点是正向边的终点
96         int v = e[i].first;
97         T flow = e[i ^ 1].second; // 反向边的容量就是正向边的流量
98         edge.push_back({u, v, flow});
99     }
100     return edge;
101 }
102 };

```

3.12.3 最小费用最大流

Dinic(dijkstra)

```

1 template <class T>
2 struct MinCostFlow
3 {
4     struct _Edge
5     {
6         int to;
7         T cap;
8         T cost;
9         _Edge(int to_, T cap_, T cost_) : to(to_), cap(cap_), cost(cost_) {}
10    };
11    int n;

```

```

12    std::vector<_Edge> e;
13    std::vector<std::vector<int>> g;
14    std::vector<T> h, dis;
15    std::vector<int> pre;
16    bool dijkstra(int s, int t)
17    {
18        dis.assign(n + 1, std::numeric_limits<T>::max());
19        pre.assign(n + 1, -1);
20        std::priority_queue<std::pair<T, int>, std::vector<std::pair<T, int>>, std::greater<std::pair<T, int>>> que;
21        dis[s] = 0;
22        que.emplace(0, s);
23        while (!que.empty())
24        {
25            T d = que.top().first;
26            int u = que.top().second;
27            que.pop();
28            if (dis[u] != d)
29            {
30                continue;
31            }
32            for (int i : g[u])
33            {
34                int v = e[i].to;
35                T cap = e[i].cap;
36                T cost = e[i].cost;
37                if (cap > 0 && dis[v] > d + h[u] - h[v] + cost)
38                {
39                    dis[v] = d + h[u] - h[v] + cost;
40                    pre[v] = i;
41                    que.emplace(dis[v], v);
42                }
43            }
44        }
45        return dis[t] != std::numeric_limits<T>::max();
46    }
47    MinCostFlow() {}
48    MinCostFlow(int n_)
49    {
50        init(n_);
51    }
52    void init(int n_)
53    {
54        n = n_;
55        e.clear();
56        g.assign(n + 1, {});
57    }
58    void add(int u, int v, T cap, T cost)
59    {
60        g[u].push_back(e.size());
61        e.emplace_back(v, cap, cost);

```

```

62     g[v].push_back(e.size());
63     e.emplace_back(u, 0, -cost);
64 }
65 std::pair<T, T> work(int s, int t)
66 {
67     T flow = 0;
68     T cost = 0;
69     h.assign(n + 1, 0);
70     while (dijkstra(s, t))
71     {
72         for (int i = 0; i <= n; ++i)
73         {
74             h[i] += dis[i];
75         }
76         T aug = std::numeric_limits<int>::max();
77         for (int i = t; i != s; i = e[pre[i] ^ 1].to)
78         {
79             aug = std::min(aug, e[pre[i]].cap);
80         }
81         for (int i = t; i != s; i = e[pre[i] ^ 1].to)
82         {
83             e[pre[i]].cap -= aug;
84             e[pre[i] ^ 1].cap += aug;
85         }
86         flow += aug;
87         cost += aug * h[t];
88     }
89     return std::make_pair(flow, cost);
90 }
91 struct Edge
92 {
93     int from;
94     int to;
95     T cap;
96     T cost;
97     T flow;
98 };
99 std::vector<Edge> edges()
100 {
101     std::vector<Edge> a;
102     for (int i = 0; i < e.size(); i += 2)
103     {
104         Edge x;
105         x.from = e[i + 1].to;
106         x.to = e[i].to;
107         x.cap = e[i].cap + e[i + 1].cap;
108         x.cost = e[i].cost;
109         x.flow = e[i + 1].cap;
110         a.push_back(x);
111     }
112     return a;

```

```

113 }
114
115 void printFlowEdges()
116 {
117     std::cout << "Flow edges (from -> to: flow / cost):" << std::endl;
118     auto allEdges = edges();
119     for (const auto &edge : allEdges)
120     {
121         if (edge.flow > 0)
122         {
123             std::cout << edge.from << " -> " << edge.to << ": "
124                 << edge.flow << " / " << edge.cost << std::endl;
125         }
126     }
127 }
128 };

```

Dinic(SPFA)

```

1 template <class T>
2 struct MinCostFlow
3 {
4     struct _Edge
5     {
6         int to;
7         T cap;
8         T cost;
9         _Edge(int to_, T cap_, T cost_) : to(to_), cap(cap_), cost(cost_) {}
10    };
11
12    int n;
13    std::vector<_Edge> e;
14    std::vector<std::vector<int>>> g;
15    std::vector<T> dis;
16    std::vector<int> pre;
17
18    MinCostFlow() {}
19    MinCostFlow(int n_) { init(n_); }
20
21    void init(int n_)
22    {
23        n = n_;
24        e.clear();
25        g.assign(n + 1, {});
26    }
27
28    void add(int u, int v, T cap, T cost)
29    {
30        g[u].push_back((int)e.size());
31        e.emplace_back(v, cap, cost);
32        g[v].push_back((int)e.size());

```

```

33     e.emplace_back(u, 0, -cost);
34 }
35
36 /// 用 SPFA 找最短费用增广路
37 bool spfa(int s, int t)
38 {
39     const T INF = std::numeric_limits<T>::max() / 4;
40     dis.assign(n + 1, INF);
41     pre.assign(n + 1, -1);
42     std::vector<bool> inq(n + 1, false);
43
44     std::queue<int> q;
45     dis[s] = 0;
46     q.push(s);
47     inq[s] = true;
48
49     while (!q.empty())
50     {
51         int u = q.front();
52         q.pop();
53         inq[u] = false;
54
55         for (int i : g[u])
56         {
57             int v = e[i].to;
58             if (e[i].cap > 0 && dis[v] > dis[u] + e[i].cost)
59             {
60                 dis[v] = dis[u] + e[i].cost;
61                 pre[v] = i;
62                 if (!inq[v])
63                 {
64                     q.push(v);
65                     inq[v] = true;
66                 }
67             }
68         }
69     }
70     return dis[t] != INF;
71 }
72
73 /// 主过程
74 std::pair<T, T> work(int s, int t)
75 {
76     T flow = 0, cost = 0;
77     const T INF = std::numeric_limits<T>::max() / 4;
78
79     while (spfa(s, t))
80     {
81         // 找到最小残量 (可增广的流量)
82         T aug = INF;
83         for (int i = t; i != s; i = e[pre[i] ^ 1].to)

```

```

84     {
85         aug = std::min(aug, e[pre[i]].cap);
86     }
87     // 更新残量网络
88     for (int i = t; i != s; i = e[pre[i] ^ 1].to)
89     {
90         e[pre[i]].cap -= aug;
91         e[pre[i] ^ 1].cap += aug;
92     }
93     flow += aug;
94     cost += aug * dis[t];
95 }
96 return {flow, cost};
97 }
98
99 struct Edge
100 {
101     int from;
102     int to;
103     T cap;
104     T cost;
105     T flow;
106 };
107
108 std::vector<Edge> edges()
109 {
110     std::vector<Edge> a;
111     for (int i = 0; i < (int)e.size(); i += 2)
112     {
113         Edge x;
114         x.from = e[i ^ 1].to;
115         x.to = e[i].to;
116         x.cap = e[i].cap + e[i ^ 1].cap;
117         x.cost = e[i].cost;
118         x.flow = e[i ^ 1].cap;
119         a.push_back(x);
120     }
121     return a;
122 }
123
124 void printFlowEdges()
125 {
126     std::cout << "Flow edges (from -> to: flow / cost):" << std::endl;
127     auto allEdges = edges();
128     for (const auto &edge : allEdges)
129     {
130         if (edge.flow > 0)
131         {
132             std::cout << edge.from << " -> " << edge.to << ": "
133                 << edge.flow << " / " << edge.cost << std::endl;
134         }

```



```

135     }
136 }
137 };

EK(SPFA)
1 template <class T>
2 struct MinCostFlow
3 {
4     // ===== 内部边结构: 存储邻接表索引指向的真实边 =====
5     struct _Edge
6     {
7         int to;      // 终点
8         T cap;       // 残量容量
9         T cost;      // 单位费用
10        _Edge(int to_, T cap_, T cost_) : to(to_), cap(cap_), cost(cost_) {}
11    };
12
13    int n = 0;          // 点数 (默认按 1..n 使用)
14    std::vector<_Edge> e; // 边表 (正反向成对存储)
15    std::vector<std::vector<int>>> g; // 邻接表: 存储边表下标
16    std::vector<T> dis;    // 最短费用距离
17    std::vector<int> pre;  // 记录最短路上的边的编号 (边表下标)
18
19    MinCostFlow() {}
20    MinCostFlow(int n_) { init(n_); }
21
22    // ===== 初始化 =====
23    void init(int n_)
24    {
25        n = n_;
26        e.clear();
27        g.assign(n + 1, {});
28    }
29
30    // ===== 加边: 自动加反向边 =====
31    void add(int u, int v, T cap, T cost)
32    {
33        g[u].push_back((int)e.size());
34        e.emplace_back(v, cap, cost);
35        g[v].push_back((int)e.size());
36        e.emplace_back(u, 0, -cost);
37    }
38
39    // ===== SPFA: 在残量网络上找费用最短增广路 (可含负边费用) =====
40    bool spfa(int s, int t)
41    {
42        const T INF = std::numeric_limits<T>::max() / 4;
43        dis.assign(n + 1, INF);
44        pre.assign(n + 1, -1);
45        std::vector<bool> inq(n + 1, false);

```

```

46        std::queue<int> q;
47        dis[s] = 0;
48        q.push(s);
49        inq[s] = true;
50
51
52        while (!q.empty())
53        {
54            int u = q.front();
55            q.pop();
56            inq[u] = false;
57
58            for (int idx : g[u])
59            {
60                const _Edge &ed = e[idx];
61                if (ed.cap <= 0) continue;
62                int v = ed.to;
63                T nd = dis[u] + ed.cost;
64                if (nd < dis[v])
65                {
66                    dis[v] = nd;
67                    pre[v] = idx;
68                    if (!inq[v])
69                    {
70                        q.push(v);
71                        inq[v] = true;
72                    }
73                }
74            }
75        }
76        return dis[t] != INF;
77    }
78
79    // ===== 主过程: 不断用 SPFA 找最短费用增广路, 直至无法增广 =====
80    std::pair<T, T> work(int s, int t)
81    {
82        T flow = 0;      // 总流量
83        T cost = 0;      // 总费用
84        const T INF = std::numeric_limits<T>::max() / 4;
85
86        while (spfa(s, t))
87        {
88            // 计算本次可增广的流量
89            T aug = INF;
90            for (int v = t; v != s; v = e[pre[v]].to)
91            {
92                aug = std::min(aug, e[pre[v]].cap);
93            }
94
95            // 沿最短路增广
96            for (int v = t; v != s; v = e[pre[v]].to)

```

```

97     {
98         e[pre[v]].cap -= aug;
99         e[pre[v] ^ 1].cap += aug;
100     }
101
102     flow += aug;
103     cost += aug * dis[t]; // dis[t] 为本次单位费用
104 }
105
106 return std::make_pair(flow, cost);
107 }
108
109 // ===== 对外可视边结构 (便于调试/打印) =====
110 struct Edge
111 {
112     int from; // 起点
113     int to;   // 终点
114     T cap;    // 原始容量
115     T cost;   // 单位费用
116     T flow;   // 已经流过的流量
117 };
118
119 // 返回所有原始边的信息 (以正向边为准)
120 std::vector<Edge> edges()
121 {
122     std::vector<Edge> a;
123     a.reserve(e.size() / 2);
124     for (int i = 0; i < (int)e.size(); i += 2)
125     {
126         Edge x;
127         x.from = e[i ^ 1].to;
128         x.to   = e[i].to;
129         // 正反向残量之和等于原始容量
130         x.cap  = e[i].cap + e[i ^ 1].cap;
131         x.cost = e[i].cost;
132         // 反向边的容量就是已流过的流量
133         x.flow = e[i ^ 1].cap;
134         a.push_back(x);
135     }
136     return a;
137 }
138
139 // 打印所有有流量的边
140 void printFlowEdges()
141 {
142     std::cout << "Flow edges (from -> to: flow / cost):" << std::endl;
143     auto all = edges();
144     for (const auto &ed : all)
145     {
146         if (ed.flow > 0)
147         {

```

```

148             std::cout << ed.from << " -> " << ed.to << ": "
149                 << ed.flow << " / " << ed.cost << std::endl;
150         }
151     }
152 }
153 };

```

3.12.4 原始对偶优化

如果有负权边，在调用 *Work* 时需要先跑一遍最短路算法计算势能 h

这里考虑采用 *SPFA/topoDP*

这里优化了最短路算法的复杂度（即通过对偶的方式让 *dijkstra* 可行），但事实上不加优化的 *SPFA* 极快，这里的优化可能起到负作用

Dinic

```

1 template <class T>
2 struct MinCostFlow
3 {
4     struct _Edge
5     {
6         int to;
7         T cap;
8         T cost;
9         _Edge(int to_, T cap_, T cost_) : to(to_), cap(cap_), cost(cost_) {}
10    };
11
12    int n;
13    std::vector<_Edge> e;
14    std::vector<std::vector<int>>> g;
15    std::vector<T> h; // 节点势 (potential)
16    std::vector<T> dis; // Dijkstra 或 SPFA 计算出的距离
17    std::vector<int> dep; // 用于 Dinic 的分层
18    std::vector<int> cur; // 当前弧优化 (在 dfs 中隐式使用)
19
20    bool ok = false; // 自动检测负权边的标志
21    const T INF = std::numeric_limits<T>::max() / 4;
22
23    MinCostFlow() {}
24    MinCostFlow(int n_) { init(n_); }
25
26    void init(int n_)
27    {
28        n = n_;
29        e.clear();
30        g.assign(n + 1, {});
31        h.assign(n + 1, 0);
32        ok = false; // 重置标志
33    }
34

```

```

35 void add(int u, int v, T cap, T cost)
36 {
37     ok |= (cost < 0);
38     g[u].push_back((int)e.size());
39     e.emplace_back(v, cap, cost);
40     g[v].push_back((int)e.size());
41     e.emplace_back(u, 0, -cost);
42 }
43
44 private:
45 // 仅在有权边时，用于计算初始势的 SPFA
46 void SPFA(int s)
47 {
48     dis.assign(n + 1, INF);
49     std::vector<bool> inq(n + 1, false);
50     std::queue<int> q;
51     dis[s] = 0;
52     q.push(s);
53     inq[s] = true;
54     while (!q.empty())
55     {
56         int u = q.front();
57         q.pop();
58         inq[u] = false;
59         for (int i : g[u])
60         {
61             auto &t = e[i];
62             if (t.cap > 0 && dis[t.to] > dis[u] + t.cost)
63             {
64                 dis[t.to] = dis[u] + t.cost;
65                 if (!inq[t.to])
66                 {
67                     q.push(t.to);
68                     inq[t.to] = true;
69                 }
70             }
71         }
72     }
73 }
74 // 使用 Dijkstra 寻找最短路，边权为 cost + h[u] - h[v]
75 bool dijkstra(int s, int t)
76 {
77     dis.assign(n + 1, INF);
78     dis[s] = 0;
79     using pti = std::pair<T, int>;
80     std::priority_queue<pti, std::vector<pti>, std::greater<pti>> pq;
81     pq.push({0, s});
82
83     while (!pq.empty())
84     {
85         auto [d, u] = pq.top();

```

```

86         pq.pop();
87         if (dis[u] < d)
88             continue;
89         for (int i : g[u])
90         {
91             auto &t = e[i];
92             T c = t.cost + h[u] - h[t.to];
93             if (t.cap > 0 && dis[t.to] > dis[u] + c)
94             {
95                 dis[t.to] = dis[u] + c;
96                 pq.push({dis[t.to], t.to});
97             }
98         }
99     }
100     return dis[t] != INF;
101 }
102 // 在可接受网络上（缩减成本为0的边）构建分层图
103 bool bfs(int s, int t)
104 {
105     dep.assign(n + 1, 0);
106     dep[s] = 1;
107     std::queue<int> q;
108     q.push(s);
109     while (!q.empty())
110     {
111         int u = q.front();
112         q.pop();
113         for (int i : g[u])
114         {
115             auto &t = e[i];
116             if (t.cap > 0 && !dep[t.to] && t.cost + h[u] - h[t.to] == 0)
117             {
118                 dep[t.to] = dep[u] + 1;
119                 q.push(t.to);
120             }
121         }
122     }
123     return dep[t] != 0;
124 }
125 T dfs(int u, int t, T limit)
126 {
127     if (u == t)
128         return limit;
129     T used = 0;
130     for (int i : g[u])
131     {
132         if (used == limit)
133             break;
134         auto &E = e[i];
135         if (E.cap > 0 && dep[E.to] == dep[u] + 1 && E.cost + h[u] - h[E.to] == 0)
136

```

```

137         T f = dfs(E.to, t, std::min(limit - used, E.cap));
138         if (f > 0)
139         {
140             E.cap -= f;
141             e[i ^ 1].cap += f;
142             used += f;
143         }
144     }
145 }
146 if (used == 0)
147     dep[u] = 0;
148 return used;
149 }
150
151 public:
152     // 主过程：原始对偶算法
153     std::pair<T, T> work(int s, int t)
154     {
155         T flow = 0, cost = 0;
156         // **智能初始化**：如果检测到负权边，则运行一次 SPFA 初始化势 h
157         if (ok)
158         {
159             SPFA(s);
160             for (int i = 0; i <= n; ++i)
161             {
162                 if (dis[i] < INF)
163                     h[i] = dis[i];
164             }
165         }
166         // 主循环：不断用 Dijkstra 寻找增广路
167         while (dijkstra(s, t))
168         {
169             for (int i = 0; i <= n; ++i)
170             {
171                 if (dis[i] < INF)
172                 {
173                     h[i] += dis[i];
174                 }
175             }
176             T f = 0;
177             while (bfs(s, t))
178             {
179                 f += dfs(s, t, std::numeric_limits<T>::max());
180             }
181
182             flow += f;
183             cost += f * h[t];
184         }
185         return {flow, cost};
186     }
187 }

```

```

188 struct Edge
189 {
190     int from, to;
191     T cap, cost, flow;
192 };
193 std::vector<Edge> edges()
194 {
195     std::vector<Edge> a;
196     for (size_t i = 0; i < e.size(); i += 2)
197     {
198         a.push_back({e[i ^ 1].to, e[i].to, e[i].cap + e[i ^ 1].cap, e[i].cost, e[
199             i ^ 1].cap});
200     }
201     return a;
202 };

```

EK

```

1 template <class T>
2 struct MinCostFlowEK
3 {
4     struct _Edge
5     {
6         int to;
7         T cap;
8         T cost;
9         _Edge(int to_, T cap_, T cost_) : to(to_), cap(cap_), cost(cost_) {}
10    };
11
12    int n;
13    std::vector<_Edge> e;
14    std::vector<std::vector<int>>> g;
15    std::vector<T> h; // 节点势
16    std::vector<T> dis; // Dijkstra 或 SPFA 计算出的距离
17    std::vector<int> pre; // BFS/增广路径前驱
18    bool ok = false;
19    const T INF = std::numeric_limits<T>::max() / 4;
20
21    MinCostFlowEK() {}
22    MinCostFlowEK(int n_) { init(n_); }
23
24    void init(int n_)
25    {
26        n = n_;
27        e.clear();
28        g.assign(n + 1, {});
29        h.assign(n + 1, 0);
30        ok = false;
31    }
32
33    void add(int u, int v, T cap, T cost)

```

```

34 {
35     ok |= (cost < 0);
36     g[u].push_back((int)e.size());
37     e.emplace_back(v, cap, cost);
38     g[v].push_back((int)e.size());
39     e.emplace_back(u, 0, -cost);
40 }
41
42 private:
43 void SPFA(int s)
44 {
45     dis.assign(n + 1, INF);
46     std::vector<bool> inq(n + 1, false);
47     std::queue<int> q;
48     dis[s] = 0;
49     q.push(s);
50     inq[s] = true;
51     while (!q.empty())
52     {
53         int u = q.front();
54         q.pop();
55         inq[u] = false;
56         for (int i : g[u])
57         {
58             auto &E = e[i];
59             if (E.cap > 0 && dis[E.to] > dis[u] + E.cost)
60             {
61                 dis[E.to] = dis[u] + E.cost;
62                 if (!inq[E.to])
63                 {
64                     q.push(E.to);
65                     inq[E.to] = true;
66                 }
67             }
68         }
69     }
70 }
71
72 // Dijkstra + potentials
73 bool dijkstra(int s, int t)
74 {
75     dis.assign(n + 1, INF);
76     using pti = std::pair<T, int>;
77     std::priority_queue<pti, std::vector<pti>, std::greater<pti>> pq;
78     dis[s] = 0;
79     pq.push({0, s});
80     while (!pq.empty())
81     {
82         auto [d, u] = pq.top();
83         pq.pop();
84         if (dis[u] < d)

```

```

85         continue;
86         for (int i : g[u])
87         {
88             auto &E = e[i];
89             T c = E.cost + h[u] - h[E.to];
90             if (E.cap > 0 && dis[E.to] > dis[u] + c)
91             {
92                 dis[E.to] = dis[u] + c;
93                 pre[E.to] = i; // 记录前驱边
94                 pq.push({dis[E.to], E.to});
95             }
96         }
97     }
98     return dis[t] != INF;
99 }
100
101 public:
102 std::pair<T, T> work(int s, int t)
103 {
104     T flow = 0, cost = 0;
105     if (ok)
106     {
107         SPFA(s);
108         for (int i = 0; i <= n; i++)
109             if (dis[i] < INF)
110                 h[i] = dis[i];
111     }
112
113     pre.assign(n + 1, -1);
114
115     while (dijkstra(s, t))
116     {
117         // 更新势
118         for (int i = 0; i <= n; i++)
119             if (dis[i] < INF)
120                 h[i] += dis[i];
121
122         // 找到最小增广量
123         T f = INF;
124         for (int u = t; u != s; u = e[pre[u] ^ 1].to)
125             f = std::min(f, e[pre[u]].cap);
126
127         // 增广
128         for (int u = t; u != s; u = e[pre[u] ^ 1].to)
129         {
130             e[pre[u]].cap -= f;
131             e[pre[u] ^ 1].cap += f;
132         }
133
134         flow += f;
135         cost += f * h[t]; // 原始对偶特性: h[t] = 本次增广的真实成本

```

```

136     }
137
138     return {flow, cost};
139 }
140
141 struct Edge
142 {
143     int from, to;
144     T cap, cost, flow;
145 };
146
147 std::vector<Edge> edges()
148 {
149     std::vector<Edge> res;
150     for (size_t i = 0; i < e.size(); i += 2)
151         res.push_back({e[i ^ 1].to, e[i].to, e[i].cap + e[i ^ 1].cap, e[i].cost,
152                        e[i ^ 1].cap});
153     return res;
154 }
155 };

```

3.12.5 上下界网络流

注意在这一模块我们区分源点汇点和超级源点汇点，前者指原本的流量网络中的源汇，后者则是新的流量网络中的源汇。为了方便表示，我们分别表示为 S, T, SS, TT

无源汇可行流 无源汇可行流是指给你一个流量网络 g ，每条边上有上界 $C(u, v)$ ，也有下界 $B(u, v)$ ，我们要为每条边构造一个流量函数 $f(u, v)$ 使得该网络满足流量网络的基本特性，即流量守恒和斜对称性等等。

我们不妨先把 $f(u, v)$ 设为 $B(u, v)$ ，这样对于某个点它的流量差可以确定 $w_i = \sum B(u, i) - \sum B(i, u)$ 当 $w > 0$ 说明流入的过多，反之则说明流出的过多。根据斜对称性我们有 $\sum W_i = 0$ ，我们不妨设 $\delta = \sum_{w_i > 0} w_i = \sum_{w_i < 0} -w_i$ 。这启发我们建立新的流量网络 G ，每条边的流量上界为 $C(u, v) - B(u, v)$ ，我们建立超级源点和汇点 SS, TT ，如果 $W_i > 0$ ，我们连边 $SS \rightarrow (i, W_i)$ ，否则我们连边 $i \rightarrow (TT, -W_i)$ ，我们跑最大流得到最大流量 max ，如果 $max == \delta$ ，就说明可以构造出一条可行流。每条边真实的流量函数 $f(u, v) = B(u, v) + F(u, v)$ ，其中 $F(u, v)$ 指 G 中的流量

```

1 /**
2  * @brief 无源汇可行流求解器
3  */
4 template <class G>
5 class FeasibleFlow
6 {
7 public:
8     /**
9      * @brief 构造函数
10     * @param n 图中节点数量（不包括超级源汇）
11     * @param m 图中边数量

```

```

12 */
13 FeasibleFlow(int n, int m) : n(n), m(m), val(n + 2), ans(m + 1) {}
14
15 /**
16  * @brief 添加一条边
17  * @param u 边的起点
18  * @param v 边的终点
19  * @param lower 流量下界
20  * @param upper 流量上界
21  * @param id 边的标识符（用于结果输出）
22  */
23 void addEdge(int u, int v, G lower, G upper, int id)
24 {
25     edges.emplace_back(u, v, lower, upper, id);
26 }
27
28 /**
29  * @brief 求解可行流
30  * @return 是否存在可行流
31  */
32 bool solve()
33 {
34     // 初始化最大流算法（包含超级源汇S=0和T=n+1）
35     Flow<G> flow(n + 2);
36     const int S = 0, T = n + 1;
37
38     // 第一步：构建网络流图
39     for (const auto &[u, v, b, c, id] : edges)
40     {
41         // 添加边，容量为上界减下界
42         flow.add(u, v, c - b);
43         // 初始流量设为下界
44         ans[id] = b;
45         // 维护节点流量平衡（入流-出流）
46         val[v] += b;
47         val[u] -= b;
48     }
49
50     // 第二步：处理流量平衡约束
51     G sum = 0; // 记录需要补偿的总流量
52     for (int i = 1; i <= n; ++i)
53     {
54         if (val[i] > 0)
55         {
56             // 该节点需要额外流入val[i]流量
57             sum += val[i];
58             flow.add(S, i, val[i]);
59         }
60         else if (val[i] < 0)
61         {
62             // 该节点需要额外流出-val[i]流量

```

```

63         flow.add(i, T, -val[i]);
64     }
65 }
66
67 // 第三步：计算最大流
68 G f = flow.work(S, T);
69
70 // 判断是否存在可行流
71 if (f != sum)
72 {
73     return false; // 无法满足流量平衡约束
74 }
75
76 // 第四步：重构实际流量值
77 auto flowEdges = flow.getFlowEdges();
78 for (const auto &[from, to, flowVal] : flowEdges)
79 {
80     // 只处理原始图中的边（排除与超级源汇相连的边）
81     if (1 <= from && from <= n && 1 <= to && to <= n)
82     {
83         // ans[id]+=flowVal; id得自行修改传入
84         // 实际流量 = 下界 + 附加流量
85     }
86 }
87
88 return true;
89 }
90
91 /**
92  * @brief 获取每条边的实际流量
93  * @return 包含各边流量的数组（按添加顺序）
94  */
95 std::vector<G> getFlow() const
96 {
97     return std::vector<G>(ans.begin() + 1, ans.end());
98 }
99
100 private:
101     int n; // 原始图中节点数
102     int m; // 原始图中边数
103     std::vector<G> val; // 节点流量平衡值（入流-出流）
104     std::vector<G> ans; // 各边的实际流量
105     std::vector<std::tuple<int, int, G, G, int>> edges; // 存储所有边(u,v,下界,上界,id)
106 };

```

注意代码中 *ans* 数组表示每条边的可行流量

有源汇可行流 我们只需要连一条新边 $T \rightarrow (S, inf)$ ，即可转化为无源汇可行流

有源汇最大流 首先基于一个事实：只要给定的 $f(u, v)$ 流函数是一组可行流，我们就一定能找到最大流。

我们首先考虑求出可行流，也就是执行上述过程，在这个过程中我们可以得到一组可行流函数 $f(u, v)$ ，并且此时已有的流量是 $T \rightarrow S$ 的反边流量。然后我们对 G 撤去 SS 和 TT 并且删掉边 $T \rightarrow S$ ，再次从 $S \rightarrow T$ 跑最大流。将得到的最大流和一开始的可行流相加即为答案。我们称撤去 $SS TT T \rightarrow S$ 为初步调整，再次跑最大流为二次调整。

注意一定得按顺序来，必须先跑 $SS \rightarrow TT$ ，然后删去 $T \rightarrow S$ 再跑 $S \rightarrow T$

```

1 template <class G>
2 class BoundedMaxFlow
3 {
4 public:
5     BoundedMaxFlow(int n, int s, int t) : n(n), s(s), t(t), val(n + 2, 0) {}
6
7     void addEdge(int u, int v, G b, G c)
8     {
9         edges.emplace_back(u, v, b, c);
10    }
11
12    G compute()
13    {
14        Flow<G> flow(n + 2); // 包含超级源汇 S=0, T=n+1
15        const int S = 0, T = n + 1;
16
17        // 构建基础网络
18        for (const auto &[u, v, b, c] : edges)
19        {
20            flow.add(u, v, c - b);
21            val[v] += b;
22            val[u] -= b;
23        }
24
25        // 添加补偿边
26        G sum = 0;
27        for (int i = 1; i <= n; ++i)
28        {
29            if (val[i] > 0)
30            {
31                sum += val[i];
32                flow.add(S, i, val[i]);
33            }
34            else if (val[i] < 0)
35            {
36                flow.add(i, T, -val[i]);
37            }
38        }
39
40        // 添加 t->s 的无限边
41        flow.add(t, s, std::numeric_limits<G>::max() / 2);
42        // 检查可行性

```

```

43     if (flow.work(S, T) != sum)
44     {
45         return -1; // 无解
46     }
47
48     // 计算最大流
49     G initialFlow = flow.e.back().second; // 最后一条边 t-s 的流量
50     flow.g[s].pop_back();
51     flow.g[t].pop_back();
52     G additionalFlow = flow.work(s, t);
53     return initialFlow + additionalFlow;
54 }
55
56 private:
57     int n; // 节点数
58     int s, t; // 源汇点
59     std::vector<G> val; // 流量平衡
60     std::vector<std::tuple<int, int, G, G>> edges; // u,v,b,c
61 };

```

有源汇最小流 根据结论 $S- > T$ 的最小流等于 $T- > S$ 的最大流的相反数，用可行流流量减去 G 中 $T- > S$ 的最大流

所以最终最小流为 $minflow(s, t) = -maxflow(t, s)$

其中 s, t 为给定的源和汇

有源汇费用流 我们把最大流算法换成费用流算法，关于 SS, TT 的所有连边代价均为 0，初始的费用为 $\sum B(u, v)W(u, v)$ ，然后我们要考虑撤去初步调整的影响，即加上 $SS- > TT$ 最小费用最大流产生的费用，然后二次调整也要加上 $S- > T$ 最小费用最大流的费用。

有负圈的费用流 我们考虑让负边强制满流，也就是让 $B(u, v) = C(u, v)$ ，并且让 $C(v, u) = C(u, v)$ 表示退流，这样 $u- > v$ 这条边就不会出现在流量网络 G 中，就可以正常跑费用流了。

```

1  /**
2  * @brief 带负圈处理的最小费用流求解器
3  * @tparam G 费用流数据类型（通常为int或long long）
4  * @tparam MinCostFlow 最小费用流算法模板（需提供相同接口）
5  */
6  template <typename G>
7  class NegativeCycleFlow
8  {
9  public:
10     /**
11     * @brief 构造函数
12     * @param n 原始图中节点数（不包括超级源汇）
13     * @param s 原始源点
14     * @param t 原始汇点
15     */
16     NegativeCycleFlow(int n, int s, int t) : n(n), s(s), t(t), val(n + 1, 0) {}

```

```

17
18  /**
19  * @brief 添加有上下界和费用的边
20  * @param u 起点
21  * @param v 终点
22  * @param minFlow 最小流量（下界）
23  * @param maxFlow 最大流量（上界）
24  * @param cost 单位费用
25  */
26  void addEdge(int u, int v, G minFlow, G maxFlow, G cost)
27  {
28      if (cost < 0)
29      {
30          edges.push_back({u, v, maxFlow, maxFlow, cost});
31          edges.push_back({v, u, 0, maxFlow, -cost});
32      }
33      else
34      {
35          edges.push_back({u, v, minFlow, maxFlow, cost});
36      }
37  }
38
39  /**
40  * @brief 计算带负圈的最小费用流
41  * @return 返回{总流量, 总费用}, 若无解则返回{-1, -1}
42  */
43  std::pair<G, G> compute()
44  {
45      // 初始化费用流（包含超级源汇 S=0 和 T=n+1）
46      MinCostFlow<G> flow(n + 2);
47      const int S = 0, T = n + 1;
48
49      // 第一步：处理负费用边（强制流满上界消除负圈）
50      G totalCost = 0;
51      for (const auto &[u, v, minF, maxF, c] : edges)
52      {
53          totalCost += minF * c; // 计算强制流量的费用
54          val[u] -= minF; // 调整节点流量平衡
55          val[v] += minF;
56          // 添加剩余容量边
57          flow.add(u, v, maxF - minF, c);
58      }
59
60      // 第二步：处理流量平衡约束
61      G totalFlow = 0;
62      for (int i = 1; i <= n; ++i)
63      {
64          if (val[i] > 0)
65          {
66              flow.add(S, i, val[i], 0);
67              totalFlow += val[i];

```



```

68     }
69     else
70     {
71         flow.add(i, T, -val[i], 0);
72     }
73 }
74
75 // 第三步：添加t->s的无限容量边（转化为循环流）
76 flow.add(t, s, std::numeric_limits<G>::max() / 2, 0);
77
78 // 第四步：检查可行性并计算初始费用
79 auto [f1, c1] = flow.work(S, T);
80 if (f1 != totalFlow)
81 {
82     return {-1, -1}; // 无可行流
83 }
84 totalCost += c1;
85
86 // 第五步：计算实际最大流
87 // 1. 获取初始可行流值（t->s边的流量）
88 G initialFlow = flow.e.back().cap;
89 flow.g[s].pop_back();
90 flow.g[t].pop_back();
91 // 2. 在残余网络上继续增广
92 auto [f2, c2] = flow.work(s, t);
93 totalCost += c2;
94
95 return {initialFlow + f2, totalCost};
96 }
97
98 private:
99     int n; // 原始节点数
100     int s, t; // 原始源汇点
101     std::vector<G> val; // 节点流量平衡值
102     std::vector<std::tuple<int, int, G, G, G>> edges; // 边信息[u, v, min, max, cost]
103 };

```

最大费用最大流 我们把所有边权取反，跑（可能有负圈）的最小费用最大流即可。

具体的说，对于一条边 $(u, v, b, c, cost)$ 来说如果 $cost$ 为负，我们把该边拆成两条分别是 $(u, v, c, c, cost)$ 以及 $(v, u, 0, c, -cost)$ ，这样在加入上下界网络流图中时负边权必须流满。

3.12.6 Stoer-Wagner 无向图全局最小割

定义无向联通图 G 的最小割为，边权和最小的边集 E ，满足去掉 E 之后可以使 G 变成两个联通分量。

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4 const int N = 610; // 最大顶点数

```

```

5 const ll INF = 1e18; // 代表无穷大
6 ll w[N][N]; // 邻接矩阵，存储边的权值
7 bool deleted[N]; // 标记顶点是否已被合并
8 int n; // 图的顶点数
9 // Stoer-Wagner 全局最小割算法
10 // 返回一个 pair，包含最小割的值和构成割的一个顶点集合
11 pair<ll, vector<int>> stoer_wagner()
12 {
13     ll min_cut = INF;
14     vector<int> best_partition;
15
16     // `nodes` 存储当前图中还存在的顶点
17     vector<int> nodes(n);
18     iota(nodes.begin(), nodes.end(), 0); // 初始化 nodes 为 {0, 1, ..., n-1}
19
20     for (int phase = n; phase > 1; --phase)
21     {
22         // Minimum Cut Phase: 类似 Prim 算法，找出 s 和 t
23         vector<ll> dist(n, 0);
24         vector<bool> in_set(n, false);
25         int s = -1, t = -1;
26
27         for (int i = 0; i < phase; ++i)
28         {
29             int max_j = -1;
30             // 找到不在集合中且 dist 最大的顶点
31             for (int u : nodes)
32             {
33                 if (!in_set[u] && (max_j == -1 || dist[u] > dist[max_j]))
34                 {
35                     max_j = u;
36                 }
37             }
38             if (i == phase - 2)
39                 s = max_j; // 倒数第二个加入集合的顶点是 s
40             if (i == phase - 1)
41                 t = max_j; // 最后一个加入集合的顶点是 t
42             in_set[max_j] = true;
43             // 更新所有不在集合中的顶点的 dist 值
44             for (int u : nodes)
45             {
46                 if (!in_set[u])
47                 {
48                     dist[u] += w[max_j][u];
49                 }
50             }
51         }
52         // 更新全局最小割
53         if (dist[t] < min_cut)
54         {
55             min_cut = dist[t];

```

```

56 // 找到构成当前割的原始顶点集合
57 // 需要遍历所有原始顶点, 判断它们最终被合并到了 t 还是其他节点
58 // 但一个更简单的方法是直接知道 t 本身代表了哪些原始顶点
59 // 这里我们偷个懒, 只返回了 t 对应的划分, 需要更复杂的逻辑来追踪所有合并
60 // 对于只求最小割值的场景, 这样已经足够。
61 // 为了得到正确的划分, 我们需要手动维护每个节点所代表的原始节点集合
62 // 对于竞赛来说, 通常只要求割的值
63 best_partition = {t}; // 注意: 这只是最简单的划分表示
64 }
65 // Contraction Phase: 合并 s 和 t
66 // 将 t 的所有边权加到 s 上
67 for (int u : nodes)
68 {
69     if (u != s && u != t)
70     {
71         w[s][u] += w[t][u];
72         w[u][s] += w[t][u]; // 关键: 维持对称性
73     }
74 }
75
76 // 从 nodes 向量中移除 t
77 nodes.erase(remove(nodes.begin(), nodes.end(), t), nodes.end());
78 }
79 return {min_cut, best_partition};
80 }
81
82 int main()
83 {
84     ios::sync_with_stdio(false);
85     cin.tie(nullptr);
86
87     int m;
88     cin >> n >> m;
89     // 初始化邻接矩阵
90     for (int i = 0; i < n; ++i)
91     {
92         for (int j = 0; j < n; ++j)
93         {
94             w[i][j] = 0;
95         }
96     }
97     for (int i = 0; i < m; ++i)
98     {
99         int u, v;
100         ll c;
101         cin >> u >> v >> c;
102         --u;
103         --v;
104         // 无向图, 边权累加
105         w[u][v] += c;
106         w[v][u] += c;

```

```

107     }
108
109     auto [min_cut_val, partition] = stoer_wagner();
110     cout << min_cut_val << '\n';
111
112     return 0;
113 }

```

3.12.7 其他总结

残余网络 即跑完当前流以后所有容量 >0 的边的集合, 注意无论正反边只要有容量即可 (正边可以推流, 反边可以退流)。

最小割集 求最小割集的方法: 在残余网络中从源点开始 *DFS* (只走容量 >0 的边), 能到达的所有点即为 S , S 及其补集为最小割集。

最小割集必须点 残余网络中 S 直接连向的点必在 S 的割集中, 直接连向 T 的点必在 T 的割集中; 若这些点的并集为全集, 则最小割方案唯一。

(这里可到是指只走容量 >0 的边)

最小割可行边 在残余网络中跑 *SCC*, 缩点后剩余的边是可行边, 这些边必然满流

最小割必须边 仍然在残余网络中跑 *SCC*, 如果一条边 $u \rightarrow v$, 满足 $bel[u] = bel[s]$ and $bel[v] = bel[t]$, 那么是必须边

最大权闭合子图 即选了 u 必须选 u 能到达的所有点, 以此类推。

S 向正权点 x 连流量为 $a[x]$ 的边, 负权点 y 向 T 连流量为 $-a[y]$ 的边, 每条有向边加入, 流量无穷大即可。

求最小割后, S 所在的最小割集即为选择方案。

分组问题 有 n 个 *bool* 型变量, 你需要把他们分为 01 两组, 每个点有为 0 的代价 l_i , 有为 1 的代价 r_i , 有 m 个特殊限制, 如果 i, j 同时在左侧会产生代价 $L_{i,j}$, 同理有 $R_{i,j}$, 我们考虑建立源点 S , 汇点 T , S 向每个点连流量为 l_i 的边, 每个点向 T 连流量为 r_i 的边。对于左侧代价 $L_{i,j}$, 我们建立一个新点 x , 连边 $(S, x, L_{i,j}), (x, i, inf), (x, j, inf)$, 对于右侧同理建新点 x , 连边 $(x, T, R_{i,j}), (i, x, inf), (j, x, inf)$, 跑最小割即可。

3.13 二分图

二分图是无向图的概念, 二分图是一类特殊的无向图. 其特点是可以把点分成左右两堆, 满足每堆内部的点两两不存在有边, 特别的我们称为左部点和右部点

3.13.1 二分图的判断

一个无向图是二分图的充要条件是不存在奇环我们有两种方法判断二分图

- 1 我们可以通过二分图染色来判断是否为二分图, 如果是二分图, 染色过程中还帮我们分出了左部点和右部点, 时间复杂度 $O(n + m)$
- 2 如果像线段树分治那样, 需要动态判断二分图, 我们可以采用扩展域并查集, 对于边 (u, v) 我们 $merge(u, v + n), merge(v, u + n)$, 加边 (u, v) 时我们先 $check(u, v)$ 是否连通, 如果联通则说明有奇环, 为什么呢? 注意我们的连边方式, 想要直接连通必须经过偶数条边, 当再出现一条直接联通的边时, 就存在了奇环

二分图染色

```

1 struct BiGraphColor
2 {
3     std::vector<int> col;
4     bool isBiGraph;
5     int cnt1 = 0, cnt2 = 0;
6     BiGraphColor(const std::vector<std::vector<int>> &e) : col(e.size(), -1),
7         isBiGraph(true)
8     {
9         int n = e.size() - 1;
10        std::function<void(int)> dfs = [&](int x)
11        {
12            if (!isBiGraph)
13            {
14                return;
15            }
16            for (int y : e[x])
17            {
18                if (col[y] == -1)
19                {
20                    col[y] = col[x] ^ 1;
21                    if (col[y] == 0)
22                    {
23                        cnt1++;
24                    }
25                    else
26                    {
27                        cnt2++;
28                    }
29                    dfs(y);
30                }
31                else if (col[y] == col[x])
32                {
33                    isBiGraph = false;
34                    return;
35                }
36            }
37        };
38        for (int i = 1; i <= n; i++)

```

```

39    {
40        if (col[i] == -1)
41        {
42            col[i] = 0;
43            dfs(i);
44        }
45        if (!isBiGraph)
46            return;
47    }
48 }
49 };

```

二分图匹配问题留在图匹配中统一说明

3.13.2 二分图上博弈

在一张二分图上, *Alice* 和 *Bob* 从某个起点开始轮流移动, *Alice* 先手, 不能重复移动到同一节点, 无法移动者输? 问必胜策略

结论: 如果起点是最大匹配的关键点, 那么 *Alice* 必胜

证明: *Alice* 只要按匹配边不断走就可以, 我们可以证明 *Bob* 无法在过程中走到未盖点, 假如 *Bob* 走到未盖点, 那我们翻转 *Alice* 的起点出发的路径, 匹配数不变, 但此时起点不再属于匹配点, 与起点是关键点矛盾。因此 *Bob* 只能跟随 *Alice* 不断走交错路, 直至无路可走

我们只需找到关键点即可, 在二分图匹配中我们已经详细阐述了如何寻找关键点

3.13.3 删边二分图

给你一个无向图问删除一条边是不是二分图?

我们从维护奇环的角度来考虑, 没有奇环是二分图的充要条件, 我们考虑对图跑出 *dfs* 树, 然后考虑非树边在树上构成的环路长度, 我们每找到一个奇环, 就对这条路径上的树边进行区间加, 并标记该非树边, 同理我们每找到一个偶环, 也对这条路径上的树边区间加, 我们用树上差分实现。十分关键的一点是, 我们为什么要统计偶环呢? 我们发现这种操作方式并不能找到所有的环, 对于环套环的图即可发现, 也就是说我们可能遗漏某些环的信息, 具体的说, 如果两个奇环合并又构成一个偶环, 我们只会统计到一个奇环和一个偶环, 我们知道只有两个奇环共用的边才是合法答案, 而我们知道奇环 + 偶环 = 奇环, 如果一条边既被奇环覆盖, 又被偶环覆盖, 说明他可以衍生出新的奇环, 这样该边就不合法了, 因为我们删去这条边需要删掉所有的奇环

所以我们得到统计方法:

对于没有奇环的图, 每条边都是合法边

对于存在奇环的图, 对于树边, 如果该边被奇环覆盖的次数等于统计的奇环个数, 且该边不被偶环覆盖, 那么是合法边。对于非树边, 当且仅当只有一个奇环, 且该树边构成这个奇环时才合法

3.14 图匹配

3.14.1 基础概念

匹配:

对于一张图 $G = (V, E)$, 我们有一个边集 $E' \subseteq E$ 且满足 E' 中的边两两之间无公共点, 那么 E' 称作该图的一个匹配。在匹配中的边称作匹配边, 否则称作非匹配边, 同理我们称为已盖点和未盖点增广路:

两端都是未盖点的简单路径, 且路径上的边满足匹配边和非匹配边交替的路径称作增广路。用边集表示类似于 01010, 用点集表示即 0111110 增广路的意义在于你翻转所有匹配边和非匹配边, 得到的匹配边加一。

最大匹配:

在图的所有匹配中, 如果一个匹配 E' 包含的元素数量是最多的, 那么我们称作最大匹配, 特别的, 一个图不一定只有一个最大匹配

匹配关键点

在一个图的所有最大匹配中都存在的点称作匹配的关键点

增广路定理:

一个匹配是图的最大匹配当且仅当前残图上的未盖点和非匹配边不存在增广路

完美匹配:

如果一个匹配 E' 中的边已经涵盖了图上原有的所有点, 那么该匹配称作完美匹配

最大权匹配:

当我们的边存在边权时, 匹配中边权和最大的匹配称作最大权匹配

最大权完美匹配

即边权和最大的完美匹配

最小点覆盖

对一个图来说, 我们用最少的点覆盖所有的边, 此时点的数量称作最小点覆盖

最小边覆盖

对一个图来说, 我们用最少的边覆盖所有的点, 此时边的数量称作最小边覆盖

最大独立集

对一个图来说, 我们选出最多的点, 满足这些点之间没有边直接相连, 此时点的数量称作最大独立集

3.14.2 二分图上的匹配

匹配在二分图中将有很好的特殊性质, 因此我们把二分图和一般图分开讨论, 我们用 $G(V1, V2, E)$ 表示二分图, $V1$ 表示左部点的集合, $V2$ 表示右部点的集合

特别的如果题目没有帮我们区分左部点和右部点, 我们可以二分图染色得到

二分图最大匹配 匈牙利算法: 根据增广路的概念和增广路定理, 我们只需要朴素的不断寻找增广路即可, 直至我们找不到增广路

由此诞生了匈牙利算法, 匈牙利算法的原理是对每一个左部的未盖点, 我去右部同样寻找一个未盖点与其配对形成增广路, 这样匹配数 +1, 但算法过程略有不同, 对于左部点 $V1$, 假设找到了右部点 $V2$ 但

$V2$ 已经匹配, 我们尝试递归 $V2$ 已匹配的左部点 $V3$ 使其更换匹配点, 如果 $V3$ 能找到新匹配点 $V4$, 那么 $V3$ 和 $V4$ 配对, $V1$ 和 $V2$ 配对。对每一个左部点都进行此操作, 最终成功配对数量即最大匹配, 复杂度 $O(n * E + m)$ 其中 n 为左部点数量, E 为边的数量, m 为右部点数量, 不难发现我们可以使点数少的点为左部来优化时间

在题目中, 如果明确了左部点右部点, 不能进行二分图染色!

第一个是手动二分图染色版本

```

1 struct Hungary
2 {
3     int n;
4     const std::vector<std::vector<int>> &g;
5     std::vector<int> left, right, col;
6     bool isBigrph;
7     int maxMatch;
8     std::vector<int> vis, match; // match记每个右部点匹配的左部点是谁
9     int cnt = 0;
10    void dfs1(int u) // 二分图染色
11    {
12        for (auto v : g[u])
13        {
14            if (col[v] == -1)
15            {
16                col[v] = col[u] ^ 1;
17                if (col[v] == 1)
18                {
19                    right.push_back(v);
20                }
21                else
22                {
23                    left.push_back(v);
24                }
25                dfs1(v);
26            }
27            else if (col[u] == col[v])
28            {
29                isBigrph = false;
30                return;
31            }
32        }
33    }
34    bool dfs2(int u, int num) // 找增广路
35    {
36        for (auto v : g[u])
37        {
38            if (vis[v] == num)
39            {
40                continue;
41            }
42            vis[v] = num;
43            if (!match[v] or dfs2(match[v], num))

```

```

44     {
45         match[v] = u;
46         return true;
47     }
48 }
49 return false;
50 }
51 Hungary(const std::vector<std::vector<int>> &g) : g(g), n(g.size() - 1), vis(n +
52     1), match(n + 1)
53 {
54     isBigraph = 1;
55     maxMatch = 0;
56     col.assign(n + 1, -1);
57     for (int i = 1; i <= n; ++i)
58     {
59         if (col[i] == -1)
60         {
61             col[i] = 0;
62             left.push_back(i);
63             dfs1(i);
64         }
65     }
66     if (isBigraph)
67     {
68         if (left.size() > right.size())
69         {
70             std::swap(left, right);
71         }
72         for (auto i : left)
73         {
74             ++cnt; // 时间戳优化
75             if (dfs2(i, cnt))
76             {
77                 ++maxMatch;
78             }
79         }
80     }
81 };

```

第二个是传入图和左部点集版本

```

1 struct Hungary
2 {
3     int n;
4     const std::vector<std::vector<int>> &g;
5     std::vector<int> left;
6     int maxMatch;
7     std::vector<int> vis, match; // match记每个右部点匹配的左部点是谁
8     int cnt = 0;
9     bool dfs2(int u, int num) // 找增广路
10    {

```

```

11        for (auto v : g[u])
12        {
13            if (vis[v] == num)
14            {
15                continue;
16            }
17            vis[v] = num;
18            if (!match[v] or dfs2(match[v], num))
19            {
20                match[v] = u;
21                return true;
22            }
23        }
24        return false;
25    }
26    Hungary(const std::vector<std::vector<int>> &g, std::vector<int> L) : g(g), n(g.
27        size() - 1), vis(n + 1), match(n + 1)
28    {
29        left = L;
30        maxMatch = 0;
31        for (auto i : left)
32        {
33            ++cnt; // 时间戳优化
34            if (dfs2(i, cnt))
35            {
36                ++maxMatch;
37            }
38        }
39    };

```

最大流做法，我们建立超级源点 S 和超级汇点 T ， S 向每一个左部点连流量为 1 的边，每一个右部点向 T 连流量为 1 的边，二分图原有边流量置为 1，从 S 到 T 的最大流即最大匹配，正确性显然。重边并不会影响算法的正确性，因为源点汇点的流量卡死了。

匹配的关键点 首先我们有朴素算法，即去掉每个点后再次跑最大匹配，看匹配的大小是否改变，但是我们有更优秀的方法

我们给出一个引理：

一个匹配点 P 不是关键点，当且仅当存在一条以 P 为端点，以匹配边出发的交替链，使得终点为某个未盖点

因此我们得出做法：先跑出最大匹配，然后我们从左边的每一个未盖点出发，给路径上的点打标记，然后左边未被标记的匹配点就是关键点，同理我们可以得到右边的关键点。复杂度 $O(V + E)$

我们也可以用最大流跑出流网络然后再使用同样的方法得到关键点。

```

1 class HungaryCriticalPoints
2 {
3 private:
4     int n; // 顶点数量
5     const std::vector<std::vector<int>> &g; // 图的邻接表表示

```

```

6  std::vector<int> left;           // 左部点集合
7  std::vector<int> matchl;        // 左部点的匹配点 (matchl[u] = v 表示左部
   点u匹配右部点v)
8  std::vector<int> matchr;        // 右部点的匹配点 (matchr[v] = u 表示右部
   点v匹配左部点u)
9  std::vector<int> pos;           // 标记是左部点 还是右部点 左部点为1 右部
   点为0
10
11 public:
12  /**
13   * @brief 构造函数，初始化匈牙利算法关键点查找器
14   * @param g 图的邻接表表示，g.size() = n+1, 顶点编号从1开始
15   * @param left 左部点集合
16   */
17  HungaryCriticalPoints(const std::vector<std::vector<int>> &g, std::vector<int>
   left)
18   : g(g), n(g.size() - 1), left(left)
19  {
20
21   // 初始化匹配数组
22   pos.resize(n + 1, 0);
23   matchl.resize(n + 1, 0);
24   matchr.resize(n + 1, 0);
25   for (auto x : left) // 左部点标记为1
26   {
27     pos[x] = 1;
28   }
29   // 运行匈牙利算法找到最大匹配
30   Hungary hungary(g, left);
31
32   // 将匹配结果存入matchl和matchr
33   for (int v = 1; v <= n; ++v)
34   {
35     if (hungary.match[v] != 0)
36     {
37       matchl[hungary.match[v]] = v;
38       matchr[v] = hungary.match[v];
39     }
40   }
41 }
42
43 /**
44  * @brief 查找所有关键点
45  * @return 返回一个pair, first是左部关键点, second是右部关键点
46  */
47 std::pair<std::vector<int>, std::vector<int>> findCriticalPoints()
48 {
49   std::vector<bool> markl(n + 1, false); // 标记左部点是否被访问 未被访问是关键
   点
50   std::vector<bool> markr(n + 1, false); // 标记右部点是否被访问
51

```

```

52 // 从所有未匹配点出发进行BFS 左右两边分别做
53 std::queue<int> ql, qr;
54 for (int i = 1; i <= n; ++i)
55 {
56   if (pos[i] and matchl[i] == 0)
57   {
58     ql.push(i);
59     markl[i] = 1;
60   }
61   else if (!pos[i] and matchr[i] == 0) // 未盖点
62   {
63     qr.push(i);
64     markr[i] = 1;
65   }
66 }
67 // BFS标记交替路径上的点
68 while (!ql.empty())
69 {
70   int u = ql.front();
71   ql.pop();
72   for (auto v : g[u])
73   {
74     v = matchr[v];
75     if (!markl[v])
76     {
77       ql.push(v);
78       markl[v] = 1;
79     }
80   }
81 }
82 while (!qr.empty())
83 {
84   int u = qr.front();
85   qr.pop();
86   for (auto v : g[u])
87   {
88     v = matchl[v];
89     if (!markr[v])
90     {
91       qr.push(v);
92       markr[v] = 1;
93     }
94   }
95 }
96 // 收集关键点
97 std::vector<int> leftCritical, rightCritical;
98 for (int i = 1; i <= n; ++i)
99 {
100   if (pos[i] and !markl[i])
101   {
102     leftCritical.push_back(i);

```

```

103     }
104     else if (!pos[i] and !markr[i])
105     {
106         rightCritical.push_back(i);
107     }
108 }
109 return {leftCritical, rightCritical};
110 }
111 };

```

二分图最大匹配的性质

最大匹配 = 最小点覆盖

最大匹配 = n - 最大独立集

最大匹配 = n - 最小边覆盖 - 孤立点数

Hall 定理:

对于一个二分图 $G = (V_1, V_2, E)$, 我们设 S 为 V_1 的一个子集, $F(S)$ 为 S 的一个邻集, 即 S 所有相邻点的集合, 总有 $|F(S)| \geq |S|$, 也就是说左部点的子集的邻集元素数量不小于子集的元素数量

Hall 定理加强描述:

更强的结论是, G 的最大匹配为 $|V_1| - \max_{S \subseteq V_1} (|S| - |F(S)|)$

最小点覆盖的构造方法: 对于左部每个未匹配点, 出发走交替路, 给经过的所有点打标记。那么最小点覆盖的点集为: 左部所有不可达点和右部所有可达点

最大独立集的构造方法: 即最小点覆盖的补集

最小边覆盖的构造方法: 未匹配点随机选一条出边即可

二分图最大多重匹配 也即我们对 u 点设一个变量 $h(u)$, 代表 u 点不能与超过 $h(u)$ 条边相连, 一般的最大匹配中 $h(u) = 1$

我们只需要把 $S = V_1, V_2 - T$ 的流量改为对应的 $h(u)$ 跑最大流即可

二分图带权最大匹配 对于最大权或者最小权最大匹配, 我们改最大流为最大费用最大流或最小费用最大流

二分图最大权完美匹配 我们用 KM 算法在 $O(n^3)$ 的时间复杂度内解决该问题

我们默认左部右部点数量相等, 否则将是最大权匹配

需要注意的是, 如果题目存在负边权, 那么不存在的边视作负无穷大, 否则视作 0

首先我们为每个点赋一个权值 $h(u)$ 称作顶标, 要求对于所有的边 (u, v, w) , 都满足 $h(u) + h(v) \geq w$

然后我们引出相等子图的概念, 相等子图即只包含满足 $h(u) + h(v) = w$ 的边的子图

然后我们有定理: 如果相等子图存在完美匹配, 那么该匹配为最大权完美匹配

所以求解最大权完美匹配的过程实际上是一个不断调整顶标的过程

我们初始化左部点顶标为对应出边最大值, 右部点顶标为 0

我们先给出类似匈牙利算法的朴素思想, 每次搜索假设我们左部点为 u , 我们遍历其所有出边, 如果右部点 v 已被访问则跳过, 否则检测该边是否在相等子图中, 如果在子图中未被匹配或者递归可以更换匹配点的话 (和匈牙利类似), 我们返回增广成功的信息。如果不在子图中, 我们更新右部点的 $slack$ 值

然后取 d 为所有未访问右部点的最小 $slack$ 值, 访问过的左部点顶标减去 d , 右部点加上 d , 对每个点进行此循环直至成功为止, 最终的所有顶标和即为最大权, $match$ 数组即为匹配情况, 用 dfs 实现复杂度 $O(n^4)$

```

1 template <class T>
2 class KuhnMunkres
3 {
4 private:
5     const T low = std::numeric_limits<T>::min() / 2;
6     const T high = std::numeric_limits<T>::max() / 2;
7     int n; // 顶点数量
8     std::vector<std::vector<T>>> graph; // 邻接矩阵表示的带权二分图
9     std::vector<T> A; // 左部顶点的顶标
10    std::vector<T> B; // 右部顶点的顶标
11    std::vector<int> match; // 匹配结果, match[v] = u 表示右部点v匹配左部点u
12    std::vector<T> slack; // 松弛量数组
13    std::vector<int> vis;
14    // 访问标记数组
15
16    /**
17     * @brief DFS寻找增广路
18     * @param u 当前尝试匹配的左部顶点
19     * @return 是否找到增广路
20     */
21    bool dfs(int u)
22    {
23        for (int v = 1; v <= n; ++v)
24        {
25            if (vis[v])
26                continue; // 右部顶点已访问过
27
28            if (A[u] + B[v] == graph[u][v])
29            { // 满足相等子图条件
30                vis[v] = true;
31                if (!match[v] || dfs(match[v]))
32                { // 如果v未匹配或可以找到增广路
33                    match[v] = u; // 更新匹配
34                    return true;
35                }
36            }
37            else
38            {
39                // 更新松弛量
40                slack[v] = std::min(slack[v], A[u] + B[v] - graph[u][v]);
41            }
42        }
43        return false;
44    }
45
46 public:

```



```

47  /**
48  * @brief 构造函数，初始化KM算法
49  * @param n 顶点数量
50  * @param edges 边的三元组(u, v, w)列表
51  */
52  KuhnMunkres(int n, const std::vector<std::tuple<int, int, int>> &edges)
53      : n(n), graph(n + 1, std::vector<T>(n + 1, low)),
54        A(n + 1, low), B(n + 1, 0),
55        match(n + 1, 0), slack(n + 1, high), vis(n + 1, 0)
56  {
57
58      // 初始化邻接矩阵
59      for (const auto &[u, v, w] : edges)
60      {
61          graph[u][v] = w;
62      }
63
64      // 初始化左部顶标为出边最大权值
65      for (int u = 1; u <= n; ++u)
66      {
67          for (int v = 1; v <= n; ++v)
68          {
69              A[u] = std::max(A[u], graph[u][v]);
70          }
71      }
72  }
73
74  /**
75  * @brief 计算最大权完美匹配
76  * @return 匹配结果和总权重
77  */
78  std::pair<std::vector<int>, T> solve()
79  {
80      for (int u = 1; u <= n; ++u)
81      { // 为每个左部顶点寻找匹配
82          while (true)
83          {
84              // 每次尝试前重置访问标记和松弛量
85              std::fill(vis.begin(), vis.end(), 0);
86              std::fill(slack.begin(), slack.end(), high);
87
88              if (dfs(u))
89                  break; // 找到增广路则退出
90
91              // 计算最小的松弛量d
92              T d = high;
93              for (int v = 1; v <= n; ++v)
94              {
95                  if (!vis[v])
96                  {
97                      d = std::min(d, slack[v]);

```

```

98          }
99      }
100
101      // 调整顶标
102      for (int v = 1; v <= n; ++v)
103      {
104          if (vis[v])
105          { // 在增广路中的右部顶点
106              B[v] += d;
107              A[match[v]] -= d;
108          }
109      }
110      A[u] -= d; // 调整当前左部顶点的顶标
111  }
112
113      // 计算总权重
114      T totalWeight = 0;
115      for (int v = 1; v <= n; ++v)
116      {
117          if (match[v] > 0)
118          {
119              totalWeight += graph[match[v]][v];
120          }
121      }
122
123      return {match, totalWeight};
124  }
125  };
126

```

我们考虑继续优化，发现调整顶标后重新搜索十分浪费时间，我们考虑改 *dfs* 为 *bfs*，具体的说就是利用了交错树遗留的信息优化了复杂度，时间复杂度为 $O(n^3)$ ，实际上 *bfs* 的过程并不好理解，卡 *dfs* 的题目也并不多

```

1  template <class T>
2  class KuhnMunkresBFS
3  {
4  private:
5      const T low = std::numeric_limits<T>::min() / 2;
6      const T high = std::numeric_limits<T>::max() / 2;
7      int n; // 顶点数量
8      std::vector<std::vector<T>> graph; // 邻接矩阵表示的带权二分图
9      std::vector<T> A; // 左部顶点的顶标
10     std::vector<T> B; // 右部顶点的顶标
11     std::vector<int> match; // 匹配结果，match[v] = u表示右部点v匹配左部点u
12     std::vector<T> slack; // 松弛量数组
13     std::vector<int> vis; // 访问标记数组
14     std::vector<int> pre; // 前驱数组，用于记录增广路径
15
16     /**

```



```

17  * @brief BFS寻找增广路
18  * @param u 起始的左部顶点
19  */
20  void bfs(int u)
21  {
22      std::fill(vis.begin(), vis.end(), 0); // 重置访问标记
23      std::fill(slack.begin(), slack.end(), high); // 重置松弛量
24
25      int x = u, y = 0; // x:当前左部点, y:当前右部点
26      match[0] = u; // 哨兵值, 便于后续处理
27
28      while (true)
29      {
30          vis[y] = true; // 标记当前右部点已访问
31
32          // 寻找最小松弛量和对应的右部点
33          T minSlack = high;
34          int nextY = 0;
35
36          for (int i = 1; i <= n; ++i)
37          {
38              if (vis[i])
39                  continue; // 跳过已访问的右部点
40
41              // 计算当前松弛量
42              T currentSlack = A[x] + B[i] - graph[x][i];
43
44              // 更新松弛量数组和前驱
45              if (currentSlack < slack[i])
46              {
47                  slack[i] = currentSlack;
48                  pre[i] = y; // 记录前驱
49              }
50
51              // 跟踪最小松弛量
52              if (slack[i] < minSlack)
53              {
54                  minSlack = slack[i];
55                  nextY = i;
56              }
57          }
58
59          // 调整顶标
60          A[u] -= minSlack; // 调整起始左部点顶标
61
62          for (int i = 1; i <= n; ++i)
63          {
64              if (vis[i])
65              {
66                  // 对于已访问的右部点, 调整其和匹配左部点的顶标
67                  B[i] += minSlack;

```

```

68          A[match[i]] -= minSlack;
69      }
70      else
71      {
72          // 对于未访问的右部点, 减少松弛量
73          slack[i] -= minSlack;
74      }
75  }
76
77  y = nextY; // 移动到最小松弛量对应的右部点
78
79  if (match[y] == 0)
80      break; // 找到未匹配点, 可以增广
81
82  x = match[y]; // 继续从匹配的左部点开始搜索
83  }
84
85  // 更新匹配关系
86  while (y != 0)
87  {
88      match[y] = match[pre[y]]; // 沿着前驱链回溯更新匹配
89      y = pre[y];
90  }
91  }
92
93  public:
94      /**
95       * @brief 构造函数, 初始化KM算法
96       * @param n 顶点数量
97       * @param edges 边的三元组(u, v, w)列表
98       */
99      KuhnMunkresBFS(int n, const std::vector<std::tuple<int, int, int>> &edges)
100      : n(n), graph(n + 1, std::vector<T>(n + 1, low)),
101        A(n + 1, low), B(n + 1, 0),
102        match(n + 1, 0), slack(n + 1), vis(n + 1), pre(n + 1)
103      {
104
105          // 初始化邻接矩阵
106          for (const auto &[u, v, w] : edges)
107          {
108              graph[u][v] = w;
109          }
110
111          // 初始化左部顶标为出边最大权值
112          for (int u = 1; u <= n; ++u)
113          {
114              for (int v = 1; v <= n; ++v)
115              {
116                  if (graph[u][v] > A[u])
117                  {
118                      A[u] = graph[u][v];

```

```

119     }
120 }
121 }
122 }
123
124 /**
125  * @brief 计算最大权完美匹配
126  * @return 匹配结果和总权重
127  */
128 std::pair<std::vector<int>, T> solve()
129 {
130     for (int u = 1; u <= n; ++u)
131     {
132         bfs(u); // 为每个左部顶点寻找匹配
133     }
134
135     // 计算总权重
136     T totalWeight = 0;
137     for (int v = 1; v <= n; ++v)
138     {
139         if (match[v] > 0)
140         {
141             totalWeight += graph[match[v]][v];
142         }
143     }
144     return {match, totalWeight};
145 }
146 }
147 };

```

二分图最大权匹配 我们只需要将点数少的一边补足点，并且补满边权为 0 或者负无穷大的边，即可跑 *KM* 算法求最大权完美匹配

3.14.3 一般图最大匹配

在最大匹配的基础上，代码求出了匹配必需点（可能用于一般图上博弈）。

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Edmonds
5 {
6     int n;
7     vector<vector<int>> g;
8     vector<int> match, base, p, q, inq, inb, ed;
9
10     Edmonds(int n) : n(n), g(n) {}
11
12     void addEdge(int u, int v)

```

```

13 {
14     g[u].push_back(v);
15     g[v].push_back(u);
16 }
17
18 int lca(int a, int b, vector<int> &mark)
19 {
20     fill(mark.begin(), mark.end(), 0);
21     while (true)
22     {
23         a = base[a];
24         mark[a] = 1;
25         if (match[a] == -1)
26             break;
27         a = p[match[a]];
28     }
29     while (true)
30     {
31         b = base[b];
32         if (mark[b])
33             return b;
34         if (match[b] == -1)
35             break;
36         b = p[match[b]];
37     }
38     return -1;
39 }
40
41 void markPath(int v, int b, int x)
42 {
43     while (base[v] != b)
44     {
45         inb[base[v]] = inb[base[match[v]]] = 1;
46         p[v] = x;
47         x = match[v];
48         v = p[match[v]];
49     }
50 }
51
52 int findPath(int root)
53 {
54     inq.assign(n, 0);
55     p.assign(n, -1);
56     iota(base.begin(), base.end(), 0);
57     int qh = 0, qt = 0;
58     q[qt++] = root;
59     inq[root] = 1;
60     while (qh < qt)
61     {
62         int v = q[qh++];
63         for (int u : g[v])

```

```

64     if (base[v] != base[u] && match[v] != u)
65     {
66         if ((u == root) || (match[u] != -1 && p[match[u]] != -1))
67         {
68             int curbase = lca(v, u, ed);
69             fill(inb.begin(), inb.end(), 0);
70             markPath(v, curbase, u);
71             markPath(u, curbase, v);
72             for (int i = 0; i < n; i++)
73                 if (inb[base[i]])
74                 {
75                     base[i] = curbase;
76                     if (!inq[i])
77                         q[qt++] = i, inq[i] = 1;
78                 }
79         }
80         else if (p[u] == -1)
81         {
82             p[u] = v;
83             if (match[u] == -1)
84             {
85                 while (u != -1)
86                 {
87                     v = p[u];
88                     int w = match[v];
89                     match[v] = u;
90                     match[u] = v;
91                     u = w;
92                 }
93                 return 1;
94             }
95             else
96             {
97                 q[qt++] = match[u];
98                 inq[match[u]] = 1;
99             }
100         }
101     }
102 }
103 return 0;
104 }
105
106 int maxMatching()
107 {
108     match.assign(n, -1);
109     base.resize(n);
110     q.resize(n);
111     inb.resize(n);
112     ed.resize(n);
113     int res = 0;
114     for (int i = 0; i < n; i++)

```

```

115         if (match[i] == -1)
116             res += findPath(i);
117     return res;
118 }
119
120 // 返回所有必需点
121 vector<int> essentialVertices()
122 {
123     int M = maxMatching();
124
125     // Dulmage - Mendelsohn 分解
126     vector<int> vis(n, 0);
127     queue<int> que;
128     for (int i = 0; i < n; i++)
129         if (match[i] == -1)
130         {
131             que.push(i);
132             vis[i] = 1;
133         }
134
135     while (!que.empty())
136     {
137         int v = que.front();
138         que.pop();
139         for (int u : g[v])
140         {
141             int m = match[u];
142             if (!vis[u])
143             {
144                 vis[u] = 1;
145                 if (m != -1 && !vis[m])
146                 {
147                     vis[m] = 1;
148                     que.push(m);
149                 }
150             }
151         }
152     }
153
154     // 必需点: 未被交替树覆盖的点
155     vector<int> ans;
156     for (int i = 0; i < n; i++)
157     {
158         if (!vis[i])
159             ans.push_back(i);
160     }
161     return ans;
162 }
163 };
164
165 int main()

```

```

166 {
167     ios::sync_with_stdio(false);
168     cin.tie(nullptr);
169
170     int n, m;
171     cin >> n >> m;
172     Edmonds solver(n);
173     for (int i = 0; i < m; i++)
174     {
175         int u, v;
176         cin >> u >> v;
177         --u;
178         --v;
179         solver.addEdge(u, v);
180     }
181     std::cout << solver.maxMatching() << '\n';
182     auto t = solver.match;
183     for (int i = 0; i < t.size(); ++i)
184     {
185         std::cout << t[i] + 1 << " \n"[i + 1 == t.size()];
186     }
187 }

```

3.15 环计数问题

3.15.1 无向图三元环计数

首先给所有边定向。我们规定从度数小的点指向度数大的点，度数相同就从编号小的点指向编号大的点。

那么此时此图是一张有向无环图。

然后我们枚举所有的点 u 和 u 的出点 v ，再枚举 v 的出点 w ，检查 w 和 u 是否联通即可。

可以证明复杂度为 $O(m\sqrt{m})$ 。

```

1 class TriangleCounter
2 {
3 private:
4     std::vector<std::vector<int>> adj;
5     int n;
6     int m;
7
8 public:
9     // 构造函数：接收顶点数和边数
10    TriangleCounter(int vertices, int edges) : n(vertices), m(edges), adj(n + 1) {}
11
12    // 添加边到图中
13    void addEdge(int u, int v)
14    {
15        adj[u].push_back(v);
16        adj[v].push_back(u);
17    }

```

```

18
19 // 计算图中三角形的数量
20 int countTriangles() const
21 {
22     int ans = 0;
23     std::vector<int> vis(n + 1, 0);
24
25     // 遍历每个顶点
26     for (int u = 1; u <= n; ++u)
27     {
28         // 第一阶段：标记满足条件的邻接点
29         for (int v : adj[u])
30         {
31             if (shouldPointTo(u, v))
32             {
33                 vis[v] = u;
34             }
35         }
36
37         // 第二阶段：查找三角形
38         for (int v : adj[u])
39         {
40             if (shouldPointTo(u, v))
41             {
42                 for (int w : adj[v])
43                 {
44                     if (shouldPointTo(v, w) && vis[w] == u)
45                     {
46                         ++ans;
47                     }
48                 }
49             }
50         }
51     }
52
53     return ans;
54 }
55
56 private:
57 // 判断边的方向：度数大的点指向度数小的点，或者度数相同则编号大的点指向编号小的点
58 bool shouldPointTo(int u, int v) const
59 {
60     return adj[u].size() > adj[v].size() ||
61            (adj[u].size() == adj[v].size() && u > v);
62 }
63 };

```

3.15.2 无向图四元环计数

类似地，四元环就是指四个点 a, b, c, d 满足 $(a, b)(b, c)(c, d)(d, a)$ 均有边连接。

考虑先对点进行排序。度数小的排在前面，度数大的排在后面。

考虑枚举排在最后面的点 a ，此时只需要对于每个比 a 排名更前的点 c ，都求出有多少个排名比 a 前的点 b 满足 $(a,b)(c,b)$ 有边。然后只需要这些中任取两个都能成为一个四元环。求 b 的数量只需要遍历一遍 b 和 c 即可。

注意到我们枚举的复杂度本质上与枚举三元环等价，所以时间复杂度相同。

值得注意的是， (a,b,c,d) 和 (a,c,b,d) 可以是两个不同的四元环。

另外，度数相同的结点的排名将不相同，并且需要注意判断 $a \neq c$ 。

```

1 class SquareCounter
2 {
3 private:
4     int n; // 顶点数
5     std::vector<int> deg; // 顶点度数
6     std::vector<std::vector<int>> E; // 原始无向图邻接表
7     std::vector<std::vector<int>> E1; // 有向图邻接表（度数大→小，或编号大→小）
8     std::vector<int> cnt; // 临时计数器
9
10 public:
11     // 构造函数：初始化图的大小
12     explicit SquareCounter(int vertices)
13         : n(vertices), deg(n + 1), E(n + 1), E1(n + 1), cnt(n + 1) {}
14
15     // 添加边到图中
16     void addEdge(int u, int v)
17     {
18         E[u].push_back(v);
19         E[v].push_back(u);
20         deg[u]++;
21         deg[v]++;
22     }
23
24     // 计算图中三角形数量
25     int count()
26     {
27         buildDirectedGraph();
28         return calc();
29     }
30
31 private:
32     // 构建有向图E1：度数大的点指向度数小的点，度数相同则编号大的点指向编号小的点
33     void buildDirectedGraph()
34     {
35         for (int u = 1; u <= n; ++u)
36         {
37             for (int v : E[u])
38             {
39                 if (deg[u] > deg[v] || (deg[u] == deg[v] && u > v))
40                 {
41                     E1[u].push_back(v);
42

```

```

43         }
44     }
45 }
46
47 // 处理并计算三角形数量
48 int calc()
49 {
50     int ans = 0;
51     for (int a = 1; a <= n; ++a)
52     {
53         // 遍历a的所有有向边邻居b
54         for (int b : E1[a])
55         {
56             // 遍历b的所有邻居c
57             for (int c : E[b])
58             {
59                 // 检查c是否满足条件：度数小于a或度数相同但编号小于a
60                 if (deg[c] < deg[a] || (deg[c] == deg[a] && c < a))
61                 {
62                     ans += cnt[c]++;
63                 }
64             }
65         }
66         // 重置计数器
67         for (int b : E1[a])
68         {
69             for (int c : E[b])
70             {
71                 cnt[c] = 0;
72             }
73         }
74     }
75     return ans;
76 }
77 };
78

```

3.16 最大团与最大独立集

考虑到最大独立集是补图的最大团，因此我们只需要考虑怎么求最大团。

我们考虑求最大团并计数

Solution1 折半搜索

我们考虑折半搜索，对前半部分，我们求出每个子集 S 对应的最大团，以及对应的数量。这个操作可以使用高维前缀和完成，复杂度为 $O(2^{n/2} * n/2)$ ，但我们发现我们可以考虑线性递推，具体的说，我们每次拿出 S 对应最低的位，考虑该位对应的点选不选即可。因此复杂度降为 $O(2^{n/2})$ 。对后半部分，我们只考虑判断每个子集是否为团，然后找出和前半部分的兼容掩码，更新答案即可，最终复杂度 $O(2^{n/2})$

```

1 using ll = long long;

```

```

2 struct Result
3 {
4     int size;           // 最大规模（顶点数量）
5     ll count;           // 解的总数量（所有满足最大规模的子集个数）
6     std::vector<int> nodes; // 其中一个具体解（顶点编号集合）
7 };
8
9 // 图求解器类：封装最大团和最大独立集的求解逻辑
10 class GraphSolver
11 {
12 private:
13     int n;               // 图的顶点总数（顶点编号从1开始）
14     std::vector<std::vector<bool>> adj; // 邻接矩阵: adj[u][v] = true 表示u和v之间有边
15     std::vector<int> pop; // 预处理的位计数表: popCount[mask] = mask二进制中1的个数（快速计算子集大小）
16     std::vector<int> log2; // 预处理的低位1位置表: log2[lowbit] = 该低位1对应的幂次+1（快速定位顶点编号）
17     const int bit = 25; // 最大位宽限制：确保log2表覆盖足够的掩码范围（1<=25可满足n≤50的场景）
18
19 public:
20     // 构造函数：初始化顶点数量和邻接矩阵
21     // 参数n: 图的顶点总数
22     GraphSolver(int n) : n(n), adj(n + 1, std::vector<bool>(n + 1, false))
23     {
24         // 1. 预处理位计数表popCount
25         // 最大掩码长度为前半部分顶点数: (n+1)/2（折半搜索的前半段）
26         int maxMask = 1 << ((n + 1) / 2);
27         pop.resize(maxMask, 0);
28         // 递推计算每个掩码的1的个数: mask的1的个数 = (mask去掉最低位1后的掩码)的1的个数 + 1
29         for (int i = 1; i < maxMask; ++i)
30         {
31             pop[i] = pop[i & (i - 1)] + 1;
32         }
33         // 2. 预处理log2表（定位最低位1对应的顶点）
34         log2.resize(1 << bit, 0);
35         // 对于2^i的掩码，其最低位1在第i位，对应顶点编号i+1（因顶点从1开始）
36         for (int i = 0; i < bit; ++i)
37         {
38             log2[1 << i] = i + 1;
39         }
40     }
41     void addEdge(int u, int v)
42     {
43         adj[u][v] = true; // u到v有边
44         adj[v][u] = true; // 无向图，v到u也有边
45     }
46     Result solve0() // 最大团
47     {

```

```

48         return solve(false);
49     }
50     Result solve1()
51     {
52         return solve(true);
53     }
54
55 private:
56     // 核心求解函数：统一处理最大团（原图）和最大独立集（补图）
57     Result solve(bool ok)
58     {
59         std::vector<std::vector<bool>> cur; // 当前使用的邻接矩阵（原图或补图）
60         if (ok)
61         {
62             // 构建补图：补图中u和v有边 ⇔ 原图中u和v无边（自身环设为true，不影响团的判断）
63             cur.resize(n + 1, std::vector<bool>(n + 1, false));
64             for (int i = 1; i <= n; ++i)
65             {
66                 for (int j = 1; j <= n; ++j)
67                 {
68                     if (i == j)
69                     {
70                         cur[i][j] = true; // 自身环不影响团（团要求任意两点有边，自身无需考虑）
71                     }
72                     else
73                     {
74                         cur[i][j] = !adj[i][j]; // 补图边 = 原图边取反
75                     }
76                 }
77             }
78         }
79         else
80         {
81             cur = adj; // 求解最大团，直接使用原图邻接矩阵
82         }
83
84         // ----- 步骤2: 预处理顶点邻接掩码 -----
85         int mid = n / 2; // 折半搜索：将顶点分为前半段（1~mid）和后半段（mid+1~n）
86         std::vector<ll> mask(n + 1, 0); // mask[i]: 顶点i的邻接顶点掩码（二进制位j=1表示i与j+1有边）
87         for (int i = 1; i <= n; ++i)
88         {
89             for (int j = 1; j <= n; ++j)
90             {
91                 if (cur[i][j])
92                 {
93                     // 将j-1位设为1（因顶点j对应二进制位j-1，从0开始计数）

```

```

94         mask[i] |= 1LL << (j - 1);
95     }
96 }
97
98 // ----- 步骤3: 处理前半段顶点 (动态规划预处理)
99 -----
100 int x = 1 << mid; // 前半段顶点的所有子集数 (2^mid个)
101 // f[s]: 前半段子集s对应的"最优团信息"
102 // first: 该子集能扩展出的最大团的掩码, second: 该最大团的计数
103 std::vector<std::pair<int, ll>> f(x);
104 f[0] = {0, 1}; // 空集的初始状态: 最大团为空 (掩码0), 计数1
105 // 枚举前半段所有非空子集
106 for (int s = 1; s < x; ++s)
107 {
108     int lowbit = s & -s; // 取子集s的最低位1 (对应一个顶点)
109     int prev = s ^ lowbit; // 去掉最低位1后的子集 (前一个状态)
110     int v = log2[lowbit]; // 最低位1对应的顶点编号 (从1开始)
111     // 两种选择:
112     // op1: 不选顶点v, 直接沿用prev子集的最优团信息
113     auto op1 = f[prev];
114     // op2: 选顶点v, 此时需取prev与v邻接的子集 (确保v与团中所有顶点有边), 再
115     // 加入v
116     auto op2 = f[prev & mask[v]];
117     op2.first |= lowbit; // 将v对应的位加入团掩码
118     // 选择更优的方案 (优先按团大小, 再按计数累加)
119     if (pop[op2.first] > pop[op1.first])
120     {
121         // op2的团更大, 更新为op2
122         f[s] = op2;
123     }
124     else if (pop[op2.first] == pop[op1.first])
125     {
126         // 团大小相同, 计数累加
127         f[s].first = op1.first;
128         f[s].second = op1.second + op2.second;
129     }
130     else
131     {
132         // op1的团更大, 更新为op1
133         f[s] = op1;
134     }
135 }
136
137 // ----- 步骤4: 处理后半段顶点 (筛选有效团)
138 -----
139 int y = 1 << (n - mid); // 后半段顶点的所有子集数 (2^(n-mid)个)
140 std::vector<bool> valid(y, false); // valid[s]: 标记后半段子集s是否为团
141 valid[0] = true; // 空集是团
142 // 枚举后半段所有非空子集
143 for (int s = 1; s < y; ++s)

```

```

142 {
143     int lowbit = s & -s; // 取子集s的最低位1
144     int prev = s ^ lowbit; // 去掉最低位1后的子集
145     int v = log2[lowbit] + mid; // 转换为原顶点编号 (后半段顶点从mid+1开始)
146     // 判断子集s是否为团:
147     // 1. 前一个状态prev是团 (valid[prev]为true)
148     // 2. 新加入的顶点v与s中所有其他顶点有边 (通过掩码判断: s去掉v后的部分都在v的邻接集中)
149     valid[s] = valid[prev] && ((prev & ~(mask[v] >> mid)) == 0);
150 }
151
152 // ----- 步骤5: 合并前后半段结果, 找全局最优解
153 -----
154 int maxSize = 0; // 全局最大团/独立集的规模
155 ll count = 0; // 全局最大解的总数量
156 ll bestMask = 0; // 全局最优解的顶点掩码 (二进制位i=1表示顶点i+1在解中)
157 std::vector<int> to(y, 0); // to[b]: 后半段子集b对应的前半段兼容最大团的掩码
158
159 // 枚举所有有效的后半段团
160 for (int b = 0; b < y; ++b)
161 {
162     if (!valid[b])
163         continue; // 跳过非团的子集
164
165     // 计算前半段中与后半段子集b兼容的最大团掩码t:
166     // 兼容指: t中的所有顶点与b中的所有顶点都有边
167     int t;
168     if (b == 0)
169     {
170         // 后半段是空集, 前半段可兼容的最大团是前半段所有顶点 (掩码(1<<mid)-1)
171         t = (1 << mid) - 1;
172     }
173     else
174     {
175         int lowbit = b & -b; // 取后半段子集b的最低位1
176         int prev = b ^ lowbit; // 去掉最低位1后的子集
177         int v = log2[lowbit] + mid; // 转换为原顶点编号
178         // 前半段兼容掩码 = 前一个状态的兼容掩码 & v的邻接掩码 (确保与新加入的v兼容)
179         t = to[prev] & mask[v];
180     }
181     to[b] = t; // 记录当前后半段子集b对应的前半段兼容掩码
182     // 计算当前合并后的团规模和计数
183     int curSize = pop[f[t].first] + pop[b]; // 前半段最大团大小 + 后半段团大小
184     ll curCount = f[t].second; // 前半段最大团的计数 (后半段团只有1种)
185     // 更新全局最优解
186     if (curSize > maxSize)

```

```

186     {
187         // 找到更大的解，更新规模、计数和掩码
188         maxSize = curSize;
189         count = curCount;
190         // 合并前后半段掩码：前半段掩码直接用，后半段掩码左移mid位（对应原顶
            点编号）
191         bestMask = f[t].first | (ll(b) << mid);
192     }
193     else if (curSize == maxSize)
194     {
195         // 与当前最大规模相同，累加计数
196         count += curCount;
197     }
198 }
199 // ----- 步骤6：从最优掩码中提取顶点编号
    -----
200 std::vector<int> nodes;
201 for (int i = 1; i <= n; ++i)
202 {
203     // 检查第i个顶点是否在最优掩码中（对应二进制位i-1）
204     if (bestMask & (1ULL << (i - 1)))
205     {
206         nodes.push_back(i);
207     }
208 }
209 // 返回求解结果
210 return {maxSize, count, nodes};
211 }
212 };

```

Solution2 启发式搜索

我们考虑每次找到最大的度数 max ，如果 $max \leq 2$ ，说明是环或者链，可以通过预处理好的 f 和 g 解决，否则我们选择度数最大的点递归搜索。

复杂度可以证明，上限为 $O(1.38^n)$ ，实际运行速度极快

```

1 #include <bits/stdc++.h>
2
3 using u64 = unsigned long long;
4 const int N = 55; // 最大顶点数量
5 int n, m;         // 顶点数和边数
6 int f[N], g[N];   // 存储链和环的方案数
7 u64 U;            // 表示所有顶点的集合（位掩码）
8 u64 e[N];         // 邻接表（位掩码形式）
9
10 /**
11  * 深度优先搜索寻找最大独立集
12  * @param S 当前待处理的顶点集合（位掩码）
13  * @return pair<u64, int> 最大独立集及其方案数
14  */
15 std::pair<u64, int> dfs(u64 S)
16 {

```

```

17     if (!S)
18         return {0, 1}; // 空集处理
19
20     // 选择当前集合中度数最大的顶点作为分支点
21     int w = 0, d = -1;
22     for (u64 i = S; i; i &= (i - 1))
23     {
24         int x = __builtin_ctzll(i);
25         int t = __builtin_popcountll(e[x] & S); // 计算当前顶点在集合S中的度数
26         if (t > d)
27         {
28             d = t;
29             w = x;
30         }
31     }
32
33     // 处理度数≤2的特殊情况（链或环）
34     if (d <= 2)
35     {
36         u64 v = S, x = 0, s1, s2;
37         int y = 1;
38         bool o;
39
40         // 内部DFS用于探索连通分量并进行二分图着色
41         auto dfs = [&](auto &&sf, int x, bool o) -> void
42         {
43             v ^= (1ull << x); // 标记顶点x为已处理
44             (o ? s2 : s1) |= 1ull << x; // 按颜色分组
45             o &= (__builtin_popcountll(e[x] & S) == 2); // 检查是否为环
46             // 递归访问所有未处理的邻居
47             while (v & e[x])
48             {
49                 sf(sf, __builtin_ctzll(v & e[x]), o ^ 1);
50             }
51         };
52
53         // 处理所有连通分量
54         while (v)
55         {
56             s1 = s2 = 0;
57             o = 1; // 初始假设为环
58             dfs(dfs, __builtin_ctzll(v), 0); // 从最低位顶点开始处理
59
60             int a = __builtin_popcountll(s1);
61             int b = __builtin_popcountll(s2);
62             int num = a + b;
63
64             // 选择最大独立集并累计算法数
65             if (0 && (num & 1))
66             { // 奇环
67                 x |= (a < b ? s1 : s2);

```



```

68     }
69     else
70     { // 偶环或链
71         x |= (a > b ? s1 : s2);
72     }
73     y *= (0 ? g : f)[num];
74 }
75 return {x, y};
76 }
77
78 // 常规情况：分支递归
79 u64 W = 1ull << w;
80 std::pair<u64, int> nw = dfs(S ^ W); // 不选顶点w
81
82 // 选择顶点w（需排除其邻居）
83 auto [x, y] = dfs(S & (~(e[w] | W)));
84 x |= W;
85
86 // 比较两种情况，选择最优解
87 if (__builtin_popcountll(x) > __builtin_popcountll(nw.first))
88 {
89     nw = {x, y};
90 }
91 else if (__builtin_popcountll(x) == __builtin_popcountll(nw.first))
92 {
93     nw.second += y;
94 }
95
96 return nw;
97 }
98 // 预处理f和g数组（链和环的方案数）
99 void preprocess()
100 {
101     // f[i]: i个顶点的链的最大独立集方案数
102     // g[i]: i个顶点的环的最大独立集方案数
103     for (int i = 1; i <= n; ++i)
104     {
105         f[i] = (i & 1) ? 1 : (i / 2 + 1); // 链的方案数公式
106         g[i] = (i & 1) ? i : 2;          // 环的方案数公式
107     }
108 }
109 void exchange() // 翻转图
110 {
111     for (int i = 0; i < n; ++i)
112     {
113         // 补图的邻接集 = 全集排除自身和原图邻接集
114         e[i] = ((~e[i]) & U) ^ (1ull << i);
115     }
116 }
117
118 int main()

```

```

119 {
120     std::ios::sync_with_stdio(0);
121     std::cin.tie(0);
122
123     std::cin >> n >> m;
124     U = (1ull << n) - 1;
125     while (m--)
126     {
127         int u, v;
128         std::cin >> u >> v;
129         --u;
130         --v;
131         e[u] |= (1ull << v);
132         e[v] |= (1ull << u);
133     }
134     preprocess();
135     exchange();
136     auto [val, cnt] = dfs(U);
137     std::cout << __builtin_popcountll(val) << ' ' << cnt << '\n';
138     while (val)
139     {
140         std::cout << __builtin_ctzll(val) + 1 << ' ';
141         val ^= (val & (-val));
142     }
143     std::cout << '\n';
144     exchange();
145     auto [x, y] = dfs(U);
146     val = x, cnt = y;
147     std::cout << __builtin_popcountll(val) << ' ' << cnt << '\n';
148     while (val)
149     {
150         std::cout << __builtin_ctzll(val) + 1 << ' ';
151         val ^= (val & (-val));
152     }
153 }

```

3.17 LCA | 最近公共祖先

```

1 struct LCA {
2     int n;
3     vector<int> dfn;
4     vector<int> data[20];
5
6     LCA(int n, int root, const auto& G) : dfn(n + 1) {
7         for (int i = 0; i <= __lg(n); i++)
8             data[i].assign(n - (1 << i) + 2, 0);
9         int t = 0;
10        const auto dfs = [&](auto&& self, int u, int p) -> void {
11            data[0][dfn[u] = ++t] = p;

```

```

12     for (auto v : G[u]) if (v != p) self(self, v, u);
13 };
14 dfs(dfs, root, 0);
15 for (int k = 1; k <= __lg(n); k++) {
16     for (int i = 1; i + (1 << k) - 1 <= n; i++) {
17         data[k][i] = cmp(data[k - 1][i], data[k - 1][i + (1 << (k - 1))]);
18     }
19 }
20 }
21
22 int cmp(int l, int r) {
23     return dfn[l] < dfn[r] ? l : r;
24 }
25 int ask(int u, int v) {
26     if (u == v) return u;
27     u = dfn[u], v = dfn[v];
28     if (u < v) swap(u, v);
29     int k = __lg(u - v);
30     return cmp(data[k][v + 1], data[k][u - (1 << k) + 1]);
31 }
32 };

```

```

3 vector<int> low(n + 1);
4 vector<int> cut(2 * m + 2);
5 int timer = 0;
6 const auto tarjan = [&](auto&& self, int u, int pid) -> void {
7     dfn[u] = low[u] = ++timer;
8     for (auto to : G[u]) {
9         if (to.id == (pid ^ 1)) continue;
10        if (!dfn[to.v]) {
11            self(self, to.v, to.id);
12            low[u] = min(low[u], low[to.v]);
13            if (low[to.v] > dfn[u])
14                cut[to.id] = cut[to.id ^ 1] = 1;
15        } else {
16            low[u] = min(low[u], dfn[to.v]);
17        }
18    }
19 };
20 for (int u = 1; u <= n; u++)
21     if (!dfn[u]) tarjan(tarjan, u, 0);
22 return move(cut);
23 };

```

3.18 Cycle count | 普通环计数

时间复杂度 $O(2^n \cdot m)$, 注意需要是 0-based。

```

1 i64 cycle_count(int n, int m, const vector<vector<int>>& G) {
2     i64 res = 0;
3     vector<vector<i64>> f((1 << n), vector<i64>(n));
4     for (int i = 0; i < n; i++) f[1 << i][i] = 1;
5     for (int s = 1; s < (1 << n); s++) {
6         for (int i = 0; i < n; i++) {
7             if (!f[s][i]) continue;
8             for (auto j : G[i]) {
9                 if ((s & -s) > (1 << j)) continue;
10                if (s >> j & 1) {
11                    if ((s & -s) == (1 << j)) res += f[s][i];
12                } else f[s | (1 << j)][j] += f[s][i];
13            }
14        }
15    }
16    return (res - m) >> 1;
17 }

```

3.19 Cut edge | 割边

```

1 auto tarjan_cut_edge(int n, int m, const auto& G) {
2     vector<int> dfn(n + 1);

```

3.20 Cut node | 割点

```

1 auto tarjan_cut_node(int n, const auto& G) {
2     vector<int> dfn(n + 1);
3     vector<int> low(n + 1);
4     vector<int> cut(n + 1);
5     int timer = 0, root = 0;
6     const auto tarjan = [&](auto&& self, int u) -> void {
7         int cnt_son = 0;
8         dfn[u] = low[u] = ++timer;
9         for (auto to : G[u]) {
10            if (!dfn[to.v]) {
11                ++cnt_son;
12                self(self, to.v);
13                low[u] = min(low[u], low[to.v]);
14                if (low[to.v] >= dfn[u] && u != root)
15                    cut[u] = 1;
16            } else {
17                low[u] = min(low[u], dfn[to.v]);
18            }
19        }
20        if (cnt_son >= 2 && u == root)
21            cut[u] = 1;
22    };
23    for (int u = 1; u <= n; u++)
24        if (!dfn[u]) tarjan(tarjan, root = u);
25    return move(cut);

```

26 }

3.21 SCC | 强连通分量

```

1 auto tarjan_scc(int n, const auto& G) {
2     vector<int> dfn(n + 1);
3     vector<int> low(n + 1);
4     vector<int> scc(n + 1);
5     int timer = 0, scc_cnt = 0;
6     const auto tarjan = [&](auto&& self, int u) -> void {
7         static vector<int> stk;
8         dfn[u] = low[u] = ++timer;
9         stk.push_back(u);
10        for (auto to : G[u]) {
11            if (!dfn[to.v]) {
12                self(self, to.v);
13                low[u] = min(low[u], low[to.v]);
14            } else if (!scc[to.v]) {
15                low[u] = min(low[u], dfn[to.v]);
16            }
17        }
18        if (dfn[u] == low[u] && ++scc_cnt) {
19            while (!stk.empty()) {
20                auto v = stk.back();
21                stk.pop_back();
22                scc[v] = scc_cnt;
23                if (v == u) break;
24            }
25        }
26    };
27    for (int u = 1; u <= n; u++)
28        if (!dfn[u]) tarjan(tarjan, u);
29    return move(scc);
30 }

```

3.22 EBCC | 边双连通分量

```

1 auto tarjan_ebcc(int n, const auto& G) {
2     vector<int> dfn(n + 1);
3     vector<int> low(n + 1);
4     vector<int> ebcc(n + 1);
5     int timer = 0, ebcc_cnt = 0;
6     const auto tarjan = [&](auto&& self, int u, int pid) -> void {
7         static vector<int> stk;
8         dfn[u] = low[u] = ++timer;
9         stk.push_back(u);
10        for (auto to : G[u]) {

```

```

11        if (to.id == (pid ^ 1)) continue;
12        if (!dfn[to.v]) {
13            self(self, to.v, to.id);
14            low[u] = min(low[u], low[to.v]);
15        } else if (!ebcc[to.v]) {
16            low[u] = min(low[u], dfn[to.v]);
17        }
18    }
19    if (dfn[u] == low[u] && ++ebcc_cnt) {
20        while (!stk.empty()) {
21            auto v = stk.back();
22            stk.pop_back();
23            ebcc[v] = ebcc_cnt;
24            if (v == u) break;
25        }
26    }
27 };
28 for (int u = 1; u <= n; u++)
29     if (!dfn[u]) tarjan(tarjan, u, 0);
30 return move(ebcc);
31 }

```

3.23 VBCC | 点双连通分量 (点集)

```

1 auto tarjan_vbcc(int n, const auto& G) {
2     vector<int> dfn(n + 1);
3     vector<int> low(n + 1);
4     vector<vector<int>> vbcc;
5     int timer = 0;
6     const auto tarjan = [&](auto&& self, int u) -> void {
7         static vector<int> stk;
8         dfn[u] = low[u] = ++timer;
9         if (G[u].empty()) return vbcc.push_back({u});
10        else stk.push_back(u);
11        for (auto to : G[u]) {
12            if (!dfn[to.v]) {
13                self(self, to.v);
14                low[u] = min(low[u], low[to.v]);
15                if (low[to.v] >= dfn[u]) {
16                    vbcc.push_back({});
17                    while (!stk.empty()) {
18                        auto p = stk.back();
19                        stk.pop_back();
20                        vbcc.back().push_back(p);
21                        if (p == to.v) break;
22                    }
23                    vbcc.back().push_back(u);
24                }
25            } else {

```

```

26     low[u] = min(low[u], dfn[to.v]);
27 }
28 }
29 };
30 for (int u = 1; u <= n; u++)
31     if (!dfn[u]) tarjan(tarjan, u);
32 return move(vbcc);
33 }

```

3.24 VBCC | 点双连通分量 (边集)

```

1 auto tarjan_vbcc(int n, const auto& G) {
2     vector<int> dfn(n + 1);
3     vector<int> low(n + 1);
4     vector<vector<int>> vbcc;
5     int timer = 0;
6     const auto tarjan = [&](auto&& self, int u, int preid) -> void {
7         static vector<int> stk;
8         dfn[u] = low[u] = ++timer;
9         for (auto to : G[u]) {
10             if (to.id == preid) continue;
11             if (!dfn[to.v]) {
12                 stk.push_back(to.id);
13                 self(self, to.v, to.id);
14                 low[u] = min(low[u], low[to.v]);
15                 if (low[to.v] >= dfn[u]) {
16                     vbcc.push_back({});
17                     while (!stk.empty()) {
18                         auto p = stk.back();
19                         stk.pop_back();
20                         vbcc.back().push_back(p);
21                         if (p == to.id) break;
22                     }
23                 }
24             } else if (dfn[to.v] < dfn[u]) {
25                 stk.push_back(to.id);
26                 low[u] = min(low[u], dfn[to.v]);
27             }
28         }
29     };
30     for (int u = 1; u <= n; u++)
31         if (!dfn[u]) tarjan(tarjan, u, -1);
32     return move(vbcc);
33 }

```

3.25 2-SAT | 2-SAT

```

1 vector<int> g[N];
2 int dfn[N], low[N], ins[N], idx, cnt;
3 int col[N];
4 stack<int> st;
5 void add(int x, int y) {
6     g[(int)(abs(x)) + n * (x < 0)].push_back((int)(abs(y)) + n * (y > 0));
7     g[(int)(abs(y)) + n * (y < 0)].push_back((int)(abs(x)) + n * (x > 0));
8     // 由正负号判断是否取反。
9 }
10 void dfs(int u) {
11     dfn[u] = low[u] = ++idx;
12     ins[u] = true;
13     st.push(u);
14     for (auto v : g[u]) {
15         if (dfn[v] == 0) dfs(v);
16         if (ins[v]) low[u] = min(low[u], low[v]);
17     }
18     if (dfn[u] == low[u]) {
19         vector<int> t;
20         cnt++;
21         while (true) {
22             int x = st.top();
23             t.push_back(x);
24             ins[x] = false;
25             col[x] = cnt;
26             st.pop();
27             if (u == x) break;
28         }
29     }
30 }
31
32 for (int i = 1; i <= m; i++) {
33     int a, x, b, y;
34     cin >> a >> x >> b >> y;
35     if (x == 0 && y == 0) {
36         g[a + n].push_back(b);
37         g[b + n].push_back(a);
38     }
39     if (x == 0 && y == 1) {
40         g[a + n].push_back(b + n);
41         g[b].push_back(a);
42     }
43     if (x == 1 && y == 0) {
44         g[a].push_back(b);
45         g[b + n].push_back(a + n);
46     }
47     if (x == 1 && y == 1) {
48         g[a].push_back(b + n);
49         g[b].push_back(a + n);
50     }
51 }

```

```

52 for(int i =1;i<=2*n;i++){
53     if(!dfn[i]) dfs(i);
54 }
55 for(int i=1;i<=n;i++){
56     if(col[i] == col[i + n]){
57         cout << "IMPOSSIBLE\n";
58         return;
59     }
60 }
61 cout << "POSSIBLE\n";
62 for(int i=1;i<=n;i++){
63     cout << (col[i] > col[i + n]) << ' ';
64 }

```

3.26 Virtual tree | 虚树

```

1 auto build_virtual_tree = [&]() {
2     vector<int> a = h;
3     auto cmp = [&](int l, int r) { return dfn[l] < dfn[r]; };
4     sort(h.begin(), h.end(), cmp);
5     for (int i = 1; i < a.size(); i++) a.push_back(lca(h[i], h[i - 1]));
6     sort(a.begin(), a.end(), cmp);
7
8     a.erase(unique(a.begin(), a.end(), a.end());
9     for (int i = 1; i < a.size(); i++) add_edge(lca(a[i], a[i - 1]], a[i]);
10 };

```

4 Dynamic Programming | 动态规划

4.1 Subsequence DP | 子序列问题

4.1.1 LIS / LDS / 不升 / 不降子序列

最长上升 / 下降 / 不升 / 不降子序列可用二分 + 贪心在 $O(n \log n)$ 内求出。典型做法维护一个数组 f , $f[k]$ 为长度为 k 的子序列的最小末尾值。

4.1.2 LCS | 最长公共子序列

给定字符串 A, B , 常见做法有:

1. 经典二维 DP: $dp[i][j]$ 表示 A 前 i 位与 B 前 j 位的 LCS 长度, 复杂度 $O(nm)$ 。
2. 若某一串中字符不重复, 可用 LIS 化简: 将另一串按该串重标并删去无关字符, 在新序列上做 LIS。
3. bitset + DP 的 bit-parallel 做法, 可降到 $O(|A| \cdot |B|/w)$ 。下面给出字母集为小写英文字母的 bit-parallel LCS 模板。

```

1 using u64 = unsigned long long;
2
3 // 64 位块数表示的 bitset LCS (Hunt - Szymanski 风格 bit-parallel)
4 // 时间  $O(|a| * ((|b| + 63) / 64))$ , 空间  $O(26 * \text{blocks})$ 
5 int LCS(const std::string &a, const std::string &b) {
6     int n = (int)a.size(), m = (int)b.size();
7     if (n == 0 || m == 0) return 0;
8
9     int W = (m + 63) / 64; // 每个 bitset 需要的 u64 个数
10    std::vector<std::vector<u64>> pos(26, std::vector<u64>(W, 0ULL));
11
12    // 构造 pos: pos[c] 在 b 中字符 c 出现位置的位图
13    for (int j = 0; j < m; ++j) {
14        int c = b[j] - 'a';
15        pos[c][j >> 6] |= (1ULL << (j & 63));
16    }
17
18    std::vector<u64> dp(W, 0ULL), U(W), V(W), SUB(W);
19    // 最后一块只保留到 m 的有效位
20    u64 last_mask = (m % 64 == 0) ? ~0ULL : ((1ULL << (m % 64)) - 1ULL);
21
22    for (char ch : a) {
23        int idx = ch - 'a';
24
25        // U = dp | pos[idx]
26        for (int i = 0; i < W; ++i) {
27            U[i] = dp[i] | pos[idx][i];
28        }
29
30        // V = (dp << 1) | 1
31        u64 carry = 0;
32        for (int i = 0; i < W; ++i) {
33            u64 next_carry = dp[i] >> 63;
34            V[i] = (dp[i] << 1) | carry;
35            carry = next_carry;
36        }
37        V[0] |= 1ULL; // 位置 0 视作空前缀
38
39        // SUB = U - V (多块减法, 考虑借位)
40        u64 borrow = 0ULL;
41        for (int i = 0; i < W; ++i) {
42            unsigned __int128 v_with_borrow =
43                (unsigned __int128)V[i] + (unsigned __int128)borrow;
44            u64 v64 = (u64)v_with_borrow;
45            u64 res = (u64)((unsigned __int128)U[i] - v_with_borrow);
46            SUB[i] = res;
47            borrow = (v_with_borrow > (unsigned __int128)U[i]) ? 1ULL : 0ULL;
48        }
49
50        // dp = U & ~SUB
51        for (int i = 0; i < W; ++i) {

```

```

52     dp[i] = U[i] & ~SUB[i];
53 }
54 // 清除最后块中超出 m 的位
55 dp[W - 1] &= last_mask;
56 }
57
58 // 统计 dp 中 1 的个数就是 LCS 长度
59 int ans = 0;
60 for (int i = 0; i < W; ++i) {
61     ans += __builtin_popcountll(dp[i]);
62 }
63 return ans;
64 }

```

4.2 Partition of integers | 自然数划分问题

考虑 n 个自然数之和为 m 的方案数。设 $f(i, j)$ 表示选了 i 个数总和为 j 的方案数。

4.2.1 可重复

讨论是否包含数字 1:

1. 至少包含一个 1, 则去掉一个 1 后为 $f(i-1, j-1)$ 。
2. 不包含 1, 则让当前 i 个数全部减去 1, 和变为 $j-i$, 即 $f(i, j-i)$ 。

转移为 $f(i, j) = f(i-1, j-1) + f(i, j-i)$, 复杂度 $O(nm)$ 。

更高阶做法是利用五边形数定理, 复杂度 $O(m\sqrt{m})$ 。下面是一个以五边形数为核心的划分数模板 (需配合取模函数 `norm`)。

```

1 std::vector<int> calcPartition(int n) {
2     std::vector<long long> a(n), p(n);
3     p[0] = 1;
4     if (n > 1) p[1] = 1;
5     if (n > 2) p[2] = 2;
6     for (int i = 1; 2 * i + 1 < n; ++i) {
7         // 五边形数 1, 5, 12, 22, ...
8         a[2 * i] = 1LL * i * (3 * i - 1) / 2;
9         a[2 * i + 1] = 1LL * i * (3 * i + 1) / 2;
10    }
11    for (int i = 3; i < n; ++i) {
12        for (int j = 2; a[j] <= i; ++j) {
13            // p[n] = p[n-1] + p[n-2] - p[n-5] - p[n-7] + ...
14            p[i] += ((j & 2) ? 1 : -1) * p[i - a[j]];
15            norm(p[i]); // 视需要取模
16        }
17    }
18    std::vector<int> res(n);
19    for (int i = 0; i < n; ++i) res[i] = (int)p[i];
20    return res;
21 }

```

4.2.2 不可重复

同样按是否包含 1 分类:

1. 恰好包含一个 1, 则把每个数加 1, 相当于 $f(i-1, j-i)$ 。
2. 不包含 1, 则同样每个数加 1, 对应 $f(i, j-i)$ 。

于是转移为 $f(i, j) = f(i-1, j-i) + f(i, j-i)$ 。

4.3 Knapsack DP | 背包 DP

4.3.1 Multiple knapsack with monotone queue | 多重背包 (单调队列优化)

经典多重背包可以拆成若干 01 背包, 或用单调队列对每个模 w 的同余类优化。下面是按体积模 w 分组、对每个组使用单调队列的模板。

```

1 using i64 = long long;
2
3 auto multiBag(std::vector<std::array<int, 3>> &goods, int S) {
4     // S 为总背包容量, goods: {value, weight, count}
5     std::vector<i64> f(S + 1, 0);
6     std::vector<int> q(S + 2, 0);
7
8     for (auto [v, w, m] : goods) {
9         auto calc = [&](int i, int j) {
10             // i, j 为同余类中的两个体积, i >= j
11             return f[j] + 1LL * (i - j) / w * v;
12         };
13         for (int up = S; up + w > S; --up) {
14             int l = 1, r = 0, k = up;
15             for (int x = up; x > 0; x -= w) {
16                 for (; k >= std::max(0LL, 1LL * x - 1LL * m * w); k -= w) {
17                     while (l <= r && calc(x, k) > calc(x, q[r])) --r;
18                     q[++r] = k;
19                 }
20                 f[x] = calc(x, q[l]);
21                 if (q[l] == x) ++l;
22             }
23         }
24     }
25     return f;
26 }

```

4.3.2 Tree knapsack | 树上背包

树上背包常见于“在树上选点 / 边满足某种限制”的计数或最值题。一般在 DFS 时, 用临时数组 `temp` 合并子树贡献, 避免子树间的重复转移。

```

1 template <class T>
2 class PackageOnTree {
3     const T INF = std::numeric_limits<T>::max() / 2;
4
5 public:
6     std::vector<std::vector<T>> dp;
7
8     PackageOnTree(int n, int m, int root)
9         : n(n), g(n + 1), siz(n + 1),
10           infCapacity(m), root(root) {
11         dp.assign(n + 1, std::vector<T>(m + 1, 0));
12     }
13
14     void addEdge(int u, int v, T w) {
15         g[u].emplace_back(v, w);
16         g[v].emplace_back(u, w);
17     }
18
19     void work() { dfs(root, 0); }
20
21 private:
22     int infCapacity;
23     int n, root;
24     std::vector<std::vector<std::pair<int, T>>> g;
25     std::vector<int> siz;
26
27     // u 子树内选 cnt 个点, 父边权为 w 的贡献
28     T calc(int u, T w, int cnt) {
29         return w * (cnt * (infCapacity - cnt)
30             + (siz[u] - cnt) * (n - infCapacity - (siz[u] - cnt)));
31     }
32
33     void dfs(int u, int fa) {
34         dp[u][0] = 0;
35         siz[u] = 1;
36         for (auto [v, w] : g[u]) {
37             if (v == fa) continue;
38             dfs(v, u);
39             siz[u] += siz[v];
40             std::vector<T> temp(std::min(siz[u], infCapacity) + 1, 0);
41             for (int j = std::min(infCapacity, siz[u]); j >= 0; --j) {
42                 for (int k = std::max(0, siz[v] + j - siz[u]);
43                     k <= std::min(siz[v], j); ++k) {
44                     temp[j] = std::max(
45                         temp[j],
46                         dp[u][j - k] + dp[v][k] + calc(v, w, k)
47                     );
48                 }
49             }
50             for (int j = 0; j <= std::min(siz[u], infCapacity); ++j) {
51                 dp[u][j] = temp[j];

```

```

52     }
53 }
54 };
55

```

4.4 Interval DP | 区间 DP

区间 DP 一般形如:

$$dp[i][j] = \min_{i \leq k < j} (dp[i][k] + dp[k+1][j] + \text{cost}(i, j)),$$

复杂度通常为 $O(n^3)$ 。常用技巧包括四边形不等式 / 决策单调性优化。

4.5 Monotone queue optimization | 单调队列优化

若 DP 形如 $dp[i] = \min_j (dp[j] + \text{cost}(j, i))$, 且对每个 i 可行转移 j 在某个滑动窗口内, 则可用单调队列维护“候选 j 的代价”, 将复杂度降为线性。

4.6 Monge decision optimization | 决策单调性优化

4.6.1 2D form

典型二维形式:

$$dp[i][k] = \min_{1 \leq j < i} \{dp[j][k-1] + w(j+1, i)\},$$

若满足决策单调性, 即最优决策点在层间单调, 则可用分治或二分队列优化。

Divide and conquer 优化 在第 k 层中, 用分治求出区间 $[l, r]$ 的最优决策点在 $[L, R]$ 区间内, 复杂度 $O(nk \log n)$ 。

```

1 template <class T>
2 class OptimizeDP2D {
3     const T INF = std::numeric_limits<T>::max() / 2;
4
5 public:
6     std::vector<std::vector<T>> dp;
7     int n, K;
8
9     OptimizeDP2D(int n, int k) : n(n), K(k) {
10         dp.assign(n + 1, std::vector<T>(k + 1, 0));
11         for (int j = 1; j <= k; ++j) {
12             work(1, n, 1, n, j);
13         }
14     }
15
16 private:
17     // 代价函数 w(l, r) 需按题意实现
18     T calcW(int l, int r) {

```

```

19 // TODO: 按题目实现
20 return 0;
21 }
22
23 // 在第 dep 层, 求解区间 [l,r], 决策点限制在 [L,R]
24 void work(int l, int r, int L, int R, int dep) {
25     if (l > r) return;
26     int mid = (l + r) >> 1;
27     T best = -INF;
28     int opt = -1;
29     for (int i = std::min(R, mid); i >= L; --i) {
30         T val = dp[i - 1][dep - 1] + calcW(i, mid);
31         if (val > best) {
32             best = val;
33             opt = i;
34         }
35     }
36     dp[mid][dep] = best;
37     work(l, mid - 1, L, opt, dep);
38     work(mid + 1, r, opt, R, dep);
39 }
40 };

```

Binary-search queue 优化 也可用“决策区间 + 队列”维护三元组 $[j, l, r]$, 在队列上二分找到每个位置的最优决策点, 复杂度同样为 $O(nk \log n)$ 。代码与 1D 形式极为相似, 此处不再赘述。

4.6.2 1D form

一维形式:

$$dp[i] = \min_{1 \leq j < i} \{dp[j] + w(j+1, i)\},$$

决策单调性出现在同一层内部, 无法分治, 只能使用二分队列方法维护一个凸壳队列。

```

1 template <class T>
2 class OptimizeDP1D {
3     const T INF = std::numeric_limits<T>::max() / 2;
4
5 public:
6     std::vector<T> dp;
7
8     explicit OptimizeDP1D(int n) : n(n) {
9         dp.assign(n + 1, INF);
10        dp[0] = 0;
11        work();
12    }
13
14 private:
15     int n;
16

```

```

17 T getW(int l, int r) {
18     // TODO: 按题意实现 w(l, r)
19     return 0;
20 }
21
22 T getDP(int l, int r) {
23     return dp[l] + getW(l + 1, r);
24 }
25
26 void work() {
27     std::vector<std::array<int, 3>> q(n + 1);
28     // q[t] = {j, L, R} 表示在 [L,R] 内的最优转移点为 j
29     int hh = 1, tt = 0;
30     q[++tt] = {0, 1, n};
31     for (int i = 1; i <= n; ++i) {
32         while (hh < tt && q[hh][2] < i) ++hh;
33         q[hh][1] = i;
34         int k = q[hh][0];
35         dp[i] = getDP(k, i);
36
37         while (hh < tt && getDP(q[tt][0], q[tt][1]) >= getDP(i, q[tt][1])) {
38             --tt;
39         }
40         q[tt][2] = n;
41         int L = q[tt][1], R = q[tt][2];
42         int pos = n + 1;
43         while (L <= R) {
44             int mid = (L + R) >> 1;
45             if (getDP(q[tt][0], mid) >= getDP(i, mid)) {
46                 R = mid - 1;
47                 pos = mid;
48             } else {
49                 L = mid + 1;
50             }
51         }
52         if (pos != n + 1) {
53             q[tt][2] = pos - 1;
54             q[++tt] = {i, pos, n};
55         }
56     }
57 }
58 };

```

4.7 Slope trick / Convex hull optimization | 斜率优化

斜率优化适用于形如

$$dp[i] = \min_j (dp[j] + w_1(i) + w_2(j) + f(i)g(j))$$

的 DP。令 $y = dp[j] + w_2(j)$, $x = g(j)$, $k = -f(i)$, $b = dp[i] - w_1(i)$, 可化为直线形式 $y = kx + b$,

问题转化为在一堆点 (x, y) 中找使得 b 最优的点。若 x 与 k 都具备单调性，可用单调队列维护凸壳；若只有 x 单调，则使用二分队列。

4.7.1 Monotone queue version

```

1 template <class T>
2 class ConvexHullTrick {
3     const T INF = std::numeric_limits<T>::max() / 2;
4
5 public:
6     std::vector<T> dp;
7
8     explicit ConvexHullTrick(int n) : n(n) {
9         dp.assign(n + 1, INF);
10        dp[0] = 0;
11        work();
12    }
13
14 private:
15     int n;
16
17     T calcX(int x) {
18         // TODO: 按题意实现
19         return 0;
20     }
21
22     T calcY(int x) {
23         // TODO: 按题意实现
24         return 0;
25     }
26
27     T calcK(int x) {
28         // TODO: 按题意实现斜率 k(i)
29         return 0;
30     }
31
32     long double calcSlope(int l, int r) {
33         if (calcX(r) == calcX(l)) return INF;
34         return (long double)(calcY(r) - calcY(l)) /
35             (long double)(calcX(r) - calcX(l));
36     }
37
38     T calcVal(int i, int j) {
39         // TODO: 返回用 j 转移到 i 的代价
40         return 0;
41     }
42
43     void work() {
44         int hh = 0, tt = -1;
45         std::vector<int> q(n + 1, 0);

```

```

46         q[+tt] = 0;
47         for (int i = 1; i <= n; ++i) {
48             T k = calcK(i);
49             while (hh < tt && calcSlope(q[hh], q[hh + 1]) <= k) ++hh;
50             int j = q[hh];
51             dp[i] = dp[j] + calcVal(i, j);
52             while (hh < tt &&
53                 calcSlope(q[tt - 1], q[tt]) >= calcSlope(q[tt], i)) {
54                 --tt;
55             }
56             q[+tt] = i;
57         }
58     }
59 };

```

4.7.2 Binary-search queue version

```

1 template <class T>
2 class ConvexHullTrickBS {
3     const T INF = std::numeric_limits<T>::max() / 2;
4
5 public:
6     std::vector<T> dp;
7
8     explicit ConvexHullTrickBS(int n) : n(n) {
9         dp.assign(n + 1, INF);
10        dp[0] = 0;
11        work();
12    }
13
14 private:
15     int n;
16
17     T calcX(int x) { return 0; } // TODO
18     T calcY(int x) { return 0; } // TODO
19     T calcK(int x) { return 0; } // TODO
20
21     long double calcSlope(int l, int r) {
22         if (calcX(r) == calcX(l)) return INF;
23         return (long double)(calcY(r) - calcY(l)) /
24             (long double)(calcX(r) - calcX(l));
25     }
26
27     T calcVal(int i, int j) { return 0; } // TODO
28
29     void work() {
30         int hh = 0, tt = -1;
31         std::vector<int> q(n + 1, 0);
32         q[+tt] = 0;
33     }

```

```

34 auto findOptimizePoint = [&](int L, int R, T k) {
35     int ans = R;
36     while (L <= R) {
37         int mid = (L + R) >> 1;
38         if (calcSlope(q[mid], q[mid + 1]) > k) {
39             R = mid - 1;
40             ans = mid;
41         } else {
42             L = mid + 1;
43         }
44     }
45     return q[ans];
46 };
47
48 for (int i = 1; i <= n; ++i) {
49     T k = calcK(i);
50     int j = findOptimizePoint(0, tt - 1, k);
51     dp[i] = dp[j] + calcVal(i, j);
52     while (hh < tt &&
53           calcSlope(q[tt - 1], q[tt]) >= calcSlope(q[tt], i)) {
54         --tt;
55     }
56     q[++tt] = i;
57 }
58 }
59 };

```

4.8 WQS binary / weight binary search | WQS 二分

WQS（二分斜率）用于解决“强制选恰好 m 个物品”的最优方案，前提是 $f(m)$ 对 m 是凸的（一次差分单调）。

通过给每个被选物品减去一个惩罚权重 λ ，将“选 m 个”的限制吸收到代价中，变成不限制个数的简单 DP，并根据最后选的个数与目标 m 的关系二分 λ 。

4.9 Dynamic DP | 动态 DP

引入广义矩阵乘法 $C = A * B$ ，定义

$$C_{ij} = \max_k (A_{ik} + B_{kj})$$

很多从左到右的线性 DP 可写成广义矩阵乘，与线段树结合可支持区间修改与查询。例：

$$f(i) = \max(f(i-1) + a_i, b_i),$$

可写作

$$\begin{bmatrix} a_i & b_i \\ -\infty & 0 \end{bmatrix} * \begin{bmatrix} f(i-1) \\ 0 \end{bmatrix} = \begin{bmatrix} f(i) \\ 0 \end{bmatrix}.$$

4.10 Bitmask DP | 状压 DP

4.10.1 SOS DP（子集和 DP）

要在所有 $0 \leq i < 2^n$ 上求 $\sum_{j \subseteq i} a_j$ ，朴素枚举子集复杂度 $O(3^n)$ 。SOS DP 的优化为：

```

1 // 枚举子集的朴素写法
2 for (int s = 0; s < (1 << n); ++s) {
3     for (int T = s; T; T = (T - 1) & s) {
4         dp[s] += a[T];
5     }
6 }
7
8 // SOS DP 优化: dp[i] 最终为所有子集的和
9 for (int j = 0; j < n; ++j) {
10     for (int i = 0; i < (1 << n); ++i) {
11         if (i >> j & 1) {
12             dp[i] += dp[i ^ (1 << j)];
13         }
14     }
15 }

```

4.11 Digit DP | 数位 DP

数位 DP 一般按位从高到低枚举，用记忆化搜索解决。常见状态参数包括：

1. pos: 当前处理的位数。
2. limit: 本位是否受上界限制。
3. lead0: 前缀是否全是前导零。
4. last: 上一位的取值（如有相邻位约束）。
5. sum / r / st: 数位和、模值、状态压缩等。

```

1 class DigitDP {
2 public:
3     DigitDP(int maxLen, int base)
4         : a(maxLen + 1), base(base) {
5         // f 按题意开维度并初始化为 -1
6     }
7
8     int solve(int L, int R) {
9         return calc(R) - calc(L - 1);
10    }
11
12 private:
13     std::vector<int> a;
14     // 具体形状视题目而定，例如 f[pos][sum][r][...]
15     // std::vector<std::vector<int>>> f;
16     int base;
17
18     // pos: 当前位（从高到低），limit: 是否受上界限制

```

```

19 // lead0: 是否仍在先导零, r/sum/last/st 等按题目添加
20 int dfs(int pos, bool limit, bool lead0,
21         int r, int last, int sum) {
22     if (pos == 0) {
23         // 判断当前数是否满足题目要求
24         if (/* condition */) {
25             return 1; // 或返回 sum 等属性
26         } else {
27             return 0;
28         }
29     }
30     // 若不受限制且已记忆, 直接返回
31     // if (!limit && f[pos][...] != -1) return f[pos][...];
32
33     int up = limit ? a[pos] : base - 1;
34     int res = 0;
35     for (int d = 0; d <= up; ++d) {
36         bool nlimit = limit && (d == up);
37         bool nlead0 = lead0 && (d == 0);
38         if (nlead0) {
39             // 仍在先导零
40             res += dfs(pos - 1, nlimit, nlead0,
41                       /* r */, /* last */, /* sum */);
42         } else {
43             if (/* transition is valid */) {
44                 res += dfs(pos - 1, nlimit, nlead0,
45                           /* new r */, /* new last */, /* new sum */);
46             }
47         }
48     }
49     // if (!limit) f[pos][...] = res;
50     return res;
51 }
52
53 int calc(int x) {
54     if (x < 0) return 0;
55     int len = 0;
56     while (x) {
57         a[++len] = x % base;
58         x /= base;
59     }
60     return dfs(len, true, true, 0, -1, 0);
61 }
62 };

```

1. 两次 DFS + 一个数组：第一次以固定根算出子树 DP，第二次换根传递。
2. 两次 DFS，维护 $F(u)$ （子树内答案）与 $G(u)$ （子树外答案），最后将两者合并。

前后缀积 Trick 当转移为子树乘积形式且需要取模时，用前后缀积避免除法与逆元。例子：

$$f[u] = \prod_{v \in \text{son}(u)} (f[v] + 1).$$

```

1 template <class T>
2 class PreSufOptimizeDP {
3 public:
4     std::vector<T> dp;
5
6     explicit PreSufOptimizeDP(std::vector<std::vector<int>> &g)
7         : g(g), dp(g.size()),
8           pre(g.size()), suf(g.size()) {
9         dfs1(1, 0);
10        dfs2(1, 0);
11    }
12
13 private:
14     std::vector<std::vector<int>> g;
15     std::vector<std::vector<T>> pre, suf;
16
17     void dfs1(int u, int fa) {
18         dp[u] = 1;
19         pre[u].push_back(1);
20         for (auto v : g[u]) {
21             if (v == fa) continue;
22             dfs1(v, u);
23             dp[u] = dp[u] * (dp[v] + 1);
24             pre[u].push_back(dp[v] + 1);
25             suf[u].push_back(dp[v] + 1);
26         }
27         suf[u].push_back(1);
28         for (int i = 1; i < (int)pre[u].size(); ++i) {
29             pre[u][i] = pre[u][i] * pre[u][i - 1];
30         }
31         for (int i = (int)suf[u].size() - 2; i >= 0; --i) {
32             suf[u][i] = suf[u][i] * suf[u][i + 1];
33         }
34     }
35
36     // 原式: dp[v] = dp[v] * (dp[u] / (dp[v] + 1) + 1)
37     void dfs2(int u, int fa) {
38         int idx = 0;
39         for (auto v : g[u]) {
40             if (v == fa) continue;
41             dp[v] = dp[v] * (pre[u][idx] * suf[u][idx + 1] + 1);
42             ++idx;
43             dfs2(v, u);
44         }
45     }
46 };

```

4.12 Tree DP | 树形 DP

4.12.1 Rerooting DP | 换根 DP

换根 DP 分为“点换根”和“边换根”，常见做法有两种：

```

44     }
45 }
46 };

```

4.13 DP with after-effect | 带后效性的 DP

当转移存在环路或无法直接拓扑化时，DP 方程本质上是一组线性方程，可用待定系数或高斯消元求解。

4.13.1 Method of undetermined coefficients | 待定系数法

典型应用是树上随机游走的期望问题。设 $DP(x)$ 为从 x 开始到达叶子的期望步数，转移大致形如

$$DP(x) = \frac{1 + \sum_{y \in N(x)} DP(y)}{\deg(x)},$$

由于包含父子互相依赖，可设 $DP(x) = k_x \cdot DP(fa[x]) + b_x$ ，把子树的 DP 形式带回父节点方程中，递推得到所有 k, b ，最终在根处解出答案。

4.13.2 Gaussian elimination | 高斯消元

更一般的线性 DP 可直接写成线性方程组，用高斯消元在 $O(n^3)$ 时间内解出。若系数矩阵为带状，可降到 $O(nd^2)$ 。

例如在一般图上随机游走的期望次数，可对每个点写出

$$DP(i) = \sum_{(i,j) \in E} \frac{DP(j)}{\deg(j)} + c_i,$$

将 $DP(n)$ 固定为边界条件，在其他点上联立方程，用高斯消元即可。

5 String | 字符串

5.1 String Hash | 字符串哈希

注意哈希传入的字符串都是 1-index。

5.1.1 Ordered string hash | 有序字符串哈希

考虑哈希函数 $f(n) = \sum_{i=1}^n s[i] \cdot \text{Base}^{n-i}$ 。预处理前缀哈希 $H(i)$ 与幂数组 $\text{Hash}(i)$ 后，区间 $[l, r]$ 的哈希值为

$$f[l, r] = H(r) - H(l-1) \cdot \text{Base}^{r-l+1}.$$

```

1 struct StringHash {
2     int n;
3     std::vector<int> Hash;
4     std::vector<int> H;
5     int Base, P;

```

```

6
7 // 默认字符串加了前导字符，s[0] 不使用
8 StringHash(const std::string &s, int Base, int P)
9 : n(s.size()), Hash(n), H(n), Base(Base), P(P) {
10     Hash.assign(n, 0);
11     H.assign(n, 0);
12     Hash[0] = 1;
13     for (int i = 1; i <= n - 1; ++i) {
14         Hash[i] = 1LL * Hash[i - 1] * Base % P;
15     }
16     for (int i = 1; i <= n - 1; ++i) {
17         H[i] = (1LL * H[i - 1] * Base % P + s[i]) % P;
18     }
19 }
20
21 int operator()(int l, int r) const {
22     return (H[r] - 1LL * H[l - 1] * Hash[r - l + 1] % P + P) % P;
23 }
24 };

```

5.1.2 Unordered string hash | 无序字符串哈希

考虑哈希函数 $f(n) = \sum_{i=1}^n h(s[i])$ ，其中 $h(x)$ 为预先随机的权值，只要求相同字符对应的 $h(x)$ 相同即可。

```

1 struct StringHash {
2     int n;
3     std::vector<int> Hash;
4     std::vector<int> H;
5     int Base, P;
6
7 // a 为 1-index 的离散值数组，max 为值域上界
8 StringHash(std::vector<int> &a, int maxv, int Base, int P)
9 : n(a.size()), Base(Base), P(P) {
10     Hash.assign(maxv + 1, 0);
11     H.assign(n, 0);
12     Hash[0] = 1;
13     for (int i = 1; i <= maxv; ++i) {
14         Hash[i] = 1LL * Hash[i - 1] * Base % P;
15     }
16     for (int i = 1; i <= n - 1; ++i) {
17         H[i] = (H[i - 1] + Hash[a[i]]) % P;
18     }
19 }
20
21 int operator()(int l, int r) const {
22     return (H[r] - H[l - 1] + P) % P;
23 }
24 };

```

5.1.3 2D ordered hash | 二维有序字符串哈希

```

1 using u64 = unsigned long long;
2
3 struct Hash2D {
4     int n, m;
5     std::vector<u64> hX;           // 行方向 Base 的幂
6     std::vector<u64> hY;           // 列方向 Base 的幂
7     std::vector<std::vector<u64>> A; // 哈希前缀和
8     int P, Q;                     // 两个 Base, 分别为行和列
9
10    Hash2D(const std::vector<std::vector<char>> &a,
11            int n, int m, int P, int Q)
12        : n(n), m(m), hX(n + 1), hY(m + 1), A(n + 1, std::vector<u64>(m + 1, 0)),
13          P(P), Q(Q) {}
14    hX[0] = 1;
15    hY[0] = 1;
16    for (int i = 1; i <= n; ++i) {
17        hX[i] = hX[i - 1] * P;
18    }
19    for (int j = 1; j <= m; ++j) {
20        hY[j] = hY[j - 1] * Q;
21    }
22    for (int i = 1; i <= n; ++i) {
23        for (int j = 1; j <= m; ++j) {
24            A[i][j] = A[i][j - 1] * Q
25                    + A[i - 1][j] * P
26                    - A[i - 1][j - 1] * P * Q
27                    + (a[i][j] - 'a' + 1);
28        }
29    }
30 }
31
32 u64 query(int x1, int y1, int x2, int y2) const {
33     return A[x2][y2]
34         - A[x2][y1 - 1] * hY[y2 - y1 + 1]
35         - A[x1 - 1][y2] * hX[x2 - x1 + 1]
36         + A[x1 - 1][y1 - 1] * hX[x2 - x1 + 1] * hY[y2 - y1 + 1];
37 }
38 };

```

5.2 Minimal representation | 最小表示法

返回一个序列字典序最小的循环同构串。常用于最小循环同构、项链问题等。

```

1 std::vector<int> minimalString(std::vector<int> &a) {
2     int n = a.size();
3     int i = 0, j = 1, k = 0;
4     while (k < n && i < n && j < n) {
5         if (a[(i + k) % n] == a[(j + k) % n]) {

```

```

6             ++k;
7         } else {
8             if (a[(i + k) % n] > a[(j + k) % n]) {
9                 i += k + 1;
10            } else {
11                j += k + 1;
12            }
13            if (i == j) ++i;
14            k = 0;
15        }
16    }
17    int start = std::min(i, j);
18    std::vector<int> ans(n);
19    for (int t = 0; t < n; ++t) {
20        ans[t] = a[(t + start) % n];
21    }
22    return ans;
23 }

```

Trick: 若有一堆字符串, 按某种顺序拼接使得整体字典序最小, 只需按比较规则 $a + b < b + a$ 排序。

5.3 KMP

对于字符串 S , KMP 通过前缀函数 (失配函数) 维护 $fail(i)$: 表示以 i 为结尾的前缀串的最长相等前后缀的长度。

模式串 t 在文本串 s 中的出现位置, 可在 $t\#s$ 上跑一遍前缀函数, 凡是 $fail(i) = |t|$ 的位置均为一次匹配。

```

1 class KMP {
2 public:
3     KMP() = default;
4
5     std::vector<int> getPreFunction(std::string &s) {
6         return kmp(s);
7     }
8
9     // 在 s 中找 t, 返回所有匹配起始下标 (0-index)
10    std::vector<int> getMatchPos(std::string &s, std::string &t) {
11        std::vector<int> pos;
12        auto link = kmp(t + "#" + s); // 在 s 中找 t
13        for (int i = (int)t.size() + 1; i < (int)link.size(); ++i) {
14            if (link[i] == (int)t.size()) {
15                pos.push_back(i - 2 * (int)t.size() + 1);
16            }
17        }
18        return pos;
19    }
20
21 private:
22    std::vector<int> kmp(const std::string &s) {

```

```

23     int n = s.size();
24     std::vector<int> link(n);
25     for (int i = 1, j = 0; i < n; ++i) {
26         while (j && s[i] != s[j]) {
27             j = link[j - 1];
28         }
29         if (s[i] == s[j]) ++j;
30         link[i] = j;
31     }
32     return link;
33 }
34 };

```

5.4 Z-function (扩展 KMP)

对于字符串 S , $Z(i)$ 表示后缀 $S[i, \dots]$ 与整个串 S 的最长公共前缀长度。实现时维护一个当前最右的 Z-box 区间 $[l, r]$ 以加速转移, 约定 $Z(0) = 0$ 。

```

1 class ZFunction {
2 public:
3     ZFunction() = default;
4
5     // 单串 Z 函数
6     std::vector<int> getZFunction(const std::string &s) {
7         return zFunction(s);
8     }
9
10    // 得到字符串 s 的每个后缀与 t 的前缀匹配长度
11    std::vector<int> getMatch(const std::string &s, const std::string &t) {
12        auto z = zFunction(t + "#" + s);
13        return std::vector<int>(z.begin() + t.size() + 1, z.end());
14    }
15
16 private:
17    std::vector<int> zFunction(const std::string &s) {
18        int n = s.size();
19        std::vector<int> z(n);
20        int l = 0, r = 0;
21        for (int i = 1; i < n; ++i) {
22            if (i < r) {
23                z[i] = std::min(z[i - l], r - i);
24            }
25            while (i + z[i] < n && s[i + z[i]] == s[z[i]]) {
26                ++z[i];
27            }
28            if (i + z[i] > r) {
29                l = i;
30                r = i + z[i];
31            }
32        }
33    }
34 }

```

```

33     return z;
34 }
35 };

```

5.5 Manacher

Manacher 算法在线性时间内求出每个位置为中心的最长回文半径。常见做法是在原串中间插入分隔符, 统一奇偶回文的处理。

```

1 std::vector<int> manacher(const std::string &s) {
2     std::string t;
3     t.reserve(s.size() * 2 + 1);
4     t.push_back('#');
5     for (char c : s) {
6         t.push_back(c);
7         t.push_back('#');
8     }
9     int n = t.size();
10    std::vector<int> r(n);
11    int center = 0;
12    for (int i = 0; i < n; ++i) {
13        if (2 * center - i >= 0 && center + r[center] > i) {
14            r[i] = std::min(r[2 * center - i], center + r[center] - i);
15        }
16        while (i - r[i] >= 0 && i + r[i] < n && t[i - r[i]] == t[i + r[i]]) {
17            ++r[i];
18        }
19        if (i + r[i] > center + r[center]) {
20            center = i;
21        }
22    }
23    for (auto &x : r) --x; // 去掉分隔符的贡献
24    return r;
25 }

```

5.6 Minimal period | 最小循环节

判断一个串是否有长度为 d 的循环节, 只需比较区间 $[l, r - d]$ 与 $[l + d, r]$ 是否完全相同。

最小循环节长度一定是 len 的因子, 可用字符串哈希配合质因数分解不断尝试缩小周期。

```

1 int findMinimalCycle(std::string s) {
2     int len = (int)s.length();
3     s = '#' + s; // 1-index
4     int l = 1, r = len;
5     StringHash hash(s, 13331, 998244353);
6     int ans = len;
7     int tmp = len;
8     while (tmp != 1) {

```

```

9     int p = minPrimeFactor(tmp);
10    int d = ans / p;
11    if (hash(l, r - d) == hash(l + d, r)) {
12        ans = d;
13    }
14    tmp /= p;
15 }
16 return ans;
17 }

```

5.7 Lyndon decomposition | Lyndon 分解

Lyndon 串定义为本身严格小于其所有非空真后缀的串。任意串都存在唯一的 Lyndon 分解

$$s = w_1 w_2 \dots w_k, \quad w_1 \geq w_2 \geq \dots \geq w_k,$$

其中每个 w_i 是 Lyndon 串。

```

1 // 返回每个 Lyndon 块的右端点 (1-index, 下标相对原串)
2 std::vector<int> Lyndon(std::string s) {
3     std::vector<int> res;
4     int n = s.length();
5     s = ' ' + s;
6     for (int i = 1; i <= n; ) {
7         int j = i, k = i + 1;
8         while (k <= n && s[j] <= s[k]) {
9             if (s[j] < s[k]) {
10                 j = i;
11             } else {
12                 ++j;
13             }
14             ++k;
15         }
16         while (i <= j) {
17             res.push_back(i + k - j - 1);
18             i += k - j;
19         }
20     }
21     return res;
22 }

```

5.8 Subsequence automaton | 子序列自动机

子序列自动机预处理 $next[i][c]$: 从位置 i 开始往后, 第一个字符 c 出现的位置。朴素做法时间复杂度 $O(n \cdot \Sigma)$ 。

```

1 template <int Z, char Base>
2 struct SubSequenceAutomaton {
3     std::vector<std::array<int, Z>> next;

```

```

4     int n;
5
6     // s 默认加过前缀空字符, 1-index
7     explicit SubSequenceAutomaton(const std::string &s)
8         : n((int)s.size() - 1), next(n + 1) {
9         for (int c = 0; c < Z; ++c) {
10             next[n][c] = n + 1;
11         }
12         for (int i = n - 1; i >= 0; --i) {
13             next[i] = next[i + 1];
14             next[i][s[i + 1] - Base] = i + 1;
15         }
16     }
17
18     int getNextPos(int pos, char c) const {
19         return next[pos][c - Base];
20     }
21 };

```

当字符集很大时, 可以用可持久化线段树压缩值域, 仅在发生改变的位置更新, 查询复杂度降为 $O(\log n)$ 。

5.8.1 Subsequence automaton with President tree | 主席树优化子序列自动机

```

1 struct PresidentTree {
2     int idx = 0;
3     int n;
4     std::vector<int> root, lc, rc;
5     std::vector<int> sum;
6
7     void pushup(int u) {
8         sum[u] = sum[lc[u]] + sum[rc[u]];
9     }
10
11     // a 为值域数组, len 为时间轴长度
12     PresidentTree(const std::vector<int> &a, int len)
13         : n(a.size()), root(len + 1),
14           lc(len * 40), rc(len * 40), sum(len * 40) {
15         // 传入的 a 实际上是值域
16         std::function<void(int &, int, int)> build =
17             [&](int &u, int l, int r) {
18                 u = ++idx;
19                 if (l == r) {
20                     sum[u] = a[l];
21                     return;
22                 }
23                 int mid = (l + r) >> 1;
24                 build(lc[u], l, mid);
25                 build(rc[u], mid + 1, r);
26                 pushup(u);

```

```

27     };
28     build(root[len], 1, n - 1); // 倒着建树
29 }
30
31 void insert(int &u, int v, int l, int r, int pos, int x) {
32     u = ++idx;
33     lc[u] = lc[v];
34     rc[u] = rc[v];
35     sum[u] = sum[v];
36     if (l == r) {
37         sum[u] = x;
38         return;
39     }
40     int mid = (l + r) >> 1;
41     if (pos <= mid) {
42         insert(lc[u], lc[v], l, mid, pos, x);
43     } else {
44         insert(rc[u], rc[v], mid + 1, r, pos, x);
45     }
46     pushup(u);
47 }
48
49 void insert(int now, int pre, int pos, int x) {
50     insert(root[now], root[pre], 1, n - 1, pos, x);
51 }
52
53 int query(int u, int l, int r, int pos) {
54     if (l == r) {
55         return sum[u];
56     }
57     int mid = (l + r) >> 1;
58     if (pos <= mid) {
59         return query(lc[u], l, mid, pos);
60     } else {
61         return query(rc[u], mid + 1, r, pos);
62     }
63 }
64
65 int query(int t, int pos) {
66     return query(root[t], 1, n - 1, pos);
67 }
68 };
69
70 // 子序列自动机 (值域大时)
71 struct SubsequenceAutomaton {
72     const std::vector<int> &val;
73     PresidentTree Tree;
74     int n;
75
76     SubsequenceAutomaton(const std::vector<int> &a, std::vector<int> &init)
77         : val(a), Tree(init, (int)a.size() - 1),

```

```

78     n((int)a.size() - 1) {
79         for (int i = n - 1; i >= 0; --i) {
80             // 更新子序列自动机
81             Tree.insert(i, i + 1, val[i + 1], i + 1);
82         }
83     }
84
85     // pos 后面第一个值为 val 的位置
86     int query(int pos, int v) {
87         return Tree.query(pos, v);
88     }
89 };

```

5.9 AC automaton | AC 自动机

AC 自动机是在 trie 树基础上加上 fail 指针得到的有向图。

对于节点 p ，若沿字符 c 没有儿子，则跳到 $fail(p)$ 再尝试 c ；fail 树上父亲表示更短的后缀。多模式匹配时在 AC 图上跑一遍文本串，并把贡献沿 fail 树向上累加即可。

```

1 template <int Z, char Base>
2 struct AcAutomaton {
3     std::vector<int> fail; // fail 指针
4     std::vector<std::vector<int>> ch; // trie 图
5     std::vector<std::vector<int>> ID; // 终止串编号
6     int idx = 0;
7     int sum = 0;
8     int n; // 模式串个数
9
10    explicit AcAutomaton(std::vector<std::string> &s)
11        : n((int)s.size()) {
12        for (auto &x : s) {
13            sum += (int)x.size();
14        }
15        ch.assign(sum + 1, std::vector<int>(Z, 0));
16        fail.assign(sum + 1, 0);
17        ID.resize(sum + 1);
18        for (int i = 0; i < n; ++i) {
19            insert(i, s[i]);
20        }
21        build();
22    }
23
24    void insert(int id, const std::string &s) {
25        int p = 0;
26        for (char c : s) {
27            int x = c - Base;
28            if (!ch[p][x]) {
29                ch[p][x] = ++idx;
30            }
31            p = ch[p][x];

```



```

32     }
33     ID[p].push_back(id);
34 }
35
36 void build() {
37     std::queue<int> q;
38     for (int i = 0; i < Z; ++i) {
39         if (ch[0][i]) {
40             q.push(ch[0][i]);
41         }
42     }
43     while (!q.empty()) {
44         int u = q.front();
45         q.pop();
46         for (int i = 0; i < Z; ++i) {
47             int v = ch[u][i];
48             if (v) {
49                 fail[v] = ch[fail[u]][i];
50                 q.push(v);
51             } else {
52                 ch[u][i] = ch[fail[u]][i];
53             }
54         }
55     }
56 }
57 };

```

5.10 Suffix automaton | 后缀自动机

后缀自动机 (SAM) 是一个 DAG, 加上一棵 *fail* 树。

每个状态代表一个 endpos 等价类, *len[u]* 为该状态表示的最长子串长度, *fail[u]* 指向最长真后缀所在的状态。

```

1  template <int Z, char Base>
2  struct Sam {
3      std::vector<std::array<int, Z>> ch; // 转移
4      std::vector<std::vector<int>>> g; // fail 树
5      std::vector<int> fail, len;
6      std::vector<long long> cnt; // endpos 大小
7
8      int last; // 上一个添加的状态
9      int tot; // 总状态数
10     long long num; // 不同子串数量
11
12     explicit Sam(int n)
13     : ch((n + 1) << 1),
14       g((n + 1) << 1),
15       fail((n + 1) << 1),
16       len((n + 1) << 1),
17       cnt((n + 1) << 1, 0),

```

```

18     last(1), tot(1), num(0) {
19         fail[1] = -1;
20     }
21
22     explicit Sam(const std::string &s)
23     : Sam((int)s.size()) {
24         for (char c : s) {
25             add(c);
26         }
27         for (int i = 2; i <= tot; ++i) {
28             g[fail[i]].push_back(i);
29         }
30     }
31
32     void add(char c) {
33         c -= Base;
34         int cur = ++tot;
35         len[cur] = len[last] + 1;
36         cnt[cur] = 1;
37         int v = last;
38         while (v != -1 && !ch[v][c]) {
39             ch[v][c] = cur;
40             v = fail[v];
41         }
42         if (v == -1) {
43             fail[cur] = 1;
44         } else {
45             int q = ch[v][c];
46             if (len[v] + 1 == len[q]) {
47                 fail[cur] = q;
48             } else {
49                 int clone = ++tot;
50                 len[clone] = len[v] + 1;
51                 ch[clone] = ch[q];
52                 fail[clone] = fail[q];
53                 while (v != -1 && ch[v][c] == q) {
54                     ch[v][c] = clone;
55                     v = fail[v];
56                 }
57                 fail[q] = fail[cur] = clone;
58             }
59         }
60         last = cur;
61         num += len[cur] - (fail[cur] == -1 ? 0 : len[fail[cur]]);
62     }
63 };

```

5.10.1 Dynamic SAM with LCT (sketch) | LCT 维护 SAM 链 / 子树信息

当需要在线维护串的改变时，可以用 LCT 维护 fail 树上的链或子树信息。核心是在 `add` 中对克隆节点做 `splay` 并复制节点信息，保证历史贡献不丢失。此处省略 LCT 具体实现，只保留 `add` 思路。

5.11 Generalized SAM | 广义后缀自动机

广义 SAM 在一棵 trie 上建多个字符串的 SAM，每个状态维护一个长度为串数的数组 `ID[u][i]`，表示该状态代表的子串在第 i 个串中出现次数。

在 fail 树上自底向上累加可以得到每个状态在每个串中的 `endpos` 大小，从而解决多串最长公共子串等问题。

```
1 template <int Z, char Base>
2 struct ExSam {
3     std::vector<std::array<int, Z>> ch;
4     std::vector<int> len, fail;
5     std::vector<std::vector<int>> ID;
6     std::vector<std::vector<int>> g;
7     int cnt;          // 字符串数量
8
9     int tot = 0, last = 0;
10    long long num = 0;
11
12    explicit ExSam(const std::vector<std::string> &S) {
13        cnt = (int)S.size();
14        int LEN = 0;
15        for (auto &str : S) {
16            LEN += (int)str.size();
17        }
18        len.assign(LEN << 1, 0);
19        fail.assign(LEN << 1, 0);
20        ch.assign(LEN << 1, {});
21        g.assign(LEN << 1, {});
22        ID.assign(LEN << 1, std::vector<int>(cnt, 0));
23        fail[0] = -1;
24
25        for (int i = 0; i < cnt; ++i) {
26            last = 0;
27            for (char c : S[i]) {
28                exadd(c, i);
29            }
30        }
31    }
32
33    void dfs(int u) {
34        for (int v : g[u]) {
35            dfs(v);
36            for (int i = 0; i < cnt; ++i) {
```

```
37            ID[u][i] += ID[v][i];
38        }
39    }
40 }
41
42 bool check(int p) {
43     for (int i = 0; i < cnt; ++i) {
44         if (!ID[p][i]) return false;
45     }
46     return true;
47 }
48
49 int querylen(const std::string &s) {
50     for (int i = 1; i <= tot; ++i) {
51         g[fail[i]].push_back(i);
52     }
53     dfs(0);
54     int p = 0, now = 0, res = 0;
55     for (char c : s) {
56         int x = c - Base;
57         if (check(ch[p][x])) {
58             p = ch[p][x];
59             ++now;
60         } else {
61             while (p != -1 && !check(ch[p][x])) {
62                 p = fail[p];
63             }
64             if (p == -1) {
65                 now = 0;
66                 p = 0;
67             } else {
68                 now = len[p] + 1;
69                 p = ch[p][x];
70             }
71         }
72         res = std::max(res, now);
73     }
74     return res;
75 }
76
77 private:
78     void assign(int cur, int id) {
79         ID[cur][id] += 1;
80     }
81
82     void exadd(char c, int id) {
83         int x = c - Base;
84         if (ch[last][x]) {
85             int v = last;
86             int q = ch[v][x];
87             if (len[q] != len[v] + 1) {
```

```

88     int cl = ++tot;
89     len[cl] = len[v] + 1;
90     fail[cl] = fail[q];
91     ch[cl] = ch[q];
92     while (v != -1 && ch[v][x] == q) {
93         ch[v][x] = cl;
94         v = fail[v];
95     }
96     fail[q] = cl;
97     q = cl;
98 }
99 int cur = last = q;
100 assign(cur, id);
101 return;
102 }
103 int cur = ++tot;
104 len[cur] = len[last] + 1;
105 assign(cur, id);
106 int v = last;
107 while (v != -1 && !ch[v][x]) {
108     ch[v][x] = cur;
109     v = fail[v];
110 }
111 if (v == -1) {
112     fail[cur] = 0;
113 } else {
114     int q = ch[v][x];
115     if (len[q] == len[v] + 1) {
116         fail[cur] = q;
117     } else {
118         int cl = ++tot;
119         len[cl] = len[v] + 1;
120         fail[cl] = fail[q];
121         ch[cl] = ch[q];
122         while (v != -1 && ch[v][x] == q) {
123             ch[v][x] = cl;
124             v = fail[v];
125         }
126         fail[q] = fail[cur] = cl;
127     }
128 }
129 last = cur;
130 num += len[cur] - len[fail[cur]];
131 }
132 };

```

```

1 template <int Z, char Base>
2 struct Pam {
3     std::vector<std::array<int, Z>> ch;
4     std::vector<int> fail, len, dep, cnt;
5     std::string s;
6
7     int cur = 0, tot = 1; // 0: 偶根, 1: 奇根
8
9     explicit Pam(int n)
10         : ch(n + 2), fail(n + 2),
11           len(n + 2), dep(n + 2), cnt(n + 2, 0) {
12         fail[0] = 1;
13         len[1] = -1;
14     }
15
16     explicit Pam(const std::string &str)
17         : Pam((int)str.size()) {
18         for (int i = 0; i < (int)str.size(); ++i) {
19             add(i, str[i]);
20         }
21     }
22
23     void assign(int u, int /*id*/) {
24         cnt[u] += 1;
25     }
26
27     int getfail(int x, int i) {
28         while (i - len[x] - 1 < 0 || s[i - len[x] - 1] != s[i]) {
29             x = fail[x];
30         }
31         return x;
32     }
33
34     void add(int i, char c) {
35         int x = c - Base;
36         s.push_back(c);
37         int v = getfail(cur, i);
38         if (!ch[v][x]) {
39             fail[++tot] = ch[getfail(fail[v], i)][x];
40             ch[v][x] = tot;
41             len[tot] = len[v] + 2;
42             dep[tot] = dep[fail[tot]] + 1;
43         }
44         cur = ch[v][x];
45         assign(cur, i);
46     }
47
48     auto getFailTree() const {
49         std::vector<std::vector<int>> e(tot + 1);
50         for (int i = 0; i <= tot; ++i) {
51             if (i != 1) {

```

5.12 Palindromic automaton | 回文自动机

回文自动机 (PAM) 维护所有不同回文子串。建两棵根：偶数根 0 和奇数根 1，其中 $len[0] = 0, len[1] = -1$ ，且 $fail[0] = 1$ 。每个节点代表一个回文串， $fail[u]$ 为其最长真回文后缀。

```

52         e[fail[i]].push_back(i);
53     }
54 }
55 return e;
56 }
57 };

```

5.13 Suffix array | 后缀数组

后缀数组维护后缀的字典序排序， $sa[i]$ 为第 i 小后缀的起始下标， $rk[i]$ 为后缀 i 的排名， $h[i]$ 为 $sa[i]$ 与 $sa[i-1]$ 的 LCP。常用性质包括：

1. $LCP(sa[i], sa[j]) = \min_{k=i+1}^j h[k]$ 。 2. 不同子串数： $n(n+1)/2 - \sum_{i=1}^n h[i]$ 。

```

1 struct SuffixArray {
2     const int n;
3     std::vector<int> sa, rk, h;
4
5     template <class T>
6     explicit SuffixArray(const T &s)
7         : n((int)s.size()),
8           sa(n), rk(n),
9           id(n), tmp(n) {
10         std::iota(id.begin(), id.end(), 0);
11         for (int i = 0; i < n; ++i) {
12             rk[i] = s[i];
13         }
14         countSort();
15         for (int w = 1; w <= 1) {
16             std::iota(id.begin(), id.begin() + w + 1, n - w);
17             for (int i = 0, p = w; i < n; ++i) {
18                 if (sa[i] >= w) {
19                     id[p++] = sa[i] - w;
20                 }
21             }
22             countSort();
23             oldrk = rk;
24             rk[sa[0]] = 0;
25             for (int i = 1, p = 0; i < n; ++i) {
26                 rk[sa[i]] = equal(sa[i], sa[i-1], w) ? p : ++p;
27             }
28             if (rk[sa.back()] + 1 == n) break;
29         }
30         calcHeight(s);
31     }
32
33 private:
34     std::vector<int> oldrk, id, tmp, cnt;
35
36     template <class T>

```

```

37 void calcHeight(const T &s) {
38     h.assign(n, 0);
39     for (int i = 0, k = 0; i < n; ++i) {
40         if (!rk[i]) continue;
41         if (k) --k;
42         while (i + k < n && s[i + k] == s[sa[rk[i] - 1] + k]) {
43             ++k;
44         }
45         h[rk[i]] = k;
46     }
47 }
48
49 void countSort() {
50     int m = *std::max_element(rk.begin(), rk.end());
51     cnt.assign(m + 1, 0);
52     for (int i = 0; i < n; ++i) {
53         cnt[tmp[i] = rk[id[i]]] += 1;
54     }
55     for (int i = 1; i < (int)cnt.size(); ++i) {
56         cnt[i] += cnt[i-1];
57     }
58     for (int i = n - 1; i >= 0; --i) {
59         sa[--cnt[tmp[i]]] = id[i];
60     }
61 }
62
63 bool equal(int x, int y, int w) {
64     int rkx = (x + w < n ? oldrk[x + w] : -1);
65     int rky = (y + w < n ? oldrk[y + w] : -1);
66     return oldrk[x] == oldrk[y] && rkx == rky;
67 }
68 };

```

6 Number Theory | 数论

6.1 ModInt | 模运算

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using i64 = long long;
4 using u64 = unsigned long long;
5
6 template <int P>
7 struct modint {
8     u64 x;
9
10     constexpr modint() : x(0) {}
11     constexpr modint(auto _x) : x(_x) {}

```

```

12
13 constexpr modint pow(u64 b) const {
14     modint res{1};
15     for (modint a{*this}; b; b >= 1, a *= a) {
16         if (b & 1) res *= a;
17     }
18     return res;
19 }
20 constexpr modint inv() const { return pow(P - 2); }
21 constexpr friend istream& operator>> (istream& is, modint& val) { return is >> val.
    x; }
22 constexpr friend ostream& operator<< (ostream& os, const modint& val) { return os
    << val.x; }
23 constexpr modint& operator+= (const modint& rhs) { x += rhs.x, x = min(x, x - P);
    return *this; }
24 constexpr modint& operator-= (const modint& rhs) { x -= rhs.x, x = min(x, x + P);
    return *this; }
25 constexpr modint& operator*= (const modint& rhs) { (x *= rhs.x) %= P; return
    *this; }
26 constexpr modint& operator/= (const modint& rhs) { (x *= rhs.inv().x) %= P; return
    *this; }
27 constexpr friend modint operator+ (const modint& lhs, const modint& rhs) { return
    modint{lhs} += rhs; }
28 constexpr friend modint operator- (const modint& lhs, const modint& rhs) { return
    modint{lhs} -= rhs; }
29 constexpr friend modint operator* (const modint& lhs, const modint& rhs) { return
    modint{lhs} *= rhs; }
30 constexpr friend modint operator/ (const modint& lhs, const modint& rhs) { return
    modint{lhs} /= rhs; }
31 };

```

6.2 Prime sieve | 素数筛

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using i64 = long long;
4 using u64 = unsigned long long;
5
6 namespace prime_sieve {
7     constexpr int N = 4e6;
8     constexpr int M = 2e3;
9     vector<int> prime;
10    bitset<N + 1> is_prime;
11
12    void init() {
13        prime.reserve(N >> 3);
14        is_prime.set();
15        for (int i = 2; i <= M; i++) {
16            if (!is_prime[i]) continue;

```

```

17         for (int j = i << 1; j <= N; j += i) is_prime[j] = 0;
18     }
19     for (int i = 2; i <= N; i = is_prime._Find_next(i)) {
20         prime.push_back(i);
21     }
22 }
23 };

```

6.3 Qpow block | 快速幂分块

```

1 template <typename meta>
2 struct qpow_block {
3     static constexpr int N[4] = {0, 16, 32, 48};
4     static constexpr int B = 1 << 16;
5     static constexpr int M = B - 1;
6     meta data[4][B + 1];
7
8     qpow_block(auto x) {
9         meta base{x};
10        for (int k = 0; k < 4; k++) {
11            data[k][0] = 1;
12            for (int i = 1; i <= B; i++)
13                data[k][i] = data[k][i - 1] * base;
14        }
15    }
16 }
17
18 meta ask(u64 n) const {
19     meta res{1};
20     for (int k = 0; k < 4; k++)
21         res *= data[k][(n >> N[k]) & M];
22     return res;
23 }
24 };

```

6.4 O(1) GCD | O(1) 最大公约数

适用于 $O(n)$ 以内的数字求 fac ，我们传入的 $ifac$ 为能接受数字的上限， inv 为小表参数，复杂度为 $fac[i] = fac[i - 1] * i$ ，查询可以认为是 $ifac[i] = ifac[i + 1] * (i + 1)$ ，我们一般传入 $inv[i] = ifac[i] * fac[i - 1]$ 即可。

```

1 class FastGCD {
2     int N; // 支持的最大整数
3     int M; // 小整数表上限
4     std::vector<std::vector<int>>> f; // 小整数 GCD 表
5     std::vector<std::array<int, 3>> d; // 每个整数拆成最多 3 块质因数
6     std::vector<int> p; // 素数数组

```

```

7   std::vector<bool> v;           // 筛标记
8
9   public:
10      FastGCD(int n, int m)
11          : N(n), M(m), f(M + 1, std::vector<int>(M + 1)),
12            d(N + 1, std::array<int, 3>{0, 0, 0}),
13            v(N + 1, false) {
14          precompute();
15      }
16
17      // ===== 普通欧几里得，用于小表生成 =====
18      int gcdNormal(int a, int b) const {
19          while (b) {
20              int t = a % b;
21              a = b;
22              b = t;
23          }
24          return a;
25      }
26
27   private:
28      void precompute() {
29          p.clear();
30
31          // 1. 构建小整数 GCD 表 f[i][j]，保证安全
32          for (int i = 0; i <= M; i++) {
33              for (int j = 0; j <= M; j++) {
34                  if (i == 0 || j == 0)
35                      f[i][j] = i + j;
36                  else if (i == j)
37                      f[i][j] = i;
38                  else if (i > j)
39                      f[i][j] = f[i - j][j]; // 安全: i-j >= 1
40                  else
41                      f[i][j] = f[j][i]; // i < j, 交换
42              }
43          }
44
45          // 2. 初始化 d[1]
46          d[1] = std::array<int, 3>{1, 1, 1};
47
48          // 3. 线性筛 + 分块质因数
49          for (int i = 2; i <= N; i++) {
50              if (!v[i]) {
51                  p.push_back(static_cast<int>(i));
52                  d[i] = std::array<int, 3>{static_cast<int>(i), 1, 1};
53              }
54              for (size_t j = 0; j < p.size(); j++) {
55                  int k = i * p[j];
56                  if (k > N)
57                      break;

```

```

58              v[k] = true;
59              d[k] = d[i]; // 继承父数质因数块
60              if (d[k][0] * p[j] <= M)
61                  d[k][0] *= p[j];
62              else if (d[k][1] * p[j] <= M)
63                  d[k][1] *= p[j];
64              else
65                  d[k][2] *= p[j];
66              if (i % p[j] == 0)
67                  break; // 线性筛剪枝
68          }
69      }
70  }
71
72   public:
73      int query(int x, int y) const {
74          if (!x || !y)
75              return x + y;
76          int t = 1;
77          const auto &px = d[x];
78          for (int i = 0; i < 3 && y > 1; i++) {
79              if (px[i] <= 1)
80                  continue;
81              int k = 0;
82              if (px[i] <= M)
83                  k = f[px[i]][y % px[i]];
84              else if (y % px[i] == 0)
85                  k = px[i];
86              if (k == 0)
87                  continue; // 避免除零
88              t *= k;
89              y /= k;
90          }
91          return t;
92      }
93  };

```

6.5 Number theory basics | 数论基础知识

6.5.1 关于狄利克雷卷积的定义

6.5.2 质数表

1	101	1009	10007	100003	1000003
2	103	1013	10009	100019	1000033
3	107	1019	10037	100043	1000037
4	109	1021	10039	100049	1000039
5	113	1031	10061	100057	1000081
6	127	1033	10067	100069	1000099

7	131	1039	10069	100103	1000117
8	137	1049	10079	100109	1000121
9	139	1051	10091	100129	1000133
10	149	1061	10093	100151	1000151
11	151	1063	10099	100153	1000159
12	157	1069	10103	100169	1000171
13	163	1087	10111	100183	1000183
14	167	1091	10133	100189	1000187
15	173	1093	10139	100193	1000193
16	179	1097	10141	100207	1000199
17	181	1103	10151	100213	1000211
18	191	1109	10159	100237	1000213
19	193	1117	10163	100267	1000231
20	197	1123	10169	100271	1000249
21	199	1129	10177	100279	1000253
22	211	1151	10181	100291	1000273
23	223	1153	10193	100297	1000289
24	227	1163	10211	100313	1000291
25	229	1171	10223	100333	1000303
26	233	1181	10243	100343	1000313
27	239	1187	10247	100357	1000333
28	241	1193	10253	100361	1000357
29	251	1201	10259	100363	1000367
30	257	1213	10267	100379	1000381
31	263	1217	10271	100391	1000393
32	269	1223	10273	100393	1000397
33	271	1229	10289	100403	1000403
34	277	1231	10301	100411	1000409
35	281	1237	10303	100417	1000423
36	283	1249	10313	100447	1000427
37	293	1259	10321	100459	1000429
38	307	1277	10331	100469	1000453
39	311	1279	10333	100483	1000457
40	313	1283	10337	100493	1000507
41	317	1289	10343	100501	1000537
42	331	1291	10357	100511	1000541
43	337	1297	10369	100517	1000547
44	347	1301	10391	100519	1000577
45	349	1303	10399	100523	1000579
46	353	1307	10427	100537	1000589
47	359	1319	10429	100547	1000609
48	367	1321	10433	100549	1000619
49	373	1327	10453	100559	1000621
50	379	1361	10457	100591	1000639

52	1e7~1e12				
53	10000019	100000007	1000000007	10000000019	100000000003
		1000000000039			
54	10000079	100000037	1000000009	10000000033	100000000019
		1000000000061			
55	10000103	100000039	1000000021	10000000061	100000000057

		1000000000063			
56	10000121	100000049	1000000033	10000000069	100000000063
		1000000000091			
57	10000139	100000073	1000000087	10000000097	100000000069
		1000000000121			
58	10000141	100000081	1000000093	10000000103	100000000073
		1000000000163			
59	10000169	100000123	1000000097	10000000121	100000000091
		1000000000169			
60	10000189	100000127	1000000103	10000000141	100000000103
		1000000000177			
61	10000223	100000193	1000000123	10000000147	100000000129
		1000000000189			
62	10000229	100000213	1000000181	10000000207	100000000171
		1000000000193			
63	10000247	100000217	1000000207	10000000259	100000000183
		1000000000211			
64	10000253	100000223	1000000223	10000000277	100000000193
		1000000000271			
65	10000261	100000231	1000000241	10000000279	100000000211
		1000000000303			
66	10000271	100000237	1000000271	10000000319	100000000223
		1000000000331			
67	10000303	100000259	1000000289	10000000343	100000000237
		1000000000333			
68	10000339	100000267	1000000297	10000000391	100000000253
		1000000000339			
69	10000349	100000279	1000000321	10000000403	100000000283
		1000000000459			
70	10000357	100000357	1000000349	10000000469	100000000319
		1000000000471			
71	10000363	100000379	1000000363	10000000501	100000000363
		1000000000537			
72	10000379	100000393	1000000403	10000000537	100000000367
		1000000000543			
73	10000439	100000399	1000000409	10000000583	100000000379
		1000000000547			
74	10000451	100000421	1000000411	10000000589	100000000393
		1000000000561			
75	10000453	100000429	1000000427	10000000597	100000000403
		1000000000609			
76	10000457	100000463	1000000433	10000000601	100000000411
		1000000000661			
77	10000481	100000469	1000000439	10000000631	100000000417
		1000000000669			
78	10000511	100000471	1000000447	10000000643	100000000427
		1000000000721			
79	10000537	100000493	1000000453	10000000649	100000000447
		1000000000751			
80	10000583	100000541	1000000459	10000000667	100000000487
		1000000000787			

81	10000591	100000543	1000000483	10000000679	100000000519
	1000000000789				
82	10000609	100000561	1000000513	10000000711	100000000567
	1000000000799				
83	10000643	100000567	1000000531	10000000723	100000000579
	1000000000841				
84	10000651	100000577	1000000579	10000000741	100000000621
	1000000000903				
85	10000657	100000609	1000000607	10000000753	100000000631
	1000000000921				
86	10000667	100000627	1000000613	10000000793	100000000637
	1000000000931				
87	10000687	100000643	1000000637	10000000799	100000000669
	1000000000933				
88	10000691	100000651	1000000663	10000000807	100000000699
	1000000000949				
89	10000721	100000661	1000000711	10000000877	100000000703
	1000000000997				
90	10000723	100000669	1000000753	10000000883	100000000721
	1000000001051				
91	10000733	100000673	1000000787	10000000889	100000000739
	1000000001083				
92	10000741	100000687	1000000801	10000000949	100000000747
	1000000001123				
93	10000747	100000717	1000000829	10000000963	100000000801
	1000000001159				
94	10000759	100000721	1000000861	10000000991	100000000817
	1000000001197				
95	10000763	100000793	1000000871	10000000993	100000000819
	1000000001201				
96	10000769	100000799	1000000891	10000000999	100000000861
	1000000001213				
97	10000789	100000801	1000000901	10000001041	100000000901
	1000000001243				
98	10000799	100000837	1000000919	10000001047	100000000943
	1000000001261				
99	10000813	100000841	1000000931	10000001051	100000000951
	1000000001263				
100	10000819	100000853	1000000933	10000001057	100000001009
	1000000001293				
101	10000831	100000891	1000000993	10000001087	100000001099
	1000000001303				
102	10000849	100000921	1000001011	10000001101	100000001111
	1000000001339				
103					
104					
105	1e13~1e16				
106	10000000000037	100000000000031	1000000000000037	1000000000000061	
107	100000000000051	100000000000067	1000000000000091	1000000000000069	
108	100000000000099	100000000000097	1000000000000159	1000000000000079	
109	100000000000129	100000000000099	1000000000000187	1000000000000099	

110	100000000000183	100000000000133	1000000000000223	1000000000000453
111	100000000000259	100000000000139	1000000000000241	1000000000000481
112	100000000000267	100000000000169	1000000000000249	1000000000000597
113	100000000000273	100000000000183	1000000000000259	1000000000000613
114	100000000000279	100000000000261	1000000000000273	1000000000000639
115	100000000000283	100000000000357	1000000000000279	1000000000000669
116	100000000000313	100000000000367	1000000000000297	1000000000000753
117	100000000000343	100000000000403	1000000000000357	1000000000000793
118	100000000000391	100000000000423	1000000000000399	1000000000000819
119	100000000000411	100000000000469	1000000000000403	1000000000000861
120	100000000000433	100000000000487	1000000000000487	1000000000000897
121	100000000000453	100000000000493	1000000000000513	1000000000000909
122	100000000000591	100000000000541	1000000000000613	1000000000000931
123	100000000000609	100000000000601	1000000000000711	1000000000000949
124	100000000000643	100000000000643	1000000000000741	1000000000000957
125	100000000000649	100000000000657	1000000000000783	1000000000000991
126	100000000000657	100000000000709	1000000000000811	1000000000001029
127	100000000000687	100000000000721	1000000000000843	1000000000001101
128	100000000000691	100000000000753	1000000000000873	1000000000001129
129	100000000000717	100000000000777	1000000000000921	1000000000001239
130	100000000000729	100000000000807	1000000000001003	1000000000001293
131	100000000000751	100000000000841	1000000000001027	1000000000001311
132	100000000000759	100000000000843	1000000000001063	1000000000001321
133	100000000000777	100000000000861	1000000000001089	1000000000001377
134	100000000000853	100000000000963	1000000000001117	1000000000001423
135	100000000000883	100000000000993	1000000000001209	1000000000001477
136	100000000000943	100000000001087	1000000000001269	1000000000001549
137	100000000000957	100000000001089	1000000000001293	1000000000001551
138	100000000000987	100000000001129	1000000000001347	1000000000001567
139	100000000000993	100000000001173	1000000000001371	1000000000001657
140	100000000001011	100000000001177	1000000000001413	1000000000001659
141	100000000001023	100000000001189	1000000000001491	1000000000001719
142	100000000001087	100000000001197	1000000000001503	1000000000001723
143	100000000001093	100000000001203	1000000000001551	1000000000001731
144	100000000001141	100000000001273	1000000000001617	1000000000001743
145	100000000001161	100000000001303	1000000000001623	1000000000001747
146	100000000001177	100000000001381	1000000000001669	1000000000001771
147	100000000001191	100000000001387	1000000000001741	1000000000001789
148	100000000001207	100000000001413	1000000000001749	1000000000001833
149	100000000001239	100000000001423	1000000000001819	1000000000001873
150	100000000001261	100000000001467	1000000000001839	1000000000001939
151	100000000001267	100000000001549	1000000000001867	1000000000002013
152	100000000001269	100000000001593	1000000000001897	1000000000002049
153	100000000001291	100000000001611	1000000000002037	1000000000002053
154	100000000001309	100000000001623	1000000000002161	1000000000002083
155	100000000001339	100000000001647	1000000000002193	1000000000002137
156				
157	1e17	1e18		
158	10000000000000003	10000000000000003		
159	10000000000000013	10000000000000009		
160	10000000000000019	100000000000000031		


```

161 10000000000000021      10000000000000079
162 10000000000000049      10000000000000177
163 10000000000000081      10000000000000183
164 10000000000000099      10000000000000201
165 10000000000000141      10000000000000283
166 10000000000000181      10000000000000381
167 10000000000000337      10000000000000387
168 10000000000000339      10000000000000507
169 10000000000000369      10000000000000523
170 10000000000000379      10000000000000583
171 10000000000000423      10000000000000603
172 10000000000000519      10000000000000619
173 10000000000000543      10000000000000621
174 10000000000000589      10000000000000799
175 10000000000000591      10000000000000841
176 10000000000000609      10000000000000861
177 10000000000000669      10000000000000877
178 10000000000000691      10000000000000913
179 10000000000000781      10000000000000931
180 10000000000000787      10000000000000997
181 10000000000000817      10000000000001093
182 10000000000000819      10000000000001191
183 10000000000000871      10000000000001267
184 10000000000000889      10000000000001323
185 10000000000001053      10000000000001347
186 10000000000001071      10000000000001359
187 10000000000001093      10000000000001453
188 10000000000001099      10000000000001459
189 10000000000001107      10000000000001537
190 10000000000001201      10000000000001563
191 10000000000001219      10000000000001593
192 10000000000001221      10000000000001659
193 10000000000001243      10000000000001683
194 10000000000001249      10000000000001729
195 10000000000001287      10000000000001743
196 10000000000001293      10000000000001771
197 10000000000001327      10000000000001827
198 10000000000001399      10000000000001879
199 10000000000001429      10000000000001953
200 10000000000001449      10000000000002049
201 10000000000001459      10000000000002097
202 10000000000001573      10000000000002137
203 10000000000001603      10000000000002217
204 10000000000001623      10000000000002271
205 10000000000001641      10000000000002319
206 10000000000001903      10000000000002481
207 10000000000001921      10000000000002493

```

6.6 Number theory theorems | 数论定理

6.6.1 三平方和定理

对于任意自然数 $ifac$, 可以表示为

$$(n, m) = (n, n - m)$$

形式的

$$(n, m) = (n - 1, m) + (n - 1, m - 1)$$

的条件是

$$k(n, k) = n(n - 1, k - 1)$$

6.6.2 拉格朗日四平方和定理

对于任意自然数

$$k^i(n, k) = n^i(n - i, k - i)$$

, 都可以表示为

$$(n, m)(m, k) = (n, k)(n - k, m - k)$$

, 其中

$$\sum_{k=0}^n (m1, k)(m2, n - k) = (m1 + m2, n)$$

可以为

$$\sum_{k=0}^n (k, m1)(n - k, m2) = (n + 1, m1 + m2 + 1)$$

暴力代码大约为

$$\sum_{\sum B_i = M} \prod_{i=1}^n (B_i, A_i) = (M + n - 1, \sum A_i + n - 1)$$

优化代码大约为

$$\sum_{i=0}^n (n, i)(i, j) = (n, j) * 2^{n-j}$$

到

$$\sum_{i=0}^n (n, i)^2 = (2n, n)$$

```

1 void fourSquares(int n) {
2     for (int a = 0; a * a <= n; ++a)
3         for (int b = 0; a * a + b * b <= n; ++b)
4             for (int c = 0; a * a + b * b + c * c <= n; ++c) {
5                 int d2 = n - a * a - b * b - c * c;
6                 int d = sqrt(d2);
7                 if (d * d == d2) {
8                     std::cout << a << ' ' << b << ' ' << c << ' ' << d << '\n';
9                     return;

```

```

10     }
11 }
12 }

```

令

$$(\alpha, i) = \alpha^i / i!$$

如果要求

$$\sum_{i=0}^m (-1)^i (n, i) = (-1)^m (n-1, m)$$

, 条件是

$$[n, m] = [n-1, m-1] + (n-1) * [n-1, m]$$

和

$$[0, 0] = 1$$

至少一个是偶数且

$$\{n, m\} = \{n-1, m-1\} + m * \{n-1, m\}$$

6.6.5 预处理因数

我们可以在 $x^n = \sum_{k=0}^n \{n, k\} (x, k) k!$ 的时间内处理出 n^m 中所有数字的所有因数

```

1 for (int i = 1; i <= N; ++i) // 预处理 [1, N] 内所有数的所有因数 {
2     for (int j = i; j <= N; j += i) {
3         f[j].push_back(i);
4     }
5 }

```

6.6.6 欧拉筛

```

1 /**
2  * @struct Prime
3  * @brief 质数筛法结构体（欧拉筛/线性筛）
4  */
5 struct Prime {
6     int n; // 筛法处理的上界
7     int primeCnt = 0; // 质数计数器
8     std::vector<int> prime; // 存储所有筛出的质数（从 prime[1] 开始存储）
9     std::vector<bool> vis; // 标记数组, vis[i]=true 表示 i 是合数
10    std::vector<int> minPrimeFactor; // 记录每个数的最小质因子
11
12    /**
13     * @brief 构造函数, 执行线性筛法预处理
14     * @param n 筛法处理的上界值
15     */
16    Prime(int n) : n(n), prime(n+1), vis(n+1), minPrimeFactor(n+1) {
17        // 定义并立即执行筛法 lambda 函数
18        auto getPrime = [&]() {
19            // 0 和 1 不是质数, 直接标记
20            vis[0] = vis[1] = true;
21        };
22    }
23 };

```

6.6.3 雅可比四平方和定理

设能满足

$$\sum_{i=0}^n i(n, i) = n2^{n-1}$$

的自然数解个数为

$$\sum_{i=0}^n i^2(n, i) = n * (n+1)2^{n-2}$$

, 那么如果

$$\sum_{i=0}^n i(n, i)^2 = n * (2n-1, n-1)$$

为奇数,

$$\sum_{i=0}^n (i, k) = (n+1, k+1)$$

, 否则我们记

$$\sum_{i=0}^n (m+i, i) = (n+m+1, n)$$

, 即去掉所有

$$\sum_{i=0}^n (m+i, s+i) = (n+m+1, n+s) - (m, s-1)$$

的质因子, 此时

$$f_{n+1} = \sum_{i=0}^n (n-i, i) \text{ 其中 } f \text{ 为斐波那契数列}$$

。其中

$$\sum_{i=0}^n (n, i) x^i = (x+1)^n$$

表示

$$\sum_{i=0}^n (n, i) a^i b^{n-i} = (a+b)^n$$

的约数和

6.6.4 毕达哥拉斯三元组

$$\sum_{i=0}^{\infty} (\alpha, i) x^{\alpha-i} y^i = (x+y)^\alpha$$

```

22 // 主筛法循环：从2开始到n
23 for (int i = 2; i <= n; ++i) {
24     // 如果i未被标记为合数，则它是质数
25     if (!vis[i]) {
26         prime[++primeCnt] = i; // 存入质数表
27         minPrimeFactor[i] = i; // 质数的最小质因子是其本身
28     }
29
30     // 用当前数i与已筛出的质数相乘来标记合数
31     for (int j = 1; j <= primeCnt and i * prime[j] <= n; ++j) {
32         // 标记i*prime[j]为合数
33         minPrimeFactor[i * prime[j]] = prime[j]; // 记录最小质因子
34         vis[i * prime[j]] = true; // 标记为合数
35
36         // 关键优化：保证每个合数只被最小质因子筛一次
37         if (i % prime[j] == 0) {
38             break;
39         }
40     }
41 }
42 };
43
44 // 执行筛法
45 getPrime();
46
47 // 调整prime数组大小为实际质数数量+1（因为prime从下标1开始使用）
48 prime.resize(primeCnt + 1);
49 }
50 };

```

6.6.7 欧拉函数

欧拉函数的数论意义：

$$\{0, 0\} = 1$$

表示在

$$\{n, m\} = \sum_{i=0}^m \frac{(-1)^{m-i} i^n}{i!(m-i)!}$$

中和 $x^n = \sum_{k=0}^n \{n, k\} x^k$ 互质的数字的数量公式： $x^n = \sum_{k=0}^n [n, k] (-1)^{n-k} x^k$ 其中 $x^n = \sum_{k=0}^n \{n, k\} (-1)^{n-k} x^k$ 为 x 的质因子 $x^n = \sum_{k=0}^n [n, k] x^k$ 为质因子数量

```

1 /**
2  * @struct Euler
3  * @brief 欧拉筛法实现欧拉函数计算
4  *
5  * 该类使用欧拉筛法在线性时间内预处理欧拉函数值，
6  * 同时记录每个数的最小质因子。
7  */
8 struct Euler {

```

```

9
10 /**
11  * @brief 计算单个数的欧拉函数值（适用于超出预处理范围的大数）
12  * @param x 要计算欧拉函数的数
13  * @return x的欧拉函数值
14  */
15 int getPhi(int x) {
16     // 如果在预处理范围内，直接返回预计算值
17     if (x <= n) {
18         return phi[x];
19     }
20
21     // 对于超出预处理范围的数，直接计算欧拉函数
22     int res = x;
23     // 遍历可能的质因子
24     for (int i = 2; i <= x / i; ++i) {
25         if (x % i == 0) {
26             // 找到一个质因子i
27             res = res / i * (i - 1); // 欧拉函数计算公式
28             // 去除所有i因子
29             while (x % i == 0) {
30                 x /= i;
31             }
32         }
33     }
34     // 处理剩余的大于1的质因子
35     if (x > 1) {
36         res = res / x * (x - 1);
37     }
38     return res;
39 }
40
41 int n; // 预处理的上界
42 std::vector<int> prime; // 存储所有预处理的质数
43 std::vector<int> phi; // 存储欧拉函数值
44 std::vector<int> minPrimeFactor; // 存储每个数的最小质因子
45 std::vector<bool> vis; // 标记是否为合数
46 int primeCnt = 0; // 质数计数器
47
48 /**
49  * @brief 构造函数，执行欧拉筛预处理
50  * @param n 预处理的上界
51  */
52 Euler(int n) : n(n),
53               prime(n + 1),
54               phi(n + 1),
55               vis(n + 1, false),
56               minPrimeFactor(n + 1) {
57     // 定义并立即执行欧拉筛函数
58     auto getPhi = [&]() {
59         // 初始化特殊情况
60         phi[0] = 0; // 通常认为φ(0)未定义，这里设为0

```

```

60 phi[1] = 1; // φ(1) = 1
61 vis[0] = vis[1] = true; // 0和1不是质数
62
63 // 欧拉筛主循环
64 for (int i = 2; i <= n; ++i) {
65     if (!vis[i])
66     {
67         // i是质数
68         minPrimeFactor[i] = i; // 质数的最小质因子是其本身
69         prime[++primeCnt] = i; // 存入质数表
70         phi[i] = i - 1; // 质数的欧拉函数值为i-1
71     }
72
73     // 用当前数i与已知质数筛合数
74     for (int j = 1; j <= primeCnt && i * prime[j] <= n; ++j) {
75         vis[i * prime[j]] = true; // 标记为合数
76         minPrimeFactor[i * prime[j]] = prime[j]; // 记录最小质因子
77
78         if (i % prime[j] == 0) {
79             // i包含prime[j]因子时的欧拉函数计算
80             phi[i * prime[j]] = phi[i] * prime[j];
81             break; // 保证每个合数只被最小质因子筛一次
82         }
83         // i与prime[j]互质时的欧拉函数计算
84         phi[i * prime[j]] = phi[i] * (prime[j] - 1);
85     }
86 }
87
88 // 执行欧拉筛预处理
89 getPhi();
90
91 }
92
93 };

```

6.6.8 莫比乌斯函数

莫比乌斯函数最重要的性质:

$$n! = \sum_{k=0}^n [n, k]$$

用莫比乌斯反演来写也就是

$$\sum_{k=1}^n (n, k) \{k, m\} = \{n+1, m+1\}$$

$$\sum_{k=1}^n (n, k) [k, m] = [n+1, m+1]$$

```

1 struct Mobius {
2     int n; // 预计算的最大数值范围
3

```

```

4 // 获取x的莫比乌斯函数值
5 int getMo(int x) {
6     // 如果x在预计算范围内, 直接返回结果
7     if (x <= n) {
8         return mo[x];
9     }
10    // 否则现场计算莫比乌斯函数值
11    int res = 1; // 初始化为1
12    // 分解质因数
13    for (int i = 2; i <= x / i; ++i) {
14        if (x % i == 0) // 找到质因数 {
15            res *= -1; // 每找到一个质因数, 符号翻转
16            int tot = 0; // 计算该质因数的指数
17            while (x % i == 0) {
18                x /= i;
19                ++tot;
20            }
21            // 如果质因数的指数大于1, 莫比乌斯函数值为0
22            if (tot != 1) {
23                return 0;
24            }
25        }
26    }
27    // 处理剩余的质因数
28    if (x != 1) {
29        res *= -1;
30    }
31    return res;
32 }
33
34 // 下面是预计算莫比乌斯函数用到的成员变量
35 std::vector<int> vis; // 标记是否为合数
36 std::vector<int> prime; // 存储素数
37 std::vector<int> mo; // 存储莫比乌斯函数值
38 std::vector<int> minPrimeFactor; // 存储最小质因子
39 int primeCnt = 0; // 素数计数器
40
41 // 构造函数, 预计算1到n的莫比乌斯函数
42 Mobius(int n) : n(n), vis(n + 1), prime(n + 1), mo(n + 1), minPrimeFactor(n + 1)
43 {
44     vis[0] = vis[1] = 1; // 0和1不是素数
45     mo[0] = mo[1] = 1; // 0和1的莫比乌斯函数值设为1 (注: 实际上μ(1)=1, μ(0)通常未定义)
46
47     // 线性筛法计算莫比乌斯函数
48     for (int i = 2; i <= n; ++i) {
49         if (!vis[i]) // i是素数 {
50             prime[++primeCnt] = i; // 记录素数
51             mo[i] = -1; // 素数的莫比乌斯函数值为-1
52         }
53         // 用当前已知素数筛i的倍数

```

```

53     for (int j = 1; j <= primeCnt and i * prime[j] <= n; ++j) {
54         vis[i * prime[j]] = 1; // 标记为合数
55         if (i % prime[j] == 0) // i包含prime[j]的平方因子 {
56             mo[i * prime[j]] = 0; // 莫比乌斯函数值为0
57             break;
58         } else {
59             mo[i * prime[j]] = -mo[i]; // 新的素数因子, 符号翻转
60         }
61     }
62 }
63 }
64 };

```

6.6.9 欧拉定理

6.6.10 扩展欧拉定理

6.6.11 扩展欧几里得算法

即求

$$\{n, n-1\} = [n, n-1] = (n, 2)$$

的通解根据裴蜀定理我们知道当 $\{n, 0\}, \{n, 1\}, \dots, \{n, n\}$ 所以该二元不定方程有解当且仅当 $\{0, n\}, \{1, n\}, \dots, \{k, n\}$ 我们考虑求出简单式的解再乘 $H(n) = \sum_i \{i, n\} x^i$ 即可同时我们发现假设一组解 $H(n) = \sum_i (\{i-1, n-1\} + n\{i-1, n\}) x^i = nxH(n) + xH(n-1)$ 是方程 $H(n) = \frac{xH(n-1)}{1-nx}$ 的一组特解那么我们可以写出其通解通解形式为 $H(n) = \frac{x^n}{\prod_{i=1}^n (1-ix)}$ 那么我们考虑求出一组特解即可 *size* 再次根据裴蜀定理我们有 k 结合两个式子我们有 RE 那么根据欧几里得算法的过程一定会递归到 $\sum_{i=0}^n \sum_{j=0}^i \{i, j\}$ 那么此时结果为 $\sum_{j=0}^n \sum_{i=0}^n \sum_{k=0}^j \frac{(-1)^{j-k} k^i}{k!(j-k)!}$ 至此我们的特解求解完成最终通解仍然要乘 $\sum_{j=0}^n \sum_{k=0}^j \frac{(-1)^{j-k}}{(j-k)!} \frac{\sum_{i=0}^n k^i}{k!}$

```

1  template <class T>
2  class ExtendGcd {
3  public:
4      ExtendGcd(T a, T b, T c)
5          : a(a), b(b), c(c), hasSolution(false) {
6          Work();
7      }
8      // 传入 ax+by=c的参数 a b c
9      std::pair<T, T> particularSolution() const {
10         return std::make_pair(x_, y_);
11     }
12     std::array<T, 4> generalSolution() const {
13         auto [x, y] = particularSolution();
14         return {x, b / gcd, y, a / gcd};
15         // 返回通解 {x+k*b/gcd, y+k*a/gcd}
16     }
17
18 private:

```

```

19 T a;           // 方程系数a
20 T b;           // 方程系数b
21 T c;           // 方程右侧常数c
22 T gcd;         // a和b的最大公约数
23 T x_;          // 特解x
24 T y_;          // 特解y
25 bool hasSolution; // 方程是否有解
26
27 static T extendGcd(T a, T b, T &x, T &y) {
28     if (b == 0) {
29         x = 1;
30         y = 0;
31         return a;
32     }
33     T tx, ty;
34     T gcd = extendGcd(b, a % b, tx, ty);
35     x = ty;
36     y = tx - (a / b) * ty;
37     return gcd;
38 }
39 void Work() {
40     // 特殊情况处理
41     if (a == 0 && b == 0) {
42         hasSolution = (c == 0);
43         return;
44     }
45
46     // 计算gcd(a,b)和特解
47     gcd = extendGcd(std::abs(a), std::abs(b), x_, y_);
48     // 调整符号
49     if (a < 0)
50         x_ = -x_;
51     if (b < 0)
52         y_ = -y_;
53
54     // 检查是否有解
55     if (c % gcd != 0) {
56         hasSolution = false;
57         return;
58     }
59     // 调整特解以满足 ax + by = c
60     T factor = c / gcd;
61     x_ *= factor;
62     y_ *= factor;
63
64     hasSolution = true;
65 }
66 };

```

6.6.12 扩展中国剩余定理

扩展中国剩余定理是用来求解一次同余方程组的通解考虑到 $O(n \log n)$ 会比 $O(n)$ 适用性更广 (S 数可以不互质) 我们直接考虑 n 方程形式即 $A_1, A_2 \dots A_n$ (我们用 A_i 表示都满足) 我么考虑取出前两个式子 P_i ,

$$\left| \bigcap_{i=1}^n \overline{A_i} \right| = |S| + \sum_{T \subseteq \{1,2,\dots,n\}, T \neq \emptyset} (-1)^{|T|} \left| \bigcap_{i \in T} A_i \right|$$

$$\left| \bigcup_{i=1}^n A_i \right| = \sum_{T \subseteq \{1,2,\dots,n\}} (-1)^{|T|-1} \left| \bigcap_{i \in T} A_i \right|$$

我们做差得到 1 我们通过 Exgcd 解出 2 和 k 的通解我们得到 k 和 k 后再次代入 $\alpha(k)$ 中得到 k 其中

$$\alpha(0) = |S|$$

表示刚刚解出的

$$\alpha(1) = \sum_i |A_i|$$

的特解不难发现我们再次得到了一个同余式

$$\alpha(2) = \sum_{i,j} |A_i \cap A_j|$$

我们不断合并直至只剩一个式子即得答案复杂度

...

```

1 template <class T>
2 struct ExGcd {
3     T operator()(const T &a, const T &b, T &x, T &y) {
4         if (b == 0)
5             return (x = 1, y = 0, a);
6         T g = (*this)(b, a % b, y, x);
7         y -= a / b * x;
8         return g;
9     }
10 };
11 template <class T, class G>
12 struct ExCrt : public ExGcd<T> {
13     std::vector<std::pair<T, T>> q;
14     void insert(T a, T mod) {
15         q.push_back({a, mod});
16     }
17 };
18 // 方程组 x ≡ a (模 mod) 返回最小正解
19 // 无解返回 -1
20 std::pair<T, T> get() {
21     T res = 0, M = 1;
22     for (auto [a, mod] : q) // 返回最小正数解和其mod数 [x,mod] 即 x+k*mod 均为解 {
23         T r = (a - res) % mod;

```

```

24     r += (r < 0 ? mod : 0);
25
26     T x, y;
27     T g = (*this)(M, mod, x, y);
28     if (r % g) {
29         q.clear();
30         return {-1, -1};
31     }
32
33     x = (G(x) * r / g % (mod / g));
34     x += (x < 0 ? mod / g : 0);
35
36     T Last = M;
37     M = M / g * mod;
38     res = (G(x) * Last % M + res) % M;
39 }
40 q.clear();
41 return {res, M};
42 }
43 };
44 ExCrt<int, __int128> crt;

```

6.6.13 扩展大步小步算法

我们先提出 BSGS 算法 BSGS 的算法思想很多时候能发挥重要作用, 其中文名为大步小步算法, 实际上是在进行一种优雅的暴力枚举我们的目的是求

$$\alpha(k) = \sum_{T \subseteq \{1,2,\dots,n\}, |T|=k} \left| \bigcap_{i \in T} A_i \right|$$

的解 $\alpha(k)$ 我们设 $P(P \geq k)$ 其中 (P, k) 我们称这个 m 为大步我们移项以后 $\beta(k)$ 我们称 1 为小步我们枚举 k 从 $\beta(k)$ 到 $\alpha(k) - \sum_{i=k+1}^n (i, k) \beta(i)$, 用哈希表存下所有左边的值, 再枚举 $O(n^2)$ 同样从

$$\beta(k) = \sum_{i=k}^n (-1)^{i-k} (i, k) \alpha(i)$$

到 $O(n)$, 求出右边的值, 在哈希表中查找是否存在答案即可, 我们发现所有小步的长度和等于一大步, 我们发现这覆盖的范围是 β , 当 S , 由于扩展欧拉定理的内容, 我们发现这已经可以覆盖所有情况, 复杂度为 $\max(S)$ 然后我们考虑如何解决 (a, P) 不互质的情况得到 EXBSGS 算法考虑把同余式转化为方程式, 即 $\min(S)$ 我们设 $\max_k(s)$, 那么由裴蜀定理当 k 的时候原方程有解, 如果有解, 我们可以转化为

$$\max(S) = \sum_{T \subseteq S, T \neq \emptyset} (-1)^{|T|-1} \min(T)$$

我们重复此过程直至 $\min(S) = \sum_{T \subseteq S, T \neq \emptyset} (-1)^{|T|-1} \max(T)$ 我们设递归了 k 次, 这

$$\max_k(S) = \sum_{T \subseteq S, |T| \geq k} (-1)^{|T|-k} (|T| - 1, k - 1) \min(T)$$

次

$$\min_k(S) = \sum_{T \subseteq S, |T| \geq k} (-1)^{|T|-k} (|T| - 1, k - 1) \max(T)$$

的乘积为 $\min - \max$ 那么原方程为 $E(\max(S)) = \sum_{T \subseteq S, T \neq \emptyset} (-1)^{|T|-1} E(\min(T))$ 注意到此时变成了一个 BSGS 问题但前面多了一个常数我们可以求其逆元丢到右边即可

```

1 // 扩展BSGS算法：求解高次同余方程  $a^x \equiv b \pmod{P}$  的最小正整数解，其中p不需要是质数
2 // 参数说明：
3 //   a: 底数
4 //   b: 余数
5 //   P: 模数
6 // 返回值：
7 //   成功返回最小正整数解x，无解返回-1
8
9 template <class T, ll P>
10 class ExBsgs {
11 public:
12     ExBsgs() {};
13     T getAns(T a, T b) {
14         // 首先对参数取模处理
15         a %= P, b %= P;
16
17         // 特殊情况处理
18         if (b == 1 || P == 1)
19             { // 任何数的0次方都是1，且模1的同余式总是成立
20                 return T(0);
21             }
22
23         T d, k = 0, A = 1; // d:最大公约数, k:迭代次数, A:累积系数
24
25         // 第一阶段：消除a和p的公因数
26         while (true) {
27             d = std::gcd(a, P); // 计算a和p的最大公约数
28             if (d == 1)
29                 { // 当a和p互质时退出循环
30                     break;
31                 }
32             if (b % d)
33                 { // 如果b不能被d整除，则方程无解
34                     return T(-1);
35                 }
36             k++; // 迭代次数增加
37             b /= d; // 约简b
38             P /= d; // 约简p
39             A = A * (a / d) % P; // 更新累积系数
40
41             // 检查是否已经找到解
42             if (A == b)
43                 { // 如果  $A \equiv b \pmod{P}$ ，则当前k就是解
44                     return k;
45                 }
46         }
47     }
48 };

```

```

46 }
47
48 // 第二阶段：BSGS算法部分
49 T m = ceil(sqrt(P)); // 计算分块大小
50
51 // 预处理：构建哈希表 (baby-step)
52 T t = b; // t用于存储中间结果
53 std::unordered_map<T, int> hash; // 哈希表存储值到索引的映射
54 hash[b] = 0; // 初始状态:  $b \equiv b \pmod{P}$ 
55 for (int j = 1; j < m; ++j) {
56     t = t * a % P; // 计算  $b \cdot a^j \pmod{P}$ 
57     hash[t] = j; // 存入哈希表
58 }
59
60 // 计算  $a^m \pmod{P}$  (giant-step的基数)
61 T mi = 1;
62 for (int i = 1; i <= m; ++i) {
63     mi = mi * a % P;
64 }
65
66 // 搜索匹配项 (giant-step)
67 t = A; // 从累积系数A开始
68 for (int i = 1; i <= m; ++i) {
69     t = t * mi % P; // 计算  $A \cdot (a^m)^i \pmod{P}$ 
70     if (hash.count(t))
71         { // 如果在哈希表中找到匹配
72             // 解为:  $i \cdot m - \text{hash}[t] + k$ 
73             return T(i * m - hash[t] + k);
74         }
75 }
76 // 没有找到解
77 return T(-1);
78 }
79 };

```

6.6.14 卢卡斯定理

即 n 不难发现我们写个递归函数就行，请注意递归出口是 n ，此时返回 n 。

6.6.15 原根

阶的定义：在 k 意义下满足 $E(\min(T))$ 的最小正整数 T 称作 $E(x) = \frac{1}{P(x)}P(x)$ 意义下的阶原根的定义：一个数在 $P(\min(T))$ 意义下的阶等于 $P(\min(T)) = \sum_{X \subseteq T} P(X)$ 那么该数称作 K 的一个原根首先已有数学家证明一个数 $E(\min_k(S))$ 的最小原根不会超过 $E(\max_{n-k+1}(S))$ 所以我们考虑从小到大暴力枚举在此之前我们已经证明只有 $E(\min(T))$ 存在原根（这里 P 是质数）所以我们先通过欧拉筛预处理判断出一个数是否存在原根在存在的情况下我们先求出最小原根从 1 开始枚举然后对每个数分别 Check 假设我们枚举到数字 2^n 我们枚举 $\sum_{X \subseteq T} P(X)$ 的所有质因数 $|T|$ 如果 $dp[i][j][k]$ 也是 i 的阶那么 $\sum_{X \subseteq T} P(X)$ 就不是 j 的原根当我们找到最小原根 $|T|$ 后对于所有的 k 满

足 $\sum_{X \subseteq T} P(X)$ 的所有 1)，满足 $f(n) = \sum_{i=0}^n (n, i)g(i) \iff g(n) = \sum_{i=0}^n (-1)^{n-i} (n, i)f(i)$ 也是原根至此我们得到了求出所有原根的方法且根据最后一个定理我们知道一个数 2) 如果存在原根那么它将有 $f(n) = \sum_{i=0}^n (-1)^i (n, i)g(i) \iff g(n) = \sum_{i=0}^n (-1)^i (n, i)f(i)$ 个原根

```

1  /**
2  * @class PrimitiveRootFinder
3  * @brief 原根求解器
4  *
5  * 该类封装了求解模数x的最小原根及其所有原根的方法，
6  * 基于欧拉函数和快速幂算法实现。
7  */
8  class PrimitiveRootFinder {
9  public:
10     /**
11      * @brief 构造函数，初始化欧拉筛实例
12      * @param euler 预初始化的欧拉筛类实例
13      */
14     PrimitiveRootFinder() {}
15
16     /**
17      * @brief 判断给定数x是否存在原根
18      * @param x 要判断的数
19      * @return 如果x有原根返回true，否则false
20      *
21      * @note 根据数论理论，只有以下形式的数有原根：
22      * 1. 1, 2, 4
23      * 2. p^a, 其中p是奇素数
24      * 3. 2*p^a, 其中p是奇素数
25      */
26     bool hasRoot(int x) const {
27         // 特殊情况处理
28         if (x == 1 || x == 2 || x == 4) {
29             return true;
30         }
31
32         // 处理偶数情况
33         if (x % 2 == 0) {
34             x /= 2;
35             // 如果除以2后还是偶数，则不符合2*p^a的形式
36             if (x % 2 == 0) {
37                 return false;
38             }
39         }
40
41         // 检查是否为p^a的形式
42         int primeFactor = euler.minPrimeFactor[x];
43         if (primeFactor == 2) {
44             return false; // 不满足p是奇素数的条件
45         }
46     }
47

```

```

48     // 检查x是否能被分解为p^a
49     while (x % primeFactor == 0) {
50         x /= primeFactor;
51     }
52
53     // 如果x被完全分解，说明是p^a的形式
54     return x == 1;
55 }
56
57 /**
58 * @brief 获取模x的最小原根
59 * @param x 要求原根的模数
60 * @return 最小原根，如果不存在原根则返回-1
61 */
62 int findMinRoot(int x) const {
63     if (!hasRoot(x)) {
64         return -1;
65     }
66
67     const int phi = euler.getPhi(x);
68     const auto primeFactors = getPrimeFactor(phi);
69
70     // 枚举可能的g值寻找最小原根
71     for (int g = 1; ++g) {
72         // 检查g^phi ≡ 1 P x
73         if (fpow(g, phi, x) != 1) {
74             continue;
75         }
76
77         // 检查g^(phi/p) ≠ 1 P x 对所有phi的素因子p
78         bool isPrimitive = true;
79         for (int p : primeFactors) {
80             if (fpow(g, phi / p, x) == 1) {
81                 isPrimitive = false;
82                 break;
83             }
84         }
85
86         if (isPrimitive) {
87             return g;
88         }
89     }
90 }
91
92 /**
93 * @brief 获取模x的所有原根
94 * @param x 要求原根的模数
95 * @return 包含所有原根的vector，按升序排列
96 */
97 std::vector<int> findAllRoot(int x) const {
98     std::vector<int> roots;

```



```

99
100     const int g = findMinRoot(x);
101     if (g == -1) {
102         return roots;
103     }
104     roots.push_back(g);
105     const int phi = euler.getPhi(x);
106     int current = g;
107     // 生成所有原根: g^k, 其中gcd(k, phi) = 1
108     for (int k = 2; k < phi; ++k) {
109         current = current * g % x;
110         if (std::gcd(k, phi) == 1) {
111             roots.push_back(current);
112         }
113     }
114
115     std::sort(roots.begin(), roots.end());
116     return roots;
117 }
118
119 private:
120 /**
121  * @brief 快速幂算法
122  * @param a 底数
123  * @param b 指数
124  * @param P 模数
125  * @return (a^b) % P
126  */
127 static int fpow(int a, int b, int P) {
128     int res = 1;
129     a %= P;
130     while (b > 0) {
131         if (b & 1) {
132             res = 1ll * res * a % P;
133         }
134         b >>= 1;
135         a = 1ll * a * a % P;
136     }
137     return res;
138 }
139
140 /**
141  * @brief 获取数的所有不同质因数
142  * @param n 要分解的数
143  * @return 包含n的所有不同质因数的vector
144  */
145 std::vector<int> getPrimeFactor(int n) const {
146     std::vector<int> factors;
147     if (n == 1) {
148         return factors;
149     }

```

```

150         while (n != 1) {
151             int p = euler.minPrimeFactor[n];
152             factors.push_back(p);
153             while (n % p == 0) {
154                 n /= p;
155             }
156         }
157         return factors;
158     }
159 };

```

6.6.16 Pell 方程

即 $f(n) = \sum_{i=n}^? (i, n)g(i) \iff g(n) = \sum_{i=n}^? (-1)^{i-n} (i, n)f(i)$, 求最小正整数解 4) 由于答案可以很大, 代码由 $f(n) = \sum_{i=n}^? (-1)^i (i, n)g(i) \iff g(n) = \sum_{i=n}^? (-1)^i (i, n)f(i)$ 实现具体求解方法涉及连分数, 这里不过多介绍

```

1 n = int(raw_input())
2 j = 1
3 while j * j < n:
4     j += 1
5 if j * j == n:
6     print(j, 1)
7 if j * j > n:
8     p = [0 for i in range(0, 1001)]
9     q = [0 for i in range(0, 1001)]
10    a = [0 for i in range(0, 1001)]
11    g = [0 for i in range(0, 1001)]
12    h = [0 for i in range(0, 1001)]
13    p[1] = q[0] = h[1] = 1
14    p[0] = q[1] = g[1] = 0
15    a[2] = j - 1
16    i = 2
17    while 1:
18        g[i] = -g[i - 1] + a[i] * h[i - 1]
19        h[i] = (n - g[i] * g[i]) / h[i - 1]
20        a[i + 1] = (g[i] + a[2]) / h[i]
21        p[i] = a[i] * p[i - 1] + p[i - 2]
22        q[i] = a[i] * q[i - 1] + q[i - 2]
23        if (p[i] * p[i] - n * q[i] * q[i] == 1):
24            print(p[i], q[i])
25            break
26        i += 1

```

6.6.17 类欧几里得算法

类欧算法是一类特定式子的求和方法 5) $f(n, m) = \sum_{i=0}^n \sum_{j=0}^m (n, i)(m, j)g(i, j) \iff$
 $g(n, m) = \sum_{i=0}^n \sum_{j=0}^m (-1)^{n+m-i-j} (n, i)(m, j)f(i, j)$ 6) 注意代码并没有处理取模的

问题第一段代码只解决了 $f(n, m) = \sum_{i=0}^n \sum_{j=0}^m (-1)^{i+j} (n, i)(m, j)g(i, j) \iff$
 $g(n, m) = \sum_{i=0}^n \sum_{j=0}^m (-1)^{i+j} (n, i)(m, j)f(i, j)$ 的值递推公式为若 7) $f(n, m) =$
 $\sum_{i=n}^? \sum_{j=m}^? (i, n)(j, m)g(i, j) \iff g(n, m) = \sum_{i=n}^? \sum_{j=m}^? (-1)^{i+j-n-m} (i, n)(j, m)f(i, j)$
 否则 8), 其中 $f(n, m) = \sum_{i=n}^? \sum_{j=m}^? (-1)^{i+j} (i, n)(j, m)g(i, j) \iff g(n, m) =$
 $\sum_{i=n}^? \sum_{j=m}^? (-1)^{i+j} (i, n)(j, m)f(i, j)$ 递归出口为

1, 1, 2, 4, 9, 20, 48, 115, 286, 719, 1842, 4766

第二段代码同时解决了

$$a_{n+1} = (\sum_{k=1}^n (\sum_{d|k} d * a(d)) + a(n - k + 1)) / n$$

推导公式过于复杂不再描述

```
1
2 int calc(int a, int b, int c, int n) //(ai+b)/c 对i属于[0,n]求和 {
3     if (a == 0) {
4         return b / c * (n + 1);
5     } else if (a >= c || b >= c) {
6         return (a / c) * n * (n + 1) / 2 + (b / c) * (n + 1) + calc(a % c, b % c, c,
7             n);
8     } else {
9         return (a * n + b) / c * n - calc(c, c - b - 1, a, (a * n + b) / c - 1);
10    }
11 }
```

6.6.18 整除分块

我们考虑如何解决各式各样的整除分块我们先考虑一维的情况即 n 其中 m 是常数

$$a_n - \sum_{i=1}^{n/2} a_i a_{n-i}$$

为单元函数且单调增加, 我们只需要每次求出右端点 n , 然后

$$a_n - \sum_{i=1}^{n/2} a_i a_{n-i} + a_{n/2} (a_{n/2} + 1) / 2$$

即可, 问题在于如何求出 $prufer$, 我们来推式子首先我们已经知道左端点 1, 也就相当于知道了当前的值 $prufer$ 也就是说 1 那么 n 应该满足 $prufer$ 那么 $n - 2$ 然后我们考虑二维的情况, 一般形式为 $prufer$, 在这种情况下我们发现有两个限制, 但我们只需要每次操作取他们的交集即可, 因为有 $prufer$ 在。可以保证全覆盖也就是取 $n - 2$, 至于函数的形式我们参考一维即可上取整的整除分块

```
1 std::vector<std::pair<ll, ll>> floorBlock(ll n, ll m)
2 // n/i 从1到m的整除分块区间 {
3     std::vector<std::pair<ll, ll>> res;
4     for (ll l = 1, r; l <= std::min(n, m); l = r + 1) {
5         r = std::min(n / (n / l), std::min(n, m));
```

```
6         res.emplace_back(l, r);
7     }
8     return res;
9 }
```

6.6.19 莫比乌斯反演

其实就是一个式子如果 2 那么有 $prufer$ 当然也和 $prufer$ 等价

6.6.20 杜教筛

杜教筛是一种能在亚线性时间内求出积性函数前缀和的方法构造方法如下我们设 u 表示所求函数, $prufer$ 表示辅助函数, v 表示前缀和函数那么我们有如下式子 $u - v$ 加入我们可以构造恰当的数论函数 2 使得: 1. 可以快速计算 $prufer$ 2 可以快速计算 D_i 的前缀和, 从而用数论分块来递归求解 $D_i - 1$ 我们发现构造是一个困难的事情我们在此给出欧拉函数和莫比乌斯函数的板子对于欧拉函数我们可以构造 n 对于莫比乌斯函数我们可以构造 $prufer$ 套公式即可

```
1 // 杜教筛模板类, 用于高效计算欧拉函数φ(n)和莫比乌斯函数μ(n)的前缀和
2 struct DujiaoSieve {
3     const int N = 2e6;
4     std::vector<int> prime, phi, mo;
5     std::vector<bool> vis;
6     int primeCnt = 0;
7     // 哈希表, 用于存储已经计算过的前缀和结果 (记忆化)
8     std::unordered_map<ll, int> hashPhi; // 存储欧拉函数前缀和
9     std::unordered_map<ll, int> hashMo; // 存储莫比乌斯函数前缀和
10
11     // 构造函数, 初始化时调用init()进行预处理
12     DujiaoSieve() : prime(N + 1), phi(N + 1), mo(N + 1), vis(N + 1, 0) {
13         init();
14     }
15
16     // 初始化函数, 使用线性筛预处理前N个数的φ和μ值及其前缀和
17     void init() {
18         // 初始化1的特殊情况
19         mo[1] = phi[1] = 1;
20
21         // 线性筛法 (欧拉筛)
22         for (int i = 2; i <= N; ++i) {
23             if (!vis[i]) {
24                 // 如果i是素数
25                 prime[++primeCnt] = i; // 记录素数
26                 phi[i] = i - 1; // 素数的欧拉函数值为i-1
27                 mo[i] = -1; // 素数的莫比乌斯函数值为-1
28             }
29
30             // 用当前已知素数筛i的倍数
31             for (int j = 1; i * prime[j] <= N && j <= primeCnt; ++j) {
32                 vis[i * prime[j]] = true; // 标记为合数
```

```

33
34     if (i % prime[j] == 0)
35     {
36         mo[i * prime[j]] = 0; // i包含prime[j]的平方因子
37         // 莫比乌斯函数值为0 (可以不写, 因为默认初始化为0)
38         phi[i * prime[j]] = phi[i] * prime[j]; // 欧拉函数性质
39         break; // 保证每个数只被最小质因子筛一次
40     } else {
41         mo[i * prime[j]] = -mo[i]; // 新的素数因子, 莫比乌斯函数符号翻转
42         phi[i * prime[j]] = phi[i] * (prime[j] - 1); // 欧拉函数性质
43     }
44 }
45
46 // 计算前缀和数组
47 for (int i = 1; i <= N; ++i) {
48     phi[i] += phi[i - 1]; // 欧拉函数前缀和
49     mo[i] += mo[i - 1]; // 莫比乌斯函数前缀和
50 }
51
52 // 计算欧拉函数前缀和 (杜教筛核心)
53 ll prePhi(int n) {
54     if (n < N)
55     { // 如果在预处理范围内, 直接返回
56         return phi[n];
57     }
58     if (hashPhi.count(n))
59     { // 如果已经计算过, 直接从哈希表中取
60         return hashPhi[n];
61     }
62
63     // 初始估计值 (利用欧拉函数求和公式)
64     ll ans = 1ll * n * (n + 1) / 2;
65
66     // 分块处理, 利用数论分块优化
67     for (int l = 2, r; l <= n; l = r + 1) {
68         r = n / (n / l); // 计算当前块的右边界
69         ans -= 1ll * prePhi(n / l) * (r - l + 1); // 递归计算并减去贡献
70     }
71
72     return hashPhi[n] = ans; // 存储结果到哈希表并返回
73 }
74
75 // 计算莫比乌斯函数前缀和 (杜教筛核心)
76 ll preMo(int n) {
77     if (n < N)
78     { // 如果在预处理范围内, 直接返回
79         return mo[n];
80     }

```

```

81     }
82     if (hashMo.count(n))
83     { // 如果已经计算过, 直接从哈希表中取
84         return hashMo[n];
85     }
86
87     // 初始值为1 (因为μ(1)=1)
88     ll ans = 1;
89
90     // 分块处理, 利用数论分块优化
91     for (int l = 2, r; l <= n; l = r + 1) {
92         r = n / (n / l); // 计算当前块的右边界
93         ans -= 1ll * preMo(n / l) * (r - l + 1); // 递归计算并减去贡献
94     }
95
96     return hashMo[n] = ans; // 存储结果到哈希表并返回
97 }
98 };
99 DujiaoSieve Sieve;

```

6.6.21 Min25 筛

min25 筛是统计一类函数前缀和的方法这类函数有如下特征 $1\ n-2$ 可以被表示成简单多项式的形式 $2\ D(n)$ 需要容易求得 Min25 筛的核心思想是把质数和合数的贡献分开统计, 这样同时解决了一些函数质数前缀和的问题我们设我们要求

$$\frac{(n-2)!}{\prod_{i=1}^n (d_i-1)!}$$

我们先考虑质数如何计算设 cnt 其中 sum 共 $\sum_{i=1}^c cnt(d_i-1)$ 种取值我们考虑 DP, 设 $(n-2, sum)$ $\frac{sum!}{\prod_{i=1}^{cnt} (d_i!)} * (n-cnt)^{n-2-sum}$ 其中 n 表示 n^{n-2} 的最小质因子, $n-2$ 是另一个我们构造出的函数, 满足以下条件 $prufer\ n$ 是完全积性函数 n 可以快速求前缀和 DP 边界为 n^{n-2} 那么我们的转移有 n 因为这里显然不会多出数字满足 n^{n-1} 1 这里的意思是减去最小质因子为 n 的数字的贡献利用了完全积性函数的性质令 $prufer$ 表示小于等于 n 的质数个数, 那么 m 即为我们所求的 i 当然我们会把第一维滚掉用整除分块求出所有 $a[i]$ 并用 $m-1$ 数组映射至此我们已经完成了质数前缀和的工作, 我们想求出所有数的前缀和, 还需要二次 $n^{m-2} * \prod a[i]$ 我们设 1 那么最终答案为 2 有递推式 3 请注意对 A 求出的 $A(i, j)$ 筛只能得到所有的 i 位置的质数前缀和在模板中 j , 对于不同的函数我们只需要修改 A 函数注意取模!!!

```

1 struct Min25 {
2     int n; // 要计算前缀和的上界
3     int block; // 分块大小, 取sqrt(n)
4     int primeCnt = 0; // 素数计数器
5
6     // 存储分块信息:
7     std::vector<int> lowBlockId; // 存储小于等于block的值的id
8     std::vector<int> highBlockId; // 存储大于block的值的id
9     std::vector<int> w; // 存储所有分块点的值
10

```

```

11 std::vector<int> g; // 存储g函数的前缀和
12 std::vector<int> prime; // 存储预处理的小素数
13 std::vector<int> vis; // 筛法标记数组
14 std::vector<int> primePrefix;
15
16 // 根据值val获取其在w数组中的索引
17 int getId(int val) {
18     return (val <= block ? lowBlockId[val] : highBlockId[n / val]);
19 }
20 int calcPrefix(int x) // 获得所有位置的前缀和 {
21     return x;
22 }
23 int calcPrimePrefix(int x) // x表示是第几个质数 {
24     return primePrefix[x];
25 }
26 int calcF(int x) // 对应的函数F(x) {
27     return 1;
28 }
29
30 // 构造函数，初始化Min25筛
31 Min25(int n) : n(n),
32               block(sqrt(n)),
33               w(2 * block + 1), // 分配2*sqrt(n)的空间
34               g(2 * block + 1),
35               prime(block + 1),
36               vis(block + 1, 0),
37               lowBlockId(block + 1),
38               highBlockId(block + 1),
39               primePrefix(block + 1, 0) {
40     // 欧拉筛预处理小素数
41     vis[0] = vis[1] = true; // 0和1不是素数
42     for (int i = 2; i <= block; ++i) {
43         if (!vis[i]) {
44             prime[++primeCnt] = i; // 记录素数
45             primePrefix[primeCnt] = primePrefix[primeCnt - 1] + calcF(i);
46         }
47         // 筛去i的倍数
48         for (int j = 1; j <= primeCnt and i * prime[j] <= block; ++j) {
49             vis[i * prime[j]] = true; // 标记为合数
50             if (i % prime[j] == 0)
51                 { // 保证每个数只被最小素因子筛去
52                     break;
53                 }
54         }
55     }
56
57     // 整除分块预处理
58     int tot = 0; // 分块计数器
59     for (int l = 1, r; l <= n; l = r + 1) {
60         r = n / (n / l); // 计算当前块的右边界
61         int val = n / l; // 当前块的代表值

```

```

62         w[++tot] = val; // 存储分块值
63
64         // 根据值大小选择不同的存储位置
65         if (val <= block) {
66             lowBlockId[val] = tot;
67         } else {
68             highBlockId[n / val] = tot;
69         }
70
71         // 初始化g(0,m)
72         g[tot] = calcPrefix(val) - calcPrefix(1);
73     }
74
75     // DP计算g函数
76     for (int j = 1; j <= primeCnt; ++j)
77     { // 遍历所有小素数
78         for (int i = 1; i <= tot and prime[j] * prime[j] <= w[i]; ++i) {
79             int k = getId(w[i] / prime[j]); // 获取转移位置
80
81             g[i] -= calcF(prime[j]) * (g[k] - calcPrimePrefix(j - 1));
82         }
83     }
84 }
85
86 // 获取质数处函数值的前缀和
87 int getPrimePrefix(int n) {
88     int k = getId(n); // 获取n对应的索引
89     return g[k]; // 返回预处理好的前缀和
90 }
91
92 };

```

6.6.22 常见的积性函数

$A(i, j) \mid i \mid j \implies A(i, j) \mid i \mid j$

6.7 Combination preprocessing | 组合数预处理

6.7.1 组合数预处理

我们考虑 n 线性递推预处理出组合数，我们用 w 表示阶乘， w 表示阶乘的逆元， n 表示逆元。那么有如下递推式： $F(n) \ G(n) \ G(n) = 2^{(n,2)}$ 不难发现我们只要用快速幂求出最后一个 $G(n)$ 的值即可。

```

1 template <class T, T P>
2 struct Comb {
3     void norm(ll &x) {
4         if (x >= P) {
5             x -= P;
6         }

```

```

7     if (x < 0) {
8         x += P;
9     }
10 }
11 void norm(int &x) {
12     if (x >= P) {
13         x -= P;
14     }
15     if (x < 0) {
16         x += P;
17     }
18 }
19 ll mul(ll a, ll b) {
20     ll res = a * b - (ll)(1.L * a * b / P) * P;
21     res %= P;
22     norm(res);
23     return res;
24 }
25 int mul(int a, int b) {
26     int res = 1ll * a * b % P;
27     norm(res);
28     return res;
29 }
30 std::vector<T> fac, ifac, inv;
31 T power(T a, T b) {
32     T res = 1;
33     while (b) {
34         if (b & 1) {
35             res = mul(res, a);
36         }
37         a = mul(a, a);
38         b >>= 1;
39     }
40     return res;
41 }
42 T getInv(T a) {
43     return power(a, P - 2);
44 }
45 void init(int n) {
46     fac.resize(n + 1);
47     ifac.resize(n + 1);
48     inv.resize(n + 1);
49     fac[0] = 1; // 从0开始
50     for (T i = 1; i <= n; i++) {
51         fac[i] = mul(i, fac[i - 1]);
52     }
53     ifac[n] = getInv(fac[n]);
54     for (T i = n; i >= 1; i--) {
55         ifac[i - 1] = mul(i, ifac[i]);
56         inv[i] = mul(ifac[i], fac[i - 1]);
57     }

```

```

58 }
59 T A(int a, int b) {
60     return mul(fac[a], ifac[a - b]);
61 }
62 T C(int a, int b) {
63     if (a < b or b < 0) {
64         return 0;
65     }
66     return mul(A(a, b), ifac[b]);
67 }
68 Comb(int n) {
69     init(n);
70 }
71 };

```

6.7.2 取模类

```

1 using i64 = long long;
2 template <class T>
3 constexpr T power(T a, i64 b) {
4     T res{1};
5     for (; b; b /= 2, a *= a) {
6         if (b % 2) {
7             res *= a;
8         }
9     }
10    return res;
11 }
12
13 constexpr i64 mul(i64 a, i64 b, i64 p) {
14     i64 res = a * b - i64(1.L * a * b / p) * p;
15     res %= p;
16     if (res < 0) {
17         res += p;
18     }
19     return res;
20 }
21
22 template <i64 P>
23 struct MInt {
24     i64 x;
25     constexpr MInt() : x{0} {}
26     constexpr MInt(i64 x) : x{norm(x % getMod())} {}
27
28     static i64 Mod;
29     constexpr static i64 getMod() {
30         if (P > 0) {
31             return P;
32         } else {
33             return Mod;

```

```

34     }
35 }
36 constexpr static void setMod(i64 Mod_) {
37     Mod = Mod_;
38 }
39 constexpr i64 norm(i64 x) const {
40     if (x < 0) {
41         x += getMod();
42     }
43     if (x >= getMod()) {
44         x -= getMod();
45     }
46     return x;
47 }
48 constexpr i64 val() const {
49     return x;
50 }
51 constexpr MInt operator-() const {
52     MInt res;
53     res.x = norm(getMod() - x);
54     return res;
55 }
56 constexpr MInt inv() const {
57     return power(*this, getMod() - 2);
58 }
59 constexpr MInt &operator*=(MInt rhs) & {
60     if (getMod() < (1ULL << 31)) {
61         x = x * rhs.x % (getMod());
62 } else {
63     x = mul(x, rhs.x, getMod());
64 }
65     return *this;
66 }
67 constexpr MInt &operator+=(MInt rhs) & {
68     x = norm(x + rhs.x);
69     return *this;
70 }
71 constexpr MInt &operator-=(MInt rhs) & {
72     x = norm(x - rhs.x);
73     return *this;
74 }
75 constexpr MInt &operator/=(MInt rhs) & {
76     return *this *= rhs.inv();
77 }
78 friend constexpr MInt operator*(MInt lhs, MInt rhs) {
79     MInt res = lhs;
80     res *= rhs;
81     return res;
82 }
83 friend constexpr MInt operator+(MInt lhs, MInt rhs) {
84     MInt res = lhs;

```

```

85     res += rhs;
86     return res;
87 }
88 friend constexpr MInt operator-(MInt lhs, MInt rhs) {
89     MInt res = lhs;
90     res -= rhs;
91     return res;
92 }
93 friend constexpr MInt operator/(MInt lhs, MInt rhs) {
94     MInt res = lhs;
95     res /= rhs;
96     return res;
97 }
98 friend constexpr std::istream &operator>>(std::istream &is, MInt &a) {
99     i64 v;
100     is >> v;
101     a = MInt(v);
102     return is;
103 }
104 friend constexpr std::ostream &operator<<(std::ostream &os, const MInt &a) {
105     return os << a.val();
106 }
107 friend constexpr bool operator==(MInt lhs, MInt rhs) {
108     return lhs.val() == rhs.val();
109 }
110 friend constexpr bool operator!=(MInt lhs, MInt rhs) {
111     return lhs.val() != rhs.val();
112 }
113 friend constexpr bool operator<(MInt lhs, MInt rhs) {
114     return lhs.val() < rhs.val();
115 }
116 };
117 template <>
118 i64 MInt<0>::Mod = 998244353;
119
120 constexpr int P = 998244353;
121 using Z = MInt<P>;
122
123 struct Comb {
124     int n;
125
126     std::vector<Z> _fac;
127     std::vector<Z> _invfac;
128     std::vector<Z> _inv;
129
130     Comb() : n{0}, _fac{1}, _invfac{1}, _inv{0} {}
131
132     Comb(int n) : Comb() {
133         init(n);
134     }
135

```

```

136 void init(int m) {
137     m = std::min<int>(m, Z::getMod() - 1);
138     if (m <= n)
139         return;
140     _fac.resize(m + 1);
141     _invfac.resize(m + 1);
142     _inv.resize(m + 1);
143
144     for (int i = n + 1; i <= m; i++) {
145         _fac[i] = _fac[i - 1] * i;
146     }
147     _invfac[m] = _fac[m].inv();
148     for (int i = m; i > n; i--) {
149         _invfac[i - 1] = _invfac[i] * i;
150         _inv[i] = _invfac[i] * _fac[i - 1];
151     }
152     n = m;
153 }
154
155 Z fac(int m) {
156     if (m > n)
157         init(2 * m);
158     return _fac[m];
159 }
160 Z invfac(int m) {
161     if (m > n)
162         init(2 * m);
163     return _invfac[m];
164 }
165 Z inv(int m) {
166     if (m > n)
167         init(2 * m);
168     return _inv[m];
169 }
170 Z A(int a, int b) {
171     if (a < b or b < 0)
172         return 0;
173     return fac(a) * invfac(a - b);
174 }
175 Z C(int n, int m) {
176     if (n < m || m < 0)
177         return 0;
178     return fac(n) * invfac(m) * invfac(n - m);
179 }
180 } comb;

```

6.7.3 枚举固定大小子集

```

1 void gopersHack(int n, int k) // 一共n个位 保证其中有k个1 {
2     int cur = (1 << k) - 1;

```

```

3     int limit = 1 << n;
4     while (cur < limit) {
5         std::cerr << cur << '\n';
6         // cur是我们想要的数字
7         int lb = cur & (-cur);
8         int r = cur + lb;
9         cur = ((r ^ cur) >> __builtin_ctz(lb) + 2 | r;
10    }
11 }

```

6.8 Tricks | 技巧

6.8.1 Miller Rabin

```

1  /*
2  维基百科：
3  n < 4e9, Prime = [2, 7, 61]
4  n < 3e14, Prime = [2, 3, 5, 7, 11, 13, 17]
5  n < 3e18, Prime = [2, 3, 5, 7, 11, 13, 17, 19, 23]
6  n < 3e23, Prime = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37]
7  */
8  template <class T>
9  struct MillerRabin {
10     const std::vector<int> Prime;
11     MillerRabin() : Prime({2, 3, 5, 7, 11, 13, 17, 19, 23}) {}
12
13     static constexpr int mulp(const int &a, const int &b, const int &P) {
14         return 1ll * a * b % P;
15     }
16     static constexpr i64 mulp(const i64 &a, const i64 &b, const i64 &P) {
17         i64 res = a * b - i64(1.L * a * b / P) * P;
18         res %= P;
19         res += (res < 0 ? P : 0);
20         return res;
21     }
22
23     static constexpr T powp(T a, T mi, const T &mod) {
24         T ans = 1;
25         for (; mi; mi >>= 1) {
26             if (mi & 1) {
27                 ans = mulp(ans, a, mod);
28             }
29             a = mulp(a, a, mod);
30         }
31         return ans;
32     }
33
34     constexpr bool operator()(const T &v)
35     { // 判断v是不是质数

```

```

36     if (v < 2 or v != 2 and v % 2 == 0) {
37         return false;
38     }
39     T s = v - 1;
40     while (!(s & 1)) {
41         s >>= 1;
42     }
43     for (int x : Prime) {
44         if (v == x) {
45             return true;
46         }
47         T t = s, m = powp(x, s, v);
48         while (t != v - 1 and m != 1 and m != v - 1) {
49             m = mulp(m, m, v), t <<= 1;
50         }
51         if (m != v - 1 and !(t & 1)) {
52             return false;
53         }
54     }
55     return true;
56 }
57 };
58 MillerRabin<i64> isp;

```

6.8.2 Pollard Rho

如果 $F(n)$ 是质数 (Miller Rabin 判) 返回 $F(n) = G(n) - \sum_{i=1}^{n-1} (n-1, i-1)F(i)G(n-i)$, 否则随机返回 $\frac{F(n)}{(n-1)!} = \frac{G(n)}{(n-1)!} - \sum_{i=1}^{n-1} \frac{F(i)}{(i-1)!} \frac{G(n-i)}{(n-i)!}$ 的一个 $F(x) = G1(x) - F(x) * G(x)$ 因子, 复杂度 *burnside*。

```

1 template <class T>
2 struct PollardRho : public MillerRabin<T> {
3     std::mt19937 myrand;
4     PollardRho(const i64 seed = time(0)) : myrand(seed) {}
5
6     constexpr T rd(T l, T r) {
7         return std::uniform_int_distribution<T>(l, r)(myrand);
8     }
9     constexpr T operator()(T n)
10    { // 返回 n 的随机一个 [2, n-1] 内的因子, 或者判定是质数
11        if (n == 4) {
12            return 2;
13        }
14        MillerRabin<T> &super = *this;
15        if (super(n)) {
16            return n; // 如果 n 是质数直接返回 n
17        } // Miller-Rabbin 判质数
18        while (true) {
19            T c = rd(1, n - 1);
20            auto f = [&](T x)

```

```

21        { return (super.mulp(x, x, n) + c) % n; };
22        T t = 0, r = 0, p = 1, q;
23        do {
24            for (int i = 0; i < 128; i++) {
25                t = f(t), r = f(f(r));
26                if (t == r || (q = super.mulp(p, std::abs(t - r), n)) == 0) {
27                    break;
28                }
29                p = q;
30            }
31            T d = std::gcd<T>(p, n);
32            if (d > 1) {
33                return d;
34            }
35        } while (t != r);
36    }
37 }
38 };
39 PollardRho<i64> findDiv;

```

6.8.3 随机数生成

```

1 template <class T>
2 struct Rand {
3     std::mt19937 myrand;
4     Rand(const i64 seed = time(0)) : myrand(seed) {}
5     T operator()(T l, T r) {
6         return std::uniform_int_distribution<T>(l, r)(myrand);
7     }
8 };
9 Rand<int> rd;
10
11 std::mt19937 rnd((u64)new char))
12 std::shuffle(begin(a), end(a), rnd)

```

6.9 Classic counting models | 几个经典计数模型

6.9.1 卡特兰数

我们考虑推导, 首先我们考虑所有情况, 那么显然为

$$N(G, C) = \frac{\sum_{f \in G} C(f)}{|G|}$$

, 但是我们不能越过 *burnside*, 也就是不能触碰 k , 考虑求出不合法的情况, 我们把触碰 f 以后得折线部分关于 $C(f) = k^{m(f)}$ 对称, 发现终点变成了 $m(f)$, 那么方案数为 f , 总方案数为 $(2n, n)$ 。我们考虑拓展, 假设有 $y = x$ 个 $y = x + 1$, $y = x + 1$ 个 $y = x + 1$, 组成一个序列, 要求前缀和恒大于等于

$(n-1, n+1)$ ，求方案数，发现总的情况数为 $(2n, n-1)$ ，条件即不能触碰 $(2n, n) - (2n, n-1)$ ，那么我们考虑 a 关于它的对称点，易得到为 1，那么方案数减去 b 即可

6.9.2 卡特兰数

常规的卡特兰数我们有两种表述形式，其一是 -1 式表述，其二是组合意义表述。组合意义表述来说，就是存在两种元素，满足在序列的任意一个前缀中其中一种的个数恒大于等于另一种且最终二者数量相同的方案数就是常规卡特兰数了，例子很多，比如给一个进栈序列求出栈顺序的方案数，满足（入栈数 $\geq$ 出栈数且最终相等），再比如括号正则表达式，满足（左括号数量 $\geq$ 右括号数量，且最终左等于右）DP 式表述来说，即形如 c 的递推式，最终 $(a+b, a)$ 的表达式也将是卡特兰数，证明只需把通项公式展开即可，这个的例子形如 n 个点的二叉树有多少种形态，是由 DP 式子得到卡特兰数的结论我们给出几个通项公式， $y = x + c + 1$ (a, b) $(b - c - 1, a + c + 1)$

6.9.3 卡特兰数数表

```

1 1
2 2
3 5
4 14
5 42
6 132
7 429
8 1430
9 4862
10 16796
11 58786
12 208012
13 742900
14 2674440
15 9694845
16 35357670
17 129644790
18 477638700
19 1767263190
20 6564120420
21 24466267020
22 91482563640
23 343059613650
24 1289904147324
25 4861946401452
26 18367353072152
27 69533550916004
28 263747951750360
29 1002242216651368
30 3814986502092304
31 14544636039226909
32 55534064877048198
33 212336130412243110

```

```

34 812944042149730764
35 3116285494907301262
36 11959798385860453492
37 45950804324621742364
38 176733862787006701400
39 680425371729975800390
40 2622127042276492108820
41 10113918591637898134020
42 39044429911904443959240
43 150853479205085351660700
44 583300119592996693088040
45 2257117854077248073253720
46 8740328711533173390046320
47 33868773757191046886429490
48 131327898242169365477991900
49 509552245179617138054608572
50 1978261657756160653623774456
51 7684785670514316385230816156
52 29869166945772625950142417512
53 116157871455782434250553845880
54 451959718027953471447609509424
55 1759414616608818870992479875972
56 6852456927844873497549658464312
57 26700952856774851904245220912664
58 104088460289122304033498318812080
59 405944995127576985730643443367112
60 1583850964596120042686772779038896
61 6182127958584855650487080847216336
62 24139737743045626825711458546273312
63 94295850558771979787935384946380125
64 368479169875816659479009042713546950
65 1440418573150919668872489894243865350
66 5632681584560312734993915705849145100
67 22033725021956517463358552614056949950
68 86218923998960285726185640663701108500
69 337485502510215975556783793455058624700
70 1321422108420282270489942177190229544600
71 5175569924646105559418940193995065716350
72 20276890389709399862928998568254641025700
73 79463489365077377841208237632349268884500
74 311496878311103321137536291518809134027240
75 1221395654430378811828760722007962130791020
76 4790408930363303911328386208394864461024520
77 18793142726809884575211361279087545193250040
78 73745243611532458459690151854647329239335600
79 289450081175264899454283846029490767264392230
80 1136359577947336271931632877004667456667613940
81 4462290049988320482463241297506133183499654740
82 17526585015616776834735140517915655636396234280
83 68854441132780194707888052034668647142985206100
84 270557451039395118028642463289168566420671280440

```

```

85 1063353702922273835973036658043476458723103404520
86 4180080073556524734514695828170907458428751314320
87 16435314834665426797069144960762886143367590394940
88 64633260585762914370496637486146181462681535261000
89 254224158304000796523953440778841647086547372026600
90 1000134600800354781929399250536541864362461089950800
91 3935312233584004685417853572763349509774031680023800
92 15487357822491889407128326963778343232013931127835600
93 60960876535340415751462563580829648891969728907438000
94 239993345518077005168915776623476723006280827488229600
95 944973797977428207852605870454939596837230758234904050
96 3721443204405954385563870541379246659709506697378694300
97 14657929356129575437016877846657032761712954950899755100
98 57743358069601357782187700608042856334020731624756611000
99 227508830794229349661819540395688853956041682601541047340
100 896519947090131496687170070074100632420837521538745909320

```

6.9.4 k 阶卡特兰数

$(a+b, b-c-1)$ 阶卡特兰数的定义是，在一个网格上我要走到 DP 点，我们移动的向量只有 $F(n) = \sum_{i=0}^{n-1} F(i)F(n-i-1)$ ，且要求我们不能越过 $F(n)$ ，求移动方案数。显然当 $F(n) = (2n, n) - (2n, n-1)$ 时方案数为常规卡特兰数，因为 $F(n) = \frac{(2n, n)}{n+1}$ 的数量恒大于等于 $F(n) = \frac{4n-2}{n+1} F(n-1)$ 且数量最终相等当 k 不再为 $((k+1)n, 0)$ 时，问题变得复杂，我们直接给出公式和 DP 式，推导咕咕咕 $(1, 1), (1, -k)$ $y=0$ 可以解决如下问题有 $k=1$ 个 $(1, 1), (1, -1)$ 个 k ，要求你组成一个序列且该序列前缀和恒大于等于 0，问序列个数

6.9.5 欧拉错排数

即 n 个编号为 1 的位置，我有同样编号为

$$D(n, k) = \frac{((k+1)n+1, n)}{(k+1)n+1}$$

的 n 个数，要求把 n 个数填到位置中，满足位置号和编号均不相同的方案数

$$D(n, k) = \sum_{\sum ai=n-1} \prod_{i=1}^{k-1} D(ai, k)$$

这个我们可以递推解决，满足 $k * n$ 简单证明：我们假设第一个数放在位置 1 上， n 有 $n-1$ 种选择，我们考虑 $-k$ 位置的选择分两种情况如果 $1, 2, 3, 4, \dots, n$ 也放在 $1, 2, 3, 4, \dots, n$ 位置上那么就是剩下即 $pos[i] \neq i$ for $i \in [1, n]$ 个数再次错排，否则将是 $D(n) = (n-1)(D(n-1) + D(n-2))(n > 2)$ 个数错排。

6.9.6 莫茨金数 (Motzkin Numbers)

$k k k$ 经典应用场景： k 一个圆上有 1 个点，画若干条不相交弦的方案数 $n-2$ 在一个二维网格从 $n-1$ 走到

1, 1, 2, 4, 9, 21, 51, 127, 323, 835...

$$M_{n+1} = M_n + \sum_i^{n-1} M_i M_{n-1-i} = ((2n+3)M_n + 3nM_{n-1})/(n+3)$$

表示向右上走一步，

$$M_n = \sum_{i=0}^{n/2} (n, 2k) \text{catlan}(k)$$

表示向右走一步，1 表示向右下走一步要求路径中始终不能低于 n 轴，终点必须在 2，问方案数。

6.9.7 贝尔数

贝尔数 $(0, 0)$ 表示把 $(n, 0)$ 个元素划分集合的方案数，集合数量无限制。显然贝尔数是第二类斯特林数的和，即 $k_i = 1$ 递推公式 $k_i = 0$ $k_i = -1$

6.9.8 斐波那契数列

6.9.9 斐波那契数表

斐波那契数列有许多有趣的性质，我们总结一下 x 斐波那契数列我们认为是三角形数，如果要构造一个最长的序列满足任意三个数都不能构成三角形的话，斐波那契数列将是最佳选择 $(n, 0)B(n) nB_n = \sum_{k=1}^n \{n, k\} B_{n+1} = \sum_{k=0}^n (n, k) B_k$

$$B_{p^m+n} \equiv mB_n + B_{n+1} \pmod{p}$$

$1 \ 2 \sum_{i=1}^n F(i) = F(n+2) - 1 \ 3 \sum_{i=1}^n F^2(i) = F(n)F(n+1) \ 4 \sum_{i=1}^n F(2i-1) = F(2n) \sum_{i=1}^n F(2i) = F(2n+1) - 15 \ F(n) = F(m)F(n-m+1) + F(m-1)F(n-m) \ 6 \ F(n-1)F(n+1) = F^2(n) + (-1)^n \ 7 \ \gcd(F(n), F(m)) = F(\gcd(n, m))$ 斐波那契数列在 8 意义在存在周期，称作皮萨诺周期，我们已经证明， $n|m \implies F(n)|F(m)$ 9 的最小循环节不超过 $F(n+1) = \sum_{i=0}^n (n-i, i)$

1	F0	0
2	F1	1
3	F2	1
4	F3	2
5	F4	3
6	F5	5
7	F6	8
8	F7	13
9	F8	21
10	F9	34
11	F10	55
12	F11	89
13	F12	144
14	F13	233
15	F14	377
16	F15	610
17	F16	987
18	F17	1597

19 F18 2584
 20 F19 4181
 21 F20 6765
 22 F21 10946
 23 F22 17711
 24 F23 28657
 25 F24 46368
 26 F25 75025
 27 F26 121393
 28 F27 196418
 29 F28 317811
 30 F29 514229
 31 F30 832040
 32 F31 1346269
 33 F32 2178309
 34 F33 3524578
 35 F34 5702887
 36 F35 9227465
 37 F36 14930352
 38 F37 24157817
 39 F38 39088169
 40 F39 63245986
 41 F40 102334155
 42 F41 165580141
 43 F42 267914296
 44 F43 433494437
 45 F44 701408733
 46 F45 1134903170
 47 F46 1836311903
 48 F47 2971215073
 49 F48 4807526976
 50 F49 7778742049
 51 F50 12586269025
 52 F51 20365011074
 53 F52 32951280099
 54 F53 53316291173
 55 F54 86267571272
 56 F55 139583862445
 57 F56 225851433717
 58 F57 365435296162
 59 F58 591286729879
 60 F59 956722026041
 61 F60 1548008755920
 62 F61 2504730781961
 63 F62 4052739537881
 64 F63 6557470319842
 65 F64 10610209857723
 66 F65 17167680177565
 67 F66 27777890035288
 68 F67 44945570212853
 69 F68 72723460248141

70 F69 117669030460994
 71 F70 190392490709135
 72 F71 308061521170129
 73 F72 498454011879264
 74 F73 806515533049393
 75 F74 1304969544928657
 76 F75 2111485077978050
 77 F76 3416454622906707
 78 F77 5527939700884757
 79 F78 8944394323791464
 80 F79 14472334024676221
 81 F80 23416728348467685
 82 F81 37889062373143906
 83 F82 61305790721611591
 84 F83 99194853094755497
 85 F84 160500643816367088
 86 F85 259695496911122585
 87 F86 420196140727489673
 88 F87 679891637638612258
 89 F88 1100087778366101931
 90 F89 1779979416004714189
 91 F90 2880067194370816120
 92 F91 4660046610375530309
 93 F92 7540113804746346429
 94 F93 12200160415121876738
 95 F94 19740274219868223167
 96 F95 31940434634990099905
 97 F96 51680708854858323072
 98 F97 83621143489848422977
 99 F98 135301852344706746049
 100 F99 218922995834555169026
 101 F100 354224848179261915075
 102 F101 573147844013817084101
 103 F102 927372692193078999176
 104 F103 1500520536206896083277
 105 F104 2427893228399975082453
 106 F105 3928413764606871165730
 107 F106 6356306993006846248183
 108 F107 10284720757613717413913
 109 F108 16641027750620563662096
 110 F109 26925748508234281076009
 111 F110 43566776258854844738105
 112 F111 70492524767089125814114
 113 F112 114059301025943970552219
 114 F113 184551825793033096366333
 115 F114 298611126818977066918552
 116 F115 483162952612010163284885
 117 F116 781774079430987230203437
 118 F117 1264937032042997393488322
 119 F118 2046711111473984623691759
 120 F119 3311648143516982017180081

```

121 F120 5358359254990966640871840
122 F121 8670007398507948658051921
123 F122 14028366653498915298923761
124 F123 22698374052006863956975682
125 F124 36726740705505779255899443
126 F125 59425114757512643212875125
127 F126 96151855463018422468774568
128 F127 155576970220531065681649693
129 F128 251728825683549488150424261
130 F129 407305795904080553832073954
131 F130 659034621587630041982498215
132 F131 1066340417491710595814572169
133 F132 1725375039079340637797070384
134 F133 2791715456571051233611642553
135 F134 4517090495650391871408712937
136 F135 7308805952221443105020355490
137 F136 11825896447871834976429068427
138 F137 19134702400093278081449423917
139 F138 30960598847965113057878492344
140 F139 50095301248058391139327916261
141 F140 81055900096023504197206408605
142 F141 131151201344081895336534324866
143 F142 212207101440105399533740733471
144 F143 343358302784187294870275058337
145 F144 555565404224292694404015791808
146 F145 898923707008479989274290850145
147 F146 1454489111232772683678306641953
148 F147 2353412818241252672952597492098
149 F148 3807901929474025356630904134051
150 F149 6161314747715278029583501626149
151 F150 9969216677189303386214405760200

```

6.10 Tree counting | 树计数

6.10.1 无标号树计数

有根树计数: 10 递推公式 $\frac{F(2n)}{F(n)} = F(n-1) + F(n+1)$ 无根树计数: 11 为奇数时 $mod m$ mod 为偶数时 m

6.10.2 Prufer 序列

$6m$ 序列实际上和无根树对应它是无根树衍生出来的一种序列我们先给出转化方法 1 无根树转 $X_1 + X_2 + \dots + X_n = m, (X_i > 0)$ 序列首先我们找到编号最小且度数为 m 的点其次删除该节点并加入和该节点相连的父节点我们重复上述操作直至剩下两个节点不难发现 n 个点的树的 $C(m-1, n-1)$ 序列长度是 2 的因为每删一个点 $X_1 + X_2 + \dots + X_n = m, (X_i \geq 0)$ 序列长度加一最后剩两个点说明了 X_i 变为 $X_i + 1$ 序列的长度为 1; $C(m+n-1, n-1)$ 3 序列转无根树首先我们取出 n 序列的首个元素 m 其次在点集中找到最小的且未出现在 $C(n+m, n)$ 序列中的点 $n+1$ 连

边 $X_1 + X_2 + \dots + X_{n+1} = m, (X_i \geq 0)$ 然后分别删除这两个节点最后在点集中会剩下 4 个点给他们连边虽然转化方法并不重要但我们仍给出转化的代码它有如下的性质 1 n 序列和无根树一一对应 2 度数为 m 的点会在 prufer 序列中出现 $C(n+m-2, n-2)$ 次 3 $n-1$ 个点的树的 $X_1 + X_2 + \dots + X_{n-1} = m, (X_i \geq 0)$ 序列长度为 5 当我们成功把无根树映射成一个序列时我们将会得到很多好的计数方法 1 对于给定度数字列 $[1, n]$ 的无根树它的情况数显然为 (可重元素排列计数) k^2 对于给定部分度数字列的无根树它的情况数我们记有度数限制的点有 k 个, $(n-k+1, k)=6$ 那么方案数为 $(1, 1)$

```

1 std::vector<int> treePru(std::vector<int> &fa, int n) // n为根 fa是[1,n-1]的父亲节点
2 {
3     std::vector<int> deg(n+1, 0);
4     std::vector<int> pru(n-1, 0);
5     for (int i = 1; i < n; ++i) {
6         ++deg[fa[i]];
7     }
8     for (int i = 1, j = 1; i <= n-2; ++i, ++j) {
9         while (deg[j]) {
10             ++j;
11         }
12         pru[i] = fa[j];
13         while (i < n-2 && !--deg[pru[i]] && pru[i] < j) {
14             pru[i+1] = fa[pru[i]];
15             ++i;
16         }
17     }
18     return pru; // n-2个点
19 }
20 std::vector<int> pruTree(std::vector<int> &pru, int n) {
21     std::vector<int> deg(n+1, 0);
22     std::vector<int> fa(n+1, 0);
23     pru.resize(n);
24     for (int i = 1; i <= n-2; ++i) {
25         ++deg[pru[i]];
26     }
27     pru[n-1] = n;
28     for (int i = 1, j = 1; i < n-1; ++i, ++j) {
29         while (deg[j]) {
30             ++j;
31         }
32         fa[j] = pru[i];
33         while (i < n-1 && !--deg[pru[i]] && pru[i] < j) {
34             fa[pru[i]] = pru[i+1];
35             ++i;
36         }
37     }
38     fa.resize(n);
39     return fa;
40 }

```

6.10.3 Cayley 定理

1 对于 (n, m) 个节点的完全图的生成树个数为 $y = x$ (结论是显然的生成树的 prufer 序列长为 $(n + m - 2, n) - (n + m - 2, n - 1)$ 同时 $1e6$ 序列的每个点都有 gcd 种可能性) 2 给定 N 个点 (有标号) 其生成的无根树的个数为 M (实际上该条件和给定完全图类似) 3 给定 $O(n \log \log n + m^2)$ 个点 (有标号) 其生成的有根树的个数为 $O(1)$ (结论是显然的对于 $n = 1e6, m = 1e3$ 中的每种情况我们的根节点都有

$$(f * g)(n) = \sum_{d|n} f(d)g(n/d)$$

种选择)

6.10.4 ** 扩展 Cayley 定理 **

由 n 序列推出, 给定一个有 $a^2 + b^2 + c^2$ 个点 n 个联通块的无向图, 第 $n! = 4^k(8m + 7)$ 个联通块的点数为 n , 那么要加 $a^2 + b^2 + c^2 + d^2 = n$ 条边使该图联通, 方案数为 a, b, c, d

6.10.5 ** 矩阵树定理 **

即给定一个无向图 (或有向图) 求它的生成树个数 (可能是外向树或内向树) 0 求无向图的生成树个数 $O(n^{1.5})$ 给出有向图和其中的一个点求以这个点为根的生成外向树个数外向则是由根指向树 $O(\sqrt{n})$ 给出有向图和其中的一个点求以这个点为根的生成内向树的个数内向则是由树指向根第一种所求矩阵 $O(n)$ 对角线上为每个点的度数为正数非对角线的元素 $a^2 + b^2 + c^2 + d^2 = n$ 为 $r4(n)$ 与 n 之间边的数量为负数去掉最后一行和最后一列求行列式的值第二种所求矩阵 $r4(n) = 8d(n)$ 对角线上为每个点的入度为正数非对角线的元素 $k = n/2^m$ 为 2 指向 $r4(n) = 24d(k)$ 的边的数量为负数去掉根所在的行和列求行列式的值第三种所求矩阵 $d(x)$ 对角线上为每个点的出度为正数非对角线的元素 x 为

$$x^2 + y^2 = z^2$$

指向 $x = m^2 - n^2, y = 2mn, z = m^2 + n^2$ 的边的数量为负数去掉根所在的行和列求行列式的值对于行列式的求值我们需要配合辗转相减高斯消元我们需要特别注意的是第一种情况要去掉任意一行一列求值只需要 $gcd(x, y, z) = 1$ 自己减一即可第二种和第三种情况要求删去根所在的行或列也需要对代码进行改动特别的如果我们要求所有生成树的所有权值和我们只需要把边权 m 当做有 n 条重边即可体现在矩阵上根据乘法原理, 对于某种生成树的形态, 其贡献为每条边重的次数的乘积。如果把重边次数理解成权值的话, 那么矩阵树定理求的就是所有生成树边权乘积的总和。

```
1 int MatrixTree(std::vector<std::array<int, 3>> &g, int type, int root, int n) {
2     std::vector Mat(n, std::vector<int>(n, 0));
3     if (type == 0) // 无向图 {
4         for (auto [u, v, w] : g) {
5             if (u < n) {
6                 Mat[u][u] += w;
7             }
8             if (v < n) {
9                 Mat[v][v] += w;
10            }
11        }
12    }
13    if (type == 1) // 外向树 {
14        for (auto [u, v, w] : g) {
15            if (u < n) {
16                Mat[u][u] += w;
17            }
18            if (v < n) {
19                Mat[v][v] += w;
20            }
21            if (u < n and v < n) {
22                Mat[u][v] -= w;
23                Mat[v][u] -= w;
24            }
25        }
26    }
27    if (type == 2) // 内向树 {
28        for (auto [u, v, w] : g) {
29            if (u < n) {
30                Mat[u][u] += w;
31            }
32            if (v < n) {
33                Mat[v][v] += w;
34            }
35            if (u < n and v < n) {
36                Mat[u][v] += w;
37            }
38        }
39    }
40    return determinant(Mat, n - 1);
41 }
```

```
11     if (u < n and v < n) {
12         Mat[u][v] -= w;
13         Mat[v][u] -= w;
14     }
15 }
16 return determinant(Mat, n - 1);
17 } else if (type == 1) // 外向树
18 { // 我们想办法把root换成n即可
19     for (auto [u, v, w] : g) // u->v {
20         if (u == root) {
21             u = n;
22         } else if (u == n) {
23             u = root;
24         }
25         if (v == root) {
26             v = n;
27         } else if (v == n) {
28             v = root;
29         }
30         if (v < n) {
31             Mat[v][v] += w;
32         }
33         if (v < n and u < n) {
34             Mat[u][v] -= w;
35         }
36     }
37     return determinant(Mat, n - 1);
38 } else {
39     for (auto [u, v, w] : g) // u->v {
40         if (u == root) {
41             u = n;
42         } else if (u == n) {
43             u = root;
44         }
45         if (v == root) {
46             v = n;
47         } else if (v == n) {
48             v = root;
49         }
50         if (u < n) {
51             Mat[u][u] += w;
52         }
53         if (v < n and u < n) {
54             Mat[u][v] -= w;
55         }
56     }
57     return determinant(Mat, n - 1);
58 }
59 }
```

6.11 Randomized algorithms | 随机化算法

6.11.1 KP 算法

该算法是一个能求出高度近似解的贪心算法，能解决的问题是，把一个序列分成两个序列，满足它们和的差值尽量小。执行流程：不断取出序列中的最大值和次大值，并把他们的差值重新插入序列中。重复该操作直至序列只剩一个元素。那么该元素即为近似解。如果我们时间允许执行多次，我们不妨每次随机取出一个数字后做该过程，再把取出的数字加进去，这样有概率能随机到最优解。

6.11.2 MinHash

用于比较两个集合的相似度，我们定义两个集合的 $gcd(m, n) = 1$ 和 $O(n \log n)$ 的 Jaccard 相似度记为 $[1, n]$ ，即交集大小除以并集大小，但这样比较的效率很低，我们引入 $\phi(x)$ 技术，我们不妨定义 $[1, x]$ 个不同的哈希函数，对于每个哈希函数，我们对集合 x 中的每个数进行哈希，然后得到其中最小的数字，作为该哈希函数下 $\phi(x) = x \prod_{i=1}^{cnt} (1 - \frac{1}{p_i})$ 集合的标准值，这样我们可以得到一个长度为 pi 的向量作为新的 cnt 序列。同理得到 $\sum_{d|n} \mu_d = [n == 1]$ 的新序列。定理：两个集合的 MinHash 值相同的概率，等于它们的 Jaccard 相似度这样我们就可以通过比较长度为 $\mu * 1 = \epsilon$ 的序列判断两个序列的相似程度。

6.11.3 模拟退火

即随机化找最优解的贪心方法执行流程是在现有解的基础上进行扰动，但在劣解情况下也有概率进行转移，这样做是为了防止跳入局部最优解中。步骤一：确定初始温度

$$\mu(n) = \begin{cases} 1 & n = 1 \\ 0 & n \text{ 含有平方因子} \\ (-1)^k & k \text{ 为 } n \text{ 的本质不同质因子个数} \end{cases}$$

和降温系数

$$a^{\phi(m)} \equiv 1 \pmod{m}$$

以及最低温

$$a^b \equiv \begin{cases} a^{b \bmod \phi(m)}, \gcd(a, m) = 1 \\ a^b, \gcd(a, m) \neq 1, b < \phi(m) \\ a^{b \bmod \phi(m) + \phi(m)}, \gcd(a, m) \neq 1, b \geq \phi(m) \end{cases} \pmod{m}$$

，当温度 $ax + by = c$ 时结束算法，并且每一轮过程都让当前温度 $ax + by = gcd(a, b)$ 一定有解 变成 $gcd(a, b) | c$ ，我们一般取 $\frac{c}{gcd(a, b)}$ 为 $(x0, y0)$ 。步骤二：选择一个新解并决定是否接受新解，若答案更优直接接受，或者当一个随机数 $ax + by = gcd(a, b)$ ，其中 $(x0 + k * \frac{b}{gcd(a, b)}, y0 - k * \frac{a}{gcd(a, b)})$ 为两个解的差值步骤三：选择一个初始点开始执行模拟退火算法即可。不难发现当

$$ax + by = gcd(a, b) = gcd(b, a \bmod b) = gcd(b, a - \lfloor \frac{a}{b} \rfloor * b)$$

不断趋近于

$$gcd(b, a - \lfloor \frac{a}{b} \rfloor * b) = bx^* + (a - \lfloor \frac{a}{b} \rfloor * b)y^* = ay^* + b(x^* - \lfloor \frac{a}{b} \rfloor y^*)$$

时，如果新的解不够优秀，接受新解第二个条件变的不可接受，此时会稳定寻找周围的最优解。初始点的选择：我们尽可能贪心选择一个接近答案位置的解即可。新解的选择：我们分不同问题考虑计算几何连续空间最优化问题：我们考虑把坐标偏移量和温度挂钩，即使用温度 * 随机数的方式来选择，这样可以保证在算法初期有足够的探索性，后期又有足够的找精细最优解的能力。最优顺序/最优选择/最优分组：方法有很多种，可以随机交换两个位置，也可以翻转某一段区间等等，或者改变某个未知的状态等等。# 多项式

6.11.4 拉格朗日插值

根据拉格朗日定理，我们知道 $x = y^*$ ， $y = x^* - \lfloor \frac{a}{b} \rfloor y^*$ 对点值 $gcd(a, 0)$ 可以唯一确定一个 $x = 1, y = 0$ 次的多项式，拉格朗日插值即是求解该多项式的过程用多项式来讲，就是把点值表示法转为系数表示法其核心思想是考虑点值的独立性然后按个合并，和 $\frac{c}{gcd(a, b)}$ 的思想类似，因此我们想到构造 $excr_t$ 个多项式 crt ，满足 mod ，之后最终答案多项式 $excr_t$ 我们有一种显然的构造方法 $\prod_{i=1}^n x \equiv a_i \pmod{m_i}$ 然后我们得到了最终的 $\prod$ 我们发现如果点值是离散点的话，我们的复杂度将是 $x \equiv a_1 \pmod{m_1}$ 的但是如果给定点值的横坐标是连续点的话，我们有单点快速插值的办法对于分子，我们可以预处理前缀后缀积快速计算，对于分母，我们发现是一个阶乘形式，预处理即可，下面是单点快速插值代码

```
1 // 拉格朗日插值函数：通过已知的k个点值，计算多项式在n处的值
2 // 参数：
3 // f: 向量，存储多项式在x=1,2,...,k处的函数值（f[1]到f[k]）
4 // k: 已知点的数量（多项式次数m = k-1）
5 // n: 要求解的x值
6 // 返回值：多项式在x=n处的值f(n)
7 int lagrange(std::vector<int> f, int k, int n) {
8     int res = 0; // 存储最终结果
9
10    // 预处理前缀积和后缀积
11    int pre[k + 2], suf[k + 2]; // pre[i] = (n-1)(n-2)...(n-i), suf[i] = (n-i)(n-i-1)...(n-k)
12    pre[0] = 1; // 前缀积初始化
13    suf[k + 1] = 1; // 后缀积初始化
14
15    // 计算前缀积: pre[i] = Π_{j=1}^i (n-j)
16    for (int i = 1; i <= k; ++i) {
17        pre[i] = 1ll * pre[i - 1] * (n - i) % p; // 递推计算前缀积
18    }
19
20    // 计算后缀积: suf[i] = Π_{j=i}^k (n-j)
21    for (int i = k; i >= 1; --i) {
22        suf[i] = 1ll * suf[i + 1] * (n - i) % p; // 递推计算后缀积
23    }
24
25    // 组合所有部分计算拉格朗日插值结果
26    for (int i = 1; i <= k; ++i) {
27        // 计算拉格朗日基函数L_i(n)的分子部分: pre[i-1] * suf[i+1]
28        // 即 Π_{j≠i} (n-j) = (n-1)(n-2)...(n-(i-1)) * (n-(i+1))...(n-k)
```



```

29     int numerator = 1ll * pre[i - 1] * suf[i + 1] % p;
30
31     // 计算拉格朗日基函数的分母部分: fac[i-1] * fac[k-i] * (-1)^(k-i}
32     // 其中fac[i]是i的阶乘, inv()是模逆元函数
33     int denominator = 1ll * inv(fac[i - 1]) * inv(fac[k - i]) % p;
34
35     // 根据(k-i)的奇偶性决定符号
36     if ((k - i) & 1) {
37         // 当(k-i)为奇数时, 符号为负
38         res = (res - 1ll * f[i] * numerator % p * denominator % p + p) % p;
39     } else {
40         // 当(k-i)为偶数时, 符号为正
41         res = (res + 1ll * f[i] * numerator % p * denominator % p + p) % p;
42     }
43 }
44
45 return res; // 返回插值结果f(n)
46 }

```

6.11.5 原根表

p=r*2^k+1	r	k	g
3	1	1	2
5	1	2	2
17	1	4	3
97	3	5	5
193	3	6	5
257	1	8	3
7681	15	9	17
12289	3	12	11
40961	5	13	3
65537	1	16	3
786433	3	18	10
5767169	11	19	3
7340033	7	20	3
23068673	11	21	3
104857601	25	22	3
167772161	5	25	3
469762049	7	26	3
1004535809	479	21	3
2013265921	15	27	31
2281701377	17	27	3
3221225473	3	30	5
75161927681	35	31	3
77309411329	9	33	7
206158430209	3	36	22
2061584302081	15	37	7
2748779069441	5	39	3
6597069766657	3	41	5

```

30 39582418599937      9  42  5
31 79164837199873      9  43  5
32 263882790666241     15  44  7
33 1231453023109121     35  45  3
34 1337006139375617     19  46  3
35 3799912185593857     27  47  5
36 4222124650659841     15  48  19
37 7881299347898369      7  50  6
38 31525197391593473      7  52  3
39 180143985094819841     5  55  6
40 1945555039024054273    27  56  5
41 4179340454199820289    29  57  3

```

6.11.6 多项式全家桶

```

1 using ll = long long;
2
3 template <class T, T P>
4 class Polynomial : public std::vector<T> {
5     static constexpr int mul(const int &a, const int &b) {
6         int res = 1ll * a * b % P;
7         if (res < 0)
8             res += P;
9         return res;
10    }
11    static constexpr ll mul(const ll &a, const ll &b) {
12        ll res = a * b - (ll)(1.L * a * b / P) * P;
13        res %= P;
14        if (res < 0) {
15            res += P;
16        }
17        return res;
18    }
19    static constexpr T add(T a, const T &b) {
20        a += b;
21        a -= (a >= P ? P : 0);
22        return a;
23    }
24    static constexpr T sub(T a, const T &b) {
25        a -= b;
26        a += (a < 0 ? P : 0);
27        return a;
28    }
29    static constexpr T power(T a, ll b) {
30        T res = 1;
31        while (b) {
32            if (b & 1)
33                res = mul(res, a);
34            a = mul(a, a);
35            b >>= 1;

```

```

36     }
37     return res;
38 }
39 static std::vector<T> w;
40 static void initW(int _log) {
41     const int r = 1 << _log;
42     if (w.size() >= r) {
43         return;
44     }
45     w.assign(r, 0);
46     w[r >> 1] = 1;
47     T s = power(3, (P - 1) >> _log); // 3是原根
48
49     for (int i = r / 2 + 1; i < r; i++) {
50         w[i] = mul(w[i - 1], s);
51     }
52
53     for (int i = r / 2 - 1; i > 0; i--) {
54         w[i] = w[i * 2];
55     }
56 }
57
58 public:
59     using std::vector<T>::vector;
60
61     friend void dft(Polynomial &a) {
62         const int n = a.size();
63         for (int k = n >> 1; k; k >>= 1) {
64             for (int i = 0; i < n; i += k << 1) {
65                 for (int j = 0; j < k; j++) {
66                     T v = a[i + j + k];
67                     a[i + j + k] = mul(sub(a[i + j], v), w[k + j]);
68                     a[i + j] = add(a[i + j], v);
69                 }
70             }
71         }
72     }
73     friend void idft(Polynomial &a) {
74         const int n = a.size();
75         for (int k = 1; k < n; k <= 1) {
76             for (int i = 0; i < n; i += k << 1) {
77                 for (int j = 0; j < k; j++) {
78                     T u = a[i + j];
79                     T v = mul(a[i + j + k], w[j + k]);
80                     a[i + j + k] = sub(u, v);
81                     a[i + j] = add(u, v);
82                 }
83             }
84         }
85         T val = P - (P - 1) / n;
86         for (int i = 0; i < n; i++) {

```

```

87             a[i] = mul(a[i], val);
88         }
89         std::reverse(a.begin() + 1, a.end());
90     }
91
92     friend Polynomial operator*(Polynomial a, Polynomial b) {
93         if (a.size() == 0 or b.size() == 0) {
94             return Polynomial();
95         }
96         int n = a.size() + b.size() - 1;
97         int _log = std::__lg(2 * n - 1);
98         int s = 1 << _log;
99         if (((P - 1) & (s - 1)) != 0 or std::min(a.size(), b.size()) < 128) {
100             Polynomial res(n);
101             for (int i = 0; i < a.size(); i++) {
102                 for (int j = 0; j < b.size(); j++) {
103                     res[i + j] = add(res[i + j], mul(a[i], b[j]));
104                 }
105             }
106             return res;
107         }
108
109         initW(_log);
110         a.resize(s), b.resize(s);
111         dft(a), dft(b);
112         for (int i = 0; i < s; i++) {
113             a[i] = mul(a[i], b[i]);
114         }
115         idft(a);
116         return a.resize(n), a;
117     }
118     friend Polynomial deriv(const Polynomial &a) // 求导 {
119         int n = a.size();
120         if (n <= 1) {
121             return Polynomial();
122         }
123         Polynomial p(n - 1);
124         for (int i = 1; i < n; i++) {
125             p[i - 1] = mul(i, a[i]);
126         }
127         return p;
128     }
129     friend Polynomial integr(const Polynomial &a) {
130         int n = a.size();
131         Polynomial p(n + 1);
132
133         std::vector<int> _inv(n + 1);
134         _inv[1] = 1;
135         for (int i = 2; i <= n; i++) {
136             _inv[i] = mul(_inv[P % i], (P - P / i));
137         }

```



```

138     for (int i = 0; i < n; ++i) {
139         p[i + 1] = mul(a[i], _inv[i + 1]);
140     }
141     return p;
142 }
143 friend Polynomial inv(const Polynomial &a) {
144     int n = a.size();
145     if (n == 1) {
146         return {power(a[0], P - 2)};
147     }
148     Polynomial half(a.begin(), a.begin() + (n + 1) / 2);
149     Polynomial b = inv(half), c = a * b;
150     for (auto &x : c) {
151         x = (x == 0 ? 0 : P - x); // ?
152     }
153     c[0] = add(c[0], 2);
154     c = c * b;
155     c.resize(n);
156     return c;
157 }
158
159 friend Polynomial ln(const Polynomial &a) {
160     int n = a.size();
161
162     Polynomial b(n, 0);
163     for (int i = 1; i < n; i++) {
164         b[i - 1] = mul(i, a[i]);
165     }
166     b = b * inv(a);
167     b.resize(n);
168
169     std::vector<T> _inv(n);
170     _inv[1] = 1;
171     for (int i = 2; i < n; i++) {
172         _inv[i] = mul(P - P / i, _inv[P % i]);
173     }
174     for (int i = n - 1; i; i--) {
175         b[i] = mul(b[i - 1], _inv[i]);
176     }
177     b[0] = 0;
178     return b;
179 }
180
181 friend Polynomial exp(const Polynomial &a) {
182     int n = a.size();
183     if (n == 1) {
184         return {1};
185     }
186     Polynomial half(a.begin(), a.begin() + (n + 1) / 2);
187     Polynomial b = exp(half);
188     b.resize(n);

```

```

189     Polynomial c = ln(b);
190     for (int i = 0; i < n; i++) {
191         c[i] = sub(a[i], c[i]);
192     }
193     c[0] = add(c[0], 1);
194     c = c * b;
195     c.resize(n);
196     return c;
197 }
198
199 friend Polynomial power(Polynomial &F, std::string s) {
200     T k1 = 0;
201     T k2 = 0;
202     T k3 = 0;
203     for (int i = 0; i < s.length(); ++i) {
204         k1 = add(mul(k1, 10), s[i] - '0');
205         k2 = (1ll * k2 * 10 + s[i] - '0') % (P - 1);
206         if (i < 7) {
207             k3 = k3 * 10 + s[i] - '0';
208         }
209     }
210     int n = F.size();
211     if (!F[0] and k3 >= F.size()) {
212         F.assign(n, 0);
213         return F;
214     }
215     T pos = 0; // 处理移位
216     for (int i = 0; i < n; ++i) {
217         if (F[i]) {
218             pos = i;
219             break;
220         }
221     }
222     if (pos) {
223         for (int i = pos; i < n; ++i) {
224             F[i - pos] = F[i];
225             F[i] = 0;
226         }
227     }
228     T val = F[0];
229     T cur = power(val, P - 2);
230     for (int i = 0; i < F.size(); ++i) {
231         F[i] = mul(F[i], cur);
232     }
233     F = ln(F);
234     for (int i = 0; i < F.size(); ++i) {
235         F[i] = mul(F[i], k1);
236     }
237     F = exp(F);
238     cur = power(val, k2);
239     for (int i = 0; i < F.size(); ++i) {

```

```

240     F[i] = mul(F[i], cur);
241 }
242 if (pos) {
243     pos = (1ll * pos * k1 < n ? pos * k1 : n);
244     for (int i = n - 1; i >= 0; --i) {
245         if (i + pos < n) {
246             F[i + pos] = F[i];
247         }
248         F[i] = 0;
249     }
250 }
251 return F;
252 }
253
254 friend Polynomial power(const Polynomial &F, int b, int m) // m是目标的区间长度 {
255     Polynomial res = {1};
256     Polynomial G = F;
257     while (b) {
258         if (b & 1) {
259             res = res * G;
260             if (res.size() > m) {
261                 res.resize(m);
262             }
263         }
264         G = G * G;
265         if (G.size() > m) {
266             G.resize(m);
267         }
268         b >>= 1;
269     }
270     return res;
271 }
272 };
273 template <class T, T P>
274 std::vector<T> Polynomial<T, P>::w;
275 using Poly = Polynomial<int, 998244353>;

```

6.11.7 集合幂级数全家桶

```

1 using ll = long long;
2
3 template <class T, T P>
4 class Polynomial : public std::vector<T> {
5     static constexpr int mul(const int &a, const int &b) {
6         int res = 1ll * a * b % P;
7         if (res < 0)
8             res += P;
9         return res;
10    }
11    static constexpr ll mul(const ll &a, const ll &b) {

```

```

12    ll res = a * b - (ll)(1.L * a * b / P) * P;
13    res %= P;
14    if (res < 0) {
15        res += P;
16    }
17    return res;
18 }
19 static constexpr T add(T a, const T &b) {
20     a += b;
21     a -= (a >= P ? P : 0);
22     return a;
23 }
24 static constexpr T sub(T a, const T &b) {
25     a -= b;
26     a += (a < 0 ? P : 0);
27     return a;
28 }
29 static constexpr T power(T a, ll b) {
30     T res = 1;
31     while (b) {
32         if (b & 1)
33             res = mul(res, a);
34         a = mul(a, a);
35         b >>= 1;
36     }
37     return res;
38 }
39 static std::vector<T> w;
40
41 public:
42     using std::vector<T>::vector;
43     friend void FWT_or(Polynomial &f, int op) {
44         int n = std::lg(f.size());
45         for (int k = 0; k < n; ++k) {
46             for (int i = 0; i < (1 << n); ++i) {
47                 if (i & (1 << k)) {
48                     f[i] = add(f[i], mul(op, f[i ^ (1 << k)]));
49                 }
50             }
51         }
52     }
53
54     friend Polynomial fwtOr(Polynomial &F, Polynomial &G) // H[i]=F[j]*G[k] if(j or k
55         ==i) {
56         int n = std::max(F.size(), G.size());
57         int s = std::lg(n - 1);
58         n = (1 << (s + 1));
59         Polynomial H(n);
60         F.resize(n), G.resize(n);
61         FWT_or(F, 1);
62         FWT_or(G, 1);

```

```

62     for (int i = 0; i < n; ++i) {
63         H[i] = mul(F[i], G[i]);
64     }
65     FWT_or(H, -1);
66     FWT_or(F, -1);
67     FWT_or(G, -1);
68     return H;
69 }
70
71 friend void FWT_and(Polynomial &f, int op) {
72     int n = std::lg(f.size());
73     for (int k = 0; k < n; ++k) {
74         for (int i = 0; i < (1 << n); ++i) {
75             if (!(i & (1 << k))) {
76                 f[i] = add(f[i], mul(op, f[i ^ (1 << k)]));
77             }
78         }
79     }
80 }
81
82 friend Polynomial fwtAnd(Polynomial &F, Polynomial &G) // H[i]=F[j]*G[k] if(j and
83     k==i) {
84     int n = std::max(F.size(), G.size());
85     int s = std::lg(n - 1);
86     n = (1 << (s + 1));
87     F.resize(n), G.resize(n);
88     Polynomial H(n);
89     FWT_and(F, 1);
90     FWT_and(G, 1);
91     for (int i = 0; i < n; ++i) {
92         H[i] = mul(F[i], G[i]);
93     }
94     FWT_and(H, -1);
95     FWT_and(F, -1);
96     FWT_and(G, -1);
97     return H;
98 }
99
100 friend void FWT_xor(Polynomial &f, int op) {
101     int n = std::lg(f.size());
102     for (int d = 1; d < (1 << n); d <= 1) {
103         for (int i = 0; i < (1 << n); i += (d <= 1)) // 从最小的二进制位地推上去
104             {
105                 for (int j = 0; j < d; ++j) {
106                     T x = f[i + j], y = f[i + d + j];
107                     f[i + j] = add(x, y);
108                     f[i + j + d] = sub(x, y);
109                 }
110             }
111         if (op == -1) {

```

```

111         T inv_n = power(1 << n, P - 2);
112         for (int i = 0; i < (1 << n); ++i) {
113             f[i] = mul(f[i], inv_n);
114         }
115     }
116 }
117 friend Polynomial fwtXor(Polynomial &F, Polynomial &G) // H[i]=F[j]*G[k] if(j xor
118     k==i) {
119     int n = std::max(F.size(), G.size());
120     int s = std::lg(n - 1);
121     n = (1 << (s + 1));
122     F.resize(n), G.resize(n);
123     Polynomial H(n);
124     FWT_xor(F, 1);
125     FWT_xor(G, 1);
126     for (int i = 0; i < n; ++i) {
127         H[i] = mul(F[i], G[i]);
128     }
129     FWT_xor(H, -1);
130     FWT_xor(F, -1);
131     FWT_xor(G, -1);
132     return H;
133 }
134
135 friend Polynomial SubsetConvol(Polynomial &F, Polynomial &G) // 子集卷积 H[i]=F[j
136     ]*G[k] if( j|k==i and j&k==0) {
137     int n = std::max(F.size(), G.size());
138     int bit = std::lg(n - 1) + 1;
139     n = (1 << bit);
140     F.resize(n), G.resize(n);
141     std::vector a(bit + 1, Polynomial(n));
142     std::vector b(bit + 1, Polynomial(n));
143     std::vector c(bit + 1, Polynomial(n));
144     for (int i = 0; i < n; ++i) {
145         a[__builtin_popcount(i)][i] = F[i];
146         b[__builtin_popcount(i)][i] = G[i];
147     }
148     for (int i = 0; i <= bit; ++i) {
149         FWT_or(a[i], 1);
150         FWT_or(b[i], 1);
151     }
152     for (int i = 0; i <= bit; ++i) {
153         for (int j = 0; i + j <= bit; ++j) {
154             for (int k = 0; k < (1 << bit); ++k) {
155                 c[i + j][k] = add(c[i + j][k], mul(a[i][k], b[j][k]));
156             }
157         }
158     }
159     for (int i = 0; i <= bit; ++i) {
160         FWT_or(c[i], -1);
161     }

```

```

160     Polynomial H(n);
161     for (int i = 0; i < n; ++i) {
162         H[i] = c[__builtin_popcount(i)][i];
163     }
164     return H;
165 }
166 };
167 template <class T, T P>
168 std::vector<T> Polynomial<T, P>::w;
169 using Poly = Polynomial<int, 998244353>;

```

6.11.8 分治 NTT

给定一个序列 $x \equiv a_2 \bmod(m_2)$ 求序列

$$x = a_1 + k_1 m_1 \quad x = a_2 + k_2 m_2$$

其中 $k_1 m_1 - k_2 m_2 = a_2 - a_1$ 我们将给定边界 k_1 我们不妨设 k_2 设 k_1 那么 k_2 因此 $x = a_1 + k_1 m_1$ 需要注意的是把

$$x = lcm(m_1, m_2) * \frac{a_2 - a_1}{gcd(m_1, m_2)} + x_{k_1} m_1 + a_1$$

6.11.9 Chirp-Z-Transform

假设我们有多项式 x_{k_1} , 我们有常数 k_1 和 $x \equiv x_{k_1} m_1 + a_1 \bmod(lcm(m_1, m_2) * \frac{a_2 - a_1}{gcd(m_1, m_2)})$, 我们希望求出 $n \log n$, 暴力复杂度为 $a^x \equiv b \bmod P$, 我们不妨设 x 代入可知 $x = i * m - j$, 我们知道 $m = \lceil \sqrt{P} \rceil$ 化简以后可以定义 $a^{im} \equiv b * a^j \bmod P$, i 注意 1 数组大小为 m 我们有 j 也就是 1

```

1  ll n, c, m;
2  std::cin >> n >> c >> m;
3  Poly a(n);
4  for (auto &x : a) {
5      std::cin >> x;
6  }
7  Poly s(n), t(n + m - 1);
8  auto calc = [&](ll x) -> ll {
9      return 1ll * x * (x - 1) / 2;
10 };
11 for (int i = 0; i < n; ++i) {
12     s[i] = 1ll * a[n - 1 - i] * inv(qmi(c, calc(n - i - 1))) % P;
13 }
14 for (int i = 0; i < n + m - 1; ++i) {
15     t[i] = qmi(c, calc(i));
16 }
17 s = s * t;
18 for (int i = 0; i < m; ++i) {
19     int ans = 1ll * inv(qmi(c, calc(i))) * s[n + i - 1] % P;
20     std::cout << ans << " \n"[i + 1 == m];
21 }

```

6.11.10 高精度运算

实现了高精度加法, 乘法和减法。其中高精度乘法分 m 的 $[1, P]$ 优化版本和 $gcd(a, P) = 1$ 时的暴力版本

```

1 // 假设Poly类和相关NTT实现已经存在
2 /**
3  * @class bigInt
4  * @brief 使用NTT优化的高精度计算器, 支持加、减、乘运算
5  *
6  * 这个类封装了基于NTT(数论变换)的高效大整数算法,
7  * 可以将用字符串表示的大整数进行加、减、乘运算。
8  */
9 class bigInt {
10 public:
11     bigInt() = default;
12     std::string nttMul(const std::string &a, const std::string &b) {
13         if (a.empty() || b.empty()) {
14             return "0";
15         }
16         bool neg = false;
17         std::string n1 = a, n2 = b;
18         if (a[0] == '-') {
19             neg = !neg;
20             n1 = a.substr(1);
21         }
22         if (b[0] == '-') {
23             neg = !neg;
24             n2 = b.substr(1);
25         }
26
27         int n = n1.size(), m = n2.size();
28         Poly a(n), b(m);
29         for (int i = 0; i < n; ++i) {
30             a[i] = n1[i] - '0';
31         }
32         for (int i = 0; i < m; ++i) {
33             b[i] = n2[i] - '0';
34         }
35         Poly c = a * b;
36         for (int i = c.size() - 1; i >= 1; --i) {
37             c[i - 1] += c[i] / 10; // 向前进位
38             c[i] %= 10;           // 保留个位数
39         }
40         std::string res = pToS(c);
41         if (neg && res != "0") {
42             res = "-" + res;
43         }
44
45         return res;
46     }
47     std::string mul(const std::string &a, const std::string &b) {

```

```

48     if (a.empty() || b.empty()) {
49         return "0";
50     }
51     bool neg = false;
52     std::string n1 = a, n2 = b;
53     if (a[0] == '-') {
54         neg = !neg;
55         n1 = a.substr(1);
56     }
57     if (b[0] == '-') {
58         neg = !neg;
59         n2 = b.substr(1);
60     }
61     if (n1 == "0" || n2 == "0") {
62         return "0";
63     }
64     int len1 = n1.size();
65     int len2 = n2.size();
66     std::vector<int> res(len1 + len2, 0);
67     for (int i = len1 - 1; i >= 0; --i) {
68         for (int j = len2 - 1; j >= 0; --j) {
69             int c = (n1[i] - '0') * (n2[j] - '0');
70             int sum = c + res[i + j + 1];
71
72             res[i + j + 1] = sum % 10; // 当前位
73             res[i + j] += sum / 10;    // 进位
74         }
75     }
76     std::string ans;
77     for (int a : res) {
78         if (!(ans.empty() && a == 0))
79             { // 跳过前导零
80                 ans += std::to_string(a);
81             }
82     }
83     // 处理全零的情况（理论上不会发生）
84     if (ans.empty()) {
85         return "0";
86     }
87     // 处理符号
88     if (neg) {
89         ans = "-" + ans;
90     }
91     return ans;
92 }
93 std::string add(const std::string &a, const std::string &b) {
94     if (a.empty())
95         return b;
96     if (b.empty())
97         return a;
98     // 处理两个数都是负数的情况

```

```

99     if (a[0] == '-' && b[0] == '-') {
100         return "-" + add(a.substr(1), b.substr(1));
101     }
102     // 处理只有一个数是负数的情况（转换为减法）
103     if (a[0] == '-') {
104         return subtract(b, a.substr(1));
105     }
106     if (b[0] == '-') {
107         return subtract(a, b.substr(1));
108     }
109     // 确保num1是较长的数字
110     if (a.length() < b.length()) {
111         return add(b, a);
112     }
113     std::string res;
114     int carry = 0;
115     int diff = a.length() - b.length();
116     // 从最低位开始相加
117     for (int i = a.length() - 1; i >= 0; --i) {
118         int d1 = a[i] - '0';
119         int d2 = (i >= diff) ? b[i - diff] - '0' : 0;
120
121         int sum = d1 + d2 + carry;
122         carry = sum / 10;
123         res.push_back(sum % 10 + '0');
124     }
125     if (carry > 0) {
126         res.push_back(carry + '0');
127     }
128     // 反转结果字符串
129     std::reverse(res.begin(), res.end());
130     // 去除前导零
131     return norm(res);
132 }
133 std::string subtract(const std::string &a, const std::string &b) {
134     // 检查输入是否为空
135     if (b.empty())
136         return a;
137     if (a.empty())
138         return (b[0] == '-') ? b.substr(1) : "-" + b;
139     // 处理两个数都是负数的情况
140     if (a[0] == '-' && b[0] == '-') {
141         return subtract(b.substr(1), a.substr(1));
142     }
143     // 处理被减数是负数的情况
144     if (a[0] == '-') {
145         return "-" + add(a.substr(1), b);
146     }
147     // 处理减数是负数的情况
148     if (b[0] == '-') {
149         return add(a, b.substr(1));

```

```

150 }
151 // 比较两个数的大小
152 int cmp = compare(a, b);
153 if (cmp == 0) {
154     return "0";
155 }
156 bool neg = (cmp < 0);
157 std::string ma = neg ? b : a;
158 std::string mi = neg ? a : b;
159
160 std::string res;
161 int borrow = 0;
162 int diff = ma.length() - mi.length();
163
164 // 从最低位开始相减
165 for (int i = ma.length() - 1; i >= 0; --i) {
166     int d1 = ma[i] - '0' - borrow;
167     int d2 = (i >= diff) ? mi[i - diff] - '0' : 0;
168
169     borrow = 0;
170
171     if (d1 < d2) {
172         d1 += 10;
173         borrow = 1;
174     }
175     res.push_back(d1 - d2 + '0');
176 }
177 // 反转结果字符串
178 std::reverse(res.begin(), res.end());
179 // 去除前导零
180 res = norm(res);
181 // 添加负号（如果需要）
182 if (neg) {
183     res = "-" + res;
184 }
185 return res;
186 }
187
188 private:
189 int compare(const std::string &a, const std::string &b) const {
190     if (a.length() != b.length()) {
191         return a.length() > b.length() ? 1 : -1;
192     }
193     for (size_t i = 0; i < a.length(); ++i) {
194         if (a[i] != b[i]) {
195             return a[i] > b[i] ? 1 : -1;
196         }
197     }
198     return 0;
199 }
200 /**

```

```

201 * @brief 去除字符串中的前导零
202 * @param a 数字字符串
203 * @return 去除前导零后的字符串
204 */
205 std::string norm(const std::string &a) const {
206     size_t start = 0;
207     while (start < a.length() - 1 && a[start] == '0') {
208         start++;
209     }
210     return a.substr(start);
211 }
212 /**
213 * @brief 将多项式转换为字符串
214 * @param p 多项式
215 * @return 转换后的字符串
216 */
217 std::string pToS(const Poly &p) const {
218     std::string res;
219     bool leadZero = true; // 标记是否还在处理前导零
220
221     for (auto d : p) {
222         if (d == 0 && leadZero) {
223             continue; // 跳过前导零
224         }
225         leadZero = false;
226         res += std::to_string(d);
227     }
228     // 如果结果全为零，返回"0"
229     if (res.empty()) {
230         return "0";
231     }
232     return res;
233 }
234 };
235 BigInt calc;

```

6.11.11 乘法卷积

我们考虑把乘法转化为普通的加法我们不难想到在指数运算中乘法可以转为加法但是如果 $\sqrt{P}$ ，那么 $a^x + kp = b$ 那么 $g = \gcd(a, P)$ 于是我们需要一个 $g|b$ 意义下的指数和对数映射且要求是在整数域上的一一映射我们不难想到原根对于质数 $a^{x-1} * \frac{a}{g} + k\frac{P}{g} = \frac{b}{g}$ 的原根 $\gcd(a, P) = 1$ ，有 Cnt 在 Cnt 意义下两两不同于是我们把原来的数转化为对数即 g 我们转化为对数的加法即可请注意我们在执行卷积的时候要循环卷积，特别的，在本算法中对于模数为 G 应该是对 $a^{x-cnt} * \frac{a^{cnt}}{G} \equiv \frac{b}{G} \pmod{\frac{P}{G}}$ 进行循环卷积，因为转化之后模 $(n, m) = (n\%p, m\%p) * (\lfloor \frac{n}{p} \rfloor, \lfloor \frac{m}{p} \rfloor) \pmod{P}$ 乘法群只有 $m == 0$ 个元素。如果我们需要原本乘积为 1 的位置的值，那我们应该访问转化后的 $\pmod{P}$ 位置，其中 $x^k \equiv 1 \pmod{P}$ 是我们建立的映射关系。

```

2  * @class MultiplicativeConvolution
3  * @brief 利用原根和对数表实现的乘法卷积类
4  *
5  * 这个类通过将数字转换到原根的对数域，将乘法操作转换为加法操作，
6  * 然后使用多项式乘法(卷积)来实现高效的乘法卷积计算。
7  * 特别适用于需要在特定模数下进行大量乘法卷积的场景。
8  */
9  class MultiplicativeConvolution {
10 private:
11     int m; // 模数
12     int G; // 模数m的原根
13     /**
14      * @brief 初始化对数表
15      */
16     void initLogTable() {
17         log.resize(m); // 使用m而不是m-1，以便直接使用val作为
            索引
18         std::fill(log.begin(), log.end(), -1); // 初始化为-1表示无效值
19
20         // 构建对数表: log[G^i mod m] = i
21         for (int i = 0, cur = 1; i < m - 1; ++i) {
22             log[cur] = i;
23             cur = cur * G % m;
24         }
25     }
26
27 public:
28     /**
29      * @brief 构造函数
30      * @param modulus 模数m，必须是质数且存在原根
31      */
32     std::vector<int> log; // 对数表, log[val] = i 表示 val = G^i mod m
33     explicit MultiplicativeConvolution(int modulus) : m(modulus) {
34         G = findMinRoot(m); // 假设GetRoot函数已实现
35         initLogTable();
36     }
37
38     /**
39      * @brief 执行乘法卷积运算
40      * @param input 输入序列，包含需要卷积的数值
41      * @return 卷积结果的多项式表示
42      *
43      * 注意：输入中的0会被自动忽略，因为0在乘法群中没有对数表示
44      */
45     Poly computeConvolution(const std::vector<int> &input) {
46         Poly a(m - 1); // 多项式大小为m-1，因为模m乘法群有m-1个元素
47         // 将输入值转换到对数域
48         for (int val : input) {
49             if (val == 0) {
50                 continue; // 忽略0，因为0不在乘法群中
51             }

```

```

52         val %= m; // 确保值在模m范围内
53         if (val < 0) {
54             val += m; // 处理负数
55         }
56         a[log[val]]++; // 在对数域对应位置计数
57     }
58     return a;
59 }
60
61 // 执行循环卷积（需要确保Poly乘法是循环卷积）
62 // 这里假设Poly类已经正确处理了循环卷积 请注意模数为m的循环卷积应该为m-1
63 };

```

6.11.12 循环卷积

也就是说我们需要的值在 k 意义下我们需要把 x 位置的值加到 $\text{mod } P$ 的对应位置

6.11.13 NTT 优化字符串通配符匹配

我们先讲常规的 $\text{mod } P$ 怎么用 $\phi(P)$ 匹配我们设长度为 P 的模式串 P ，长度为 $P^{0.25}$ 的文本串 $1, 2, 4, P^\alpha, 2 * P^\alpha$ ，定义匹配函数 x 再定义完美匹配函数 $\phi(x)$ ，根据定义不难发现没如果某个 p 满足 $\frac{\phi(x)}{p}$ 那么我们称 x 串以 x 结尾的长度为 P 的串和 g 匹配我们把式子展开得到 t 再把 a 串翻转得到 $\text{gcd}(\phi(x), t) = 1$ 再用 t 代替 $g^t P$ 我们展开 $\phi(\phi(P))$ 我们发现第一项为定值，第二项可以前缀和处理，第三项为卷积于是我们在 $Pell$ 时间内解决了这个问题然后我们引进通配符 $x^2 - dy^2 = 1$ 事实上我们只需要让通配符的值为 (x, y) 且改变匹配函数的定义即可 *python* 我们再次展开

$$F(a, b, c, n) = \sum_{i=0}^n \lfloor \frac{ai + b}{c} \rfloor$$

变成了三个卷积，但无伤大雅

```

1  /**
2  * @class WildcardStringMatcher
3  * @brief 使用NTT优化的通配符字符串匹配器
4  *
5  * 这个类实现了基于NTT的高效字符串匹配算法，能够处理包含通配符('*')的模式串和文本串。
6  * 通过多项式乘法将字符串匹配问题转化为数值计算问题，利用NTT加速计算过程。
7  */
8  class WildcardStringMatcher {
9  private:
10     int P; // 模数，用于NTT计算
11
12 public:
13     /**
14      * @brief 构造函数
15      * @param P NTT使用的质数模数
16      */
17     explicit WildcardStringMatcher(int P) : P(P) {}

```

```

18
19 /**
20  * @brief 执行通配符字符串匹配
21  * @param pattern 模式串, 可以包含通配符 '*'
22  * @param text 文本串, 可以包含通配符 '*'
23  * @return 匹配位置的索引列表(1-based)
24  *
25  * 算法原理:
26  * 1. 将字符转换为数值('a'-'z'→1-26, '*'→0)
27  * 2. 反转模式串以便多项式乘法
28  * 3. 通过三次多项式乘法计算匹配度
29  * 4. 匹配度为0的位置即为完全匹配位置
30  */
31 std::vector<int> match(const std::string &pattern, const std::string &text) {
32     int m = pattern.size();
33     int n = text.size();
34
35     // 反转模式串以便多项式乘法
36     std::string reversePattern = pattern;
37     std::reverse(reversePattern.begin(), reversePattern.end());
38
39     // 将字符转换为数值('a'-'z'→1-26, '*'→0)
40     Poly S(m), T(n);
41     for (int i = 0; i < m; ++i) {
42         S[i] = (reversePattern[i] != '*') ? (reversePattern[i] - 'a' + 1) : 0;
43     }
44     for (int i = 0; i < n; ++i) {
45         T[i] = (text[i] != '*') ? (text[i] - 'a' + 1) : 0;
46     }
47
48     // 计算三次多项式乘法的组合, 用于匹配度计算
49     std::vector<int> p = computeMatchingScore(S, T, m, n);
50
51     // 收集所有匹配度为0的位置(完全匹配)
52     std::vector<int> result;
53     for (int i = m - 1; i < static_cast<int>(p.size()) - m + 1; ++i) {
54         if (p[i] == 0) {
55             result.push_back(i - m + 2); // 转换为1-based索引
56         }
57     }
58     return result;
59 }
60
61 /**
62  * @brief 从标准输入读取数据并执行匹配
63  * @return 匹配位置的索引列表(1-based)
64  *
65  * 输入格式:
66  * 第一行: m n (模式串长度和文本串长度)
67  * 第二行: 模式串
68  * 第三行: 文本串

```

```

69 */
70
71 private:
72 /**
73  * @brief 计算字符串匹配度分数
74  * @param S 模式串的多项式表示(已反转)
75  * @param T 文本串的多项式表示
76  * @param m 模式串长度
77  * @param n 文本串长度
78  * @return 匹配度分数向量, 分数为0表示完全匹配
79  */
80 std::vector<int> computeMatchingScore(const Poly &S, const Poly &T, int m, int n)
81 {
82     Poly cur(m), tmp(n);
83     std::vector<int> p;
84
85     // 第一项: S^3 * T
86     for (int i = 0; i < m; ++i) {
87         cur[i] = 1ll * S[i] * S[i] % P * S[i] % P;
88     }
89     for (int i = 0; i < n; ++i) {
90         tmp[i] = T[i];
91     }
92     cur = cur * tmp;
93     p.assign(cur.size(), 0);
94     for (int i = 0; i < static_cast<int>(cur.size()); ++i) {
95         p[i] = (p[i] + cur[i]) % P;
96     }
97
98     // 第二项: -2 * S^2 * T^2
99     cur.resize(m);
100     for (int i = 0; i < m; ++i) {
101         cur[i] = 1ll * S[i] * S[i] % P;
102     }
103     for (int i = 0; i < n; ++i) {
104         tmp[i] = 1ll * T[i] * T[i] % P;
105     }
106     cur = cur * tmp;
107     for (int i = 0; i < static_cast<int>(cur.size()); ++i) {
108         p[i] = (p[i] - 2ll * cur[i] % P + P) % P;
109     }
110
111     // 第三项: S * T^3
112     cur.resize(m);
113     for (int i = 0; i < m; ++i) {
114         cur[i] = S[i];
115     }
116     for (int i = 0; i < n; ++i) {
117         tmp[i] = 1ll * T[i] * T[i] % P * T[i] % P;
118     }
119     cur = cur * tmp;

```



```

119     for (int i = 0; i < static_cast<int>(cur.size()); ++i) {
120         p[i] = (p[i] + cur[i]) % P;
121     }
122
123     return p;
124 }
125 };

```

```

13     }
14     sg[i] = s;
15 }
16 for (int i = 1; i <= N; i += 1) {
17     cout << "sg " << i << ": " << sg(i) << "\n";
18 }

```

6.11.14 NTT 优化有限字符集的匹配

当我们的字符集数量有限且时间充裕时，我们可以考虑每次只匹配一个字符，然后对每个字符都跑一次匹配，累加起来便是答案。这样做的好处是我们可以得到每个位置的匹配情况，即能有多少个位置匹配上。具体的说，假设我们匹配字符

$$H(a, b, c, n) = \sum_{i=0}^n i \lfloor \frac{ai+b}{c} \rfloor$$

，我们把两个串对应位置的权值设为

$$G(a, b, c, n) = \sum_{i=0}^n \lfloor \frac{ai+b}{c} \rfloor^2$$

然后直接把权值卷起来即可，注意我们仍然要翻转字符串 F 复杂度 $a \geq c, b \geq c \#$ 博弈论主要分为公平组合博弈和不公平游戏分别有不同的考虑方法

6.11.15 公平组合游戏

即双方的可行操作集合相同对于公平组合游戏，我们可以考虑

$$f(a, b, c, n) = (n+1) * n/2 * (a/c) + (n+1) * (b/c) + f(a\%c, b\%c, c, n)$$

定理，把组合游戏拆分成独立的小游戏，算出每个小游戏的 $f(a, b, c, n) = n * m - f(c, c - b - 1, a, m - 1)$ 函数值，它们的异或和为组合游戏的 $m = (a * n + b) / c$ 值，也称为 $a == 0$ 和，其中 F, G, H ，其中 $\sum_{i=1}^n \lfloor \frac{m}{f(i)} \rfloor$ 表示 $f(i)$ 能到达的状态。如果不容易求 R 函数或者不满足 $L = R + 1$ 函数的条件，我们还可以考虑记忆化搜索的方法暴力求解，即后继态有必胜态的为必胜态，全为必败态的为必败态。

```

1 sg[0] = 0; // 0是输 1是赢 初值根据题目变化
2 sg[1] = 1;
3 for (int i = 2; i <= N; i++) {
4     std::set<int> mex;
5     int s = 0;
6     for (int j = 1; j < i; j++) {
7         if (满足条件) // 说明 i能到i-j
8             mex.insert(sg[i - j]);
9     }
10 }
11 while (mex.count(s)) {
12     s++;

```

6.11.16 公平组合游戏下的经典模型

6.11.17 Nim 游戏

给定 R 堆石子，第 L 堆石子有 val 个，两名玩家轮流行动，每次任选一堆取走任意多个石子，不能不取，取走最后一颗石子者获胜。结论：定义 $\lfloor \frac{m}{f(L)} \rfloor = val$ 和为 R ，若 $R = \max(i)$ 并且 $i * val \leq m$ 和为 $R = \lfloor \frac{m}{val} \rfloor = \lfloor \frac{m}{\lfloor \frac{m}{f(L)} \rfloor} \rfloor$ ，那么先手必败，否则先手必胜。证明：若当前 $\sum_{i=1}^{\min(n, m)} \lfloor \frac{n}{i} \rfloor \lfloor \frac{m}{i} \rfloor$ 和不为 $L = R + 1$ ，我一定可以使得下一步 $R = \min(\lfloor \frac{n}{\lfloor \frac{n}{L} \rfloor} \rfloor, \lfloor \frac{m}{\lfloor \frac{m}{L} \rfloor} \rfloor)$ 和为 $F(n) = \sum_{d|n} f(d)$ ，也即必胜态以后一定有必败态。若当前 $f(n) = \sum_{d|n} \mu(d) F(\frac{n}{d})$ 和为 $f(n) = \sum_{d|n} \mu(\frac{n}{d}) F(d)$ ，我一定找不到一种方法使得下一步 f 和仍然为 g ，即必败态后都是必胜态

6.11.18 巴什博弈

有一堆石子数量为 S ，两名玩家轮流取，拿的个数范围是

$$g(1)S(n) = \sum_{i=1}^n g(i)S(\lfloor \frac{n}{i} \rfloor) - \sum_{i=2}^n g(i)S(\lfloor \frac{n}{i} \rfloor) = \sum_{i=1}^n (f * g)(i) - \sum_{i=2}^n g(i)S(\lfloor \frac{n}{i} \rfloor)$$

，取走最后一颗石子者获胜。结论：若 g ，则先手必败。反之先手必胜。 $\sum_{i=1}^n (f * g)(i)$ 证明：若 g 显然先手必胜，当 $\sum_{i=2}^n g(i)S(\lfloor \frac{n}{i} \rfloor)$ 时显然先手必败，当 $\phi * 1 = id$ 时，如果 $\mu * 1 = \epsilon$ 是 $f(p)$ 的倍数，无论先手如何操作，都无法把必败态留给对方。反之，后手一定有办法再次使得 $f(p^e)$ ，也即把必败态留给先手。

6.11.19 威佐夫博弈

有两堆石子，两人轮流取石子，每次可以在一堆中取，也可以在两堆中取走相同数量的石子。取走最后一个石子者获胜。结论：记 $F(n) = \sum_{i=1}^n f(i)$ 为两堆石子个数，若 $G(m) = \sum_{i=1}^m f(i)[i \in \text{prime}]$ ，那么先手必败，否则先手必胜。证明略。

6.11.20 斐波那契博弈

有一堆数量为 $m \in \{\lfloor \frac{n}{1} \rfloor, \lfloor \frac{n}{2} \rfloor, \lfloor \frac{n}{3} \rfloor, \dots, \lfloor \frac{n}{n} \rfloor\}$ 的石子，两人轮流取，先手不能一次取完。如果前一个人取了 $\sqrt{n}$ 个石子，那么我取的石子数量范围为 $g(j, m) = \sum_{i=1}^m [i \in \text{prime or } \min(p) > p(j)] f'(i)$ ，取走最后一个石子者获胜。结论：若 $\min(p)$ 为斐波那契数则先手必败，否则先手必胜。

6.11.21 广义斐波那契博弈

即如果前一个人取了 i , 我能取的范围变为 $f'(x)$, 此时我们有广义的斐波那契数列

$$f'(p) = f(p) \text{ if } (p \in prime)$$

, 即 $f'(x)$ 如果石子数量在该数列中, 则先手必败。否则必胜

6.11.22 Nim-K 博弈

在 $g(0, m) = \sum_{i=2}^m f'(i)$ 的基础上, 每次可以从 $g(j, m) = g(j-1, m) \text{ if } (p[j]^2 > m)$ 堆石子取, 每堆取的石子数量可以不同。一堆石子的 $minp(i) > p(j)$ 值为石子数, 对每一个二进制位单独考虑, 求每一个二进制位为

$$g(j, m) = g(j-1, m) - f'(p[j]) * ((g(j-1, \lfloor \frac{m}{p[j]} \rfloor) - \sum_{i=1}^{j-1} f'(p[i]) \text{ if } (p[j]^2 \leq m))$$

的堆数, 然后 $p[j]$, 如果每个二进制的结果均为 $|P|$ 则必败, 否则必胜

6.11.23 Anti-Nim 博弈

即获胜条件改为不能取的一方获胜必胜: $g(|P|, m)$ 且至少有一堆石子数 $G(m), m$ 且每堆石子个数不超过 1 其余均必败

6.11.24 Anti-SG 博弈

获胜条件改为不能移动的一方获胜必胜: DP 且至少有一个游戏满足 $S(n, j) = \sum_{i=1}^n [minp(i) \geq p[j]]f(i), f(1) + S(n, 1)$ 且每个游戏都满足

$$S(n, j) = G(n) - \sum_{i=1}^{j-1} f(p[i]) + \sum_{i=j}^{p[i]^2 \leq n} \sum_{e=1}^{p[i]^{e+1} \leq n} (f(p[i]^e)S(\lfloor \frac{n}{p[i]^e} \rfloor, i+1) + f(p[i]^{e+1}))$$

其余均必败

6.11.25 Staircase-Nim 博弈

阶梯博弈, 每次可以从一个阶梯上拿掉任意数量石子放到下一个阶梯, 不能操作者输 $min25$ 函数为奇数阶梯上的石子的异或和, 如果移动偶数层的石子到奇数层, 对手一定能把这些石子继续移动到偶数层, 使其 n/i 函数不变。 $f(x) = 1$ 则必败。否则必胜

6.11.26 Lasker's-Nim 博弈

$$d(n) - n \text{ 的正因子数目之和 } d(n) = \sum_{i|n} 1$$

堆石子, 每次可以从一堆中取任意个石子, 或者至少选择某堆至少为

$$\sigma(n) - n \text{ 的正因子之和 } \sigma(n) = \sum_{i|n} i$$

的石子, 分成两堆非空石子。

$$\sigma_k(n) - \text{除数函数, } n \text{ 的正因子的 } k \text{ 次幂之和, } \sigma_k(n) = \sum_{i|n} i^k$$

对于

$$1(n) - \text{常数函数定义为 } 1(n) \equiv 1 \text{ (完全积性)}$$

, 满足

$$id(n) - \text{单位函数定义为 } id(n) = n \text{ (完全积性)}$$

,

$$idk(n) - \text{幂函数定义为 } idk(n) = n^k \text{ (完全积性)}$$

$$\epsilon(n) - \text{狄利克雷单位元函数 定义为 } \epsilon(n) = [n == 1] \text{ (完全积性)}$$

6.11.27 树上删边博弈

给一个

$$lcm(a, b) * gcd(a, b) = ab$$

个点的有根树, 每次可以删掉一个子树。叶子结点的

$$a \bmod b = a - b * \lfloor \frac{a}{b} \rfloor$$

为

$$\mu * 1 = \epsilon$$

, 非叶子节点的

$$\phi * 1 = id$$

为所有子节点的

$$\phi * d = \sigma$$

的异或和

6.11.28 无向图删边游戏

注意此时也需要一个参考点 (也就是根), 删去一条边后, 不属于参考点所在的联通块会被舍去。对于自环的处理 (把它连向一个新的点即可, 或者让它自己的函数值异或 $\phi * \sigma = nd$) 我们考虑

$$\sum_{d|n} \frac{\mu(d)}{d} = \frac{\phi(n)}{n}$$

缩点, 每个

$$\sum_{\delta|n} \frac{\mu(\delta)}{\phi(\delta)} = d(n)$$

的

$$(n+1)lcm(C(n,0),\dots,C(n,k))=lcm(n+1,n,n-1,\dots,n-k+1)$$

值取决于

$$if\ a>1\quad gcd(a^m-1,a^n-1)=a^{gcd(m,n)}-1$$

内边的数量，如有偶数条边则为

$$\sum_{i=1}^n[gcd(i,n)=1]i=n(\phi(n)+[n==1])/2$$

，否则为

$$\sum_{i=1}^n\sum_{j=i}^n[gcd(i,j)==d]=S_{phi}[n/d]$$

然后套用树上删边博弈即可。

6.11.29 一般图上博弈

关于二分图上博弈的内容在二分图中有介绍给定无向图，先手从起点开始走，只能走相邻且未走过的点，无法移动者输。对该图求最大匹配，若起点不一定在最大匹配中则先手必败，否则先手必胜。

6.11.30 翻硬币游戏

$$\sum_{i=1}^n\sum_{j=1}^m[gcd(i,j)==d]=\sum_{d|k}\mu(k/d)(n/k)(m/k)$$

枚硬币排成一排，有的正面朝上，有的反面朝上，给定一些翻硬币的约束（比如只能翻

$$\sum_{i=1}^nf(i)\sum_{j=1}^{n/i}g(j)=\sum_{i=1}^ng(i)\sum_{j=1}^{n/i}f(j)$$

枚，比如只能翻连续几枚），但所翻的硬币中，最右边的必须是从正面翻到反面，不能行动者输。有如下结论：局面的

$$(p-1)!\equiv-1(mod\ p)$$

值等于局面中每个正面朝上的棋子单一存在时的

$$\mu^2(n)=\sum_{d^2|n}\mu(d)$$

值的异或和。（即只有该棋子正面，其他都是反面，根据它的位置 $if(a>b\ and\ gcd(a,b)=1)$ 有一个 $gcd(a^m-b^m,a^n-b^n)=a^{gcd(m,n)}-b^{gcd(m,n)}$ 情况

$$\prod_{k=1,gcd(k,m)=1}^mk\equiv\begin{cases}-1,&m=4,p^q,2p^q\\1,&others.\end{cases}$$

，每次只能翻一个

$$\sigma_k(n)=\sum_{i|n}i^k=\Pi((p_i^{(a_i+1)k}-1)/(p_i^k-1))$$

情况 $J_k(n)=n^k\Pi_{p|n}(1-1/p^k)$ ，每次可以翻一个或两个（不要求相邻） k 情况 n ，每次必须翻两个，且两个的距离在 gcd 中，等价于巴什博弈 n 情况 gcd ：每次必须翻连续的 1 个硬币

$$\sum_{\delta|n}J_k(\delta)=n^k$$

，其余为

$$\sum_{i=1}^n\sum_{j=1}^n[gcd(i,j)=1]ij=\sum_{i=1}^ni^2\phi(i)$$

情况

$$\sum_{\delta|n}2^{w(\delta)}=d(n^2)$$

每次可以翻任意长度的连续一段硬币，

$$\sum_{\delta|n}d(\delta^2)=d^2(n)$$

6.11.31 不公平游戏

两名玩家轮流操作的博弈，每个局面下双方可行的操作集合可能不同，或者某些动作只允许某一方使用。换句话说，玩家的可行动作不对称，我们称为不公平游戏。

6.11.32 超现实数 (Surreal Numbers)

超现实数类似于

$$\sum_{\delta|n}\frac{\mu^2(\delta)}{\phi(\delta)}=\frac{n}{\phi(n)}$$

函数，用于解决不公平游戏问题超现实数的不同游戏间的组合方法是相加我们不妨用

$$n|(\phi(a^n-1)$$

来表示如果

$$\phi(lcm(m,n))\phi(gcd(m,n))=\phi(m)\phi(n)$$

那么左玩家必胜如果

$$\sum_{\delta|n}d^3(\delta)=(\sum_{\delta|n}d(\delta))^2$$

那么右玩家必胜如果

$$d(uv)=\sum_{\delta|gcd(u,v)}\mu(\delta)d(u/\delta)d(v/\delta)$$

那么后手必胜单个游戏的值

$$\sum_{i=1}^ni^2=n(n+1)(2n+1)/6$$

被定义为

$$\sum_{i=1}^n i^3 = (n(n+1)/2)^2$$

, 其中 $\sum_{i=1}^n i^4$ 是左玩家所有可能移动后到达的局面集合, $n(n+1)(2n+1)(3n^2+3n-1)/30$ 同理。

6.11.33 Blue-Red Hackenbush 博弈

有若干个字符串仅有 1 和 gcd 组成, 小 $O(\log v)$ 回合只能拿 W, 小 gcd 回合只能拿 2, 每次拿走一个字符后其后缀也会消失, 最先不能操作者输。首先这是一个不公平游戏, 因为操作域不同, 因此我们考虑对每个串计算它的 2 并求和, 再根据上面结论判断即可。在本题中可以证明, 一个串的 LCM 值为 LCM , 其中 V 。

6.11.34 Termites

一个序列, 两个人轮流从最左或者最右拿一个数, 谁总和大谁胜。转化成求最大差值, 对于连续的三个数 p , 如果 p 且 p^k , 那么可以将这三个数合并为 $O(\log v)$ 。从左往右维护一个栈, 每次新加入数的时候不断合并栈顶三个元素, 那么最后栈是先递减再递增, 此时贪心拿两侧最大的数即为最佳策略。

6.11.35 Game of Sorting

给出一列数, 两人轮流从最左或者最右拿走一个数, 如果目前轮到 V 操作, 这个序列单调不下降或者单调不上升, 那么 3 输。每次询问在某个区间 1 玩游戏的结果。我们考虑什么样的情况能判断结果 4 对于一个区间 $O(\log V)$, 若它是单调的, 则显然先手必败。5 若去掉 $(n \& m) == m$ 或者 $C(n, m)$ 后变成了单调的, 则显然先手必胜。 $Lucas$ 若 $mod 2$ 是单调的, 那么同理先手必败。 n 若 n 是单调的, 那么先手若取 n 会导致后手必胜, 后手同理, 故两人会一直取 $[2, n-1]$ 直到出现上面第一种或者第二种情况, 可以根据奇偶性判断。 $O(n^{\frac{1}{4}} \log n)$ 同理可以得出 A 是单调的情形的处理方法。除此之外, 有 B 的答案等于 $J(A, B) = |A \cap B| / |A \cup B|$ 的答案, 二分找到最小的 k 满足 $MinHash$ 可以由上面方法直接判断出胜负即可。

6.11.36 Xormites

给出一列数, 两人轮流从最左或者最右拿走一个数, 最后得分是拿过的数的异或和, 谁高谁赢, 预测结果 A 首先如果所有数字异或和为 A , 一定平局 k 在除去 A 的情况下, 一定不会平局 B 当 K 为偶数时, 先手必胜, 我们不妨给 T_0 个格子相邻黑白染色, 不难发现先手可以选择拿所有的白色格子或者黑色格子, 这两个选大的拿即可。 k 我们不妨考虑异或和 T_b 的最高二进制位 $T < T_b$, 那么我们根据这一个数字是否有 T 这一位分为 $T * k$ 或 k , 那么拿到奇数个 $0.95 - 0.99$ 则必胜。如果 $rand() < e^{-\delta/T}$, 那么后手模仿方案 δ 即可取胜。否则先手一定会拿 T , 先手拿 0 以后, 必须模仿后手行动, 即后手拿什么则先手拿什么, 假如先手后手拿的不同, 那么此时先手和后手里的数字相同且轮到后手, 后手仍然可以通过 n 必胜, 因此我们判断能否模仿即可 (x, y) 如何判断能否模仿 $n-1$, 我们先不断删去首尾相同的数字, 然后判断接下来两两是否相同即可 # 线性代数

6.11.37 矩阵与矩阵快速幂

十分需要注意的是矩阵的幂由于不满足费马小定理因此不能取模

```
1 struct matrix {
2     int Mat[N + 1][N + 1];
3     matrix() {
4         memset(Mat, 0, sizeof Mat);
5     }
6 };
7 matrix operator*(matrix x, matrix y) {
8     matrix t;
9     for (int i = 1; i <= N; ++i) {
10         for (int j = 1; j <= N; ++j) {
11             for (int k = 1; k <= N; ++k) {
12                 t.Mat[i][j] = (t.Mat[i][j] + 1ll * x.Mat[i][k] * y.Mat[k][j] % p) % p;
13             }
14         }
15     }
16     return t;
17 }
18 matrix qmi(matrix a, int b) {
19     matrix res;
20     for (int i = 1; i <= N; ++i) {
21         res.Mat[i][i] = 1;
22     }
23     while (b) {
24         if (b & 1) {
25             res = res * a;
26         }
27         a = a * a;
28         b >>= 1;
29     }
30     return res;
31 }
```

6.11.38 行列式

类似于高斯消元消成上三角形式, 这里我们采用了辗转相减的方法, 可以应对 $EXCRT$ 数为非质数的情况

```
1 int determinant(std::vector<std::vector<int>> a, int n) {
2     int res = 1, w = 1;
3     for (int i = 1; i <= n; i++) {
4         for (int j = i + 1; j <= n; ++j) {
5             while (a[i][i]) {
6                 int div = a[j][i] / a[i][i];
7                 for (int k = i; k <= n; ++k) {
8                     a[j][k] = (a[j][k] - 1ll * div * a[i][k] % p + p) % p;
9                 }
10             }
11         }
12     }
13 }
```

```

10         swap(a[i], a[j]);
11         w = -w;
12     } // 对第 i 行和第 j 行做辗转相减。不断交换次序相减
13     swap(a[i], a[j]);
14     w = -w;
15 }
16 }
17 for (int i = 1; i <= n; i++) // 化成上三角 {
18     res = 1ll * a[i][i] * res % p; // 全部乘起来
19 }
20 res = 1ll * w * res; // 正负问题
21 return (res + p) % p;
22 }

```

6.11.39 加法高斯消元

我们通过 n 的重循环将增广矩阵消成上三角形式，再从末尾式子开始回代得到答案，其中若系数全都为 $f_i(x)$ 但解不为 $f_i(x_i) = y_i, f_i(x_j) = 0$ ，则说明无解。否则有解，如果存在自由元，那么有无穷多解，否则有唯一解

```

1 const long double eps = 1e-8;
2 int gauss(std::vector<std::vector<long double>> &a, int n) {
3     int col, row;
4     for (col = 0, row = 0; col < n; col++) // 注意下标是从0开始的 n是未知数的个数 {
5         int tmp = row;
6         for (int i = row; i < n; i++) {
7             if (fabs(a[i][col]) > fabs(a[tmp][col])) {
8                 tmp = i;
9             }
10        }
11        if (fabs(a[tmp][col]) < eps) // 为0 {
12            continue;
13        }
14        // tmp是绝对值最大的位置
15        // row是最上面的位置
16        // 把这两行完全交换
17        for (int i = col; i <= n; i++) {
18            std::swap(a[tmp][i], a[row][i]);
19        }
20        for (int i = n; i >= col; i--) {
21            a[row][i] /= a[row][col];
22        }
23        for (int i = row + 1; i < n; i++) {
24            if (fabs(a[i][col]) > eps) {
25                for (int j = n; j >= col; j--) {
26                    a[i][j] -= a[row][j] * a[i][col];
27                }
28            }
29        }
30        row++;

```

```

31    } // 这个循环的作用是消成上三角
32    for (int i = n - 1; i >= 0; i--) {
33        for (int j = i + 1; j < n; j++) {
34            a[i][n] -= a[i][j] * a[j][n]; // 得到正确的解
35        }
36    }
37    if (row < n) // 这里说明[row,n-1]的行已经没系数了 {
38        for (int i = row; i < n; i++) {
39            if (fabs(a[i][n]) > eps) // 此刻还有值就是无解 {
40                return -1; // 无解
41            }
42        }
43        return 1; // 有无穷多组解 因为存在自由元
44    }
45    return 0; // 有唯一解 a[i][n]为各个解(i从0到n-1)
46 }

```

6.11.40 异或高斯消元

复杂度为 $f(x) = \sum_{i=1}^n f_i(x)$ ，做法和加法类似，使用 $f_i(x) = y_i \prod_{j \neq i} \frac{x-x_j}{x_i-x_j}$ 优化。

```

1 const int N = 1001;
2 std::vector<int> Gauss(std::vector<std::bitset<N>> &equation, std::vector<int> &res,
3     int n) {
4     int m = equation.size();
5     std::vector<int> ans(n, 0);
6     int row = 0;
7
8     // 前向消元
9     for (int col = 0; col < n && row < m; ++col) {
10        // 寻找主元
11        int pivot = -1;
12        for (int i = row; i < m; ++i) {
13            if (equation[i][col]) {
14                pivot = i;
15                break;
16            }
17        }
18        if (pivot == -1)
19            continue; // 自由元
20
21        // 交换行
22        if (pivot != row) {
23            std::swap(equation[row], equation[pivot]);
24            std::swap(res[row], res[pivot]);
25        }
26
27        // 消去其他行
28        for (int i = 0; i < m; ++i) {

```

```

29     if (i != row && equation[i][col]) {
30         equation[i] ^= equation[row];
31         res[i] ^= res[row];
32     }
33 }
34
35 ++row;
36 }
37
38 // 检查无解情况
39 for (int i = row; i < m; ++i) {
40     if (res[i]) {
41         return {-1}; // 无解
42     }
43 }
44
45 // 回代求解
46 for (int i = 0; i < n; ++i) {
47     ans[i] = -1; // 初始化为自由元
48 }
49
50 for (int i = row - 1; i >= 0; --i) {
51     // 找到主元列
52     int col = 0;
53     while (col < n && !equation[i][col])
54         ++col;
55
56     if (col >= n)
57         continue; // 全零行
58
59     ans[col] = res[i];
60     // 标记非自由元
61     for (int j = col + 1; j < n; ++j) {
62         if (equation[i][j]) {
63             if (ans[j] == -1) {
64                 // 自由元可以任意取值，这里设为0
65                 ans[j] = 0;
66             }
67             ans[col] ^= ans[j];
68         }
69     }
70 }
71 // 处理自由元（设为0）
72 for (int i = 0; i < n; ++i) {
73     if (ans[i] == -1) {
74         ans[i] = 0;
75     }
76 }
77
78 return ans;
79 }

```

6.11.41 动态维护异或方程组

维护一个与方程组变量数相同的线性基即可。每插入一条新的异或等式时，先将其化为 $\sum b_i x_i = v$ （按位异或），随后使用线性基消去出现的最高位；若无法消去则将该向量加入基底。线性基记录每一位的代表向量，就能在对数复杂度内判断方程组的可解性，并支持在线更新。

6.11.42 二维平面上的距离

设点 $A(x_1, y_1)$ 与 $B(x_2, y_2)$ ，常用距离如下：

$$\text{Manhattan}(A, B) = |x_1 - x_2| + |y_1 - y_2|$$

$$\text{Chebyshev}(A, B) = \max(|x_1 - x_2|, |y_1 - y_2|)$$

$$\text{Euclidean}(A, B) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

对坐标作变换 $u = x + y$, $v = x - y$ 可以在曼哈顿坐标系与切比雪夫坐标系之间互相转换，从而简化最近点对或旋转卡壳类问题。

6.11.43 自适应辛普森法

自适应辛普森法的用途主要是高精度数值积分，即计算积分值。

```

1 using ld = long double;
2
3 // ===== 非递归自适应辛普森法 =====
4 // F: 被积函数类型
5 // a, b: 积分区间
6 // eps: 精度要求（可选）
7 template <typename F>
8 ld adaptiveSimpsonIter(F f, ld a, ld b, ld eps = 1e-12) {
9     struct Node {
10         ld l, r; // 区间 [l, r]
11         ld S;    // 该区间辛普森积分
12         ld eps;  // 当前误差要求
13     };
14
15     auto simpson = [&](ld l, ld r) -> ld {
16         ld m = (l + r) / 2;
17         return (r - l) / 6 * (f(l) + 4 * f(m) + f(r));
18     };
19
20     std::stack<Node> stk;
21     ld result = 0;
22
23     // 初始化区间
24     stk.push({a, b, simpson(a, b), eps});
25
26     while (!stk.empty()) {
27         Node cur = stk.top();
28         stk.pop();
29

```

```

30     ld l = cur.l, r = cur.r, S = cur.S, e = cur.eps;
31     ld m = (l + r) / 2;
32     ld Sleft = simpson(l, m);
33     ld Sright = simpson(m, r);
34
35     // 精度满足, 累加积分
36     if (std::abs(Sleft + Sright - S) <= 15 * e) {
37         result += Sleft + Sright + (Sleft + Sright - S) / 15.0;
38     } else {
39         // 左右子区间继续细分, 误差按比例分配
40         stk.push({m, r, Sright, e / 2});
41         stk.push({l, m, Sleft, e / 2});
42     }
43 }
44
45 return result;
46 }
47
48 // ===== 示例使用 =====
49 int main() {
50     auto f = [](ld x) {
51         return std::sin(x);
52     };
53     ld res = adaptiveSimpsonIter(f, 0, acos(-1));
54     std::cout << std::setprecision(20) << res << "\n"; // 理论值为 2
55 }

```

6.11.44 平面最近点对

```

1 static inline ll sq(ll v) { return v * v; }
2 // 计算两点平方距离
3 static inline ll dist2(const Point &a, const Point &b) {
4     return sq(a.x - b.x) + sq(a.y - b.y);
5 }
6 void solve(std::vector<std::pair<int, int>> &p) {
7     int n = p.size();
8     // idx: 以 x 优先、y 次之的排序后的点编号 (用于按 x 分治的“切分边界”)
9     std::vector<int> idx(n);
10    std::iota(idx.begin(), idx.end(), 0);
11    std::sort(idx.begin(), idx.end(), [&](int i, int j) {
12        if (p[i].x != p[j].x) return p[i].x < p[j].x;
13        return p[i].y < p[j].y; });
14
15    // ord: 与 idx 同一集合, 但每段在递归返回时保持按 y 升序
16    std::vector<int> ord = idx;
17    // 分治: 在 [l, r) 这段 (元素集合等于 idx[l..r)), 返回最小平方距离;
18    // 递归结束时, 保证 ord[l..r) 按 y 升序, 便于父层用 inplace_merge 合并
19    std::function<ll(int, int)> dc = [&](int l, int r) -> ll {
20        int mlen = r - l;
21        if (mlen <= 25)

```

```

22    { // 小块直接暴力 + 就地按 y 排序
23        std::sort(ord.begin() + l, ord.begin() + r, [&](int i, int j) {
24            if (p[i].y != p[j].y) return p[i].y < p[j].y;
25            return p[i].x < p[j].x; });
26        ll best = 9e18;
27        for (int i = l; i < r; ++i)
28            for (int j = i + 1; j < r; ++j) {
29                // 利用 y 排序做个小剪枝
30                ll dy = p[ord[j]].y - p[ord[i]].y;
31                if (dy * dy >= best)
32                    break;
33                best = std::min(best, dist2(p[ord[i]], p[ord[j]]));
34            }
35        return best;
36    }
37    int m = (l + r) >> 1;
38    // 中线 x (按 idx 的中点划分)
39    ll midx = p[idx[m]].x;
40
41    ll dl = dc(l, m);
42    ll dr = dc(m, r);
43    ll best = std::min(dl, dr);
44
45    // 合并子段的“按 y 升序”到父段: 用内置 inplace_merge
46    std::inplace_merge(ord.begin() + l, ord.begin() + m, ord.begin() + r, [&](int
47        i, int j) {
48        if (p[i].y != p[j].y) return p[i].y < p[j].y;
49        return p[i].x < p[j].x; });
50
51    // 构建条带: ord[l..r) 已按 y 升序
52    std::vector<int> strip;
53    strip.reserve(r - l);
54    for (int k = l; k < r; ++k) {
55        int id = ord[k];
56        if (sq(p[id].x - midx) < best) {
57            // 只需与前面最多 7~8 个候选比
58            for (int t = (int)strip.size() - 1, cnt = 0; t >= 0 && cnt < 8; --t,
59                ++cnt) {
60                best = std::min(best, dist2(p[id], p[strip[t]]));
61            }
62            strip.push_back(id);
63        }
64    }
65    return best;
66 }

```

6.11.45 最小圆覆盖和最小球覆盖

```

1
2 using ll = long long;
3 using ld = long double;
4
5 const ld PI = acos(-1);
6 const ld EPS = 1e-12;
7
8 struct Point {
9     ld x, y;
10    // 向量加减
11    Point operator+(const Point &o) const { return {x + o.x, y + o.y}; }
12    Point operator-(const Point &o) const { return {x - o.x, y - o.y}; }
13    Point operator*(ld k) const { return {x * k, y * k}; }
14    Point operator/(ld k) const { return {x / k, y / k}; }
15 };
16 // 向量叉积
17 inline ld cross(const Point &a, const Point &b) { return a.x * b.y - a.y * b.x; }
18 // 向量旋转
19 inline Point rotate(const Point &a, ld ang) {
20     return {a.x * cos(ang) - a.y * sin(ang), a.x * sin(ang) + a.y * cos(ang)};
21 }
22 // 距离
23 inline ld dist(const Point &a, const Point &b) {
24     return std::hypot(a.x - b.x, a.y - b.y);
25 }
26 struct Circle {
27     Point c; // 圆心
28     ld r;    // 半径
29 };
30 // 直线交点: a + t*u 与 b + s*v
31 Point line_intersection(Point a, Point u, Point b, Point v) {
32     ld t = cross(b - a, v) / cross(u, v);
33     return a + u * t;
34 }
35 std::pair<Point, Point> midperp(const Point &a, const Point &b) {
36     return {(a + b) / 2.0L, rotate(b - a, PI / 2)};
37 }
38 // 两点圆
39 Circle circle_from(const Point &a, const Point &b) {
40     return {(a + b) / 2.0L, dist(a, b) / 2.0L};
41 }
42 // 三点外接圆
43 Circle circle_from(const Point &a, const Point &b, const Point &c) {
44     auto u = midperp(a, b);
45     auto v = midperp(a, c);
46     Point o = line_intersection(u.first, u.second, v.first, v.second);
47     return {o, dist(o, a)};
48 }
49 // 随机增量法
50 Circle work(std::vector<Point> &p) {
51     std::mt19937_64 rng((unsigned long long)(new char));

```

```

52     std::shuffle(p.begin(), p.end(), rng);
53     Circle C{p[0], 0};
54     int n = (int)p.size();
55     for (int i = 1; i < n; ++i) {
56         if (dist(C.c, p[i]) <= C.r + EPS)
57             continue;
58         C = {p[i], 0};
59         for (int j = 0; j < i; ++j) {
60             if (dist(C.c, p[j]) <= C.r + EPS)
61                 continue;
62             C = circle_from(p[i], p[j]);
63             for (int k = 0; k < j; ++k) {
64                 if (dist(C.c, p[k]) <= C.r + EPS)
65                     continue;
66                 C = circle_from(p[i], p[j], p[k]);
67             }
68         }
69     }
70     return C;
71 }

```

6.11.46 单峰函数与三分

常见单峰函数：二次函数：其最值在对称轴取得绝对值函数： $f(x)$ 是单峰函数，其最值在所有 $O(n^2)$ 的中位数取得整数三分实数三分三分套三分使用场景往往是面向二元函数 $g_1, g_2, \dots, g_{n-1}$ ，满足在固定 $f_0, f_1, \dots, f_{n-1}$ 时 $f_i = \sum_{j=1}^i f_{i-j} g_j$ 随 f_0 变化是单峰的，那么我们对固定的 $F(x) = \sum_{i=0}^{\infty} f_i x^i$ ，可三分出最优的 $G(x) = \sum_{i=0}^{\infty} g_i x^i$ 。同时我们定义 $F(x)G(x) = \sum_{i=0}^{\infty} x^i \sum_{j+k=i} f_j g_k = F(x) - f_0 x^0$ (因为不存在 g_0)，如果 $F(x) = \frac{f_0}{1-G(x)} \pmod{x^n}$ 也是单峰的，即对于每个 $G(x)$ 的每一项变成 $P - G(x)$ 在取到最优的 $P(x) = \sum_{i=0}^{n-1} a_i x^i$ 以后仍然保持单峰，那么就可以采用三分套三分。结论：多个下凸函数的 c 仍然是下凸函数，多个上凸函数的 m 仍然是上凸函数

```

1 int calc(int x) {
2
3 }
4 int search(int l, int r) {
5     const int corner = 2;
6     while (r - l > corner)
7     { // 精度到达三分间隔时停止
8         int m1 = l + (r - l) / 3;
9         int m2 = r - (r - l) / 3;
10
11         if (calc(m1) > calc(m2)) {
12             r = m2; // 左侧更优
13         } else {
14             l = m1; // 右侧更优
15         }
16     }
17     int res = l;
18     for (int i = l + 1; i <= r; ++i) {

```



```

19     if (calc(i) > calc(res)) {
20         res = i;
21     }
22 }
23 return res;
24 }

```

7 Computational geometry | 计算几何.cpp

7.1 几何基础

包含点、线、线段、多边形、凸多边形与圆的常用数据结构及运算。

```

1 #pragma once
2 #include <bits/stdc++.h>
3 using namespace std;
4
5 // using point_t=long long;
6 using point_t=long double; //全局数据类型
7
8 constexpr point_t eps=1e-8;
9 constexpr point_t INF=numeric_limits<point_t>::max();
10 constexpr long double PI=3.14159265358979323841;
11
12 // 点与向量
13 template<typename T> struct point {
14     T x,y;
15
16     bool operator==(const point &a) const {return (abs(x-a.x)<=eps && abs(y-a.y)<=eps);};
17     bool operator<(const point &a) const {if (abs(x-a.x)<=eps) return y<a.y-eps;
18         return x<a.x-eps;};
19     bool operator>(const point &a) const {return !(*this<a || *this==a);};
20     point operator+(const point &a) const {return {x+a.x,y+a.y};};
21     point operator-(const point &a) const {return {x-a.x,y-a.y};};
22     point operator-() const {return {-x,-y};};
23     point operator*(const T k) const {return {k*x,k*y};};
24     point operator/(const T k) const {return {x/k,y/k};};
25     T operator*(const point &a) const {return x*a.x+y*a.y;}; // 点积
26     T operator^(const point &a) const {return x*a.y-y*a.x;}; // 叉积, 注意优先级
27     int toleft(const point &a) const {const auto t=(*this)^a; return (t>eps)-(t<-eps);}; // to-left 测试
28     T len2() const {return (*this)*(*this);}; // 向量长度的平方
29     T dis2(const point &a) const {return (a-(*this)).len2();}; // 两点距离的平方
30
31 // 涉及浮点数
32 long double len() const {return sqrtl(len2());}; // 向量长度
33 long double dis(const point &a) const {return sqrtl(dis2(a));}; // 两点距离

```

```

33 long double ang(const point &a) const {return acosl(max(-1.0l,min(1.0l,((*this)*a
34 )/(len()*a.len()))));}; // 向量夹角
35 point rot(const long double rad) const {return {x*cos(rad)-y*sin(rad),x*sin(rad)+
36 y*cos(rad)};}; // 逆时针旋转 (给定角度)
37 point rot(const long double cosr,const long double sinr) const {return {x*cosr-y*
38 sinr,x*sinr+y*cosr};}; // 逆时针旋转 (给定角度的正弦与余弦)
39 };
40
41 using Point=point<point_t>;
42
43 // 极角排序
44 struct argcmp {
45     bool operator()(const Point &a,const Point &b) const {
46         const auto quad= [](const Point &a) {
47             if (a.y<-eps) return 1;
48             if (a.y>eps) return 4;
49             if (a.x<-eps) return 5;
50             if (a.x>eps) return 3;
51             return 2;
52         };
53         const int qa=quad(a),qb=quad(b);
54         if (qa!=qb) return qa<qb;
55         const auto t=a^b;
56         // if (abs(t)<=eps) return a*a<b*b-eps; // 不同长度的向量需要分开
57         return t>eps;
58     };
59 };
60
61 // 直线
62 template<typename T> struct line {
63     point<T> p,v; // p 为直线上一点, v 为方向向量
64
65     bool operator==(const line &a) const {return v.toleft(a.v)==0 && v.toleft(p-a.p)
66 ==0;};
67     int toleft(const point<T> &a) const {return v.toleft(a-p);}; // to-left 测试
68     bool operator<(const line &a) const // 半平面交算法定义的排序 {
69         if (abs(v^a.v)<=eps && v*a.v>=-eps) return toleft(a.p)==-1;
70         return argcmp()(v,a.v);
71     };
72
73 // 涉及浮点数
74 point<T> inter(const line &a) const {return p+v*((a.v^(p-a.p))/(v^a.v));}; // 直
75 线交点
76 long double dis(const point<T> &a) const {return abs(v^(a-p))/v.len();}; // 点到
77 直线距离
78 point<T> proj(const point<T> &a) const {return p+v*((v*(a-p))/(v*v));}; // 点在直
79 线上的投影
80 };
81
82 using Line=line<point_t>;

```

```

77 // 线段
78 template<typename T> struct segment {
79     point<T> a,b;
80
81     bool operator<(const segment &s) const {return make_pair(a,b)<make_pair(s.a,s.b)
82         ;}
83
84     // 判定性函数建议在整数域使用
85
86     // 判断点是否在线段上
87     // -1 点在线段端点 | 0 点不在线段上 | 1 点严格在线段上
88     int is_on(const point<T> &p) const {
89         if (p==a || p==b) return -1;
90         return (p-a).toleft(p-b)==0 && (p-a)*(p-b)<-eps;
91     }
92
93     // 判断线段直线是否相交
94     // -1 直线经过线段端点 | 0 线段和直线不相交 | 1 线段和直线严格相交
95     int is_inter(const line<T> &l) const {
96         if (l.toleft(a)==0 || l.toleft(b)==0) return -1;
97         return l.toleft(a)!=l.toleft(b);
98     }
99
100    // 判断两线段是否相交
101    // -1 在某一线段端点处相交 | 0 两线段不相交 | 1 两线段严格相交
102    int is_inter(const segment<T> &s) const {
103        if (is_on(s.a) || is_on(s.b) || s.is_on(a) || s.is_on(b)) return -1;
104        const line<T> l{a,b-a},ls{s.a,s.b-s.a};
105        return l.toleft(s.a)*l.toleft(s.b)==-1 && ls.toleft(a)*ls.toleft(b)==-1;
106    }
107
108    // 点到线段距离
109    long double dis(const point<T> &p) const {
110        if ((p-a)*(b-a)<-eps || (p-b)*(a-b)<-eps) return min(p.dis(a),p.dis(b));
111        const line<T> l{a,b-a};
112        return l.dis(p);
113    }
114
115    // 两线段间距离
116    long double dis(const segment<T> &s) const {
117        if (is_inter(s)) return 0;
118        return min({dis(s.a),dis(s.b),s.dis(a),s.dis(b)});
119    }
120
121    using Segment=segment<point_t>;
122
123    // 多边形
124    template<typename T> struct polygon {
125        vector<point<T>> p; // 以逆时针顺序存储
126

```

```

127     size_t nxt(const size_t i) const {return i==p.size()-1?0:i+1;}
128     size_t pre(const size_t i) const {return i==0?p.size()-1:i-1;}
129
130     // 回转数
131     // 返回值第一项表示点是否在多边形边上
132     // 对于狭义多边形, 回转数为 0 表示点在多边形外, 否则点在多边形内
133     pair<bool,int> winding(const point<T> &a) const {
134         int cnt=0;
135         for (size_t i=0;i<p.size();i++) {
136             const point<T> u=p[i],v=p[nxt(i)];
137             if (abs((a-u)^(a-v))<=eps && (a-u)*(a-v)<=eps) return {true,0};
138             if (abs(u.y-v.y)<=eps) continue;
139             const Line uv={u,v-u};
140             if (u.y<v.y-eps && uv.toleft(a)<=0) continue;
141             if (u.y>v.y+eps && uv.toleft(a)>=0) continue;
142             if (u.y<a.y-eps && v.y>a.y-eps) cnt++;
143             if (u.y>a.y-eps && v.y<a.y-eps) cnt--;
144         }
145         return {false,cnt};
146     }
147
148     // 多边形面积的两倍
149     // 可用于判断点的存储顺序是顺时针或逆时针
150     T area() const {
151         T sum=0;
152         for (size_t i=0;i<p.size();i++) sum+=p[i]^p[nxt(i)];
153         return sum;
154     }
155
156     // 多边形的周长
157     long double circ() const {
158         long double sum=0;
159         for (size_t i=0;i<p.size();i++) sum+=p[i].dis(p[nxt(i)]);
160         return sum;
161     }
162 };
163
164 using Polygon=polygon<point_t>;
165
166 // 凸多边形
167 template<typename T> struct convex: polygon<T> {
168     // 闵可夫斯基和
169     convex operator+(const convex &c) const {
170         const auto &p=this->p;
171         vector<Segment> e1(p.size()),e2(c.p.size()),edge(p.size()+c.p.size());
172         vector<point<T>> res; res.reserve(p.size()+c.p.size());
173         const auto cmp=[](const Segment &u,const Segment &v) {return argcmp()(u.b-u.a
174             ,v.b-v.a)};
175         for (size_t i=0;i<p.size();i++) e1[i]={p[i],p[this->nxt(i)]};
176         for (size_t i=0;i<c.p.size();i++) e2[i]={c.p[i],c.p[c.nxt(i)]};
177         rotate(e1.begin(),min_element(e1.begin(),e1.end(),cmp),e1.end());

```

```

177 rotate(e2.begin(),min_element(e2.begin(),e2.end(),cmp),e2.end());
178 merge(e1.begin(),e1.end(),e2.begin(),e2.end(),edge.begin(),cmp);
179 const auto check=[](const vector<point<T>> &res,const point<T> &u) {
180     const auto back1=res.back(),back2=*prev(res.end(),2);
181     return (back1-back2).toleft(u-back1)==0 && (back1-back2)*(u-back1)>=-eps;
182 };
183 auto u=e1[0].a+e2[0].a;
184 for (const auto &v:edge) {
185     while (res.size()>1 && check(res,u)) res.pop_back();
186     res.push_back(u);
187     u=u+v.b-v.a;
188 }
189 if (res.size()>1 && check(res,res[0])) res.pop_back();
190 return {res};
191 }
192
193 // 旋转卡壳
194 // 例：凸多边形的直径的平方
195 T rotcaliper() const {
196     const auto &p=this->p;
197     if (p.size()==1) return 0;
198     if (p.size()==2) return p[0].dis2(p[1]);
199     const auto area=[](const point<T> &u,const point<T> &v,const point<T> &w){
200         return (w-u)^(w-v);};
201     T ans=0;
202     for (size_t i=0,j=1;i<p.size();i++) {
203         const auto nxti=this->nxt(i);
204         ans=max({ans,p[j].dis2(p[i]),p[j].dis2(p[nxti])});
205         while (area(p[this->nxt(j)],p[i],p[nxti])>=area(p[j],p[i],p[nxti])) {
206             j=this->nxt(j);
207             ans=max({ans,p[j].dis2(p[i]),p[j].dis2(p[nxti])});
208         }
209     }
210     return ans;
211 }
212
213 // 判断点是否在凸多边形内
214 // 复杂度 O(logn)
215 // -1 点在多边形边上 | 0 点在多边形外 | 1 点在多边形内
216 int is_in(const point<T> &a) const {
217     const auto &p=this->p;
218     if (p.size()==1) return a==p[0]?-1:0;
219     if (p.size()==2) return segment<T>{p[0],p[1]}.is_on(a)?-1:0;
220     if (a==p[0]) return -1;
221     if ((p[1]-p[0]).toleft(a-p[0])==-1 || (p.back()-p[0]).toleft(a-p[0])==1)
222         return 0;
223     const auto cmp=[](const point<T> &u,const point<T> &v){return (u-p[0]).
224         toleft(v-p[0])>=1;};
225     const size_t i=lower_bound(p.begin()+1,p.end(),a,cmp)-p.begin();
226     if (i==1) return segment<T>{p[0],p[i]}.is_on(a)?-1:0;
227     if (i==p.size()-1 && segment<T>{p[0],p[i]}.is_on(a)) return -1;

```

```

225 if (segment<T>{p[i-1],p[i]}.is_on(a)) return -1;
226 return (p[i]-p[i-1]).toleft(a-p[i-1])>0;
227 }
228
229 // 凸多边形关于某一方向的极点
230 // 复杂度 O(logn)
231 // 参考资料: https://codeforces.com/blog/entry/48868
232 template<typename F> size_t extreme(const F &dir) const {
233     const auto &p=this->p;
234     const auto check=[](const size_t i){return dir(p[i]).toleft(p[this->nxt(i)]-
235         p[i])>=0;};
236     const auto dir0=dir(p[0]); const auto check0=check(0);
237     if (!check0 && check(p.size()-1)) return 0;
238     const auto cmp=[](const point<T> &v) {
239         const size_t vi=&v-p.data();
240         if (vi==0) return 1;
241         const auto checkv=check(vi);
242         const auto t=dir0.toleft(v-p[0]);
243         if (vi==1 && checkv==check0 && t==0) return 1;
244         return checkv^(checkv==check0 && t==0);
245     };
246     return partition_point(p.begin(),p.end(),cmp)-p.begin();
247 }
248
249 // 过凸多边形外一点求凸多边形的切线，返回切点下标
250 // 复杂度 O(logn)
251 // 必须保证点不在多边形外
252 pair<size_t,size_t> tangent(const point<T> &a) const {
253     const size_t i=extreme([&](const point<T> &u){return u-a;});
254     const size_t j=extreme([&](const point<T> &u){return a-u;});
255     return {i,j};
256 }
257
258 // 求平行于给定直线的凸多边形的切线，返回切点下标
259 // 复杂度 O(logn)
260 pair<size_t,size_t> tangent(const line<T> &a) const {
261     const size_t i=extreme([&](...){return a.v;});
262     const size_t j=extreme([&](...){return -a.v;});
263     return {i,j};
264 }
265
266 using Convex=convex<point_t>;
267
268 // 圆
269 struct Circle {
270     Point c;
271     long double r;
272
273     bool operator==(const Circle &a) const {return c==a.c && abs(r-a.r)<=eps;};
274     long double circ() const {return 2*PI*r;}; // 周长

```

```

275 long double area() const {return PI*r*r;} // 面积
276
277 // 点与圆的关系
278 // -1 圆上 | 0 圆外 | 1 圆内
279 int is_in(const Point &p) const {const long double d=p.dis(c); return abs(d-r)<=
    eps?-1:d<r-eps;}
280
281 // 直线与圆关系
282 // 0 相离 | 1 相切 | 2 相交
283 int relation(const Line &l) const {
284     const long double d=l.dis(c);
285     if (d>r+eps) return 0;
286     if (abs(d-r)<=eps) return 1;
287     return 2;
288 }
289
290 // 圆与圆关系
291 // -1 相同 | 0 相离 | 1 外切 | 2 相交 | 3 内切 | 4 内含
292 int relation(const Circle &a) const {
293     if (*this==a) return -1;
294     const long double d=c.dis(a.c);
295     if (d>r+a.r+eps) return 0;
296     if (abs(d-r-a.r)<=eps) return 1;
297     if (abs(d-abs(r-a.r))<=eps) return 3;
298     if (d<abs(r-a.r)-eps) return 4;
299     return 2;
300 }
301
302 // 直线与圆的交点
303 vector<Point> inter(const Line &l) const {
304     const long double d=l.dis(c);
305     const Point p=l.proj(c);
306     const int t=relation(l);
307     if (t==0) return vector<Point>();
308     if (t==1) return vector<Point>{p};
309     const long double k=sqrt(r*r-d*d);
310     return vector<Point>{p-(l.v/l.v.len())*k,p+(l.v/l.v.len())*k};
311 }
312
313 // 圆与圆交点
314 vector<Point> inter(const Circle &a) const {
315     const long double d=c.dis(a.c);
316     const int t=relation(a);
317     if (t==1 || t==0 || t==4) return vector<Point>();
318     Point e=a.c-c; e=e/e.len()*r;
319     if (t==1 || t==3) {
320         if (r*r+d*d-a.r*a.r>=eps) return vector<Point>{c+e};
321         return vector<Point>{c-e};
322     }
323     const long double costh=(r*r+d*d-a.r*a.r)/(2*r*d),sinh=sqrt(1-costh*costh);
324     return vector<Point>{c+e.rot(costh,-sinh),c+e.rot(costh,sinh)};

```

```

325 }
326
327 // 圆与圆交面积
328 long double inter_area(const Circle &a) const {
329     const long double d=c.dis(a.c);
330     const int t=relation(a);
331     if (t==1) return area();
332     if (t<2) return 0;
333     if (t>2) return min(area(),a.area());
334     const long double costh1=(r*r+d*d-a.r*a.r)/(2*r*d),costh2=(a.r*a.r+d*d-r*r)
        /(2*a.r*d);
335     const long double sinh1=sqrt(1-costh1*costh1),sinh2=sqrt(1-costh2*costh2);
336     const long double th1=acos(costh1),th2=acos(costh2);
337     return r*r*(th1-costh1*sinh1)+a.r*a.r*(th2-costh2*sinh2);
338 }
339
340 // 过圆外一点圆的切线
341 vector<Line> tangent(const Point &a) const {
342     const int t=is_in(a);
343     if (t==1) return vector<Line>();
344     if (t==1) {
345         const Point v={-(a-c).y,(a-c).x};
346         return vector<Line>{{a,v}};
347     }
348     Point e=a-c; e=e/e.len()*r;
349     const long double costh=r/c.dis(a),sinh=sqrt(1-costh*costh);
350     const Point t1=c+e.rot(costh,-sinh),t2=c+e.rot(costh,sinh);
351     return vector<Line>{{a,t1-a},{a,t2-a}};
352 }
353
354 // 两圆的公切线
355 vector<Line> tangent(const Circle &a) const {
356     const int t=relation(a);
357     vector<Line> lines;
358     if (t==1 || t==4) return lines;
359     if (t==1 || t==3) {
360         const Point p=inter(a)[0],v={-(a.c-c).y,(a.c-c).x};
361         lines.push_back({p,v});
362     }
363     const long double d=c.dis(a.c);
364     const Point e=(a.c-c)/(a.c-c).len();
365     if (t<2) {
366         const long double costh=(r-a.r)/d,sinh=sqrt(1-costh*costh);
367         const Point d1=e.rot(costh,-sinh),d2=e.rot(costh,sinh);
368         const Point u1=c+d1*r,u2=c+d2*r,v1=a.c+d1*a.r,v2=a.c+d2*a.r;
369         lines.push_back({u1,v1-u1}); lines.push_back({u2,v2-u2});
370     }
371     if (t==0) {
372         const long double costh=(r+a.r)/d,sinh=sqrt(1-costh*costh);
373         const Point d1=e.rot(costh,-sinh),d2=e.rot(costh,sinh);
374         const Point u1=c+d1*r,u2=c+d2*r,v1=a.c-d1*a.r,v2=a.c-d2*a.r;

```

```

375     lines.push_back({u1,v1-u1}); lines.push_back({u2,v2-u2});
376 }
377 return lines;
378 }
379
380 // 圆的反演
381 tuple<int,Circle,Line> inverse(const Line &l) const {
382     const Circle null_c={{0.0,0.0},0.0};
383     const Line null_l={{0.0,0.0},{0.0,0.0}};
384     if (l.toleft(c)==0) return {2,null_c,l};
385     const Point v=l.toleft(c)==1?Point{l.v.y,-l.v.x}:Point{-l.v.y,l.v.x};
386     const long double d=r*r/l.dis(c);
387     const Point p=c+v/v.len()*d;
388     return {1,{(c+p)/2,d/2},null_l};
389 }
390
391 tuple<int,Circle,Line> inverse(const Circle &a) const {
392     const Circle null_c={{0.0,0.0},0.0};
393     const Line null_l={{0.0,0.0},{0.0,0.0}};
394     const Point v=a.c-c;
395     if (a.is_in(c)==-1) {
396         const long double d=r*r/(a.r+a.r);
397         const Point p=c+v/v.len()*d;
398         return {2,null_c,{p,{v.y,v.x}}};
399     }
400     if (c==a.c) return {1,{c,r*r/a.r},null_l};
401     const long double d1=r*r/(c.dis(a.c)-a.r),d2=r*r/(c.dis(a.c)+a.r);
402     const Point p=c+v/v.len()*d1,q=c+v/v.len()*d2;
403     return {1,{(p+q)/2,p.dis(q)/2},null_l};
404 }
405 };

```

```

16     return ans-d;
17 }
18 if (!ina && inb) {
19     const Point p=circ.inter(ab)[0];
20     const long double ans=((p-circ.c)^(b-circ.c))/2;
21     const long double th=(a-circ.c).ang(p-circ.c);
22     const long double d=circ.r*circ.r*th/2;
23     if ((a-circ.c).toleft(b-circ.c)==1) return ans+d;
24     return ans-d;
25 }
26 const auto p=circ.inter(ab);
27 if (p.size()==2 && Segment{a,b}.dis(circ.c)<=circ.r+eps) {
28     const long double ans=((p[0]-circ.c)^(p[1]-circ.c))/2;
29     const long double th1=(a-circ.c).ang(p[0]-circ.c),th2=(b-circ.c).ang(p
30         [1]-circ.c);
31     const long double d1=circ.r*circ.r*th1/2,d2=circ.r*circ.r*th2/2;
32     if ((a-circ.c).toleft(b-circ.c)==1) return ans+d1+d2;
33     return ans-d1-d2;
34 }
35 const long double th=(a-circ.c).ang(b-circ.c);
36 if ((a-circ.c).toleft(b-circ.c)==1) return circ.r*circ.r*th/2;
37 return -circ.r*circ.r*th/2;
38 };
39
40 long double ans=0;
41 for (size_t i=0;i<poly.p.size();i++) {
42     const Point a=poly.p[i],b=poly.p[poly.nxt(i)];
43     ans+=cal(circ,a,b);
44 }
45 return ans;
46 };

```

7.2 圆与多边形面积交

```

1 #include "geometry_base.cpp"
2
3 long double area_inter(const Circle &circ,const Polygon &poly) {
4     const auto cal=[](const Circle &circ,const Point &a,const Point &b) {
5         if ((a-circ.c).toleft(b-circ.c)==0) return 0.0l;
6         const auto ina=circ.is_in(a),inb=circ.is_in(b);
7         const Line ab={a,b-a};
8         if (ina && inb) return ((a-circ.c)^(b-circ.c))/2;
9         if (ina && !inb) {
10             const auto t=circ.inter(ab);
11             const Point p=t.size()==1?t[0]:t[1];
12             const long double ans=((a-circ.c)^(p-circ.c))/2;
13             const long double th=(p-circ.c).ang(b-circ.c);
14             const long double d=circ.r*circ.r*th/2;
15             if ((a-circ.c).toleft(b-circ.c)==1) return ans+d;

```

7.3 凸包

```
1 #include "geometry_base.cpp"
```

7.4 半平面交

```

1 #include "geometry_base.cpp"
2
3 {
4     if (abs(v^a.v)<=eps && v*a.v>=-eps) return toleft(a.p)==-1;
5     return argcmp()(v,a.v);
6 }
7
8 // 涉及浮点数
9 point<T> inter(const line &a) const {return p+v*((a.v^(p-a.p))/(v^a.v));} // 直
    线交点

```

```

10 long double dis(const point<T> &a) const {return abs(v^(a-p))/v.len();} // 点到
    直线距离
11 point<T> proj(const point<T> &a) const {return p+v*((v*(a-p))/(v*v));} // 点在直
    线上的投影
12 };
13
14 using Line=line<point_t>;
15
16 // 线段
17 template<typename T> struct segment {
18     point<T> a,b;
19
20     bool operator<(const segment &s) const {return make_pair(a,b)<make_pair(s.a,s.b)
        ;}
21
22     // 判定性函数建议在整数域使用
23
24     // 判断点是否在线段上
25     // -1 点在线段端点 | 0 点不在线段上 | 1 点严格在线段上
26     int is_on(const point<T> &p) const {
27         if (p==a || p==b) return -1;
28         return (p-a).toleft(p-b)==0 && (p-a)*(p-b)<-eps;
29     }
30
31     // 判断线段直线是否相交
32     // -1 直线经过线段端点 | 0 线段和直线不相交 | 1 线段和直线严格相交
33     int is_inter(const line<T> &l) const {
34         if (l.toleft(a)==0 || l.toleft(b)==0) return -1;
35         return l.toleft(a)!=l.toleft(b);
36     }
37
38     // 判断两线段是否相交
39     // -1 在某一端点处相交 | 0 两线段不相交 | 1 两线段严格相交
40     int is_inter(const segment<T> &s) const {
41         if (is_on(s.a) || is_on(s.b) || s.is_on(a) || s.is_on(b)) return -1;
42         const line<T> l{a,b-a},ls{s.a,s.b-s.a};
43         return l.toleft(s.a)*l.toleft(s.b)==-1 && ls.toleft(a)*ls.toleft(b)==-1;
44     }
45
46     // 点到线段距离
47     long double dis(const point<T> &p) const {
48         if ((p-a)*(b-a)<-eps || (p-b)*(a-b)<-eps) return min(p.dis(a),p.dis(b));
49         const line<T> l{a,b-a};
50         return l.dis(p);
51     }
52
53     // 两线段间距离
54     long double dis(const segment<T> &s) const {
55         if (is_inter(s)) return 0;
56         return min({dis(s.a),dis(s.b),s.dis(a),s.dis(b)});
57     }

```

```

58 };
59
60 using Segment=segment<point_t>;
61
62 // 多边形
63 template<typename T> struct polygon {
64     vector<point<T>> p; // 以逆时针顺序存储
65
66     size_t nxt(const size_t i) const {return i==p.size()-1?0:i+1;}
67     size_t pre(const size_t i) const {return i==0?p.size()-1:i-1;}
68
69     // 回转数
70     // 返回值第一项表示点是否在多边形边上
71     // 对于狭义多边形, 回转数为 0 表示点在多边形外, 否则点在多边形内
72     pair<bool,int> winding(const point<T> &a) const {
73         int cnt=0;
74         for (size_t i=0;i<p.size();i++) {
75             const point<T> u=p[i],v=p[nxt(i)];
76             if (abs((a-u)^(a-v))<=eps && (a-u)*(a-v)<=eps) return {true,0};
77             if (abs(u.y-v.y)<=eps) continue;
78             const Line uv={u,v-u};
79             if (u.y<v.y-eps && uv.toleft(a)<=0) continue;
80             if (u.y>v.y+eps && uv.toleft(a)>=0) continue;
81             if (u.y<a.y-eps && v.y>=a.y-eps) cnt++;
82             if (u.y>=a.y-eps && v.y<a.y-eps) cnt--;
83         }
84         return {false,cnt};
85     }
86
87     // 多边形面积的两倍
88     // 可用于判断点的存储顺序是顺时针或逆时针
89     T area() const {
90         T sum=0;
91         for (size_t i=0;i<p.size();i++) sum+=p[i]^p[nxt(i)];
92         return sum;
93     }
94
95     // 多边形的周长
96     long double circ() const {
97         long double sum=0;
98         for (size_t i=0;i<p.size();i++) sum+=p[i].dis(p[nxt(i)]);
99         return sum;
100     }
101 };
102
103 using Polygon=polygon<point_t>;
104
105 // 凸多边形
106 template<typename T> struct convex: polygon<T> {
107     // 闵可夫斯基和
108     convex operator+(const convex &c) const {

```

```

109 const auto &p=this->p;
110 vector<Segment> e1(p.size()),e2(c.p.size()),edge(p.size()+c.p.size());
111 vector<point<T>> res; res.reserve(p.size()+c.p.size());
112 const auto cmp=[](const Segment &u,const Segment &v) {return argcmp()(u.b-u.a
    ,v.b-v.a);};
113 for (size_t i=0;i<p.size();i++) e1[i]={p[i],p[this->nxt(i)]};
114 for (size_t i=0;i<c.p.size();i++) e2[i]={c.p[i],c.p[c.nxt(i)]};
115 rotate(e1.begin(),min_element(e1.begin(),e1.end(),cmp),e1.end());
116 rotate(e2.begin(),min_element(e2.begin(),e2.end(),cmp),e2.end());
117 merge(e1.begin(),e1.end(),e2.begin(),e2.end(),edge.begin(),cmp);
118 const auto check=[](const vector<point<T>> &res,const point<T> &u) {
119     const auto back1=res.back(),back2=*prev(res.end(),2);
120     return (back1-back2).toleft(u-back1)==0 && (back1-back2)*(u-back1)>=-eps;
121 };
122 auto u=e1[0].a+e2[0].a;
123 for (const auto &v:edge) {
124     while (res.size()>1 && check(res,u)) res.pop_back();
125     res.push_back(u);
126     u=u+v.b-v.a;
127 }
128 if (res.size()>1 && check(res,res[0])) res.pop_back();
129 return {res};
130 }
131
132 // 旋转卡壳
133 // 例: 凸多边形的直径的平方
134 T rotcaliper() const {
135     const auto &p=this->p;
136     if (p.size()==1) return 0;
137     if (p.size()==2) return p[0].dis2(p[1]);
138     const auto area=[](const point<T> &u,const point<T> &v,const point<T> &w){
139         return (w-u)^(w-v);};
140     T ans=0;
141     for (size_t i=0,j=1;i<p.size();i++) {
142         const auto nxti=this->nxt(i);
143         ans=max({ans,p[j].dis2(p[i]),p[j].dis2(p[nxti])});
144         while (area(p[this->nxt(j)],p[i],p[nxti])>=area(p[j],p[i],p[nxti])) {
145             j=this->nxt(j);
146             ans=max({ans,p[j].dis2(p[i]),p[j].dis2(p[nxti])});
147         }
148     }
149     return ans;
150 }
151
152 // 判断点是否在凸多边形内
153 // 复杂度 O(logn)
154 // -1 点在多边形边上 | 0 点在多边形外 | 1 点在多边形内
155 int is_in(const point<T> &a) const {
156     const auto &p=this->p;
157     if (p.size()==1) return a==p[0]?-1:0;
158     if (p.size()==2) return segment<T>{p[0],p[1]}.is_on(a)?-1:0;

```

```

158 if (a==p[0]) return -1;
159 if ((p[1]-p[0]).toleft(a-p[0])==-1 || (p.back()-p[0]).toleft(a-p[0])==1)
160     return 0;
161 const auto cmp=[](const point<T> &u,const point<T> &v){return (u-p[0]).
    toleft(v-p[0])==1;};
162 const size_t i=lower_bound(p.begin()+1,p.end(),a,cmp)-p.begin();
163 if (i==1) return segment<T>{p[0],p[i]}.is_on(a)?-1:0;
164 if (i==p.size()-1 && segment<T>{p[0],p[i]}.is_on(a)) return -1;
165 if (segment<T>{p[i-1],p[i]}.is_on(a)) return -1;
166 return (p[i]-p[i-1]).toleft(a-p[i-1])>0;
167 }
168
169 // 凸多边形关于某一方向的极点
170 // 复杂度 O(logn)
171 // 参考资料: https://codeforces.com/blog/entry/48868
172 template<typename F> size_t extreme(const F &dir) const {
173     const auto &p=this->p;
174     const auto check=[](const size_t i){return dir(p[i]).toleft(p[this->nxt(i)]-
175         p[i])>=0;};
176     const auto dir0=dir(p[0]); const auto check0=check(0);
177     if (!check0 && check(p.size()-1)) return 0;
178     const auto cmp=[](const point<T> &v) {
179         const size_t vi=&v-p.data();
180         if (vi==0) return 1;
181         const auto checkv=check(vi);
182         const auto t=dir0.toleft(v-p[0]);
183         if (vi==1 && checkv==check0 && t==0) return 1;
184         return checkv^(checkv==check0 && t<=0);
185     };
186     return partition_point(p.begin(),p.end(),cmp)-p.begin();
187 }
188
189 // 过凸多边形外一点求凸多边形的切线, 返回切点下标
190 // 复杂度 O(logn)
191 // 必须保证点在多边形外
192 pair<size_t,size_t> tangent(const point<T> &a) const {
193     const size_t i=extreme([&](const point<T> &u){return u-a;});
194     const size_t j=extreme([&](const point<T> &u){return a-u;});
195     return {i,j};
196 }
197
198 // 求平行于给定直线的凸多边形的切线, 返回切点下标
199 // 复杂度 O(logn)
200 pair<size_t,size_t> tangent(const line<T> &a) const {
201     const size_t i=extreme([&](...){return a.v;});
202     const size_t j=extreme([&](...){return -a.v;});
203     return {i,j};
204 }
205
206 using Convex=convex<point_t>;

```



```

206 // 圆
207 struct Circle {
208     Point c;
209     long double r;
210
211     bool operator==(const Circle &a) const {return c==a.c && abs(r-a.r)<=eps;}
212     long double circ() const {return 2*PI*r;} // 周长
213     long double area() const {return PI*r*r;} // 面积
214
215     // 点与圆的关系
216     // -1 圆上 | 0 圆外 | 1 圆内
217     int is_in(const Point &p) const {const long double d=p.dis(c); return abs(d-r)<=
218         eps?-1:d<r-eps;}
219
220     // 直线与圆关系
221     // 0 相离 | 1 相切 | 2 相交
222     int relation(const Line &l) const {
223         const long double d=l.dis(c);
224         if (d>r+eps) return 0;
225         if (abs(d-r)<=eps) return 1;
226         return 2;
227     }
228
229     // 圆与圆关系
230     // -1 相同 | 0 相离 | 1 外切 | 2 相交 | 3 内切 | 4 内含
231     int relation(const Circle &a) const {
232         if (*this==a) return -1;
233         const long double d=c.dis(a.c);
234         if (d>r+a.r+eps) return 0;
235         if (abs(d-r-a.r)<=eps) return 1;
236         if (abs(d-abs(r-a.r))<=eps) return 3;
237         if (d<abs(r-a.r)-eps) return 4;
238         return 2;
239     }
240
241     // 直线与圆的交点
242     vector<Point> inter(const Line &l) const {
243         const long double d=l.dis(c);
244         const Point p=l.proj(c);
245         const int t=relation(l);
246         if (t==0) return vector<Point>();
247         if (t==1) return vector<Point>{p};
248         const long double k=sqrt(r*r-d*d);
249         return vector<Point>{p-(l.v/l.v.len())*k,p+(l.v/l.v.len())*k};
250     }
251
252     // 圆与圆交点
253     vector<Point> inter(const Circle &a) const {
254         const long double d=c.dis(a.c);
255         const int t=relation(a);

```

```

256     if (t==1 || t==0 || t==4) return vector<Point>();
257     Point e=a.c-c; e=e/e.len()*r;
258     if (t==1 || t==3) {
259         if (r*r+d*d-a.r*a.r>=eps) return vector<Point>{c+e};
260         return vector<Point>{c-e};
261     }
262     const long double costh=(r*r+d*d-a.r*a.r)/(2*r*d),sinth=sqrt(1-costh*costh);
263     return vector<Point>{c+e.rot(costh,-sinth),c+e.rot(costh,sinth)};
264 }
265
266 // 圆与圆交面积
267 long double inter_area(const Circle &a) const {
268     const long double d=c.dis(a.c);
269     const int t=relation(a);
270     if (t==1) return area();
271     if (t<2) return 0;
272     if (t>2) return min(area(),a.area());
273     const long double costh1=(r*r+d*d-a.r*a.r)/(2*r*d),costh2=(a.r*a.r+d*d-r*r)
274         /(2*a.r*d);
275     const long double sinth1=sqrt(1-costh1*costh1),sinth2=sqrt(1-costh2*costh2);
276     const long double th1=acos(costh1),th2=acos(costh2);
277     return r*r*(th1-costh1*sinth1)+a.r*a.r*(th2-costh2*sinth2);
278 }
279
280 // 过圆外一点圆的切线
281 vector<Line> tangent(const Point &a) const {
282     const int t=is_in(a);
283     if (t==1) return vector<Line>();
284     if (t==1) {
285         const Point v={-(a-c).y,(a-c).x};
286         return vector<Line>{{a,v}};
287     }
288     Point e=a-c; e=e/e.len()*r;
289     const long double costh=r/c.dis(a),sinth=sqrt(1-costh*costh);
290     const Point t1=c+e.rot(costh,-sinth),t2=c+e.rot(costh,sinth);
291     return vector<Line>{{a,t1-a},{a,t2-a}};
292 }
293
294 // 两圆的公切线
295 vector<Line> tangent(const Circle &a) const {
296     const int t=relation(a);
297     vector<Line> lines;
298     if (t==1 || t==4) return lines;
299     if (t==1 || t==3) {
300         const Point p=inter(a)[0],v={-(a.c-c).y,(a.c-c).x};
301         lines.push_back({p,v});
302     }
303     const long double d=c.dis(a.c);
304     const Point e=(a.c-c)/(a.c-c).len();
305     if (t<2) {
306         const long double costh=(r-a.r)/d,sinth=sqrt(1-costh*costh);

```



```

306     const Point d1=e.rot(costh,-sinh),d2=e.rot(costh,sinh);
307     const Point u1=c+d1*r,u2=c+d2*r,v1=a.c+d1*a.r,v2=a.c+d2*a.r;
308     lines.push_back({u1,v1-u1}); lines.push_back({u2,v2-u2});
309 }
310 if (t==0) {
311     const long double costh=(r+a.r)/d,sinh=sqrt(1-costh*costh);
312     const Point d1=e.rot(costh,-sinh),d2=e.rot(costh,sinh);
313     const Point u1=c+d1*r,u2=c+d2*r,v1=a.c-d1*a.r,v2=a.c-d2*a.r;
314     lines.push_back({u1,v1-u1}); lines.push_back({u2,v2-u2});
315 }
316 return lines;
317 }
318
319 // 圆的反演
320 tuple<int,Circle,Line> inverse(const Line &l) const {
321     const Circle null_c={{0.0,0.0},{0.0}};
322     const Line null_l={{0.0,0.0},{0.0,0.0}};
323     if (l.toleft(c)==0) return {2,null_c,l};
324     const Point v=l.toleft(c)==1?Point{l.v.y,-l.v.x}:Point{-l.v.y,l.v.x};
325     const long double d=r*r/l.dis(c);
326     const Point p=c+v/v.len()*d;
327     return {1,{(c+p)/2,d/2},null_l};
328 }
329
330 tuple<int,Circle,Line> inverse(const Circle &a) const {
331     const Circle null_c={{0.0,0.0},{0.0}};
332     const Line null_l={{0.0,0.0},{0.0,0.0}};
333     const Point v=a.c-c;
334     if (a.is_in(c)==-1) {
335         const long double d=r*r/(a.r+a.r);
336         const Point p=c+v/v.len()*d;
337         return {2,null_c,{p},{-v.y,v.x}};
338     }
339     if (c==a.c) return {1,{c,r*r/a.r},null_l};
340     const long double d1=r*r/(c.dis(a.c)-a.r),d2=r*r/(c.dis(a.c)+a.r);
341     const Point p=c+v/v.len()*d1,q=c+v/v.len()*d2;
342     return {1,{(p+q)/2,p.dis(q)/2},null_l};
343 }
344 };
345
346 // 圆与多边形面积交
347 long double area_inter(const Circle &circ,const Polygon &poly) {
348     const auto cal=[](const Circle &circ,const Point &a,const Point &b) {
349         if ((a-circ.c).toleft(b-circ.c)==0) return 0.0l;
350         const auto ina=circ.is_in(a),inb=circ.is_in(b);
351         const Line ab={a,b-a};
352         if (ina && inb) return ((a-circ.c)^(b-circ.c))/2;
353         if (ina && !inb) {
354             const auto t=circ.inter(ab);
355             const Point p=t.size()==1?t[0]:t[1];
356             const long double ans=((a-circ.c)^(p-circ.c))/2;

```

```

357         const long double th=(p-circ.c).ang(b-circ.c);
358         const long double d=circ.r*circ.r*th/2;
359         if ((a-circ.c).toleft(b-circ.c)==1) return ans+d;
360         return ans-d;
361     }
362     if (!ina && inb) {
363         const Point p=circ.inter(ab)[0];
364         const long double ans=((p-circ.c)^(b-circ.c))/2;
365         const long double th=(a-circ.c).ang(p-circ.c);
366         const long double d=circ.r*circ.r*th/2;
367         if ((a-circ.c).toleft(b-circ.c)==1) return ans+d;
368         return ans-d;
369     }
370     const auto p=circ.inter(ab);
371     if (p.size()==2 && Segment{a,b}.dis(circ.c)<=circ.r+eps) {
372         const long double ans=((p[0]-circ.c)^(p[1]-circ.c))/2;
373         const long double th1=(a-circ.c).ang(p[0]-circ.c),th2=(b-circ.c).ang(p[1]-circ.c);
374         const long double d1=circ.r*circ.r*th1/2,d2=circ.r*circ.r*th2/2;
375         if ((a-circ.c).toleft(b-circ.c)==1) return ans+d1+d2;
376         return ans-d1-d2;
377     }
378     const long double th=(a-circ.c).ang(b-circ.c);
379     if ((a-circ.c).toleft(b-circ.c)==1) return circ.r*circ.r*th/2;
380     return -circ.r*circ.r*th/2;
381 };
382
383 long double ans=0;
384 for (size_t i=0;i<poly.p.size();i++) {
385     const Point a=poly.p[i],b=poly.p[poly.nxt(i)];
386     ans+=cal(circ,a,b);
387 }
388 return ans;
389 }
390
391 // 点集的凸包
392 // Andrew 算法, 复杂度  $O(n\log n)$ 
393 Convex convexhull(vector<Point> p) {
394     vector<Point> st;
395     if (p.empty()) return Convex{st};
396     sort(p.begin(),p.end());
397     const auto check=[](const vector<Point> &st,const Point &u) {
398         const auto back1=st.back(),back2=*prev(st.end(),2);
399         return (back1-back2).toleft(u-back1)<=0;
400     };
401     for (const Point &u:p) {
402         while (st.size()>1 && check(st,u)) st.pop_back();
403         st.push_back(u);
404     }
405     size_t k=st.size();
406     p.pop_back(); reverse(p.begin(),p.end());

```

```

407 for (const Point &u:p) {
408     while (st.size()>k && check(st,u)) st.pop_back();
409     st.push_back(u);
410 }
411 st.pop_back();
412 return Convex{st};
413 }
414
415 // 半平面交
416 // 排序增量法, 复杂度  $O(n\log n)$ 
417 // 输入与返回值都是用直线表示的半平面集合
418 vector<Line> halfinter(vector<Line> l, const point_t lim=1e9) {
419     const auto check=[](const Line &a,const Line &b,const Line &c){return a.toleft(b.
420         inter(c))<0;};
421     // 无精度误差的方法, 但注意取值范围会扩大到三次方
422     /*const auto check=[](const Line &a,const Line &b,const Line &c) {
423         const Point p=a.v*(b.v^c.v),q=b.v*(b.v^c.v)+b.v*(c.v^(b.p-c.p))-a.p*(b.v^c.v)
424         ;
425         return p.toleft(q)<0;
426     };*/
427     l.push_back({{-lim,0},{0,-1}}); l.push_back({{0,-lim},{1,0}});
428     l.push_back({{lim,0},{0,1}}); l.push_back({{0,lim},{-1,0}});
429     sort(l.begin(),l.end());
430     deque<Line> q;
431     for (size_t i=0;i<l.size();i++) {
432         if (i>0 && l[i-1].v.toleft(l[i].v)==0 && l[i-1].v*l[i].v>eps) continue;
433         while (q.size()>1 && check(l[i],q.back(),q[q.size()-2])) q.pop_back();
434         while (q.size()>1 && check(l[i],q[0],q[1])) q.pop_front();
435         if (!q.empty() && q.back().v.toleft(l[i].v)<=0) return vector<Line>();
436         q.push_back(l[i]);
437     }
438     while (q.size()>1 && check(q[0],q.back(),q[q.size()-2])) q.pop_back();
439     while (q.size()>1 && check(q.back(),q[0],q[1])) q.pop_front();
440     return vector<Line>(q.begin(),q.end());
441 }

```

7.5 点集形成的最小最大三角形

```

1 #include "geometry_base.cpp"
2
3 // 极角序扫描线, 复杂度  $O(n^2\log n)$ 
4 // 最大三角形问题可以使用凸包与旋转卡壳做到  $O(n^2)$ 
5 pair<point_t,point_t> minmax_triangle(const vector<Point> &vec) {
6     if (vec.size()<=2) return {0,0};
7     vector<pair<int,int>> evt;
8     evt.reserve(vec.size()*vec.size());
9     point_t maxans=0,minans=INF;
10    for (size_t i=0;i<vec.size();i++) {
11        for (size_t j=0;j<vec.size();j++) {

```

```

12            if (i==j) continue;
13            if (vec[i]==vec[j]) minans=0;
14            else evt.push_back({i,j});
15        }
16    }
17    sort(evt.begin(),evt.end(),[&](const pair<int,int> &u,const pair<int,int> &v) {
18        const Point du=vec[u.second]-vec[u.first],dv=vec[v.second]-vec[v.first];
19        return argcmp()({du.y,-du.x},{dv.y,-dv.x});
20    });
21    vector<size_t> vx(vec.size()),pos(vec.size());
22    for (size_t i=0;i<vec.size();i++) vx[i]=i;
23    sort(vx.begin(),vx.end(),[&](int x,int y){return vec[x]<vec[y];});
24    for (size_t i=0;i<vx.size();i++) pos[vx[i]]=i;
25    for (auto [u,v]:evt) {
26        const size_t i=pos[u],j=pos[v];
27        const size_t l=min(i,j),r=max(i,j);
28        const Point vecu=vec[u],vecv=vec[v];
29        if (l>0) minans=min(minans,abs((vec[vx[l-1]]-vecu)^(vec[vx[l-1]]-vecv)));
30        if (r<vx.size()-1) minans=min(minans,abs((vec[vx[r+1]]-vecu)^(vec[vx[r+1]]-
31            vecv)));
32        maxans=max({maxans,abs((vec[vx[0]]-vecu)^(vec[vx[0]]-vecv)),abs((vec[vx.back
33            ()]-vecu)^(vec[vx.back()-vecv]))});
34        if (i<j) swap(vx[i],vx[j]),pos[u]=j,pos[v]=i;
35    }
36    return {minans,maxans};
37 }

```

7.6 平面最近点对

```

1 #include "geometry_base.cpp"
2
3 // 扫描线, 复杂度  $O(n\log n)$ 
4 point_t closest_pair(vector<Point> points) {
5     sort(points.begin(),points.end());
6     const auto cmpy=[](const Point &a,const Point &b){if (abs(a.y-b.y)<=eps) return a
7         .x<b.x-eps; return a.y<b.y-eps;};
8     multiset<Point,decltype(cmpy)> s{cmpy};
9     point_t ans=INF;
10    for (size_t i=0,l=0;i<points.size();i++) {
11        const point_t sqans=sqrtl(ans)+1; // 整数情况
12        // const point_t sqans=sqrtl(ans)+1; // 浮点数情况
13        while (l<i && points[l].x-points[i].x>=sqans) s.erase(s.find(points[l]));
14        for (auto it=s.lower_bound(Point{-INF,points[i].y-sqans});it!=s.end()&&it->y-
15            points[i].y<=sqans;it++) {
16            ans=min(ans,points[i].dis2(*it));
17        }
18        s.insert(points[i]);
19    }
20    return ans;
21 }

```

19 }

7.7 线段扫线判交

```

1 #include "geometry_base.cpp"
2
3 // 扫描线, 复杂度 O(nlogn)
4 bool segs_inter(const vector<Segment> &segs) {
5     if (segs.empty()) return false;
6     using seq_t=tuple<point_t,int,Segment>;
7     const auto seqcmp=[](const seq_t &u, const seq_t &v) {
8         const auto [u0,u1,u2]=u;
9         const auto [v0,v1,v2]=v;
10        if (abs(u0-v0)<=eps) return make_pair(u1,u2)<make_pair(v1,v2);
11        return u0<v0-eps;
12    };
13    vector<seq_t> seq;
14    for (auto seg:segs) {
15        if (seg.a.x>seg.b.x+eps) swap(seg.a,seg.b);
16        seq.push_back({seg.a.x,0,seg});
17        seq.push_back({seg.b.x,1,seg});
18    }
19    sort(seq.begin(),seq.end(),seqcmp);
20    point_t x_now;
21    auto cmp=[](const Segment &u, const Segment &v) {
22        if (abs(u.a.x-u.b.x)<=eps || abs(v.a.x-v.b.x)<=eps) return u.a.y<v.a.y-eps;
23        return ((x_now-u.a.x)*(u.b.y-u.a.y)+u.a.y*(u.b.x-u.a.x))*(v.b.x-v.a.x)<(((
24            x_now-v.a.x)*(v.b.y-v.a.y)+v.a.y*(v.b.x-v.a.x))*(u.b.x-u.a.x)-eps;
25    });
26    multiset<Segment,decltype(cmp)> s{cmp};
27    for (const auto [x,o,seg]:seq) {
28        x_now=x;
29        const auto it=s.lower_bound(seg);
30        if (o==0) {
31            if (it!=s.end() && seg.is_inter(*it)) return true;
32            if (it!=s.begin() && seg.is_inter(*prev(it))) return true;
33            s.insert(seg);
34        }
35        else {
36            if (next(it)!=s.end() && it!=s.begin() && (*prev(it)).is_inter(*next(it))
37                ) return true;
38            s.erase(it);
39        }
40    }
41    return false;
42 }

```

7.8 多边形面积并

```

1 #include "geometry_base.cpp"
2
3 // 轮廓积分, 复杂度 O(n^2logn), n为边数
4 // ans[i] 表示被至少覆盖了 i+1 次的区域的面积
5 vector<long double> area_union(const vector<Polygon> &polys) {
6     const size_t siz=polys.size();
7     vector<vector<pair<Point,Point>>> segs(siz);
8     const auto check=[](const Point &u,const Segment &e){return !((u<e.a && u<e.b) ||
9         (u>e.a && u>e.b));};
10
11     auto cut_edge=[](const Segment &e,const size_t i) {
12         const Line le{e.a,e.b-e.a};
13         vector<pair<Point,int>> evt;
14         evt.push_back({e.a,0}); evt.push_back({e.b,0});
15         for (size_t j=0;j<polys.size();j++) {
16             if (i==j) continue;
17             const auto &pj=polys[j];
18             for (size_t k=0;k<pj.p.size();k++) {
19                 const Segment s={pj.p[k],pj.p[pj.nxt(k)]};
20                 if (le.toleft(s.a)==0 && le.toleft(s.b)==0) {
21                     evt.push_back({s.a,0});
22                     evt.push_back({s.b,0});
23                 }
24                 else if (s.is_inter(le)) {
25                     const Line ls{s.a,s.b-s.a};
26                     const Point u=le.inter(ls);
27                     if (le.toleft(s.a)<0 && le.toleft(s.b)>=0) evt.push_back({u,-1});
28                     else if (le.toleft(s.a)>=0 && le.toleft(s.b)<0) evt.push_back({u,1});
29                 }
30             }
31         }
32         sort(evt.begin(),evt.end());
33         if (e.a>e.b) reverse(evt.begin(),evt.end());
34         int sum=0;
35         for (size_t i=0;i<evt.size();i++) {
36             sum+=evt[i].second;
37             const Point u=evt[i].first,v=evt[i+1].first;
38             if (!(u==v) && check(u,e) && check(v,e)) segs[sum].push_back({u,v});
39             if (v==e.b) break;
40         }
41     };
42
43     for (size_t i=0;i<polys.size();i++) {
44         const auto &pi=polys[i];
45         for (size_t k=0;k<pi.p.size();k++) {
46             const Segment ei={pi.p[k],pi.p[pi.nxt(k)]};
47             cut_edge(ei,i);
48         }
49     }
50 }

```

```

48 }
49 vector<long double> ans(siz);
50 for (size_t i=0;i<siz;i++) {
51     long double sum=0;
52     sort(segs[i].begin(),segs[i].end());
53     int cnt=0;
54     for (size_t j=0;j<segs[i].size();j++) {
55         if (j>0 && segs[i][j]==segs[i][j-1]) segs[i++cnt].push_back(segs[i][j-1]);
56         else cnt=0,sum+=segs[i][j].first^segs[i][j].second;
57     }
58     ans[i]=sum/2;
59 }
60 return ans;
61 }

```

7.9 圆面积并

```

1 #include "geometry_base.cpp"
2
3 // 轮廓积分, 复杂度  $O(n^2 \log n)$ 
4 // ans[i] 表示被至少覆盖了 i+1 次的区域的面积
5 vector<long double> area_union(const vector<Circle> &circs) {
6     const size_t siz=circs.size();
7     using arc_t=tuple<Point,long double,long double,long double>;
8     vector<vector<arc_t>> arcs(siz);
9     const auto eq=[](const arc_t &u,const arc_t &v) {
10         const auto [u1,u2,u3,u4]=u;
11         const auto [v1,v2,v3,v4]=v;
12         return u1==v1 && abs(u2-v2)<=eps && abs(u3-v3)<=eps && abs(u4-v4)<=eps;
13     };
14
15     auto cut_circ=[](const Circle &ci,const size_t i) {
16         vector<pair<long double,int>> evt;
17         evt.push_back({-PI,0}); evt.push_back({PI,0});
18         int init=0;
19         for (size_t j=0;j<circs.size();j++) {
20             if (i==j) continue;
21             const Circle &cj=circs[j];
22             if (ci.r<cj.r-eps && ci.relation(cj)>=3) init++;
23             const auto inters=ci.inter(cj);
24             if (inters.size()==1) evt.push_back({atan2l((inters[0]-ci.c).y,(inters[0]-ci.c).x),0});
25             if (inters.size()==2) {
26                 const Point dl=inters[0]-ci.c,dr=inters[1]-ci.c;
27                 long double argl=atan2l(dl.y,dl.x),argr=atan2l(dr.y,dr.x);
28                 if (abs(argl+PI)<=eps) argl=PI;
29                 if (abs(argr+PI)<=eps) argr=PI;
30                 if (argl>argr+eps) {

```

```

31         evt.push_back({argl,1}); evt.push_back({PI,-1});
32         evt.push_back({-PI,1}); evt.push_back({argr,-1});
33     }
34     else {
35         evt.push_back({argl,1});
36         evt.push_back({argr,-1});
37     }
38 }
39
40 sort(evt.begin(),evt.end());
41 int sum=init;
42 for (size_t i=0;i<evt.size();i++) {
43     sum+=evt[i].second;
44     if (abs(evt[i].first-evt[i+1].first)>eps) arcs[sum].push_back({ci.c,ci.r,
45         evt[i].first,evt[i+1].first});
46     if (abs(evt[i+1].first-PI)<=eps) break;
47 }
48 };
49
50 const auto oint=[](const arc_t &arc) {
51     const auto [cc,cr,l,r]=arc;
52     if (abs(r-l-PI-PI)<=eps) return 2.0l*PI*cr*cr;
53     return cr*cr*(r-l)+cc.x*cr*(sin(r)-sin(l))-cc.y*cr*(cos(r)-cos(l));
54 };
55
56 for (size_t i=0;i<circs.size();i++) {
57     const auto &ci=circs[i];
58     cut_circ(ci,i);
59 }
60 vector<long double> ans(siz);
61 for (size_t i=0;i<siz;i++) {
62     long double sum=0;
63     sort(arcs[i].begin(),arcs[i].end());
64     int cnt=0;
65     for (size_t j=0;j<arcs[i].size();j++) {
66         if (j>0 && eq(arcs[i][j],arcs[i][j-1])) arcs[i++cnt].push_back(arcs[i][j-1]);
67         else cnt=0,sum+=oint(arcs[i][j]);
68     }
69     ans[i]=sum/2;
70 }
71 return ans;
72 }

```

8 Misc | 杂项

8.1 Bitset | 暴力滴神

set(x): 将 x 位设置为 1, 如果不填则全部为 1

reset(x): 将 x 位设置为 0, 如果不填则全部为 0

flip(x): 将 x 位取反, 如果不填则全部取反

count(): 返回 1 的个数

size(): 返回位数

any(): 返回是否存在 1

_Find_fisrt(): 返回第一个 1 的位置

_Find_next(x): 返回 x 之后第一个 1 的位置

9 Temp | 临时式子整理

9.1 交换律

$p(k)$ 是指标集 k 的一个排列。

$$\sum_{i \in k} a_i = \sum_{i \in p(k)} a_i$$

$$a_1 + a_2 + a_3 + a_4 = a_2 + a_1 + a_3 + a_4$$

9.2 结合律

$$\sum_{i=1}^n a_i + \sum_{i=1}^n b_i = \sum_{i=1}^n (a_i + b_i)$$

9.3 分配律

$$\sum_i c \cdot a_i = c \cdot \sum_i a_i$$

10 Summation Transformation | 和式的变换技巧

10.1 替换条件式

把写在求和符号下标位置的条件改为艾弗森括号的形式, 使总的求和式化为枚举所有变量的求和式, 便于处理。

$$\sum_{d | \gcd(i, j)} f(d) = \sum_{d=1}^{\min(i, j)} [d|i][d|j]f(d)$$

10.2 替换指标变量

$$\begin{aligned} \sum_{i=1}^n \sum_{j=1}^m [\gcd(i, j) = k] \\ &= \sum_{i=1}^{\lfloor n/k \rfloor} \sum_{j=1}^{\lfloor m/k \rfloor} [\gcd(ik, jk) = k] \\ &= \sum_{i=1}^{\lfloor n/k \rfloor} \sum_{j=1}^{\lfloor m/k \rfloor} [\gcd(i, j) = 1] \end{aligned}$$

$$\sum_{i=0}^n (-1)^{n-i} f(n-i) = \sum_{i=0}^n (-1)^i f(i)$$

10.3 改变求和次序

10.3.1 基础

$$\begin{aligned} \sum_i \sum_j f(i, j) &= \sum_i \sum_j f(i, j) [1 \leq i \leq n] [1 \leq j \leq i] \\ &= \sum_j \sum_i f(i, j) [1 \leq j \leq i] [1 \leq i \leq n] \\ &= \sum_j \sum_i f(i, j) [j \leq i \leq n] [1 \leq j \leq n] \\ &= \sum_{j=1}^n \sum_{i=j}^n f(i, j) \end{aligned}$$

10.3.2 复杂的改变求和次序

为了改变求和次序，有时候我们可能需要通过解析不等式的方式。

$$\begin{aligned} & \sum_{i=1}^n \sum_{j=i+1}^n f(i, j) \\ &= \sum_j \sum_i f(i, j) [1 \leq i \leq j-1] [2 \leq j \leq n] \\ &= \sum_{j=2}^n \sum_{i=1}^{j-1} f(i, j) \end{aligned}$$

10.4 分离变量

$$\sum_i \sum_j f(i)g(j) = \left(\sum_i f(i) \right) \left(\sum_j g(j) \right)$$

10.5 改变枚举范围

10.5.1 扩大枚举范围

扩大范围可以将变量有关的枚举上下界改变为常量，便于进行下一步处理。

$$\begin{aligned} & \sum_{i=1}^n \sum_{j=1}^m \sum_{d=1}^{\min(i, j)} [d|i][d|j]f(d) \\ &= \sum_{d=1}^n \sum_{i=1}^n \sum_{j=1}^m [d|i][d|j]f(d) \end{aligned}$$

10.5.2 缩小枚举范围

缩小枚举范围使枚举条件简化，也是一个常见技巧。

$$\begin{aligned} & \sum_{i=1}^n \sum_{j=1}^m [\gcd(i, j) = k] \\ &= \sum_{i=1}^{\lfloor n/k \rfloor} \sum_{j=1}^{\lfloor m/k \rfloor} [\gcd(i, j) = 1] \end{aligned}$$

10.6 示例：组合数

设组合数 $\binom{n}{k}$ 表示从 n 个不同物品中取出 k 个，不考虑取出的顺序，有多少种方案。

$$\begin{aligned} & \sum_{i=0}^{n-1} \binom{n-1}{i} n^{i+1} + \sum_{i=0}^n \binom{n-1}{i} n^i \\ &= \sum_{i=0}^n \binom{n-1}{i} n^{i+1} + \sum_{i=0}^n \binom{n-1}{i} n^i \\ &= \sum_{i=0}^n \binom{n-1}{i-1} n^i + \sum_{i=0}^n \binom{n-1}{i} n^i \\ &= \sum_{i=0}^n \left(\binom{n-1}{i-1} + \binom{n-1}{i} \right) n^i \\ &= \sum_{i=0}^n \binom{n}{i} n^i \end{aligned}$$

10.7 和式的变换公式

10.7.1 公式 1

这个式子似乎没什么用。

$$\left(\sum_{i=1}^n a_i \right)^2 = \sum_{i=1}^n a_i^2 + 2 \sum_{1 \leq i < j \leq n} a_i a_j$$

10.7.2 公式 2

这个公式也是需要背的，因为其实用化不了时间复杂度，直接背比较好。证明也很容易：

$$\begin{aligned} & \sum_{1 \leq i < j \leq n} (a_i - a_j)^2 \\ &= \sum_{1 \leq i < j \leq n} (a_i^2 + a_j^2 - 2a_i a_j) \\ &= \sum_{1 \leq i < j \leq n} a_i^2 + \sum_{1 \leq i < j \leq n} a_j^2 - 2 \sum_{1 \leq i < j \leq n} a_i a_j \\ &= (n-1) \sum_{i=1}^n a_i^2 - \left(\left(\sum_{i=1}^n a_i \right)^2 - \sum_{i=1}^n a_i^2 \right) \\ &= n \sum_{i=1}^n a_i^2 - \left(\sum_{i=1}^n a_i \right)^2 \end{aligned}$$

10.8 阿贝尔部分求和公式

这个式子也是中看不中用。

$$\sum_{i=1}^n a_i b_i = b_n \sum_{i=1}^n a_i - \sum_{i=1}^{n-1} \left(\sum_{j=1}^i a_j \right) (b_{i+1} - b_i)$$